

Hi, 小伙伴你好~

我们在维护者[全网最大的计算机相关编程书籍分享仓库](#)，目前已有超过 1000本 的计算机经典书籍了。

其中涉及C/C++、Java、Python、Go语言等各种编程语言，还有数据结构与算法、操作系统、后端架构、计算机系统知识、数据库、计算机网络、设计模式、前端、汇编以及校招社招各种面经等~

只有你想不到，没有我们没分享到的计算机学习书籍，如果真的有我们没能分享到的书籍或者是你所需要的，欢迎添加下方联系方式来告诉我们，期待你的到来。

在此承诺[本仓库](#)永不收费，永远免费分享给有需要的人，希望自己的**辛苦结晶**能够帮助到曾经那些像我一样的小白、新手、在校生们，为那些曾经像我一样迷茫的人指明一条路。

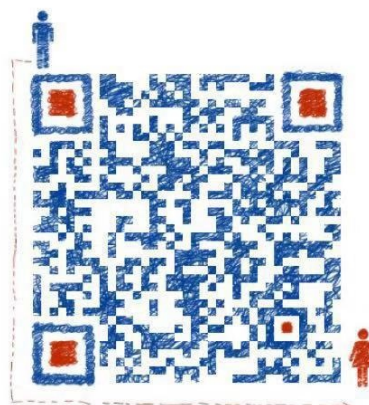
告诉他们，你是可以的！

[本仓库](#)无偿分享各种计算机书籍、各种专业PDF资料以及个人笔记资料等，所有权归仓库作者阿秀（公众号【[拓跋阿秀](#)】）所有，如有疑问提请issue或者联系本人forthespada@foxmail.com，感谢~

衷心希望我以前踩过的坑，你们不要再踩，我走过的路，你们可以照着走下来。

因为从双非二本爬到字节跳动这种大厂来，太TMD难了。

QQ群：①群:1002342950、②群:826980895



欢迎来唠嗑~



欢迎扫码关注

Learning JavaScript Data Structures and Algorithms Second Edition

学习JavaScript 数据结构与算法（第2版）

[巴西] Loiane Groner 著 邓钢 孙晓博 吴双 陈迪 袁源 译

用JavaScript学习常用的数据结构和算法，
高效解决计算机科学中的常见问题。



中国工信出版集团

图灵社区会员 sumika(575591894@qq.com) 专享 尊重版权



人民邮电出版社

POSTS & TELECOM PRESS

数字版权声明

图灵社区的电子书没有采用专有客户端，您可以在任意设备上，用自己喜欢的浏览器和PDF阅读器进行阅读。

但您购买的电子书仅供您个人使用，未经授权，不得进行传播。

我们愿意相信读者具有这样的良知和觉悟，与我们共同保护知识产权。

如果购买者有侵权行为，我们可能对该用户实施包括但不限于关闭该帐号等维权措施，并可能追究法律责任。

Loiane Groner 花旗银行软件开发经理，负责海外项目的开发和团队管理；原IBM公司系统分析师及团队负责人；巴西坎皮纳斯Java用户组（CampinasJUG）领导者、圣埃斯皮里图Java用户组（ESJUG）协调人；巴西各大型技术会议特邀发言人；Sencha和Java技术布道者，通过博客（<http://loianegroner.com>）为软件开发社区撰稿，发表关于IT职业发展和常用开发技术的文章和视频。另著有《精通Ext JS》等书。

邓钢 非专业80后程序员，爱好Web技术。曾在盛大创新院担任前端工程师，现就职于IBM中国开发中心。除了不断磨练前端技术，工作之余对算法、数据结构等计算机科学的知识也很感兴趣。

孙晓博 毕业于浙江大学，先后在百度、阿里任职。工作方向为移动前端，对WPO、组件化和移动站自动化测试有浓厚兴趣。微博：[@筱卜D](#)。

吴双 毕业于华中科技大学，现就职于饿了么大前端部。主攻大型Web应用的开发构建及新技术探索，崇尚用工具解放双手。微博：[@吴双Orange](#)。

陈迪 上海交大机械系本科、香港科技大学硕士，目前在从事招聘的创业公司上海寻仟信息科技有限公司做Web开发，热爱技术。

袁源 毕业于厦门大学。2011年起从事前端开发，目前就职于百度。



图灵程序设计丛书

Learning JavaScript Data Structures and Algorithms
Second Edition

学习JavaScript数据结构与算法 (第2版)

[巴西] Loiane Groner 著

邓钢 孙晓博 吴双 陈迪 袁源 译

人民邮电出版社
北 京

图书在版编目 (C I P) 数据

学习JavaScript数据结构与算法 : 第2版 / (巴西)
洛伊安妮·格罗纳 (Loiane Groner) 著 ; 邓钢等译. --
2版. -- 北京 : 人民邮电出版社, 2017.9
(图灵程序设计丛书)
ISBN 978-7-115-46753-9

I. ①学… II. ①洛… ②邓… III. ①JAVA语言—程
序设计 IV. ①TP312.8

中国版本图书馆CIP数据核字(2017)第213790号

内 容 提 要

本书首先介绍了 JavaScript 语言的基础知识以及 ES6 和 ES7 中引入的新功能, 接下来讨论了数组、栈、队列、链表、集合、字典、散列表、树、图等数据结构, 之后探讨了各种排序和搜索算法, 包括冒泡排序、选择排序、插入排序、归并排序、快速排序、堆排序、计数排序、桶排序、基数排序、顺序搜索、二分搜索, 然后介绍了动态规划和贪心算法等常用的高级算法以及函数式编程, 最后还介绍了如何计算算法的复杂度。

本书适用于前端 Web 开发人员, 以及所有对 JavaScript 数据结构与算法感兴趣的读者。

-
- ◆ 著 [巴西] Loiane Groner
译 邓 钢 孙晓博 吴 双 陈 迪 袁 源
责任编辑 岳新欣
责任印制 彭志环
- ◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路11号
邮编 100164 电子邮件 315@ptpress.com.cn
网址 <http://www.ptpress.com.cn>
北京 印刷
- ◆ 开本: 800×1000 1/16
印张: 14.5
字数: 342千字 2017年9月第2版
印数: 5 601-9 100册 2017年9月北京第1次印刷
著作权合同登记号 图字: 01-2017-5589号
-

定价: 49.00元

读者服务热线: (010)51095186转600 印装质量热线: (010)81055316

反盗版热线: (010)81055315

广告经营许可证: 京东工商广登字 20170147 号

版 权 声 明

Copyright © 2016 Packt Publishing. First published in the English language under the title *Learning JavaScript Data Structures and Algorithms, Second Edition*.

Simplified Chinese-language edition copyright © 2017 by Posts & Telecom Press. All rights reserved.

本书中文简体字版由Packt Publishing授权人民邮电出版社独家出版。未经出版者书面许可，不得以任何方式复制或抄袭本书内容。

版权所有，侵权必究。

致 谢

我要感谢父母这些年来对我的教育、指导和建议，帮助我成为一个更好、更专业的人。特别要感谢我的丈夫给予我的耐心、支持和鼓励。我要感谢Packt出版社给我机会，让我能够就自己真正钟爱的话题写书。感谢在图书创作、审阅和出版过程中帮助过我的所有人。

我还要感谢这本书以及我撰写的其他书的读者，感谢你们的支持和反馈。你们的反馈非常宝贵，帮助我成长为一名更优秀的作者和专业人士。非常感谢！

前言

JavaScript是当下最流行的编程语言之一。由于浏览器的原生支持（无需安装任何插件），JavaScript也被称作“互联网语言”。JavaScript的应用非常广泛，不仅被用于前端开发，也被用到服务器（Node.js）和数据库（MongoDB）环境中。

对任何专业技术人员来说，理解数据结构都非常重要。作为软件开发者，我们要能够借助编程语言来解决问题，而数据结构是这些问题的解决方案中不可或缺的一部分。如果选择了不恰当的数据结构，可能会影响所写程序的性能。因此，了解不同数据结构和它们的适用范围十分重要。

算法在计算机科学中扮演着非常重要的角色。解决一个问题有很多种方法，但有些方法会比其他方法更好。因此，了解一下最著名的算法也很重要。

本书为数据结构和算法初学者所写，也为熟悉数据结构和算法，但想在JavaScript语言中使用它们的人所写。

快乐地编码吧！

本书结构

第1章“JavaScript简介”，讲述了JavaScript的基础知识，它们可以帮助你更好地学习数据结构和算法，同时还介绍了如何搭建开发环境来运行书中的代码示例。

第2章“数组”，介绍了如何使用数组这种最基础且最常用的数据结构。这一章演示了如何对数组声明、初始化、添加和删除其中的元素，还讲述了如何使用JavaScript语言本身支持的数组方法。

第3章“栈”，介绍了栈这种数据结构，演示了如何创建栈以及怎样添加和删除元素，还讨论了如何用栈解决计算机科学中的一些问题。

第4章“队列”，详述了队列这种数据结构，演示了如何创建队列，以及如何添加和删除队列中的元素，还讨论了如何用队列解决计算机科学中的一些问题，以及栈和队列的主要区别。

第5章“链表”，讲解如何用对象和指针从头创建链表这种数据结构。这一章除了讨论如何声

明、创建、添加和删除链表元素之外，还介绍了不同类型的链表，例如双向链表和循环链表。

第6章“集合”，介绍了集合这种数据结构，讨论了如何用集合存储非重复性的元素。此外，还详述了对集合的各种操作以及相应代码的实现。

第7章“字典和散列表”，深入讲解字典、散列表及它们之间的区别。这一章介绍了这两种数据结构是如何声明、创建和使用的，还探讨了如何解决散列冲突，以及如何创建更高效的散列函数。

第8章“树”，讲解了树这种数据结构和它的相关术语，重点讨论了二叉搜索树，以及如何在树中搜索、遍历、添加和删除节点。如果想更深入地学习树（包括相关的算法），这一章还给出了一些建议。

第9章“图”，介绍了图这种数据结构和它的适用范围。这一章讲述了图的常用术语和不同表示方式，探讨了如何使用深度优先算法和广度优先算法遍历图，以及它们的适用范围。

第10章“排序和搜索算法”，探讨了常用的排序算法，如冒泡排序（包括改进版）、选择排序、插入排序、归并排序和快速排序。另外还介绍了搜索算法中的顺序搜索和二分搜索。

第11章“算法模式”，介绍了一些算法技巧和一些著名的算法。这一章讲解了什么是递归，介绍了一些高级算法，如动态规划和贪心算法。

第12章“算法复杂度”，介绍了大 O 表示法的概念，以及本书实现算法的复杂度列表。这一章还介绍了NP完全问题和启发式算法。最后，讲解了提升算法能力的诀窍。

准备工作

为学习本书，你可以设置三种不同的开发环境。你不需要设置所有这三种环境，可以选择其一，也可以逐一尝试。

方法一，你需要一个浏览器，请从下面二选一：

- ☐ Chrome（<https://www.google.com/chrome/browser/>）
- ☐ Firefox（<https://www.mozilla.org/en-US/firefox/new/>）

方法二，你需要：

- ☐ 安装方法一中的任意一个浏览器；
- ☐ 安装一个Web服务器。如果你的电脑里没有安装过Web服务器，推荐安装XAMPP（<https://www.apachefriends.org>）。

方法三，如果想安装一个纯JavaScript的环境，你需要完成下面几步。

- ❑ 安装步骤一中的任意浏览器
- ❑ 安装Node.js (<http://nodejs.org/>)
- ❑ 安装好Node.js后, 安装http-server开发包:

```
npm install http-server -g
```

第1章还会对此进行更详细的介绍。

读者对象

本书的目标读者包括计算机科学专业的学生、刚刚开启职业生涯的技术人员, 以及想探索JavaScript最优能力的朋友。要想学好书中的算法, 需要对JavaScript和编程逻辑有基本的了解。

排版约定

在本书中, 你会发现一些不同的文本样式, 用以区别不同种类的信息。下面举例说明。

正文中的代码、用户输入这样表示: “用isEmpty方法就可以判断内部数组的长度是否为0。”

代码段的格式如下:

```
function Stack() {  
    //这里是属性和方法  
}
```

如果我们想让你重点关注代码段中的某个部分, 会加粗显示:

```
class Stack {  
    constructor () { this.items = []; //{1} }  
    push(element){  
        this.items.push(element);  
    }  
    //其他方法  
}
```

所有的命令行输入或输出的格式如下:

```
stack.push(5);  
stack.push(8);
```

新术语和重点词汇以黑体标示。



这个图标表示警告或需要特别注意的内容。



这个图标表示提示或者技巧。

读者反馈

欢迎提出反馈。如果你对本书有任何想法，喜欢它什么，不喜欢它什么，请让我们知道。要写出真正对大家有帮助的书，了解读者的反馈很重要。

一般的反馈，请发送电子邮件至feedback@packtpub.com，并在邮件主题中包含书名。

如果你有某个主题的专业知识，并且有兴趣写成或帮助促成一本书，请参考我们的作者指南www.packtpub.com/authors。

客户支持

现在，你是一位自豪的Packt图书的拥有者，我们会尽全力帮你充分利用你手中的书。

下载示例代码

你可以用你的账户从<http://www.packtpub.com>下载所有已购买Packt图书的示例代码文件。如果你从其他地方购买的本书，可以访问<http://www.packtpub.com/support>并注册，我们将通过电子邮件把文件发送给你。

下载代码文件的步骤如下：

- (1) 访问<http://www.packtpub.com>，用你的邮箱和密码登录或注册；
- (2) 将鼠标指针悬停在顶部的SUPPORT标签上；
- (3) 点击Code Downloads & Errata；
- (4) 在搜索框输入书名；
- (5) 选择你要下载代码文件的图书；
- (6) 在下拉菜单中选择购书方式；
- (7) 点击Code Download。

你还可以访问本书在Packt Publishing网站上的页面，点击Code Files按钮下载代码文件。在搜索框输入书名可以访问此页。请注意，你需要登录Packt账户。

文件下载后，请使用以下软件的最新版本解压：

❑ Windows系统请使用WinRAR或7-Zip

- ❑ Mac系统请使用Zipeg、iZip或UnRarX
- ❑ Linux系统请使用7-Zip或PeaZip

本书的代码包在GitHub的托管地址是<https://github.com/loiane/javascript-datastructures-algorithms>。其他图书或视频的代码包也可以到<https://github.com/PacktPublishing>/查阅。别错过！

下载彩色插图

我们还提供了一份PDF文档，里面是本书中用到的彩色截图和图表。这些彩图能帮你更好地理解输出的变化。你可以从https://www.packtpub.com/sites/default/files/downloads/Learning-JavaScriptDataStructuresandAlgorithmsSecondEdition_ColorImages.pdf下载。

勘误表

虽然我们已尽力确保本书内容正确，但出错仍旧在所难免。如果你在我们的书中发现错误，不管是文本还是代码，希望能告知我们，我们不胜感激。这样做可以减少其他读者的困扰，帮助我们改进本书的后续版本。如果你发现任何错误，请访问<http://www.packtpub.com/submit-errata>提交，选择你的书，点击勘误表提交表单的链接，并输入详细说明。^①勘误一经核实，你的提交将被接受，此勘误将上传到本公司网站或添加到现有勘误表。从<http://www.packtpub.com/books/content/support>选择书名就可以查看现有的勘误表。

侵权行为

互联网上的盗版是所有媒体都要面对的问题。Packt非常重视保护版权和许可证。如果你发现我们的作品在互联网上被非法复制，不管以什么形式，都请立即为我们提供位置地址或网站名称，以便我们可以寻求补救。

请把可疑盗版材料的链接发到copyright@packtpub.com。

非常感谢你帮助我们保护作者，以及保护我们给你带来有价值内容的能力。

问题

如果你对本书内容存有疑问，不管是哪个方面，都可以通过questions@packtpub.com联系我们，我们将尽最大努力来解决。

^① 本书中文版的勘误请到<http://www.it-ebooks.com.cn/book/2029>查看和提交。——编者注

电子书

扫描如下二维码，即可购买本书电子版。



目 录

第 1 章 JavaScript 简介	1	2.2 创建和初始化数组	32
1.1 JavaScript 数据结构与算法	1	2.3 添加元素	33
1.2 环境搭建	2	2.3.1 使用 push 方法	33
1.2.1 最简单的环境搭建	2	2.3.2 插入元素到数组首位	34
1.2.2 使用 Web 服务器 (XAMPP)	4	2.4 删除元素	34
1.2.3 使用 Node.js 搭建 Web 服务器	5	2.5 在任意位置添加或删除元素	36
1.3 JavaScript 基础	6	2.6 二维和多维数组	36
1.3.1 变量	7	2.6.1 迭代二维数组的元素	37
1.3.2 操作符	9	2.6.2 多维数组	38
1.3.3 真值和假值	11	2.7 JavaScript 的数组方法参考	39
1.3.4 相等操作符 (==和===)	12	2.7.1 数组合并	39
1.4 控制结构	14	2.7.2 迭代器函数	40
1.4.1 条件语句	14	2.7.3 ECMAScript 6 和数组的新 功能	42
1.4.2 循环	15	2.7.4 排序元素	46
1.5 函数	16	2.7.5 搜索	48
1.6 JavaScript 面向对象编程	17	2.7.6 输出数组为字符串	49
1.7 调试工具	18	2.8 类型数组	50
1.8 ECMAScript 概述	19	2.9 小结	51
1.9 ECMAScript 6 的功能	21	第 3 章 栈	52
1.9.1 用 let 替代 var 声明变量	21	3.1 栈数据结构	52
1.9.2 常量	23	3.1.1 创建栈	53
1.9.3 模板字面量	23	3.1.2 向栈添加元素	53
1.9.4 箭头函数	24	3.1.3 从栈移除元素	53
1.9.5 函数的参数默认值	24	3.1.4 查看栈顶元素	54
1.9.6 声明展开和剩余参数	25	3.1.5 检查栈是否为空	54
1.9.7 使用类进行面向对象编程	27	3.1.6 清空和打印栈元素	54
1.10 ECMAScript 7 的功能	29	3.2 ECMAScript 6 和 Stack 类	56
1.11 小结	30	3.3 用栈解决问题	59
第 2 章 数组	31	3.4 小结	61
2.1 为什么用数组	31		

第4章 队列	62	6.3.3 差集	97
4.1 队列数据结构	62	6.3.4 子集	98
4.2 创建队列	63	6.4 ES6——Set 类	99
4.2.1 向队列添加元素	63	6.5 小结	101
4.2.2 从队列移除元素	63	第7章 字典和散列表	102
4.2.3 查看队列头元素	64	7.1 字典	102
4.2.4 检查队列是否为空	64	7.1.1 创建字典	102
4.2.5 打印队列元素	64	7.1.2 使用 Dictionary 类	105
4.3 用 ECMAScript 6 语法实现 的 Queue 类	66	7.2 散列表	106
4.4 优先队列	66	7.2.1 创建散列表	106
4.5 循环队列——击鼓传花	68	7.2.2 使用 HashTable 类	108
4.6 JavaScript 任务队列	70	7.2.3 散列表和散列集合	109
4.7 小结	70	7.2.4 处理散列表中的冲突	109
第5章 链表	71	7.2.5 创建更好的散列函数	117
5.1 链表数据结构	71	7.3 ES6——Map 类	118
5.2 创建链表	72	7.4 ES6——WeakMap 类和 WeakSet 类	118
5.2.1 向链表尾部追加元素	73	7.5 小结	119
5.2.2 从链表中移除元素	75	第8章 树	120
5.2.3 在任意位置插入元素	77	8.1 树数据结构	120
5.2.4 实现其他方法	79	8.2 树的相关术语	121
5.3 双向链表	82	8.3 二叉树和二叉搜索树	121
5.3.1 在任意位置插入新元素	82	8.3.1 创建 BinarySearchTree 类	122
5.3.2 从任意位置移除元素	85	8.3.2 向树中插入一个键	123
5.4 循环链表	87	8.4 树的遍历	126
5.5 小结	88	8.4.1 中序遍历	126
第6章 集合	89	8.4.2 先序遍历	127
6.1 构建数据集合	89	8.4.3 后序遍历	128
6.2 创建集合	89	8.5 搜索树中的值	129
6.2.1 has(value) 方法	90	8.5.1 搜索最小值和最大值	130
6.2.2 add 方法	91	8.5.2 搜索一个特定的值	131
6.2.3 remove 和 clear 方法	91	8.5.3 移除一个节点	133
6.2.4 size 方法	92	8.6 自平衡树	137
6.2.5 values 方法	93	8.6.1 Adelson-Velskii-Landi 树 (AVL 树)	137
6.2.6 使用 Set 类	93	8.6.2 更多关于二叉树的知识	143
6.3 集合操作	94	8.7 小结	143
6.3.1 并集	94	第9章 图	144
6.3.2 交集	95	9.1 图的相关术语	144

9.2 图的表示.....146	第 11 章 算法模式.....190
9.2.1 邻接矩阵.....146	11.1 递归.....190
9.2.2 邻接表.....147	11.1.1 JavaScript 调用栈大小的限制.....191
9.2.3 关联矩阵.....148	11.1.2 斐波那契数列.....191
9.3 创建 Graph 类.....148	11.2 动态规划.....193
9.4 图的遍历.....150	11.2.1 最少硬币找零问题.....194
9.4.1 广度优先搜索.....151	11.2.2 背包问题.....196
9.4.2 深度优先搜索.....156	11.2.3 最长公共子序列.....198
9.5 最短路径算法.....162	11.2.4 矩阵链相乘.....200
9.5.1 Dijkstra 算法.....163	11.3 贪心算法.....202
9.5.2 Floyd-Warshall 算法.....165	11.3.1 最少硬币找零问题.....203
9.6 最小生成树.....166	11.3.2 分数背包问题.....204
9.6.1 Prim 算法.....166	11.4 函数式编程简介.....205
9.6.2 Kruskal 算法.....168	11.4.1 函数式编程与命令式编程.....205
9.7 小结.....169	11.4.2 ES2015 和函数式编程.....206
第 10 章 排序和搜索算法.....170	11.4.3 JavaScript 函数式工具箱—— map、filter 和 reduce.....207
10.1 排序算法.....170	11.4.4 JavaScript 函数式类库和 数据结构.....209
10.1.1 冒泡排序.....171	11.5 小结.....209
10.1.2 选择排序.....174	第 12 章 算法复杂度.....210
10.1.3 插入排序.....175	12.1 大 O 表示法.....210
10.1.4 归并排序.....176	12.1.1 理解大 O 表示法.....210
10.1.5 快速排序.....179	12.1.2 时间复杂度比较.....212
10.1.6 堆排序.....183	12.1.3 NP 完全理论概述.....214
10.1.7 计数排序、桶排序和基数 排序（分布式排序）.....186	12.2 用算法娱乐身心.....216
10.2 搜索算法.....187	12.3 小结.....217
10.2.1 顺序搜索.....187	
10.2.2 二分搜索.....187	
10.3 小结.....189	

第 1 章

JavaScript简介



JavaScript是一门非常强大的编程语言。它是最流行的编程语言，也是互联网上最卓越的语言之一。在GitHub(世界上最大的代码托管站点, <https://github.com>)上, 托管了400 000多个JavaScript代码仓库(用JavaScript开发的项目数量也是最多的, 并且还在逐年增长, 参看<http://goo.gl/ZFx6mg>)。

JavaScript不仅可用于前端开发, 也适用于后端开发, 而Node.js就是其背后的技术。Node包(<http://www.npmjs.org/>)的数量也呈指数级增长。

要成为一名Web开发工程师, 掌握JavaScript必不可少。

本章, 你会学到JavaScript的语法和一些必要的基础, 这样我们就可以开始开发自己的数据结构和算法了。本章内容如下:

- ❑ 环境搭建
- ❑ JavaScript基础
- ❑ 控制结构
- ❑ 函数
- ❑ JavaScript面向对象编程
- ❑ 调试工具
- ❑ ECMAScript 6和ECMAScript 7简介

1.1 JavaScript 数据结构与算法

在本书中, 你将学习最常用的数据结构和算法。为什么用JavaScript来学习这些数据结构和算法呢? 我们已经回答了这个问题。JavaScript非常受欢迎, 作为函数式编程语言, 它非常适合用来学习数据结构和算法。通过它来学习数据结构比C或Java这些标准语言更简单, 学习新东西也会变得很有趣。谁说数据结构和算法只为C或Java这样的语言而生? 在前端开发当中, 你可能也需要实现这些语言。

学习数据结构和算法十分重要。首要原因是数据结构和算法可以很高效地解决常见问题，这对你今后所写代码的质量至关重要（也包括性能；要是用了不恰当的数据结构或算法，很可能会产生性能问题）。其次，对于计算机科学，算法是最基础的概念。最后，如果你想入职最好的IT公司（如谷歌、亚马逊、eBay等），数据结构和算法是面试问题的重头戏。

让我们开始学习吧！

1.2 环境搭建

相比其他语言，JavaScript的优势之一在于不用安装或配置任何复杂的环境就可以开始学习。每台计算机上都已具备所需的环境，哪怕使用者从未写过一行代码。有浏览器足矣！

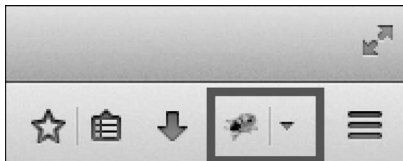
为了运行书中的示例代码，建议你做好如下准备：安装Chrome或Firefox浏览器（选择一个你最喜欢的即可），选择一个喜欢的编辑器（如Sublime Text），以及一个Web服务器（XAMPP或其他你喜欢的，这一步是可选的）。这些软件在Windows、Linux和Mac OS上均可以使用。

如果你使用Firefox，推荐你安装Firebug插件（<https://getfirebug.com>）。接下来将介绍搭建环境的三种方案。

1.2.1 最简单的环境搭建

浏览器是最简单的开发环境。

你也可以使用Firefox加Firebug。安装好Firebug后，在浏览器的右上角会看到如下图所示的图标。

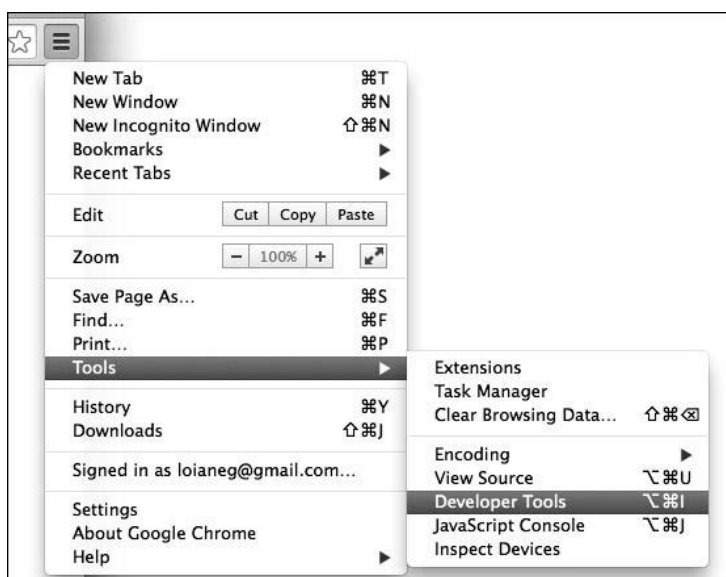


点击Firebug图标，打开它，可以看到Console标签，我们可以在其命令行区域中编写所有JavaScript代码，如下图所示（执行源代码请按Run按钮）。



也可以扩展命令行，来适应Firebug插件的整个可用区域。

你还可以使用谷歌Chrome，它已经集成了Google Developer Tools（谷歌开发者工具）。打开Chrome，点击设置及控制图标，选中Tools | Developer Tools，如下图所示。



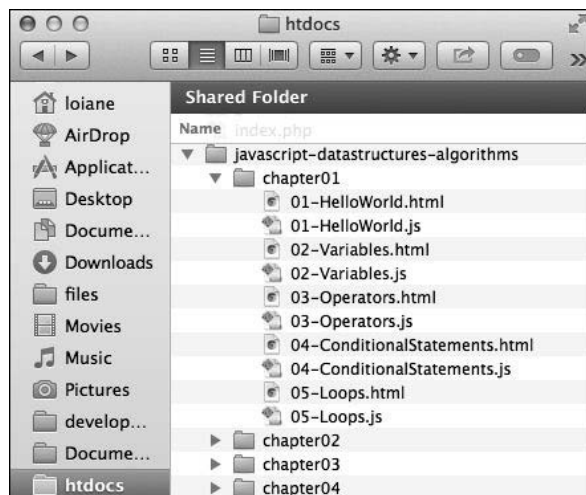
然后，就可以在Console标签页中编写JavaScript测试代码，如下所示。



1.2.2 使用 Web 服务器（XAMPP）

你可能想要安装的第二个环境是XAMPP，它的安装过程也很简单，但比只使用浏览器麻烦点儿。

安装XAMPP(<https://www.apachefriends.org>)或者你偏爱的其他Web服务器。然后，在XAMPP安装文件夹下找到htdocs目录。在该目录下新建一个文件夹，就可以在里面执行本书中所讲述的源代码；或者是直接将示例代码下载后提取到此目录，如下所示。



接下来，在启动XAMPP服务器之后，你可以通过localhost这个URL，用浏览器访问源码，如下图所示（别忘了打开Firebug或谷歌开发者工具查看输出）。



执行示例代码时，请不要忘记打开谷歌开发者工具或Firebug查看输出结果。

1.2.3 使用 Node.js 搭建 Web 服务器

第三种选择就是100%的JavaScript。我们可以使用Node.js来搭建一个JavaScript服务器，不使用XAMPP搭建的Apache服务器。

首先要到<http://nodejs.org/>下载和安装Node.js。然后，打开终端应用（如果你用的是Windows操作系统，打开Node.js的命令行），输入如下命令：

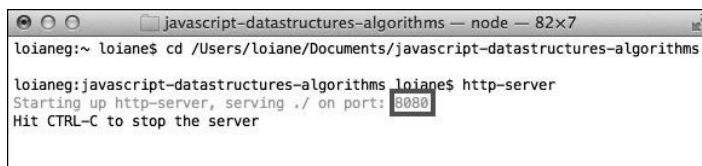
```
npm install http-server -g
```

最好手动输入这些命令，复制粘贴可能会出错。

也可以用管理员身份执行上述命令。对于Linux和Mac操作系统，使用如下命令：

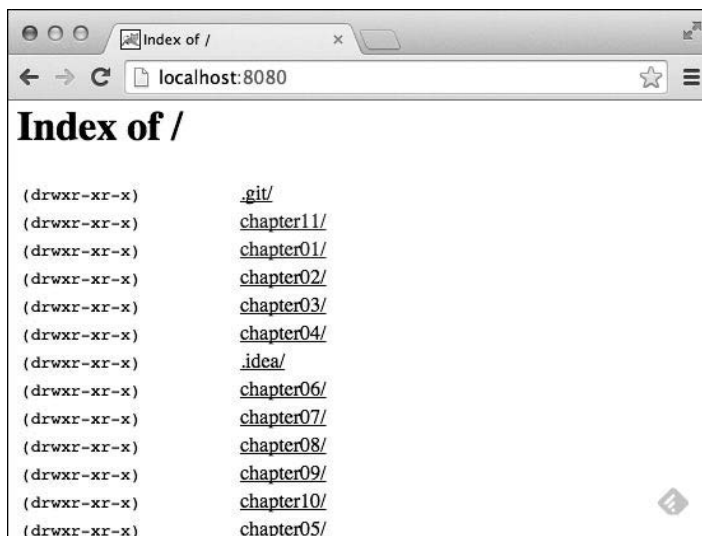
```
sudo npm install http-server -g
```

这条命令会在你的机器上安装一个JavaScript服务器：`http-server`。要启动服务器并在终端应用上运行本书中的示例代码，请将工作路径更改至示例代码文件夹，然后输入`http-server`，如下图所示，整个环境就搭建好了！



```
loianeg:~ loiane$ cd /Users/loiane/Documents/javascript-datastructures-algorithms
loianeg:javascript-datastructures-algorithms loiane$ http-server
Starting up http-server, serving ./ on port: 8080
Hit CTRL-C to stop the server
```

为执行示例，打开浏览器，通过`http-server`命令指定的端口访问：





下载代码文件的具体步骤前言中介绍过了，请翻回去看一看。本书的代码包在 GitHub 上的托管地址是 <https://github.com/loiane/javascript-datastructures-algorithms>。其他图书或视频的代码包也可以到 <https://github.com/PacktPublishing/> 查阅。别错过！

1.3 JavaScript 基础

在深入学习各种数据结构和算法前，让我们先大概了解一下JavaScript。本节教大家一些相关的基础知识，有利于学习后面各章。

首先来看在HTML中编写JavaScript的两种方式：

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    <script>
      alert('Hello, World!');
    </script>
  </body>
</html>
```

第一种方式如上面的代码所示。创建一个HTML文件，把代码写进去。在这个例子里，我们在HTML文件中声明了script标签，然后把JavaScript代码都写进这个标签。

第二种方式，我们需要创建一个JavaScript文件（比如01-HelloWorld.js），在里面写入如下代码：

```
alert('Hello, World!');
```

然后，我们的HTML文件看起来如下：

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    <script src="01-HelloWorld.js">
    </script>
  </body>
</html>
```

第二个例子展示了如何将一个JavaScript文件引入HTML文件。

这两个例子，无论执行哪个，输出都是一样的。但第二个例子是最佳实践。



可能你在网上的一些例子里看到过JavaScript的include语句，或者放在head标签中的JavaScript代码。作为最佳实践，我们会在关闭body标签前引入JavaScript代码。这样浏览器就会在加载脚本之前解析和显示HTML，有利于提升页面的性能。

1.3.1 变量

变量保存的数据可以在需要时设置、更新或提取。赋给变量的值都有对应的类型。JavaScript的类型有数字、字符串、布尔值、函数和对象。还有undefined和null，以及数组、日期和正则表达式。



尽管JavaScript有多种变量类型，然而不同于C/C++、C#或Java，它并不是一种强类型语言。在强类型语言中，声明变量时需要指定变量的类型（例如，在Java中声明一个整型变量，使用int num = 1;）。在JavaScript中，我们只需要使用关键字var，而不必指定变量类型。因此，JavaScript不是强类型语言。

下面的例子介绍如何在JavaScript里使用变量。

```
var num = 1; //{1}
num = 3; //{2}
var price = 1.5; //{3}
var name = 'Packt'; //{4}
var trueValue = true; //{5}
var nullVar = null; //{6}
var und; //{7}
```

- ❑ 在行{1}，我们展示了如何声明一个JavaScript变量（声明了一个数字类型）。虽然关键字var不是必需的，但最好每次声明一个新变量时都加上。
- ❑ 在行{2}，我们更新了已有变量。JavaScript不是强类型语言。这意味着你可以声明一个变量并初始化为一个数字类型的值，然后把它更新成字符串或者其他类型的值，不过这并不是一个好做法。
- ❑ 在行{3}，我们又声明了一个数字类型的变量，不过这次是十进制浮点数。在行{4}，声明了一个字符串；在行{5}，声明了一个布尔值；在行{6}，声明了一个null；在行{7}，声明了undefined变量。null表示变量没有值，undefined表示变量已被声明，但尚未赋值。看看下面的例子：

```
console.log("num: " + num);
console.log("name: " + name);
console.log("trueValue: " + trueValue);
console.log("price: " + price);
console.log("nullVar: " + nullVar);
console.log("und: " + und);
```

如果想看我们声明的每个变量的值,可以用`console.log`来实现,就像上面代码片段中那样。



书中示例代码会使用三种方式输出JavaScript的值。第一种是`alert('My text here')`,将输出到浏览器的警示窗口;第二种是`console.log('My text here')`,将把文本输出到调试工具的Console标签(谷歌开发者工具或是Firebug,根据你使用的浏览器而定);第三种方式是通过`document.write('My text here')`直接输出到HTML页面里并被浏览器呈现。可以选择你喜欢的方式来调试。

`console.log`方法也不止接收参数,除了`console.log("num: " + num)`还可以写成`console.log("num: ", num)`。

稍后我们会讨论函数和对象。

变量作用域

作用域指在编写的算法函数中,我们能访问的变量(在使用时,函数作用域也可以是一个函数)。有本地变量和全局变量两种。

让我们看一个例子:

```
var myVariable = 'global';
myOtherVariable = 'global';

function myFunction() {
  var myVariable = 'local';
  return myVariable;
}

function myOtherFunction() {
  myOtherVariable = 'local';
  return myOtherVariable;
}

console.log(myVariable); //{1}
console.log(myFunction()); //{2}

console.log(myOtherVariable); //{3}
console.log(myOtherFunction()); //{4}
console.log(myOtherVariable); //{5}
```

- ❑ 行{1}输出`global`,因为它是一个全局变量。
- ❑ 行{2}输出`local`,因为`myVariable`是在`myFunction`函数中声明的本地变量,所以作用域仅在`myFunction`内。
- ❑ 行{3}输出`global`,因为我们引用了在第二行初始化了的全局变量`myOtherVariable`。
- ❑ 行{4}输出`local`。在`myOtherFunction`函数里,因为没有使用`var`关键字修饰,所以这里引用的是全局变量`myOtherVariable`并将它赋值为`local`。

□ 因此，行{5}会输出local（因为在myOtherFunction里修改了myOtherVariable的值）。

你可能听其他人提过在JavaScript里应该尽量少用全局变量，这是对的。通常，代码质量可以用全局变量和函数的数量来考量（数量越多越糟）。因此，尽可能避免使用全局变量。

1.3.2 操作符

编程语言里都需要操作符。在JavaScript里有算数操作符、赋值操作符、比较操作符、逻辑操作符、位操作符、一元操作符和其他操作符。我们来看一下这些操作符：

```
var num = 0; //{1}
num = num + 2;
num = num * 3;
num = num / 2;
num++;
num--;

num += 1; //{2}
num -= 2;
num *= 3;
num /= 2;
num %= 3;

console.log('num == 1 : ' + (num == 1)); //{3}
console.log('num === 1 : ' + (num === 1));
console.log('num != 1 : ' + (num != 1));
console.log('num > 1 : ' + (num > 1));
console.log('num < 1 : ' + (num < 1));
console.log('num >= 1 : ' + (num >= 1));
console.log('num <= 1 : ' + (num <= 1));

console.log('true && false : ' + (true && false)); //{4}
console.log('true || false : ' + (true || false));
console.log('!true : ' + (!true));
```

在行{1}，我们用了算数操作符。在下面的表格里，列出了这些操作符及其描述。

算数操作符	描 述
+	加法
-	减法
*	乘法
/	除法
%	取余
++	递增
--	递减

在行{2}，我们使用了赋值操作符，在下面的表格里，列出了赋值操作符及其描述。

赋值操作符	描 述
=	赋值
+=	加/赋值 (x += y) == (x = x + y)
-=	减/赋值 (x -= y) == (x = x - y)
*=	乘/赋值 (x *= y) == (x = x * y)
/=	除/赋值 (x /= y) == (x = x / y)
%=	取余/赋值 (x %= y) == (x = x % y)

在行{3}，我们使用了比较操作符。在下面的表格里，列出了比较操作符及其描述。

比较操作符	描 述
==	相等
===	全等
!=	不等
>	大于
>=	大于等于
<	小于
<=	小于等于

在行{4}，我们使用了逻辑操作符。在下面的表格里，列出了逻辑操作符及其描述。

逻辑操作符	描 述
&&	与
	或
!	非

JavaScript也支持位操作符，如下所示：

```
console.log('5 & 1:', (5 & 1));
console.log('5 | 1:', (5 | 1));
console.log('~ 5:', (~5));
console.log('5 ^ 1:', (5 ^ 1));
console.log('5 << 1:', (5 << 1));
console.log('5 >> 1:', (5 >> 1));
```

下面的表格对位操作符做了更详细的描述。

位操作符	描 述
&	与
	或
~	非
^	异或
<<	左移
>>	右移

`typeof`操作符可以返回变量或表达式的类型。我们看下面的代码：

```
console.log('typeof num:', typeof num);
console.log('typeof Packt:', typeof 'Packt');
console.log('typeof true:', typeof true);
console.log('typeof [1,2,3]:', typeof [1,2,3]);
console.log('typeof {name:John}:', typeof {name:'John'});
```

输出如下：

```
typeof num: number
typeof Packt: string
typeof true: boolean
typeof [1,2,3]: object
typeof {name:John}: object
```

JavaScript还支持`delete`操作符，可以删除对象里的属性。看看下面的代码：

```
var myObj = {name: 'John', age: 21};
delete myObj.age;
console.log(myObj); // 输出对象{name: "John"}
```

这些操作符在后面的算法学习中可能会用到。

1.3.3 真值和假值

在JavaScript中，`true`和`false`有些复杂。在大多数编程语言中，布尔值`true`和`false`仅仅表示`true/false`。在JavaScript中，如`"Packt"`这样的字符串值，也可以看作`true`。

下面的表格能帮助我们更好地理解`true`和`false`在JavaScript中是如何转换的。

数值类型	转换成布尔值
undefined	false
null	false
布尔值	true是true, false是false
数字	+0、-0和NaN都是false, 其他都是true
字符串	如果字符串是空的（长度是0）就是false, 其他都是true
对象	true

我们来看一些代码，用输出来验证上面的总结：

```
function testTruthy(val){
  return val ? console.log('truthy') : console.log('falsy');
}

testTruthy(true); //true
testTruthy(false); //false
testTruthy(new Boolean(false)); //true (对象始终为true)

testTruthy(''); //false
testTruthy('Packt'); //true
```



```
testTruthy(new String('')); //true (对象始终为true)

testTruthy(1); //true
testTruthy(-1); //true
testTruthy(NaN); //false
testTruthy(new Number(NaN)); //true (对象始终为true)

testTruthy({}); //true (对象始终为true)

var obj = {name:'John'};
testTruthy(obj); //true
testTruthy(obj.name); //true
testTruthy(obj.age); //false (年龄不存在)
```

1.3.4 相等操作符（==和===）

当使用这两个相等操作符时，可能会引起一些困惑。

使用==时，不同类型的值也可以被看作相等。这样的结果可能会使那些资深的JavaScript开发者都感到困惑。我们用下面的表格给大家分析一下不同类型的值用相等操作符比较后的结果。

类型 (x)	类型 (y)	结 果
null	undefined	true
undefined	null	true
数字	字符串	x == toNumber(y)
字符串	数字	toNumber(x) == y
布尔值	任何类型	toNumber(x) == y
任何类型	布尔值	x == toNumber(y)
字符串或数字	对象	x == toPrimitive(y)
对象	字符串或数字	toPrimitive(x) == y

如果x和y是相同类型，JavaScript会比较它们的值或对象值。其他没有列在这个表格中的情况都会返回false。

toNumber和toPrimitive方法是内部的，并根据以下表格对其进行估值。

toNumber方法对不同类型返回的结果如下：

值类型	结 果
undefined	NaN
null	+0
布尔值	如果是true，返回1；如果是false，返回+0
数字	数字对应的值
字符串	将字符串解析成数字。如果字符串中包含字母，返回NaN；如果是由数字字符组成的，转换成数字
对象	Number(toPrimitive(vale))

toPrimitive方法对不同类型返回的结果如下：

值类型	结 果
对象	如果对象的valueOf方法的结果是原始值，返回原始值。如果对象的toString方法返回原始值，就返回这个值；其他情况都返回一个错误

用例子来验证一下表格中的结果。首先，我们知道下面的代码输出 true(字符串长度大于1)：

```
console.log('packt' ? true : false);
```

那么这行代码的结果呢？

```
console.log('packt' == true);
```

输出是false，为什么会这样呢？

- ❑ 首先，布尔值会被toNumber方法转成数字，因此得到packt == 1。
- ❑ 其次，用toNumber转换字符串值。因为字符串包含有字母，所以会被转成NaN，表达式就变成了NaN == 1，结果就是false。

那么这行代码的结果呢？

```
console.log('packt' == false);
```

输出也是false。步骤如下所示。

- ❑ 首先，布尔值会被toNumber方法转成数字，因此得到packt == 0。
- ❑ 其次，用toNumber转换字符串值。因为字符串包含有字母，所以会被转成NaN，表达式就变成了NaN == 0，结果就是false。

那么===操作符呢？简单多了。如果比较的两个值类型不同，比较的结果就是false。如果比较的两个值类型相同，结果会根据下表判断。

类型 (x)	值	结 果
数字	x和y数值相同（但不是NaN）	true
字符串	x和y是相同的字符	true
布尔值	x和y都是true或false	true
对象	x和y引用同一个对象	true

如果x和y类型不同，结果就是false。

我们来看一些例子：

```
console.log('packt' === true); //false
```

```
console.log('packt' === 'packt'); //true
```

```
var person1 = {name:'John'};
var person2 = {name:'John'};
console.log(person1 === person2); //false, 不同的对象
```

1.4 控制结构

JavaScript的控制结构和C与Java里的类似。条件语句支持if...else和switch。循环支持while、do...while和for。

1.4.1 条件语句

首先我们看一下如何构造if...else条件语句。有几种方式。

如果想让一个脚本仅当条件是true时执行，可以这样写：

```
var num = 1;
if (num === 1) {
    console.log("num is equal to 1");
}
```

如果想在条件为true的时候执行脚本A，其他情况下都执行脚本B，可以这样写：

```
var num = 0;
if (num === 1) {
    console.log("num is equal to 1");
} else {
    console.log("num is not equal to 1, the value of num is " + num);
}
```

if...else语句也可以用三元操作符替换，例如下面的if...else语句：

```
if (num === 1){
    num--;
} else {
    num++;
}
```

可以用三元操作符替换为：

```
(num === 1) ? num-- : num++;
```

如果我们有多个脚本，可以多次使用if...else，根据不同的条件执行不同的语句：

```
var month = 5;
if (month === 1) {
    console.log("January");
} else if (month === 2){
    console.log("February");
} else if (month === 3){
    console.log("March");
}
```

```
} else {  
  console.log("Month is not January, February or March");  
}
```

最后，还有switch语句。如果要判断的条件和上面的一样（但要和不同的值进行比较），可以使用switch语句：

```
var month = 5;  
switch(month) {  
  case 1:  
    console.log("January");  
    break;  
  case 2:  
    console.log("February");  
    break;  
  case 3:  
    console.log("March");  
    break;  
  default:  
    console.log("Month is not January, February or March");  
}
```

对于switch语句来说，case和break关键字的用法很重要。case判断当前switch的值是否和case分支语句的值相等。break会中止switch语句的执行。没有break会导致执行完当前的case后，继续执行下一个case，直到遇到break或switch执行结束。最后，还有default关键字，在表达式不匹配前面任何一种情形的时候，就执行default中的代码（如果有对应的，就不会执行）。

1.4.2 循环

在处理数组元素时会经常用到循环（数组是下一章的主讲内容）。在我们的算法中也会经常用到for循环。

JavaScript中的for循环与C和Java中的一样。循环的计数值通常是一个数字，然后和另一个值比较（如果条件成立就会执行for循环中的代码），之后这个数值会递增或递减。

- ❑ 在下面的代码里，我们用了个for循环。当i小于10时，会在控制台中输出其值。i的初始值是0，因此这段代码会输出0到9。

```
for (var i=0; i<10; i++) {  
  console.log(i);  
}
```

- ❑ 我们要关注的下一种循环是while循环。当while的条件判断成立时，会执行循环内的代码。下面的代码里，有一个初始值为0的变量i，我们希望在i小于10时输出它的值。输出会是0到9：

```
var i = 0;
while(i<10)
{
    console.log(i);
    i++;
}
```

- do...while循环和while循环很相似。区别是在while循环里，先进行条件判断再执行循环体中的代码，而在do...while循环里，是先执行循环体中的代码再判断循环条件。do...while循环至少会让循环体中的代码执行一次。下面的代码同样会输出0到9：

```
var i = 0;
do {
    console.log(i);
    i++;
} while (i<10)
```

1.5 函数

在用JavaScript编程时，函数很重要。在我们的例子里也用了函数。

下面的代码展示了函数的基本语法。它没有用到参数或return语句：

```
function sayHello() {
    console.log('Hello!');
}
```

要执行这个函数，只需要这样调用一下：

```
sayHello();
```

我们也可以传递参数给函数。参数是会被函数使用的变量。下面的代码展示了如何在函数中使用参数：

```
function output(text) {
    console.log(text);
}
```

我们可以通过以下代码使用该函数：

```
output('Hello!');
```

你可以传递任意数量的参数，如下所示：

```
output('Hello!', 'Other text');
```

在这个例子中，函数只使用了传入的第一个参数，第二个参数被忽略。

函数也可以返回一个值，例如：

```
function sum(num1, num2) {  
    return num1 + num2;  
}
```

这个函数计算了给定两个数字之和，并返回结果。我们可以这样使用：

```
var result = sum(1,2);  
output(result);
```

1.6 JavaScript 面向对象编程

JavaScript里的对象就是普通名值对的集合。创建一个普通对象有两种方式。第一种方式是：

```
var obj = new Object();
```

第二种方式是：

```
var obj = {};
```

也可以这样创建一个完整的对象：

```
obj = {  
    name: {  
        first: 'Gandalf',  
        last: 'the Grey'  
    },  
    address: 'Middle Earth'  
};
```

可以看到，声明JavaScript对象时，**键值对**中的键就是对象的属性，值就是对应属性的值。在本书中，我们创建的所有的类，如Stack、Set、LinkedList、Dictionary、Tree、Graph等，都是JavaScript对象。

在面向对象编程（OOP）中，对象是类的实例。一个类定义了对象的特征。我们会创建很多类来表示算法和数据结构。例如我们声明了一个类来表示书：

```
function Book(title, pages, isbn){  
    this.title = title;  
    this.pages = pages;  
    this.isbn = isbn;  
}
```

用下面的代码实例化这个类：

```
var book = new Book('title', 'pag', 'isbn');
```

然后，我们可以访问和修改对象的属性：

```
console.log(book.title); //输出书名
book.title = 'new title'; //修改书名
console.log(book.title); //输出新的书名
```

类可以包含函数。可以声明和使用函数，如下所示：

```
Book.prototype.printTitle = function(){
    console.log(this.title);
};
book.printTitle();
```

也可以直接在类的定义里声明函数：

```
function Book(title, pages, isbn){
    this.title = title;
    this.pages = pages;
    this.isbn = isbn;
    this.printIsbn = function(){
        console.log(this.isbn);
    }
}
book.printIsbn();
```



在原型的例子里，`printTitle`函数只会创建一次，在所有实例中共享。如果在类的定义里声明，就像前面的例子一样，则每个实例都会创建自己的函数副本。使用原型方法可以节约内存和降低实例化的开销。不过原型方法只能声明公共函数和属性，而类定义可以声明只在类的内部访问的私有函数和属性。ECMAScript 6 引入了一套既像类定义又基于原型的简化语法。稍后我们会进一步讨论。

1.7 调试工具

除了学会如何用JavaScript编程外，还需要了解如何调试代码。调试对于找到代码中的错误十分有帮助，也能让你低速执行代码，看到所有发生的事情（方法被调用的栈、变量赋值等）。极力推荐你花一些时间学习一下如何调试书中的源码，查看算法的每一步（这样也会让你对算法有深刻的理解）。

Firefox 和 Chrome 都支持调试。这里有一个了解谷歌开发者工具的好教程，地址是 <https://developer.chrome.com/devtools/docs/javascript-debugging>。

除了你喜好的编辑器外，这里推荐其他几个工具，可以提升编写JavaScript的效率。

- ❑ **Aptana**：这是一个开源的免费IDE，支持JavaScript、CSS3和HTML5以及其他语言（<http://www.aptna.com>）。
- ❑ **WebStorm**：这是一个很强大的IDE，支持最新的Web技术和框架。它不是免费的，但你可以下载一个30天试用版本体验一下（<http://www.jetbrains.com/webstorm>）。

- ❑ **Sublime Text**: 这是一个轻量级的文本编辑器，可以自定义插件。可以买它的许可证来支持这个工具的开发，也可以免费使用（试用版不过期），<http://www.sublimetext.com/>。
- ❑ **Atomic**: 这也是一个轻量级的文本编辑器，由GitHub创建。它对JavaScript提供了很好的支持，也可以自定义插件（<https://atom.io/>）。

1.8 ECMAScript 概述

如果你关注JavaScript的新闻和最新趋势，肯定听说过被热炒的ECMAScript 6和ECMAScript 7。ECMAScript跟JavaScript有什么关系，又有什么不同？

ECMAScript是一种脚本语言规范。JavaScript是这个规范的一个实现，Jscript和ActionScript也是如此。

ECMAScript 6 和 ECMAScript 7

我们知道，JavaScript是一种主要在浏览器中运行的语言（也可以运行于NodeJS服务端），每个浏览器都可以实现自己版本的JavaScript功能（稍后你将在本书中学习）。这个具体的实现是基于ECMAScript的。因此，浏览器提供的功能大都相同（我们的JavaScript代码可以在所有浏览器中运行）；然而，在不同的浏览器中，每个功能的行为也会存在细微的差别。

目前为止，本章给出的所有代码都是基于2009年12月发布的ECMAScript 5。编写本书时，ECMAScript的最新版本是2015年7月标准化的ECMAScript 6，与上一个版本时隔近六年。负责起草ECMAScript规范的委员会决定把定义新标准的模式改为每年更新一次，新的特性一旦通过就将加入标准。因此，ECMAScript第六版更名为ECMAScript 2015（ES6）。还有一个准备在2016年夏天发布的新版本，称为ECMAScript 2016或ECMAScript 7（ES7）。

本节，我们会学习ES6和ES7中引入的一些新功能。

兼容性列表

一定要明白，即便ES6已经发布，也不是所有的浏览器都完全支持新特性。为了更好的体验，最好使用你选择的浏览器（Firefox或Chrome）的最新版本。

通过以下链接，你可以检查在各个浏览器中哪些特性可用。

- ❑ ES6: <http://kangax.github.io/compat-table/es6/>
- ❑ ES7: <http://kangax.github.io/compat-table/es7/>

即使有些特性尚未支持，我们也可以现在就开始用新语法和新功能。

对于开发团队交付的ES6和ES7功能实现，Firefox默认开启支持。

在谷歌Chrome浏览器中,你可以如下图所示,访问chrome://flags,开启实验性JavaScript标志,启用新功能:

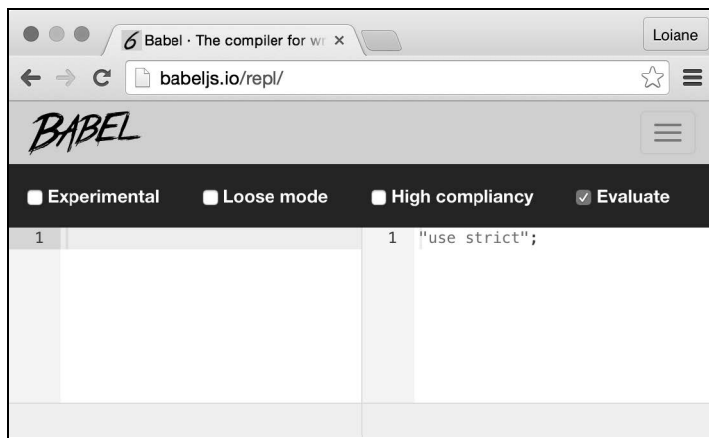


即使开启了谷歌Chrome浏览器的实验性JavaScript标志,ES6的部分特性也可能不支持,Firefox同样如此。要了解各个浏览器所支持的特性,请查看兼容性列表。

● 使用Babel.js

Babel (<https://babeljs.io>) 是一个JavaScript转译器,也称为源代码编译器。它将使用了ES6和ES7语言特性的JavaScript代码转换成只使用广泛支持的ES5特性的等价代码。

使用Babel.js的方式多种多样。其中之一是根据设置文档 (<https://babeljs.io/docs/setup/>) 进行安装。另一种方式是直接在浏览器中试用 (<https://babeljs.io/repl/>), 如下面的屏幕截图所示。



针对后续章节中出现的所有ES6和ES7的例子,我们都将提供一个在Babel中运行和测试的链接。

1.9 ECMAScript 6 的功能

本节，我们将演示如何使用ES6的一些新功能。这既对日常的JavaScript编码有用，也可以简化本书后面章节中的例子。我们将介绍以下功能。

- ❑ let和const
- ❑ 模板字面量
- ❑ 解构
- ❑ 展开操作符
- ❑ 箭头函数：=>
- ❑ 类

1.9.1 用 let 替代 var 声明变量

到ES5为止，我们可以在代码中任意位置声明变量，甚至重写已声明的变量，代码如下：

```
var framework = 'Angular';
var framework = 'React';
console.log(framework);
```

上面代码的输出是React，这个值被赋给最后声明的framework变量。这段代码中有两个同名的变量，这是非常危险的，可能会导致错误的输出。

C、Java、C#等其他语言不允许这种行为。ES6引入了一个let关键字，它是新的var，这意味着我们可以直接把var关键字都替换成let。以下代码就是一个例子：

```
let language = 'JavaScript!'; // {1}
let language = 'Ruby!'; // {2} - 抛出错误
console.log(language);
```

行{2}会抛出错误，因为在同一作用域中已经声明过language变量（行{1}）。后面会讨论let和变量作用域。



你可以访问<https://goo.gl/he0udZ>，测试和执行上面的代码。

let的变量作用域

我们通过下面这个例子（<https://goo.gl/NbsVvg>），来理解let关键字声明的变量如何工作：

```
let movie = 'Lord of the Rings'; //{1}
//var movie = 'Batman v Superman'; //抛出错误，movie变量已声明

function starWarsFan(){
```

```
    let movie = 'Star Wars'; //{2}
    return movie;
}

function marvelFan(){
    movie = 'The Avengers'; //{3}
    return movie;
}

function blizzardFan(){
    let isFan = true;
    let phrase = 'Warcraft'; //{4}
    console.log('Before if: ' + phrase);
    if (isFan){
        let phrase = 'initial text'; //{5}
        phrase = 'For the Horde!'; //{6}
        console.log('Inside if: ' + phrase);
    }
    phrase = 'For the Alliance!'; //{7}
    console.log('After if: ' + phrase);
}

console.log(movie); //{8}
console.log(starWarsFan()); //{9}
console.log(marvelFan()); //{10}
console.log(movie); //{11}
blizzardFan(); //{12}
```

以上代码的输出如下:

```
Lord of the Rings
Star Wars
The Avengers
The Avengers
Before if: Warcraft
Inside if: For the Horde!
After if: For the Alliance!
```

现在, 我们来讨论得到这些输出的原因。

- ❑ 我们在行{1}声明了一个`movie`变量并赋值为`Lord of the Rings`, 然后在行{8}输出它的值。你在1.3.1节中的“变量作用域”部分已经学过, 这个变量拥有全局作用域。
- ❑ 我们在行{9}执行了`starWarsFan`函数。在这个函数里, 我们也声明了一个`movie`变量(行{2})。这个函数的输出是`Star Wars`, 因为行{2}的变量拥有局部作用域, 也就是说它只在函数内部可见。
- ❑ 我们在行{10}执行了`marvelFan`函数。在这个函数里, 我们改变了`movie`变量的值(行{3})。这个变量是行{1}声明的全局变量。因此, 行{11}的全局变量输出和行{10}的输出相同, 都是`The Avengers`。

- ❑ 最后，我们在行{12}执行了blizzardFan函数。在这个函数里，我们声明了一个拥有函数内作用域的phrase变量（行{4}）。然后，又声明了一个phase变量（行{5}），但这个变量的作用域只在if语句内。
- ❑ 我们在行{6}改变了phrase的值。由于还在if语句内，值发生改变的是在行{5}声明的变量。
- ❑ 然后，我们在行{7}再次改变了phrase的值，但由于不是在if语句内，行{4}声明的变量的值改变了。

作用域的行为与在Java或C等其他编程语言中一样。然而，这是ES6才引入到JavaScript的。

1.9.2 常量

ES6还引入了const关键字。它的行为和let关键字一样，唯一的区别在于，用const定义的变量是只读的，也就是常量。

举例来说，考虑如下代码：

```
const PI = 3.141593;
PI = 3.0; // 抛出错误
console.log(PI);
```

当我们试图把一个新的值赋给PI，甚至只是用var PI或let PI重新声明时，代码就会抛出错误，告诉我们PI是只读的。



你可以访问<https://goo.gl/4xuWrC>执行上面的例子。

1.9.3 模板字面量

模板字面量真的很棒，因为我们创建字符串的时候不必再拼接值。

举例来说，考虑如下ES5代码：

```
var book = {
  name: '学习JavaScript数据结构与算法'
};
console.log('你正在阅读' + book.name + '。\\n这是新的一行\\n 这也是。');
```

我们可以用如下代码改进上面这个console.log输出的语法：

```
console.log(`你正在阅读${book.name}。
这是新的一行
这也是。`);
```

模板字面量用一对`包裹。要插入变量的值，只要把变量放在`\${}`里就可以了，就像例子中的`book.name`。

模板字面量也可以用于多行的字符串。再也不需要用`\n`了。只要按下键盘上的Enter就可以换一行，就像上面例子中的`这是新的一行`。

这个功能对简化我们例子的输出非常有用！



你可以访问<https://goo.gl/PTqnwO>执行上面的例子。

1.9.4 箭头函数

ES6的箭头函数极大地简化了函数的语法。考虑如下例子：

```
var circleArea = function circleArea(r) {  
  var PI = 3.14;  
  var area = PI * r * r;  
  return area;  
};  
console.log(circleArea(2));
```

上面这段代码的语法可以简化为如下代码：

```
let circleArea = (r) => { //{1}  
  const PI = 3.14;  
  let area = PI * r * r;  
  return area;  
}  
console.log(circleArea(2));
```

这个例子最大的区别在于行`{1}`，我们可以省去`function`关键字，只用`=>`。

如果函数只有一条语句，还可以变得更简单，连`return`关键字都可以省去。看看下面的代码：

```
let circleArea2 = (r) => 3.14 * r * r;  
console.log(circleArea2(2));
```



你可以访问<https://goo.gl/CigniJ>执行上面的例子。

1.9.5 函数的参数默认值

在ES6里，函数的参数还可以定义默认值。下面是一个例子：

```
function sum (x = 1, y = 2, z = 3) {  
  return x + y + z  
};  
console.log(sum(4,2)); //输出9
```

由于我们没有传入参数 z ，它的值默认为3。因此， $4 + 2 + 3 == 9$ 。

在ES6之前，上面的函数我们只能写成这样：

```
function sum (x, y, z) {  
  if (x === undefined)  
    x = 1;  
  if (y === undefined)  
    y = 2;  
  if (z === undefined)  
    z = 3;  
  return x + y + z;  
};
```

有了ES6的参数默认值，代码可以少写好几行。



你可以访问<https://goo.gl/2MiJ59>执行上面的例子。

1.9.6 声明展开和剩余参数

在ES5中，我们可以用`apply()`函数把数组转化为参数。为此，ES6有了展开操作符`(...)`。举例来说，考虑我们上一节声明的`sum`函数。可以执行如下代码来传入参数 x 、 y 和 z ：

```
var params = [3, 4, 5];  
console.log(sum(...params));
```

以上代码和下面的ES5代码的效果是相同的：

```
var params = [3, 4, 5];  
console.log(sum.apply(undefined, params));
```

在函数中，展开操作符`(...)`也可以代替`arguments`，当作剩余参数使用。考虑如下例子：

```
function restParamaterFunction (x, y, ...a) {  
  return (x + y) * a.length;  
}  
console.log(restParamaterFunction(1, 2, "hello", true, 7)); //输出9;
```

以上代码和下面代码的效果是相同的：

```
function restParamaterFunction (x, y) {  
  var a = Array.prototype.slice.call(arguments, 2);  
  return (x + y) * a.length;  
};
```



你可以访问<https://goo.gl/8equk5>执行展开操作符的例子，访问<https://goo.gl/LaJZqU>执行剩余参数的例子。

增强的对象属性

ES6引入了数组解构的概念，可以用来一次初始化多个变量。考虑如下例子：

```
var [x, y] = ['a', 'b'];
```

以上代码和下面代码的效果是相同的：

```
var x = 'a';  
var y = 'b';
```

数组解构也可以用来进行值的互换，而不需要创建临时变量，如下：

```
[x, y] = [y, x];
```

以上代码和下面代码的效果是相同的：

```
var temp = x;  
x = y;  
y = temp;
```

这对你学习排序算法会很有用，因为互换值的情况很常见。

还有一个称为属性简写的功能，它是对象解构的另一种方式。考虑如下例子：

```
var [x, y] = ['a', 'b'];  
var obj = { x, y };  
console.log(obj); // { x: "a", y: "b" }
```

以上代码和下面代码的效果是相同的：

```
var x = 'a';  
var y = 'b';  
var obj2 = { x: x, y: y };  
console.log(obj2); // { x: "a", y: "b" }
```

本节我们要讨论的最后一个功能是方法属性。这使得开发者可以在对象中声明看起来像属性的函数。下面是一个例子：

```
var hello = {  
  name: 'abcdef',  
  printHello() {  
    console.log('Hello');  
  }  
}  
console.log(hello.printHello());
```

以上代码也可以写成下面这样：

```
var hello = {  
  name: 'abcdef',  
  printHello: function printHello() {  
    console.log('Hello');  
  }  
};
```

你可以访问以下URL执行上面三个例子。

- ❑ 数组解构: <https://goo.gl/VsLecp>
- ❑ 变量互换: <https://goo.gl/EyFAII>
- ❑ 属性简写: <https://goo.gl/DKU2PN>

1.9.7 使用类进行面向对象编程

ES6还引入了一种更简洁的声明类的方式。在1.6节，你已经学习了像下面这样声明一个Book类的方式：

```
function Book(title, pages, isbn){ //{1}  
  this.title = title;  
  this.pages = pages;  
  this.isbn = isbn;  
}  
Book.prototype.printTitle = function(){  
  console.log(this.title);  
};
```

我们可以用ES6把语法简化如下：

```
class Book { //{2}  
  constructor (title, pages, isbn) {  
    this.title = title;  
    this.pages = pages;  
    this.isbn = isbn;  
  }  
  printIsbn(){  
    console.log(this.isbn);  
  }  
}
```

只需要使用class关键字，声明一个有constructor函数和诸如printIsbn等其他函数的类。行{1}声明Book类的代码与行{2}声明的代码具有相同的效果和输出：

```
let book = new Book('title', 'pag', 'isbn');  
console.log(book.title); //输出图书标题  
book.title = 'new title'; //更新图书标题  
console.log(book.title); //输出图书标题
```




你可以访问<https://goo.gl/UhK1n4>执行上面的例子。

1. 继承

除了新的声明类的方式，类的继承也有简化的语法。我们看一个例子：

```
class ITBook extends Book { //{1}
  constructor (title, pages, isbn, technology) {
    super(title, pages, isbn); //{2}
    this.technology = technology;
  }
  printTechnology(){
    console.log(this.technology);
  }
}

let jsBook = new ITBook('学习JS算法', '200', '1234567890', 'JavaScript');
console.log(jsBook.title);
console.log(jsBook.printTechnology());
```

我们可以用`extends`关键字扩展一个类并继承它的行为（行{1}）。在构造函数中，我们也可以通过`super`关键字引用父类的构造函数（行{2}）。

尽管在JavaScript中声明类的新方式的语法与Java、C、C++等其他编程语言很类似，但JavaScript面向对象编程还是基于原型实现的。



你可以访问<https://goo.gl/hgQvo9>执行上面的例子。

2. 使用属性存取器

使用新的类语法也可以为属性创建存取器函数。虽然不像其他面向对象语言（封装概念），类的属性不是私有的，但最好还是遵循一种命名模式。

下面的例子是一个声明了`get`和`set`函数的类：

```
class Person {
  constructor (name) {
    this._name = name; //{1}
  }
  get name() { //{2}
    return this._name;
  }
  set name(value) { //{3}
    this._name = value;
  }
}
```

```
}  
  
let lotrChar = new Person('Frodo');  
console.log(lotrChar.name); //{4}  
lotrChar.name = 'Gandalf'; //{5}  
console.log(lotrChar.name);  
lotrChar._name = 'Sam'; //{6}  
console.log(lotrChar.name);
```

要声明`get`和`set`函数，只需要在我们要暴露和使用的函数名前面加上`get`或`set`关键字（行{2}和行{3}）。我们可以用相同的名字声明类属性，或者在属性名前面加下划线（行{1}），让这个属性看起来像是私有的。

然后，只要像普通的属性一样，引用它们的名字（行{4}和行{5}），就可以执行`get`和`set`函数。



`_name`并非真正的私有属性，我们仍然可以引用它。本书后面的章节还会谈到这一点。

你可以访问<https://goo.gl/SMRYsv>执行上面的例子。

3. 其他功能

ES6还有其他一些功能，包括列表迭代器、类型数组、`Set`、`Map`、`WeakSet`、`WeakMap`、模块、尾调用、`Symbol`，等等。本书的其他章节会介绍其中部分功能。



更多关于ES6全部功能和规范的信息，请参考<http://www.ecma-international.org/ecma-262/6.0/>。

1.10 ECMAScript 7 的功能

本书编写时，已确定随ES7（ECMAScript 2016）发布的功能只有一个，就是`Array.prototype.includes`。

还有一些功能已经接近完成，有可能随ES7发布，但还未最终确定。

- ☐ 幂运算符
- ☐ SIMD.JS: API和填充脚本
- ☐ 异步函数
- ☐ `Object.values`和`Object.entries`
- ☐ 字符串填充
- ☐ 函数参数列表以逗号结尾

进行数学运算时，新的幂运算符**很方便，有了它，`Math.pow(2, 3)`可以简化为`2 ** 3`。

我们在第2章会进一步了解`Array.prototype.includes`的功能。



更多关于ES7的信息，请访问<https://tc39.github.io/ecma262/>。

ES6 和 ES7 的向下兼容性

我需要把现有的JavaScript代码更新到ES6或ES7吗？当然不用！ES6和ES7是JavaScript语言的子集。所有ES5规范的特性都可以继续使用。不过，你可以开始使用ES6和ES7的新语法，让代码变得更简单易读。

在本书接下来的章节中，我们会尽可能地使用ES6和ES7。如果你想使用ES5编写示例代码，可以用Babel把书中的代码转译到ES5。



我们要创建的这些例子也有ES5语法的版本，请访问GitHub仓库的master分支（<https://github.com/loiane/javascript-datastructures-algorithms>）。

好了，我们已经介绍了一些必要的JavaScript基础概念，是时候开始寻找数据结构和算法的乐趣了！

1.11 小结

本章主要讲述了如何搭建开发环境，有了这个环境就可以编写和运行书中的示例代码。

本章也讲了JavaScript语言的基础知识，这些知识会在接下来的数据结构和算法学习过程中用到。

你还学习了ECMAScript 6和ECMAScript 7的一些新功能，这会让我们接下来的例子变得更加简练。

下一章，我们要学习第一种数据结构：数组。许多语言都对数组有原生的支持（当然也包括JavaScript）。

几乎所有的编程语言都原生支持数组类型，因为**数组**是最简单的内存数据结构。JavaScript里也有数组类型，虽然它的第一个版本并没有支持数组。本章中，我们将深入学习数组数据结构和它的能力。

数组存储一系列同一种数据类型的值。但在JavaScript里，也可以在数组中保存不同类型的值。但我们还是要遵守最佳实践，别这么做（大多数语言都没这个能力）。

2.1 为什么用数组

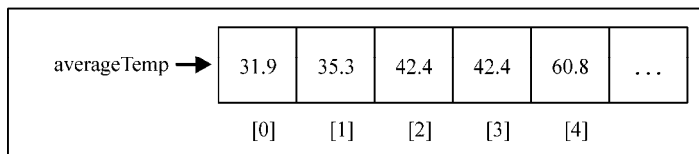
假如有这样一个需求：保存所在城市每个月的平均温度。可以这么做：

```
var averageTempJan = 31.9;
var averageTempFeb = 35.3;
var averageTempMar = 42.4;
var averageTempApr = 52;
var averageTempMay = 60.8;
```

当然，这肯定不是最好的方案。按照这种方式，如果只存一年的数据，我们能管理12个变量。但要多存几年的平均温度呢？幸运的是，我们可以用数组来解决，更加简洁地呈现同样的信息：

```
var averageTemp = [];
averageTemp[0] = 31.9;
averageTemp[1] = 35.3;
averageTemp[2] = 42.4;
averageTemp[3] = 52;
averageTemp[4] = 60.8;
```

数组averageTemp里的内容如下图所示：



2.2 创建和初始化数组

用JavaScript声明、创建和初始化数组很简单，就像下面这样：

```
var daysOfWeek = new Array(); //{1}
var daysOfWeek = new Array(7); //{2}
var daysOfWeek = new Array('Sunday', 'Monday', 'Tuesday', 'Wednesday',
    'Thursday', 'Friday', 'Saturday'); //{3}
```

使用new关键字，就能简单地声明并初始化一个数组（行{1}）。用这种方式，还可以创建一个指定长度的数组（行{2}）。另外，也可以直接将数组元素作为参数传递给它的构造器（行{3}）。

其实，用new创建数组并不是最好的方式。如果你想在JavaScript中创建一个数组，只用中括号（[]）的形式就行了，如下所示：

```
var daysOfWeek = [];
```

也可使用一些元素初始化数组，如下：

```
var daysOfWeek = ['Sunday', 'Monday', 'Tuesday', 'Wednesday',
    'Thursday', 'Friday', 'Saturday'];
```

如果想知道数组里已经存了多少个元素，可以使用数组的length属性。以下代码的输出是7：

```
console.log(daysOfWeek.length);
```

访问元素和迭代数组

要访问数组里特定位置的元素，可以用中括号传递数值位置，得到想知道的值或者赋新的值。假如我们想输出数组daysOfWeek里的所有元素，可以通过循环遍历数组，打印元素，如下所示：

```
for (var i=0; i<daysOfWeek.length; i++){
    console.log(daysOfWeek[i]);
}
```

我们来看另一个例子：求斐波那契数列的前20个数字。已知斐波那契数列中第一个数字是1，第二个是2，从第三项开始，每一项都等于前两项之和：

```
var fibonacci = []; //{1}
fibonacci[1] = 1; //{2}
fibonacci[2] = 2; //{3}

for(var i = 3; i < 20; i++){
    fibonacci[i] = fibonacci[i-1] + fibonacci[i-2]; //{4}
}

for(var i = 1; i<fibonacci.length; i++){ //{5}
    console.log(fibonacci[i]);           //{6}
}
```

- ❑ 在行{1}处, 我们声明并创建了一个数组。
- ❑ 在行{2}和行{3}, 把斐波那契数列中的前两个数字分别赋给了数组的第二和第三位 (在JavaScript中, 数组的第一位是0, 这里我们略过, 从第二位开始分别保存斐波那契数列中对应位置的元素)。
- ❑ 然后, 我们需要做的就是想办法得到斐波那契数列的第三到第二十位的数字 (前两个值我们已经初始化过了)。我们可以用循环来处理, 把数组中前两位上的元素相加, 结果赋给当前位置上的元素 (行{4}——从数组中的索引3到索引19)。
- ❑ 最后, 看看输出 (行{6}), 我们只需要循环遍历数组的各个元素 (行{5})。



示例代码里, 我们用`console.log`来输出数组中对应索引位置的值 (行{5}和行{6}), 也可以直接用`console.log(fibonacci)`输出数组。大多数浏览器都可以用这种方式, 清晰地输出数组。

现在如果想知道斐波那契数列其他位置上的值是多少, 要怎么办呢? 很简单, 把之前循环条件中的终止变量从20改成你希望的值就可以了。

2.3 添加元素

从数组中添加和删除元素也很容易, 但有时也会很棘手。假如我们有一个数组`numbers`, 初始化成0到9:

```
var numbers = [0,1,2,3,4,5,6,7,8,9];
```

如果想要给数组添加一个元素 (比如10), 只要把值赋给数组中最后一个空位上的元素即可。

```
numbers[numbers.length] = 10;
```



在JavaScript中, 数组是一个可以修改的对象。如果添加元素, 它就会动态增长。在C和Java等其他语言里, 我们要决定数组的大小, 想添加元素就要创建一个全新的数组, 不能简单地往其中添加所需的元素。

2.3.1 使用 `push` 方法

另外, 还有一个`push`方法, 能把元素添加到数组的末尾。通过`push`方法, 能添加任意个元素:

```
numbers.push(11);  
numbers.push(12, 13);
```

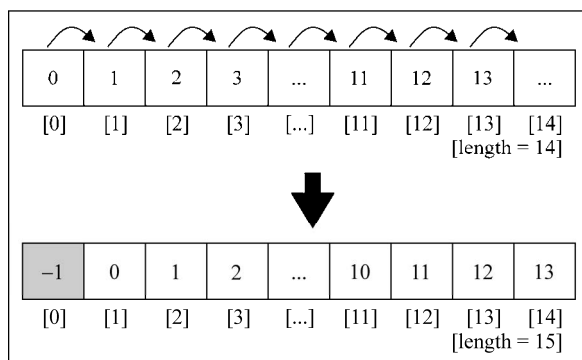
如果输出`numbers`的话, 就会看到从0到13的值。

2.3.2 插入元素到数组首位

现在，我们希望在数组中插入一个值，不像之前那样插入到最后，而是放到数组的首位。为了实现这个需求，首先我们要腾出数组里第一个元素的位置，把所有的元素向右移动一位。我们可以循环数组中的元素，从最后一位+1（长度）开始，将其对应的前一个元素的值赋给它，依次处理，最后把我们想要的值赋给第一个位置（-1）上。

```
for (var i=numbers.length; i>=0; i--){
    numbers[i] = numbers[i-1];
}
numbers[0] = -1;
```

下面这张图描述了我们刚才的操作过程：



使用unshift方法

在JavaScript里，数组有一个方法叫unshift，可以直接把数值插入数组的首位：

```
numbers.unshift(-2);
numbers.unshift(-4, -3);
```

那么，用unshift方法，我们就可以在数组的开始处添加值-2，然后添加-3、-4等。这样数组就会输出数字-4到13。

2.4 删除元素

目前为止，我们已经学习了如何给数组的开始和结尾位置添加元素。下面我们来看一下怎样从数组中删除元素。

要删除数组里最靠后的元素，可以用pop方法：

```
numbers.pop();
```



通过push和pop方法，就能用数组来模拟栈，你将会在下一章看到这部分内容。

现在，数组输出的数字是-4到12，并且数组的长度是17。

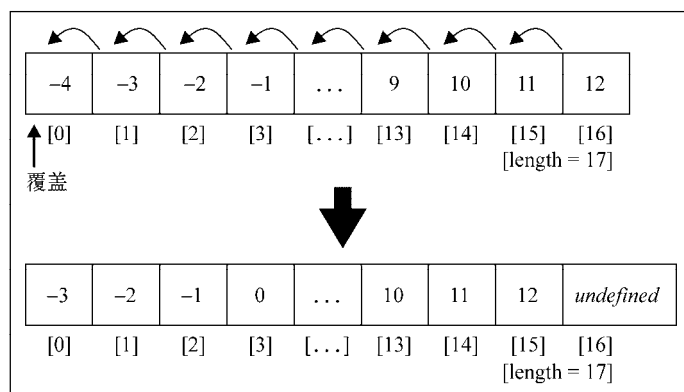
2

从数组首位删除元素

如果要移除数组里的第一个元素，可以用下面的代码：

```
for (var i = 0; i < numbers.length; i++){
  numbers[i] = numbers[i+1];
}
```

下面这张图呈现了这段代码的执行过程：



我们把数组里所有的元素都左移了一位。但数组的长度依然是17，这意味着数组中有额外的一个元素（值是`undefined`）。在最后一次循环里，`i + 1`引用了一个数组里还未初始化的位置。在Java、C/C+或C#等一些语言里，这样写可能会抛出异常了，因此不得不在`numbers.length - 1`处停止循环。

可以看到，我们只是把数组第一位的值用第二位覆盖了，并没有删除元素（因为数组的长度和之前还是一样的，并且多了多一个未定义元素）。

使用shift方法

要确实删除数组的第一个元素，可以用shift方法实现：

```
numbers.shift();
```

那么，假如本来数组中的值是从-4到12，长度为17，执行了上述代码后，数组就只有-3到12了，并且长度也会减小到16。



通过shift和unshift方法，就能用数组模拟基本的队列数据结构，第4章里会讲到。

2.5 在任意位置添加或删除元素

目前为止，我们已经学习了如何添加元素到数组的开头或结尾处，以及怎样删除数组开头和结束位置上的元素。那如何在数组中的任意位置上删除或添加元素？

我们可以使用splice方法，简单地通过指定位置/索引，就可以删除相应位置和数量的元素：

```
numbers.splice(5,3);
```

这行代码删除了从数组索引5开始的3个元素。这就意味着numbers[5]、numbers[6]和numbers[7]从数组中删除了。现在数组里的值变成了-3、-2、-1、0、1、5、6、7、8、9、10、11和12（2、3、4已经被移除）。



对于JavaScript数组和对象，我们还可以用delete操作符删除数组中的元素，例如delete numbers[0]。然而，数组位置0的值会变成undefined，也就是说，以上操作等同于numbers[0] = undefined。因此，我们应该始终使用splice、pop或shift（马上就会学到）方法来删除数组元素。

现在，我们想把数字2、3、4插入数组里，放到之前删除元素的位置上，可以再次使用splice方法：

```
numbers.splice(5,0,2,3,4);
```

splice方法接收的第一个参数，表示想要删除或插入的元素的索引值。第二个参数是删除元素的个数（这个例子里，我们的目的不是删除元素，所以传入0）。第三个参数往后，就是要添加到数组里的值（元素2、3、4）。输出会发现值又变成了从-3到12。

最后，执行下这行代码：

```
numbers.splice(5,3,2,3,4);
```

输出的值是从-3到12。原因在于，我们从索引5开始删除了3个元素，但也从索引5开始添加了元素2、3、4。

2.6 二维和多维数组

还记得本章开头平均气温测量的例子吗？现在我打算再用一下，不过把记录的数据改成数天内每小时的气温。现在我们已经知道可以用数组来保存这些数据，那么要保存两天的每小时气温

数据就可以这样：

```
var averageTempDay1 = [72,75,79,79,81,81];
var averageTempDay2 = [81,79,75,75,73,72];
```

然而，这不是最好的方法。我们可以做得更好。我们可以使用矩阵（二维数组）来存储这些信息。矩阵的行保存每天的数据，列对应小时级别的数据：

```
var averageTemp = [];
averageTemp[0] = [72,75,79,79,81,81];
averageTemp[1] = [81,79,75,75,73,72];
```

JavaScript只支持一维数组，并不支持矩阵。但是，我们可以像上面的代码一样，用数组套数组，实现矩阵或任一多维数组。代码也可以写成这样：

```
//day 1
averageTemp[0] = [];
averageTemp[0][0] = 72;
averageTemp[0][1] = 75;
averageTemp[0][2] = 79;
averageTemp[0][3] = 79;
averageTemp[0][4] = 81;
averageTemp[0][5] = 81;
//day 2
averageTemp[1] = [];
averageTemp[1][0] = 81;
averageTemp[1][1] = 79;
averageTemp[1][2] = 75;
averageTemp[1][3] = 75;
averageTemp[1][4] = 73;
averageTemp[1][5] = 72;
```

上面的代码里，我们分别指定了每天和每小时的数据。数组中的内容如下图所示：

	[0]	[1]	[2]	[3]	[4]	[5]
[0]	72	75	79	79	81	81
[1]	81	79	75	75	73	73

每行就是每天的数据，每列是当天不同时段的气温。

2.6.1 迭代二维数组的元素

如果想看这个矩阵的输出，我们可以创建一个通用函数，专门输出其中的值：

```
function printMatrix(myMatrix) {
  for (var i=0; i<myMatrix.length; i++){
```

```

    for (var j=0; j<myMatrix[i].length; j++){
        console.log(myMatrix[i][j]);
    }
}
}

```

需要遍历所有的行和列。因此，我们需要使用一个嵌套的for循环来处理，其中变量*i*为行，变量*j*为列。

使用以下代码查看矩阵averageTemp的输出：

```
printMatrix(averageTemp);
```

2.6.2 多维数组

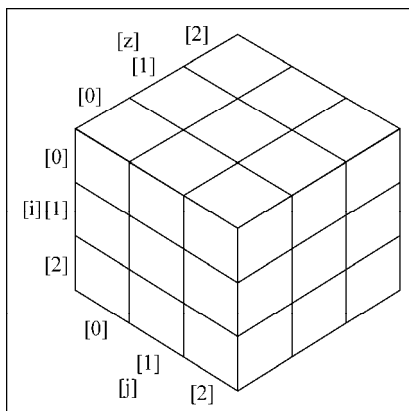
我们也可以用这种方式来处理多维数组。假如我们要创建一个3×3×3的矩阵，每一格里包含矩阵的*i*（行）、*j*（列）及*z*（深度）之和：

```

var matrix3x3x3 = [];
for (var i=0; i<3; i++){
    matrix3x3x3[i] = [];
    for (var j=0; j<3; j++){
        matrix3x3x3[i][j] = [];
        for (var z=0; z<3; z++){
            matrix3x3x3[i][j][z] = i+j+z;
        }
    }
}

```

数据结构中有几个维度都没关系，我们都可以用循环遍历每个维度来访问所有格子。3×3×3的矩阵也可用立体图表示如下：



可以用以下代码输出这个矩阵的内容：

```
for (var i=0; i<matrix3x3x3.length; i++){
  for (var j=0; j<matrix3x3x3[i].length; j++){
    for (var z=0; z<matrix3x3x3[i][j].length; z++){
      console.log(matrix3x3x3[i][j][z]);
    }
  }
}
```

如果是一个3×3×3×3的矩阵，代码中就会用四层嵌套的for语句，以此类推。

2.7 JavaScript 的数组方法参考

在JavaScript里，数组是可修改的对象，这意味着创建的每个数组都有一些可用的方法。数组很有趣，因为它们十分强大，并且相比其他语言中的数组，JavaScript中的数组有许多很好用的方法。这样就不用再为它开发一些基本功能了，例如在数据结构的中间添加或删除元素。

下面的表格中详述了数组的一些核心方法，其中的一些我们已经学习过了。

方法名	描 述
concat	连接2个或更多数组，并返回结果
every	对数组中的每一项运行给定函数，如果该函数对每一项都返回true，则返回true
filter	对数组中的每一项运行给定函数，返回该函数会返回true的项组成的数组
forEach	对数组中的每一项运行给定函数。这个方法没有返回值
join	将所有的数组元素连接成一个字符串
indexOf	返回第一个与给定参数相等的数组元素的索引，没有找到则返回-1
lastIndexOf	返回在数组中搜索到的与给定参数相等的元素的索引里最大的值
map	对数组中的每一项运行给定函数，返回每次函数调用的结果组成的数组
reverse	颠倒数组中元素的顺序，原先第一个元素现在变成最后一个，同样原先的最后一个元素变成了现在的第一个
slice	传入索引值，将数组里对应索引范围内的元素作为新数组返回
some	对数组中的每一项运行给定函数，如果任一项返回true，则返回true
sort	按照字母顺序对数组排序，支持传入指定排序方法的函数作为参数
toString	将数组作为字符串返回
valueOf	和toString类似，将数组作为字符串返回

我们已经学过了push、pop、shift、unshift和splice方法。下面来看表格中提到的方法。在本书接下来的章节里，编写数据结构和算法时会大量用到这些方法。

2.7.1 数组合并

考虑如下场景：有多个数组，需要合并起来成为一个数组。我们可以迭代各个数组，然后把每个元素加入最终的数组。幸运的是，JavaScript已经给我们提供了解决方法，叫作concat方法：

```
var zero = 0;
var positiveNumbers = [1,2,3];
var negativeNumbers = [-3,-2,-1];
var numbers = negativeNumbers.concat(zero, positiveNumbers);
```

concat方法可以向一个数组传递数组、对象或是元素。数组会按照该方法传入的参数顺序连接指定数组。在这个例子里，zero将被合并到negativeNumbers中，然后positiveNumbers继续被合并。最后输出的结果是-3、-2、-1、0、1、2、3。

2.7.2 迭代器函数

有时我们需要迭代数组中的元素。前面我们已经学过，可以用循环语句来处理，例如for语句。

JavaScript内置了许多数组可用的迭代方法。对于本节的例子，我们需要数组和函数。假如有一个数组，它值是从1到15，如果数组里的元素可以被2整除（偶数），函数就返回true，否则返回false：

```
var isEven = function (x) {
    // 如果x是2的倍数，就返回true
    console.log(x);
    return (x % 2 == 0) ? true : false;
};
var numbers = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15];
```



return (x % 2 == 0) ? true : false也可以写成return (x % 2== 0)。

1. 用every方法迭代

我们要尝试的第一个方法是every。every方法会迭代数组中的每个元素，直到返回false。

```
numbers.every(isEven);
```

在这个例子里，数组numbers的第一个元素是1，它不是2的倍数（1是奇数），因此isEven函数返回false，然后every执行结束。

2. 用some方法迭代

下一步，我们来看some方法。它和every的行为类似，不过some方法会迭代数组的每个元素，直到函数返回true：

```
numbers.some(isEven);
```

在我们的例子里，numbers数组中第一个偶数是2（第二个元素）。第一个被迭代的元素是1，

isEven会返回false。第二个被迭代的元素是2，isEven返回true——迭代结束。

3. 用forEach方法迭代

如果要迭代整个数组，可以用forEach方法。它和使用for循环的结果相同：

```
numbers.forEach(function(x){
  console.log((x % 2 == 0));
});
```

4. 使用map和filter方法

JavaScript还有两个会返回新数组的遍历方法。第一个是map：

```
var myMap = numbers.map(isEven);
```

数组myMap里的值是：[false, true, false, true, false, true, false, true, false, true, false, true, false]。它保存了传入map方法的isEven函数的运行结果。这样就很容易知道一个元素是否是偶数。比如，myMap[0]是false，因为1不是偶数；而myMap[1]是true，因为2是偶数。

还有一个filter方法。它返回的新数组由使函数返回true的元素组成：

```
var evenNumbers = numbers.filter(isEven);
```

在我们的例子里，evenNumbers数组中的元素都是偶数：[2, 4, 6, 8, 10, 12, 14]。

5. 使用reduce方法

最后是reduce方法。reduce方法接收一个函数作为参数，这个函数有四个参数：previousValue、currentValue、index和array。这个函数会返回一个将被叠加到累加器的值，reduce方法停止执行后会返回这个累加器。如果要对一个数组中的所有元素求和，这就很有用，比如：

```
numbers.reduce(function(previous, current, index){
  return previous + current;
});
```

输出将会是120。



JavaScript的Array类还有另外两个重要方法：map和reduce。这两个方法名是自解释的，这意味着map方法会依照给定函数对值进行映射，而reduce方法会依照函数规约数组包含的值。这三个方法（map、filter和reduce）是我们要在第11章学习的JavaScript函数式编程的基础。

2.7.3 ECMAScript 6 和数组的新功能

第1章提到过，ECMAScript 6（ES6或ES2015）和ECMAScript 7（ES7或ES2016）规范给JavaScript语言带来了新的功能。

下表列出了ES6和ES7新增的数组方法。

方 法	描 述
@@iterator	返回一个包含数组键值对的迭代器对象，可以通过同步调用得到数组元素的键值对
copyWithin	复制数组中一系列元素到同一数组指定的起始位置
entries	返回包含数组所有键值对的@@iterator
includes	如果数组中存在某个元素则返回true，否则返回false。ES7新增
find	根据回调函数给定的条件从数组中查找元素，如果找到则返回该元素
findIndex	根据回调函数给定的条件从数组中查找元素，如果找到则返回该元素在数组中的索引
fill	用静态值填充数组
from	根据已有数组创建一个新数组
keys	返回包含数组所有索引的@@iterator
of	根据传入的参数创建一个新数组
values	返回包含数组中所有值的@@iterator

除了这些新的方法，还有一种用for...of循环来迭代数组的新做法，以及可以从数组实例得到的迭代器对象。

在后面的主题中，我们会演示所有的新功能。

1. 使用forEach和箭头函数迭代

箭头函数可以简化使用forEach迭代数组元素的做法。代码例子如下：

```
numbers.forEach(function (x) {
  console.log(x % 2 == 0);
});
```

这段代码可以简化如下：

```
numbers.forEach(x => {
  console.log((x % 2 == 0));
});
```

2. 使用for...of循环迭代

你已经学过用for循环和forEach方法迭代数组。ES6还引入了迭代数组值的for...of循环，来看看它的用法：

```
for (let n of numbers) {
  console.log((n % 2 == 0) ? 'even' : 'odd');
}
```



可以访问<https://goo.gl/qHYAN1>运行上面的示例。

3. 使用ES6新的迭代器（@@iterator）

2

ES6还为Array类增加了一个@@iterator属性，需要通过Symbol.iterator来访问。代码如下：

```
let iterator = numbers[Symbol.iterator]();
console.log(iterator.next().value); // 1
console.log(iterator.next().value); // 2
console.log(iterator.next().value); // 3
console.log(iterator.next().value); // 4
console.log(iterator.next().value); // 5
```

然后，不断调用迭代器的next方法，就能依次得到数组中的值。numbers数组中有15个值，因此需要调用15次iterator.next().value。

数组中所有值都迭代完之后，iterator.next().value会返回undefined。

以上代码的输出和我们接下来要讲的numbers.value()是一样的。



访问<https://goo.gl/L81UQW>查看和运行示例。

● 数组的entries、keys和values方法

ES6还增加了三种从数组中得到迭代器的方法。我们首先要学习的是entries方法。

entries方法返回包含键值对的@@iterator，下面是使用这个方法的代码示例：

```
let aEntries = numbers.entries(); // 得到键值对的迭代器
console.log(aEntries.next().value); // [0, 1] - 位置0的值为1
console.log(aEntries.next().value); // [1, 2] - 位置1的值为2
console.log(aEntries.next().value); // [2, 3] - 位置2的值为3
```

numbers数组中都是数字，key是数组中的位置，value是保存在数组索引的值。

使用集合、字典、散列表等数据结构时，能够取出键值对是很有用的。这个功能会在本书后面的章节中大显身手。

keys方法返回包含数组索引的@@iterator，下面是使用这个方法的代码示例：

```
let aKeys = numbers.keys(); // 得到数组索引的迭代器
console.log(aKeys.next()); // {value: 0, done: false }
console.log(aKeys.next()); // {value: 1, done: false }
console.log(aKeys.next()); // {value: 2, done: false }
```


`keys`方法会返回`numbers`数组的索引。一旦没有可迭代的值, `aKeys.next()`就会返回一个`value`属性为`undefined`, `done`属性为`true`的对象。如果`done`属性的值为`false`, 就意味着还有可迭代的值。

`values`方法返回的`@@iterator`则包含数组的值。使用这个方法的代码示例如下:

```
let aValues = numbers.values();
console.log(aValues.next()); // {value: 1, done: false }
console.log(aValues.next()); // {value: 2, done: false }
console.log(aValues.next()); // {value: 3, done: false }
```



记住, 当前的浏览器还没有完全支持ES6所有的新功能, 因此, 测试这些代码最好的办法是使用Babel。访问<https://goo.gl/eojEGk>查看和运行示例。

4. 使用`from`方法

`Array.from`方法根据已有的数组创建一个新数组。比如, 要复制`numbers`数组, 可以这样做:

```
let numbers2 = Array.from(numbers);
```

还可以传入一个用来过滤值的函数, 例子如下:

```
let evens = Array.from(numbers, x => (x % 2 == 0));
```

上面的代码会创建一个`evens`数组, 其中只包含`numbers`数组中的偶数。



访问<https://goo.gl/n4rOY4>查看和运行示例。

5. 使用`Array.of`方法

`Array.of`方法根据传入的参数创建一个新数组。以下面的代码为例:

```
let numbers3 = Array.of(1);
let numbers4 = Array.of(1, 2, 3, 4, 5, 6);
```

它和下面这段代码的效果一样:

```
let numbers3 = [1];
let numbers4 = [1, 2, 3, 4, 5, 6];
```

我们也可以用这个方法复制已有的数组, 比如:

```
let numbersCopy = Array.of(...numbers4);
```

上面的代码和`Array.from(numbers4)`的效果是一样的, 区别只是用到了第1章讲过的展开操作符。展开操作符`(...)`会把`numbers4`数组里的值都展开成参数。



访问<https://goo.gl/FoJYNf>查看和运行示例。

6. 使用fill方法

2

fill方法用静态值填充数组。以下面的代码为例：

```
let numbersCopy = Array.of(1, 2, 3, 4, 5, 6);
```

numbersCopy数组的length是6，也就是有6个位置。再看下面的代码：

```
numbersCopy.fill(0);
```

numbersCopy数组所有位置的值都会变成0（[0, 0, 0, 0, 0, 0]）。

我们还可以指定开始填充的索引，如下：

```
numbersCopy.fill(2, 1);
```

上面的例子里，数组中从1开始的所有位置，值都是2（[0, 2, 2, 2, 2, 2]）。

同样，也可以指定结束填充的索引：

```
numbersCopy.fill(1, 3, 5);
```

上面的例子里，我们会把1填充到数组索引3到5的位置（不包括5），得到的数组为[0, 2, 2, 1, 1, 2]。

创建数组并初始化值的时候，fill方法非常好用，就像下面这样：

```
let ones = Array(6).fill(1);
```

上面的代码创建了一个长度为6，所有的值都是1的数组（[1, 1, 1, 1, 1, 1]）。



访问<https://goo.gl/sqiHSK>查看和运行示例。

7. 使用copyWithin方法

copyWithin方法复制数组中的一系列元素到同一数组指定的起始位置。看看下面这个例子：

```
let copyArray = [1, 2, 3, 4, 5, 6];
```

假如我们想把4、5、6三个值复制到数组前三个位置，得到[4, 5, 6, 4, 5, 6]这个数组。可以用下面的代码达到目的：

```
copyArray.copyWithin(0, 3);
```

假如我们想把4、5两个值（位置3和4）复制到位置1和2，可以这样做：

```
copyArray = [1, 2, 3, 4, 5, 6];  
copyArray.copyWithin(1, 3, 5);
```

这种情况下，会把从位置3开始到位置5结束（不包括5）的元素复制到位置1，结果是得到数组[1, 4, 5, 4, 5, 6]。



访问<https://goo.gl/hZhBE1>查看和运行示例。

2.7.4 排序元素

通过本书，我们能学到如何编写最常用的搜索和排序算法。其实，JavaScript里也提供了一个排序方法和一组搜索方法。让我们来看看。

首先，我们想反序输出数组numbers（它本来的排序是1, 2, 3, 4, …, 15）。要实现这样的功能，可以用reverse方法，然后数组内元素就会反序。

```
numbers.reverse();
```

现在，输出numbers的话就会看到[15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]。然后，我们用sort方法：

```
numbers.sort();
```

然而，如果输出数组，结果会是[1, 10, 11, 12, 13, 14, 15, 2, 3, 4, 5, 6, 7, 8, 9]。看起来不大对，是吧？这是因为sort方法在对数组做排序时，把元素默认成字符串进行相互比较。

我们可以传入自己写的比较函数。因为数组里都是数字，所以可以这样写：

```
numbers.sort(function(a, b) {  
    return a-b;  
});
```

这段代码，在b大于a时，会返回负数，反之则返回正数。如果相等的话，就会返回0。也就是说返回的是负数，就说明a比b小，这样sort就根据返回值的情况给数组做排序。

之前的代码也可以被表示成这样，会更清晰一些：

```
function compare(a, b) {  
    if (a < b) {  
        return -1;  
    }  
    if (a > b) {  
        return 1;  
    }  
}
```

```
// a必须等于b
return 0;
}

numbers.sort(compare);
```

这是因为JavaScript的sort方法接受compareFunction作为参数，然后sort会用它排序数组。在例子里，我们声明了一个用来比较数组元素的函数，使数组按升序排序。

2

1. 自定义排序

我们可以对任何对象类型的数组排序，也可以创建compareFunction来比较元素。例如，对象Person有名字和年龄属性，我们希望根据年龄排序，就可以这么写：

```
var friends = [
  {name: 'John', age: 30},
  {name: 'Ana', age: 20},
  {name: 'Chris', age: 25}
];

function comparePerson(a, b){
  if (a.age < b.age){
    return -1
  }
  if (a.age > b.age){
    return 1
  }
  return 0;
}

console.log(friends.sort(comparePerson));
```

在这个例子里，最后会输出Ana(20)，Chris(25)，John(30)。

2. 字符串排序

假如有这样一个数组：

```
var names = ['Ana', 'ana', 'john', 'John'];
console.log(names.sort());
```

你猜会输出什么？答案是这样的：

```
["Ana", "John", "ana", "john"]
```

既然a在字母表里排第一位，为何ana却排在了John之后呢？这是因为JavaScript在做字符比较的时候，是根据字符对应的ASCII值来比较的。例如，A、J、a、j对应的ASCII值分别是65、75、97、106。

虽然在字母表里a是最靠前的，但J的ASCII值比a的小，所以排在a前面。



想了解更多关于ASCII表的信息，请访问<http://www.asciitable.com/>。

现在，如果给`sort`传入一个忽略大小写的比较函数，将会输出`["Ana", "ana", "John", "john"]`：

```
names.sort(function(a, b){
  if (a.toLowerCase() < b.toLowerCase()){
    return -1
  }
  if (a.toLowerCase() > b.toLowerCase()){
    return 1
  }
  return 0;
});
```

假如对带有重音符号的字符做排序的话，我们可以用`localCompare`来实现：

```
var names2 = ['Maève', 'Maeve'];
console.log(names2.sort(function(a, b){
  return a.localCompare(b);
}));
```

最后输出的结果将是`["Maeve", "Maève"]`。

2.7.5 搜索

搜索有两个方法：`indexOf`方法返回与参数匹配的元素的索引，`lastIndexOf`返回与参数匹配的最后一个元素的索引。我们来看看之前用过的`numbers`数组：

```
console.log(numbers.indexOf(10));
console.log(numbers.indexOf(100));
```

在这个示例中，第一行的输出是9，第二行的输出是-1（因为100不在数组里）。

下面的代码会返回同样的结果：

```
numbers.push(10);
console.log(numbers.lastIndexOf(10));
console.log(numbers.lastIndexOf(100));
```

我们往数组里加入了一个新的元素10，因此第二行会输出15（数组中的元素是1到15，还有10），第三行会输出-1（因为100不在数组里）。

1. ECMAScript 6——`find`和`findIndex`方法

看看下面这个例子：

```
let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15];
function multipleOf13(element, index, array) {
  return (element % 13 == 0) ? true : false;
}
console.log(numbers.find(multipleOf13));
console.log(numbers.findIndex(multipleOf13));
```

`find`和`findIndex`方法接收一个回调函数，搜索一个满足回调函数条件的值。上面的例子里，我们要从数组里找一个13的倍数。

`find`和`findIndex`的不同之处在于，`find`方法返回第一个满足条件的值，`findIndex`方法则返回这个值在数组里的索引。如果没有满足条件的值，`find`会返回`undefined`，而`findIndex`返回`-1`。



访问<https://goo.gl/2vAaCh>查看和运行示例。

2. ECMAScript 7——使用`includes`方法

如果数组里存在某个元素，`includes`方法会返回`true`，否则返回`false`。使用`includes`方法的例子如下：

```
console.log(numbers.includes(15));
console.log(numbers.includes(20));
```

例子里的`includes(15)`返回`true`，`includes(20)`返回`false`，因为`numbers`数组里没有20。

如果给`includes`方法传入一个起始索引，搜索会从索引指定的位置开始：

```
let numbers2 = [7, 6, 5, 4, 3, 2, 1];
console.log(numbers2.includes(4, 5));
```

上面的例子输出为`false`，因为数组索引5之后的元素不包含4。



访问<https://goo.gl/tTY9bc>查看和运行示例。

2.7.6 输出数组为字符串

现在，我们学习最后两个方法：`toString`和`join`。

如果想把数组里所有元素输出为一个字符串，可以用`toString`方法：

```
console.log(numbers.toString());
```

1、2、3、4、5、6、7、8、9、10、11、12、13、14、15和10这些值都会在控制台中输出。

如果想用一个不同的分隔符（比如-）把元素隔开，可以用join方法：

```
var numbersString = numbers.join('-');
console.log(numbersString);
```

这将输出：

1-2-3-4-5-6-7-8-9-10-11-12-13-14-15-10

如果要把数组内容发送到服务器，或进行编码（知道了分隔符，解码也很容易），这会很有用。

有一些很棒的资源可以帮助你更深入地了解数组及其方法。

- ❑ 第一个是w3schools的数组页面：http://www.w3schools.com/js/js_arrays.asp。
- ❑ 第二个是w3schools的数组方法页面：http://www.w3schools.com/js/js_array_methods.asp。
- ❑ Mozilla的数组及其方法的页面也非常棒，还有不错的例子：https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array（<http://google/vuldiT>）。
- ❑ 在JavaScript项目中使用数组时，也有一些很棒的类库。

Underscore：<http://underscorejs.org/>

Lo-Dash：<http://lodash.com/>

2.8 类型数组

与C和Java等其他语言不同，JavaScript数组不是强类型的，因此它可以存储任意类型的数据。

而类型数组则用于存储单一类型的数据。它的语法是`let myArray = new TypedArray(length)`，其中TypedArray需替换为下表所列之一。

类型数组	数据类型
Int8Array	8位二进制补码整数
Uint8Array	8位无符号整数
Uint8ClampedArray	8位无符号整数
Int16Array	16位二进制补码整数
Uint16Array	16位无符号整数
Int32Array	32位二进制补码整数
Uint32Array	32位无符号整数
Float32Array	32位IEEE浮点数
Float64Array	64位IEEE浮点数

代码示例如下：

```
let length = 5;
let int16 = new Int16Array(length);

let array16 = [];
array16.length = length;

for (let i=0; i<length; i++){
  int16[i] = i+1;
}
console.log(int16);
```

2

使用WebGL API、进行位操作、处理文件和图像时，类型数组都可以大展拳脚。它用起来和普通数组也毫无二致，本章所学的数组方法和功能都可以用于类型数组。

<http://goo.gl/kZBsGx>是一个很好的教程，讲解了如何使用类型数组处理二进制数据，以及它在实际项目中的应用。

2.9 小结

在本章中，我们学习了最常用的数据结构：数组。我们学习了如何声明和初始化数组，给数组赋值，以及添加和移除数组元素，还学习了二维和多维数组以及数组的主要方法。这对我们在后面章节中编写自己的算法很有用。

我们还学习了ES2015和ES2016规范新增的Array方法和功能。

下一章，我们将学习栈，一种具有特殊行为的数组。

数组是计算机科学中最常用的数据结构，上一章我们学习了如何创建和使用它。我们知道，可以在数组的任意位置上删除或添加元素。然而，有时候我们还需要一种在添加或删除元素时有更多控制的数据结构。有两种数据结构类似于数组，但在添加和删除元素时更为可控。它们就是栈和队列。

本章内容包括：

- ❑ 栈数据结构
- ❑ 向栈添加元素
- ❑ 从栈移除元素
- ❑ 如何使用Stack类
- ❑ 十进制转二进制

3.1 栈数据结构

栈是一种遵从后进先出（LIFO）原则的有序集合。新添加的或待删除的元素都保存在栈的同一端，称作栈顶，另一端就叫栈底。在栈里，新元素都靠近栈顶，旧元素都接近栈底。

在现实生活中也能发现很多栈的例子。例如，下图里的一摞书或者餐厅里堆放的盘子。



栈也被用在编程语言的编译器和内存中保存变量、方法调用等。

3.1.1 创建栈

我们将创建一个类来表示栈。让我们从基础开始，先声明这个类：

```
function Stack() {  
    //各种属性和方法的声明  
}
```

首先，我们需要一种数据结构来保存栈里的元素。可以选择数组：

```
let items = [];
```

接下来，要为我们的栈声明一些方法。

- ❑ `push(element(s))`：添加一个（或几个）新元素到栈顶。
- ❑ `pop()`：移除栈顶的元素，同时返回被移除的元素。
- ❑ `peek()`：返回栈顶的元素，不对栈做任何修改（这个方法不会移除栈顶的元素，仅仅返回它）。
- ❑ `isEmpty()`：如果栈里没有任何元素就返回`true`，否则返回`false`。
- ❑ `clear()`：移除栈里的所有元素。
- ❑ `size()`：返回栈里的元素个数。这个方法和数组的`length`属性很类似。

3.1.2 向栈添加元素

我们要实现的第一个方法是`push`。这个方法负责往栈里添加新元素，有一点很重要：该方法只添加元素到栈顶，也就是栈的末尾。`push`方法可以这样写：

```
this.push = function(element){  
    items.push(element);  
};
```

因为我们使用了数组来保存栈里的元素，所以可以用上一章里学到的数组的`push`方法来实现。

3.1.3 从栈移除元素

接着，我们来实现`pop`方法。这个方法主要用来移除栈里的元素。栈遵从LIFO原则，因此移出的是最后添加进去的元素。因此，我们可以用上一章讲数组时介绍的`pop`方法。栈的`pop`方法可以这样写：

```
this.pop = function(){  
    return items.pop();  
};
```

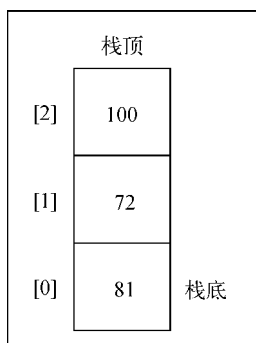
只能用`push`和`pop`方法添加和删除栈中元素，这样一来，我们的栈自然就遵从了LIFO原则。

3.1.4 查看栈顶元素

现在，为我们的类实现一些额外的辅助方法。如果想知道栈里最后添加的元素是什么，可以用`peek`方法。这个方法将返回栈顶的元素：

```
this.peek = function(){  
    return items[items.length-1];  
};
```

因为类内部是用数组保存元素的，所以访问数组的最后一个元素可以用 `length - 1`：



在上图中，有一个包含三个元素的栈，因此内部数组的长度就是3。数组中最后一项的位置是2，`length - 1` (`3 - 1`) 正好是2。

3.1.5 检查栈是否为空

下一个要实现的方法是 `isEmpty`，如果栈为空的话将返回`true`，否则就返回`false`：

```
this.isEmpty = function(){  
    return items.length == 0;  
};
```

使用`isEmpty`方法，我们能简单地判断内部数组的长度是否为0。

类似于数组的`length`属性，我们也能实现栈的`length`。对于集合，最好用`size`代替`length`。因为栈的内部使用数组保存元素，所以能简单地返回栈的长度：

```
this.size = function(){  
    return items.length;  
};
```

3.1.6 清空和打印栈元素

最后，我们来实现`clear`方法。`clear`方法用来移除栈里所有的元素，把栈清空。实现这个

方法最简单的方式是：

```
this.clear = function(){
  items = [];
};
```

另外也可以多次调用pop方法，把数组中的元素全部移除，这样也能实现clear方法。

完成了！栈已经实现。通过一个例子来放松一下：为了检查栈里的内容，我们来实现一个辅助方法，叫print。它会把栈里的元素都输出到控制台：

```
this.print = function(){
  console.log(items.toString());
};
```

这样，我们就完整创建了栈！

使用Stack类

在深入了解栈的应用前，我们先来学习如何使用Stack类。

首先，我们需要初始化Stack类。然后，验证一下栈是否为空（输出是true，因为还没有往栈里添加元素）。

```
let stack = new Stack();
console.log(stack.isEmpty()); //输出为true
```

接下来，往栈里添加一些元素（这里我们添加数字5和8；你可以添加任意类型的元素）：

```
stack.push(5);
stack.push(8);
```

如果调用peek方法，将会输出8，因为它是往栈里添加的最后一个元素：

```
console.log(stack.peek()); //输出8
```

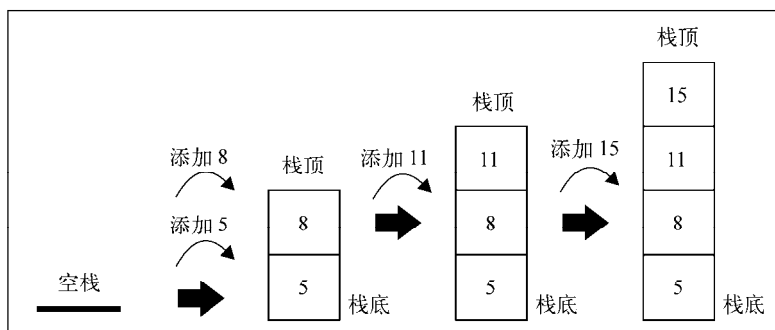
再添加一个元素：

```
stack.push(11);
console.log(stack.size()); //输出3
console.log(stack.isEmpty()); //输出false
```

我们往栈里添加了11。如果调用size方法，输出为3，因为栈里有三个元素（5、8和11）。如果我们调用isEmpty方法，会看到输出了false（因为栈里有三个元素，不是空栈）。最后，我们再添加一个元素：

```
stack.push(15);
```

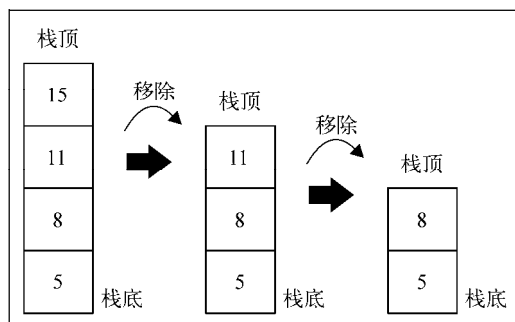
下图描绘了目前为止我们对栈的操作，以及栈的当前状态：



然后，调用两次`pop`方法从栈里移除2个元素：

```
stack.pop();
stack.pop();
console.log(stack.size()); //输出2
stack.print(); //输出[5, 8]
```

在两次调用`pop`方法前，我们的栈里有四个元素。调用两次后，现在栈里只剩下5和8了。下图描绘这个过程的执行：



3.2 ECMAScript 6 和 `stack` 类

我们花点时间分析一下代码，看看是否能用ECMAScript 6（ES6）的新功能来改进。

我们创建了一个可以当作类来使用的`Stack`函数。JavaScript函数都有构造函数，可以用来模拟类的行为。我们声明了一个私有的`items`变量，它只能被`Stack`函数/类访问。然而，这个方法为每个类的实例都创建一个`items`变量的副本。因此，如果要创建多个`Stack`实例，它就不太适合了。

看看如何用ES6新语法声明`Stack`类，并和前面的做法比较一下优缺点。

用 ES6 语法声明 `stack` 类

首先来分析下面的代码：

```
class Stack {
  constructor () {
    this.items = []; //{1}
  }
  push(element){
    this.items.push(element);
  }
  //其他方法
}
```

我们只是用ES6的简化语法把Stack函数转换成Stack类。这种方法不能像其他语言（Java、C++、C#）一样直接在类里面声明变量，只能在类的构造函数constructor里声明（行{1}），在类的其他函数里用this.nameofVariable就可以引用这个变量。

尽管代码看起来更简洁、更漂亮，变量items却是公共的。ES6的类是基于原型的。虽然基于原型的类比基于函数的类更节省内存，也更适合创建多个实例，却不能够声明私有属性（变量）或方法。而且，在这种情况下，我们希望Stack类的用户只能访问暴露给类的方法。否则，就有可能从栈的中间移除元素（因为我们用数组来存储其值），这不是我们希望看到的。

看看ES6语法有没有其他的方法可以创建私有属性。

1. 用ES6的限定作用域Symbol实现类

ES6新增了一种叫作Symbol的基本类型，它是不可变的，可以用作对象的属性。看看怎么用它在Stack类中声明items属性：

```
let _items = Symbol(); //{1}

class Stack {
  constructor () {
    this[_items] = []; //{2}
  }
  //Stack方法
}
```

在上面的代码中，我们声明了Symbol类型的变量_items（行{1}），在类的constructor函数中初始化它的值（行{2}）。要访问_items，只需把所有的this.items都换成this[_items]。

这种方法创建了一个假的私有属性，因为ES6新增的Object.getOwnPropertySymbols方法能够取到类里面声明的所有Symbols属性。下面是一个破坏Stack类的例子：

```
let stack = new Stack();
stack.push(5);
stack.push(8);
let objectSymbols = Object.getOwnPropertySymbols(stack);
console.log(objectSymbols.length); // 1
console.log(objectSymbols); // [Symbol()]
console.log(objectSymbols[0]); // Symbol()
stack[objectSymbols[0]].push(1);
stack.print(); //输出 5, 8, 1
```

从以上代码可以看到，访问`stack[objectSymbols[0]]`是可以得到`_items`的。并且，`_items`属性是一个数组，可以进行任意的数组操作，比如从中间删除或添加元素。我们操作的是栈，不应该出现这种行为。

还有第三个方案。

2. 用ES6的WeakMap实现类

有一种数据类型可以确保属性是私有的，这就是WeakMap。我们会在第7章深入探讨Map这种数据结构，现在只需要知道WeakMap可以存储键值对，其中键是对象，值可以是任意数据类型。

如果用WeakMap来存储items变量，Stack类就是这样的：

```
const items = new WeakMap(); //{1}

class Stack {
  constructor () {
    items.set(this, []); //{2}
  }
  push(element) {
    let s = items.get(this); //{3}
    s.push(element);
  }
  pop() {
    let s = items.get(this);
    let r = s.pop();
    return r;
  }
  //其他方法
}
```

- 行{1}，声明一个WeakMap类型的变量items。
- 行{2}，在constructor中，以this（Stack类自己的引用）为键，把代表栈的数组存入items。
- 行{3}，从WeakMap中取出值，即以this为键（行{2}设置的）从items中取值。

现在我们知道，items在Stack类里是真正的私有属性了，但还有一件事要做。items现在仍然是在Stack类以外声明的，因此谁都可以改动它。我们要用一个闭包（外层函数）把Stack类包起来，这样就只能在这个函数里访问WeakMap：

```
let Stack = (function () {
  const items = new WeakMap();
  class Stack {
    constructor () {
      items.set(this, []);
    }
    //其他方法
  }
})
```

```
return Stack; //{5}
})();
```

当Stack函数里的构造函数被调用时，会返回Stack类的一个实例（行{5}）。



关于JavaScript闭包，请阅读http://www.w3schools.com/js/js_function_closures.asp。

现在，Stack类有一个名为items的私有属性。虽然它很丑陋，但毕竟实现了私有属性。然而，用这种方法的话，扩展类无法继承私有属性。鱼与熊掌不可兼得！

3

把上面的代码跟本章最初实现的Stack类做个比较，我们会发现有一些相似之处：

```
function Stack() {
  let items = [];
  //其他方法
}
```

事实上，尽管ES6引入了类的语法，我们仍然不能像在其他编程语言中一样声明私有属性或方法。有很多种方法都可以达到相同的效果，但无论是语法还是性能，这些方法都有各自的优点和缺点。

哪种方法更好？这取决于你在实际项目中如何使用本书中这些算法，要处理的数据量，要创建的实例个数，以及其他约束条件。最终，还是取决于你。



在本书提供下载的代码中，所有的数据结构都会包含简单的函数类，以及ES6的WeakMap和闭包的创建方法。

3.3 用栈解决问题

栈的实际应用非常广泛。在回溯问题中，它可以存储访问过的任务或路径、撤销的操作（后面的章节讨论图和回溯问题时，我们会学习如何应用这个例子）。Java和C#用栈来存储变量和方法调用，特别是处理递归算法时，有可能抛出一个栈溢出异常（后面的章节也会介绍）。

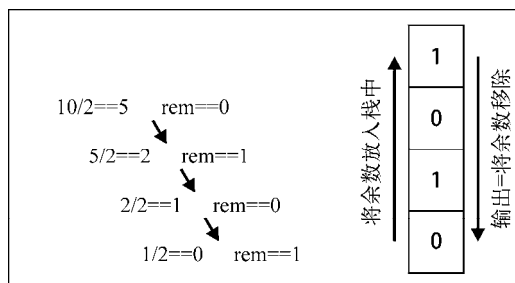
既然我们已经了解了Stack类的用法，不妨用它来解决一些计算机科学问题。本节，我们将学习使用栈的三个最著名的算法示例。首先是十进制转二进制问题，以及任意进制转换的算法；然后是平衡圆括号问题；最后，我们会学习如何用栈解决汉诺塔问题。

从十进制到二进制

现实生活中，我们主要使用十进制。但在计算科学中，二进制非常重要，因为计算机里的所有内容都是用二进制数字表示的（0和1）。没有十进制和二进制相互转化的能力，与计算机交流

就很困难。

要把十进制转化成二进制，我们可以将该十进制数字和2整除（二进制是满二进一），直到结果是0为止。举个例子，把十进制的数字10转化成二进制的数字，过程大概是这样：



大学的计算机课一般都会先教这个进制转换。下面是对应的算法描述：

```
function divideBy2(decNumber){  
  
    var remStack = new Stack(),  
        rem,  
        binaryString = '';  
  
    while (decNumber > 0){ //{1}  
        rem = Math.floor(decNumber % 2); //{2}  
        remStack.push(rem); //{3}  
        decNumber = Math.floor(decNumber / 2); //{4}  
    }  
  
    while (!remStack.isEmpty()){ //{5}  
        binaryString += remStack.pop().toString();  
    }  
  
    return binaryString;  
}
```

在这段代码里，当结果满足和2做整除的条件时（行{1}），我们会获得当前结果和2的余数，放到栈里（行{2}、{3}）。然后让结果和2做整除（行{4}）。另外请注意：JavaScript有数字类型，但是它不会区分究竟是整数还是浮点数。因此，要使用`Math.floor`函数让除法的操作仅返回整数部分。最后，用`pop`方法把栈中的元素都移除，把出栈的元素变成连接成字符串（行{5}）。

用刚才写的算法做一些测试，使用以下代码把结果输出到控制台里：

```
console.log(divideBy2(233));  
console.log(divideBy2(10));  
console.log(divideBy2(1000));
```

进制转换算法

我们很容易修改之前的算法，使之能把十进制转换成任何进制。除了让十进制数字和2整除

转成二进制数，还可以传入其他任意进制的基数为参数，就像下面算法这样：

```
function baseConverter(decNumber, base){

    var remStack = new Stack(),
        rem,
        baseString = '',
        digits = '0123456789ABCDEF'; //{6}

    while (decNumber > 0){
        rem = Math.floor(decNumber % base);
        remStack.push(rem);
        decNumber = Math.floor(decNumber / base);
    }

    while (!remStack.isEmpty()){
        baseString += digits[remStack.pop()]; //{7}
    }

    return baseString;
}
```

我们只需要改变一个地方。在将十进制转成二进制时，余数是0或1；在将十进制转成八进制时，余数是0到7之间的数；但是将十进制转成16进制时，余数是0到9之间的数字加上A、B、C、D、E和F（对应10、11、12、13、14和15）。因此，我们需要对栈中的数字做个转化才可以（行{6}和行{7}）。

可以使用之前的算法，输出结果如下：

```
console.log(baseConverter(100345, 2));
console.log(baseConverter(100345, 8));
console.log(baseConverter(100345, 16));
```



请在网上下载本书的代码，里面还有一些栈的应用实例，如平衡圆括号和汉诺塔。

3.4 小结

通过本章，我们学习了栈这一数据结构的相关知识。我们用代码自己实现了栈，还讲解了如何用push和pop往栈里添加和移除元素。

我们比较了创建Stack类的不同方法，并分别列举了优点和缺点。我们还学习了用栈来解决计算机科学中最著名的问题之一。

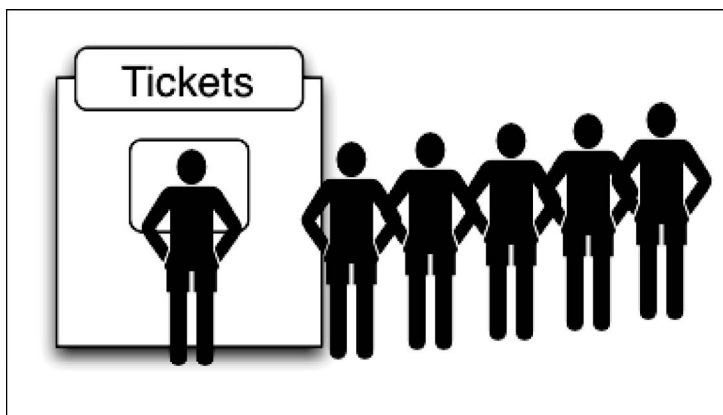
下一章将要学习队列。它和栈有很多相似之处，但有个重要区别，队列里的元素不遵循后进先出原则。

我们已经学习了栈。队列和栈非常类似，但是使用了不同的原则，而非后进先出。你将在这一章学习这些内容。

4.1 队列数据结构

队列是遵循FIFO（First In First Out，先进先出，也称为先来先服务）原则的一组有序的项。队列在尾部添加新元素，并从顶部移除元素。最新添加的元素必须排在队列的末尾。

在现实中，最常见的队列的例子就是排队：



还有，在电影院、自助餐厅、杂货店收银台，我们也都会排队。排在第一位的人会先接受服务。

在计算机科学中，一个常见的例子就是打印队列。比如说我们需要打印五份文档。我们会打开每个文档，然后点击打印按钮。每个文档都会被发送至打印队列。第一个发送到打印队列的文档会首先被打印，以此类推，直到打印完所有文档。

4.2 创建队列

我们需要创建自己的类来表示一个队列。先从最基本的声明类开始：

```
function Queue() {
  //这里是属性和方法
}
```

首先需要一个用于存储队列中元素的数据结构。我们可以使用数组，就像在上一章Stack类中那样使用（你会发现Queue类和Stack类非常类似，只是添加和移除元素的原则不同）：

```
let items = [];
```

接下来需要声明一些队列可用的方法。

- ❑ `enqueue(element(s))`：向队列尾部添加一个（或多个）新的项。
- ❑ `dequeue()`：移除队列的第一（即排在队列最前面的）项，并返回被移除的元素。
- ❑ `front()`：返回队列中第一个元素——最先得添加，也将是最先得移除的元素。队列不做任何变动（不移除元素，只返回元素信息——与Stack类的`peek`方法非常类似）。
- ❑ `isEmpty()`：如果队列中不包含任何元素，返回`true`，否则返回`false`。
- ❑ `size()`：返回队列包含的元素个数，与数组的`length`属性类似。

4.2.1 向队列添加元素

首先要实现的是`enqueue`方法。这个方法负责向队列添加新元素。这里有一个非常重要的细节，新的项只能添加到队列末尾：

```
this.enqueue = function(element){
  items.push(element);
};
```

既然我们使用数组来存储队列的元素，就可以用第2章和第3章中介绍过的JavaScript的array类的`push`方法。

4.2.2 从队列移除元素

接下来要实现`dequeue`方法。这个方法负责从队列移除项。由于队列遵循先进先出原则，最先得添加的项也是先得被移除的。可以用第2章中介绍过的JavaScript的array类的`shift`方法。如果你不记得了，这里帮你回忆一下，`shift`方法会从数组中移除存储在索引0（第一个位置）的元素：

```
this.dequeue = function(){
  return items.shift();
};
```

只有enqueue方法和dequeue方法可以添加和移除元素，这样就确保了Queue类遵循先进先出原则。

4.2.3 查看队列头元素

现在来为我们的类实现一些额外的辅助方法。如果想知道队列最前面的项是什么，可以用front方法。这个方法会返回队列最前面的项（数组的索引为0）：

```
this.front = function(){  
    return items[0];  
};
```

4.2.4 检查队列是否为空

下一个是isEmpty方法。如果队列为空，它会返回true，否则返回false（注意这个方法和Stack类里的一样）：

```
this.isEmpty = function(){  
    return items.length == 0;  
};
```

对于isEmpty方法，可以简单地验证内部数组的length是否为0。

我们也可以为Queue类实现类似于array类的length属性的方法。size方法也跟Stack类里的一样：

```
this.size = function(){  
    return items.length;  
};
```

4.2.5 打印队列元素

完成！我们的Queue类就实现好了。也可以像Stack类一样增加一个print方法：

```
this.print = function(){  
    console.log(items.toString());  
};
```

现在我们真的完成了！



Queue类和Stack类非常类似。唯一的区别是dequeue方法和front方法，这是由于先进先出和后进先出原则的不同所造成的。

使用Queue类

首先要做的是实例化我们刚刚创建的Queue类，然后就可以验证它为空（输出为true，因为我们还没有向队列添加任何元素）：

```
let queue = new Queue();
console.log(queue.isEmpty()); //输出true
```

接下来，添加一些元素（添加"John"和"Jack"两个元素——你可以向队列添加任何类型的元素）：

```
queue.enqueue("John");
queue.enqueue("Jack");
```

添加另一个元素：

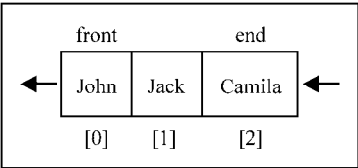
```
queue.enqueue("Camila");
```

再执行一些其他的命令：

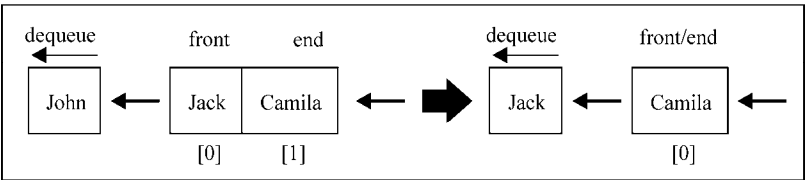
```
queue.print();
console.log(queue.size()); //输出3
console.log(queue.isEmpty()); //输出false
queue.dequeue();
queue.dequeue();
queue.print();
```

如果打印队列的内容，就会得到John、Jack和Camila这三个元素。因为我们向队列添加了三个元素，所以队列的大小为3（当然也就不为空了）。

下图展示了目前为止执行的所有入列操作，以及队列当前的状态：



然后，出列两个元素（执行两次dequeue方法）。下图展示了dequeue方法的执行过程：



最后，再次打印队列内容时，就只剩Camila一个元素了。前两个入列的元素出列了，最后入列的元素也将是最后出列的。也就是说，我们遵循了先进先出原则。

4.3 用 ECMAScript 6 语法实现的 Queue 类

和第3章一样，我们也可以用ECMAScript 6语法编写Queue类。在这种方法中，我们要用一个WeakMap来保存私有属性items，并用外层函数（闭包）来封装Queue类。

代码如下：

```
let Queue2 = (function () {  
  
  const items = new WeakMap();  
  
  class Queue2 {  
    constructor () {  
      items.set(this, []);  
    }  
    enqueue(element) {  
      let q = items.get(this);  
      q.push(element);  
    }  
    dequeue() {  
      let q = items.get(this);  
      let r = q.shift();  
      return r;  
    }  
    //其他方法  
  }  
  return Queue2;  
})();
```

我们创建的两个Queue类你都可以用，测试的输出是一样的。

4.4 优先队列

队列大量应用在计算机科学以及我们的生活中，我们在之前话题中实现的默认队列也有一些修改版本。

其中一个修改版就是**优先队列**。元素的添加和移除是基于优先级的。一个现实的例子就是机场登机的顺序。头等舱和商务舱乘客的优先级要高于经济舱乘客。在有些国家，老年人和孕妇（或带小孩的妇女）登机时也享有高于其他乘客的优先级。

另一个现实中的例子是医院的（急诊科）候诊室。医生会优先处理病情比较严重的患者。通常，护士会鉴别分类，根据患者病情的严重程度放号。

实现一个优先队列，有两种选项：设置优先级，然后在正确的位置添加元素；或者用入列操作添加元素，然后按照优先级移除它们。在这个示例中，我们将会正确的位置添加元素，因此可以对它们使用默认的出列操作：

```

function PriorityQueue() {
  let items = [];
  function QueueElement (element, priority){ // {1}
    this.element = element;
    this.priority = priority;
  }

  this.enqueue = function(element, priority){
    let queueElement = new QueueElement(element, priority);

    let added = false;
    for (let i=0; i<items.length; i++){
      if (queueElement.priority < items[i].priority){ // {2}
        items.splice(i,0,queueElement); // {3}
        added = true;
        break; // {4}
      }
    }
    if (!added){
      items.push(queueElement); // {5}
    }
  };

  this.print = function(){
    for (let i=0; i<items.length; i++){
      console.log(`${items[i].element} -
        ${items[i].priority}`);
    }
  };
  //其他方法和默认的Queue实现相同
}

```

默认的Queue类和PriorityQueue类实现上的区别是，要向PriorityQueue添加元素，需要创建一个特殊的元素（行{1}）。这个元素包含了要添加到队列的元素（它可以是任意类型）及其在队列中的优先级。

如果队列为空，可以直接将元素入列（行{2}）。否则，就需要比较该元素与其他元素的优先级。当找到一个比要添加的元素的priority值更大（优先级更低）的项时，就把新元素插入到它之前（根据这个逻辑，对于其他优先级相同，但是先添加到队列的元素，我们同样遵循先进先出的原则）。要做到这一点，我们可以用第2章学习过的JavaScript的array类的splice方法。一旦找到priority值更大的元素，就插入新元素（行{3}）并终止队列循环（行{4}）。这样，队列也就根据优先级排序了。

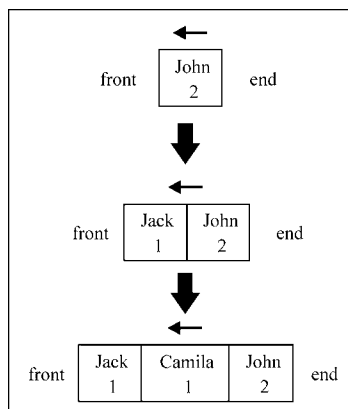
如果要添加元素的priority值大于任何已有的元素，把它添加到队列的末尾就行了（行{5}）：

```

let priorityQueue = new PriorityQueue();
priorityQueue.enqueue("John", 2);
priorityQueue.enqueue("Jack", 1);
priorityQueue.enqueue("Camila", 1);
priorityQueue.print();

```


以上代码是一个使用PriorityQueue类的示例。在下图中可以看到每条命令的结果（以上代码的结果）：



第一个被添加的元素是优先级为2的John。因为此前队列为空，所以它是队列中唯一的元素。接下来，添加了优先级为1的Jack。由于Jack的优先级高于John，它就成了队列中的第一个元素。然后，添加了优先级也为1的Camila。Camila的优先级和Jack相同，所以它会被插入到Jack之后（因为Jack先被插入队列）；Camila的优先级高于John，所以它会被插入到John之前。

我们在这里实现的优先队列称为最小优先队列，因为优先级的值较小的元素被放置在队列最前面（1代表更高的优先级）。最大优先队列则与之相反，把优先级的值较大的元素放置在队列最前面。

4.5 循环队列——击鼓传花

还有另一个修改版的队列实现，就是循环队列。循环队列的一个例子就是击鼓传花游戏（Hot Potato）。在这个游戏中，孩子们围成一个圆圈，把花尽快地传递给旁边的人。某一时刻传花停止，这个时候花在谁手里，谁就退出圆圈结束游戏。重复这个过程，直到只剩一个孩子（胜者）。

在下面这个示例中，我们要实现一个模拟的击鼓传花游戏：

```
function hotPotato (nameList, num){

    let queue = new Queue(); // {1}

    for (let i=0; i<nameList.length; i++){
        queue.enqueue(nameList[i]); // {2}
    }

    let eliminated = '';
    while (queue.size() > 1){
```

```
for (let i=0; i<num; i++){
    queue.enqueue(queue.dequeue()); // {3}
}
eliminated = queue.dequeue();// {4}
console.log(eliminated + '在击鼓传花游戏中被淘汰。');
}

return queue.dequeue();// {5}
}

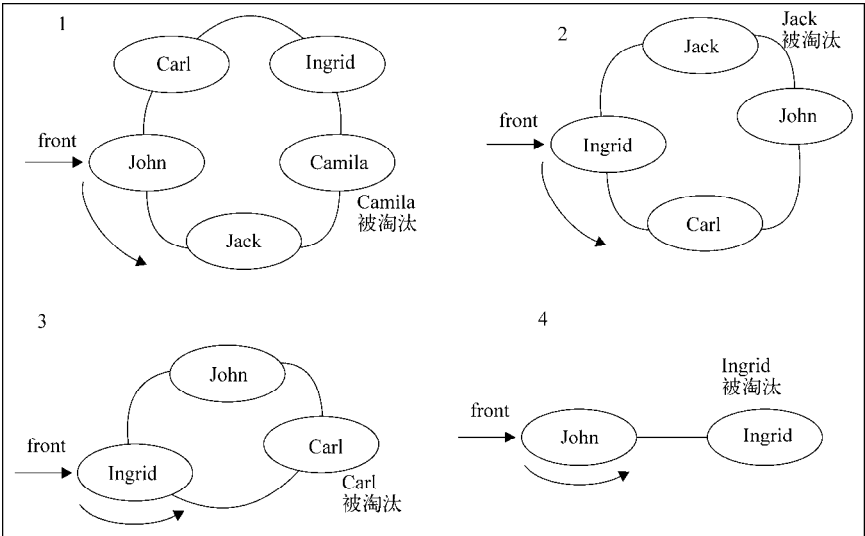
let names = ['John','Jack','Camila','Ingrid','Carl'];let winner =
hotPotato(names, 7);console.log('The winner is: ' + winner);
```

实现一个模拟的击鼓传花游戏，要用到这一章开头实现的Queue类（行{1}）。我们会得到一份名单，把里面的名字全都加入队列（行{2}）。给定一个数字，然后迭代队列。从队列开头移除一项，再将其添加到队列末尾（行{3}），模拟击鼓传花（如果你把花传给了旁边的人，你被淘汰的威胁立刻就解除了）。一旦传递次数达到给定的数字，拿着花的那个人就被淘汰了（从队列中移除——行{4}）。最后只剩一个人的时候，这个人就是胜者（行{5}）。

以上算法的输出如下：

Camila在击鼓传花游戏中被淘汰。
Jack在击鼓传花游戏中被淘汰。
Carl在击鼓传花游戏中被淘汰。
Ingrid在击鼓传花游戏中被淘汰。
胜利者： John

下图模拟了这个输出过程：



你可以改变传入hotPotato函数的数字，模拟不同的场景。

4.6 JavaScript 任务队列

既然我们在书中使用的是JavaScript，何不探索一下这门语言本身？

当我们在浏览器中打开新标签时，就会创建一个任务队列。这是因为每个标签都是单线程处理所有的任务，它被称为事件循环。浏览器要负责多个任务，如渲染HTML，执行JavaScript代码，处理用户交互（用户输入、鼠标点击等），执行和处理异步请求。如果想更多地了解事件循环，可以访问<https://goo.gl/ayF840>。

像JavaScript这样流行而强大的语言，内部控制所使用的也是如此基础的数据结构，真令人高兴。

4.7 小结

这一章我们学习了队列这种数据结构。我们实现了自己的队列算法，学习了如何通过enqueue方法和dequeue方法添加和移除元素。我们还学习了两种非常著名的特殊队列的实现：优先队列和循环队列（使用击鼓传花游戏的实现）。

在下一章中，我们将学习链表，一种比数组更复杂的数据结构。

我们在第2章中学习了数组这种数据结构。数组（或者也可以称为列表）是一种非常简单的存储数据序列的数据结构。在这一章中，你会学习如何实现和使用链表这种动态的数据结构，这意味着我们可以从中任意添加或移除项，它会按需进行扩容。

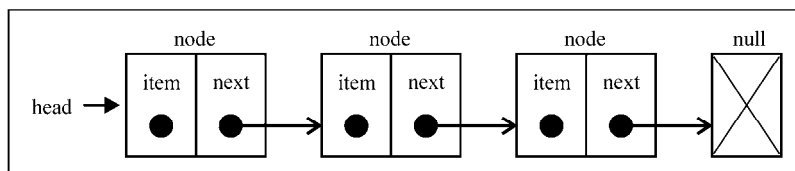
本章内容包括：

- ❑ 链表数据结构
- ❑ 向链表添加元素
- ❑ 从链表移除元素
- ❑ 使用`LinkedList`类
- ❑ 双向链表
- ❑ 循环链表

5.1 链表数据结构

要存储多个元素，数组（或列表）可能是最常用的数据结构。正如本书之前提到过的，每种语言都实现了数组。这种数据结构非常方便，提供了一个便利的`[]`语法来访问它的元素。然而，这种数据结构有一个缺点：（在大多数语言中）数组的大小是固定的，从数组的起点或中间插入或移除项的成本很高，因为需要移动元素（尽管我们已经学过的JavaScript的`array`类方法可以帮我们做这些事，但背后的情况同样是这样）。

链表存储有序的元素集合，但不同于数组，链表中的元素在内存中并不是连续放置的。每个元素由一个存储元素本身的节点和一个指向下一个元素的引用（也称指针或链接）组成。下图展示了一个链表的结构：

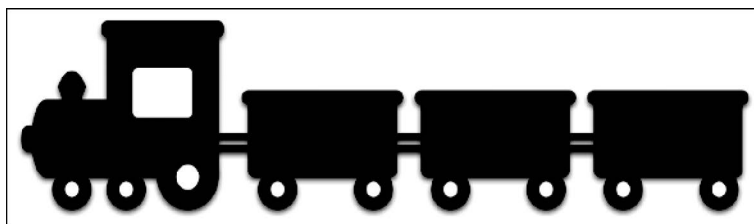


相对于传统的数组，链表的一个好处在于，添加或移除元素的时候不需要移动其他元素。然而，链表需要使用指针，因此实现链表时需要额外注意。数组的另一个细节是可以直接访问任何位置的任何元素，而要想访问链表中间的一个元素，需要从起点（表头）开始迭代列表直到找到所需的元素。

现实中也有一些链表的例子。第一个例子就是康加舞队。每个人是一个元素，手就是链向下一个人的指针。可以向队列中增加人——只需要找到想加入的点，断开连接，插入一个人，再重新连接起来。

另一个例子是寻宝游戏。你有一条线索，这条线索是指向寻找下一条线索的地点的指针。你顺着这条链接去下一个地点，得到另一条指向再下一处的线索。得到列表中间的线索的唯一办法，就是从起点（第一条线索）顺着列表寻找。

还有一个可能是用来说明链表的最流行的例子，那就是火车。一列火车是由一系列车厢（也称车皮）组成的。每节车厢或车皮都相互连接。你很容易分离一节车皮，改变它的位置，添加或移除它。下图演示了一列火车。每节车皮都是列表的元素，车皮间的连接就是指针：



在这一章，我们会介绍链表和双向链表。但还是先从最简单的数据结构开始吧。

5.2 创建链表

理解了链表是什么之后，现在就要开始实现我们的数据结构了。以下是我们的LinkedList类的骨架：

```
function LinkedList() {  
  
  let Node = function(element){ // {1}  
    this.element = element;  
    this.next = null;  
  };  
  
  let length = 0; // {2}  
  let head = null; // {3}  
  
  this.append = function(element){};  
  this.insert = function(position, element){};  
}
```

```

    this.removeAt = function(position){};
    this.remove = function(element){};
    this.indexOf = function(element){};
    this.isEmpty = function() {};
    this.size = function() {};
    this.getHead = function(){};
    this.toString = function(){};
    this.print = function(){};
}

```

LinkedList数据结构还需要一个Node辅助类（行{1}）。Node类表示要加入列表的项。它包含一个element属性，即要添加到列表的值，以及一个next属性，即指向列表中下一个节点项的指针。

LinkedList类也有存储列表项的数量的length属性（内部/私有变量）（行{2}）。

另一个重要的点是，我们还需要存储第一个节点的引用。为此，可以把这个引用存储在一个称为head的变量中（行{3}）。

然后就是LinkedList类的方法。在实现这些方法之前，先来看看它们的职责。

- ❑ append(element)：向列表尾部添加一个新的项。
- ❑ insert(position, element)：向列表的特定位置插入一个新的项。
- ❑ remove(element)：从列表中移除一项。
- ❑ indexOf(element)：返回元素在列表中的索引。如果列表中没有该元素则返回-1。
- ❑ removeAt(position)：从列表的特定位置移除一项。
- ❑ isEmpty()：如果链表中不包含任何元素，返回true，如果链表长度大于0则返回false。
- ❑ size()：返回链表包含的元素个数。与数组的length属性类似。
- ❑ toString()：由于列表项使用了Node类，就需要重写继承自JavaScript对象默认的toString方法，让其只输出元素的值。

5.2.1 向链表尾部追加元素

向LinkedList对象尾部添加一个元素时，可能有两种场景：列表为空，添加的是第一个元素，或者列表不为空，向其追加元素。

下面是我们实现的append方法：

```

this.append = function(element){

    let node = new Node(element), //{1}
        current; //{2}

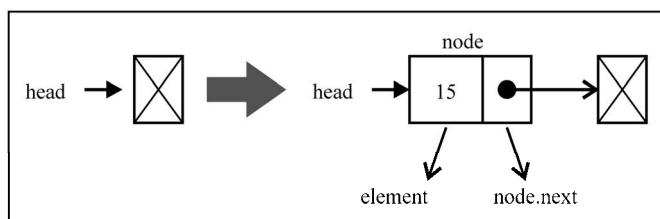
    if (head === null){ //列表中第一个节点 //{3}
        head = node;
    }
}

```

```
} else {  
    current = head; //{4}  
  
    //循环列表，直到找到最后一项  
    while(current.next){  
        current = current.next;  
    }  
  
    //找到最后一项，将其next赋为node，建立链接  
    current.next = node; //{5}  
}  
  
length++; //更新列表的长度 //{6}  
};
```

首先需要做的是把element作为值传入，创建Node项（行{1}）。

先来实现第一个场景：向为空的列表添加一个元素。当我们创建一个LinkedList对象时，head会指向null：



如果head元素为null（列表为空——行{3}），就意味着在向列表添加第一个元素。因此要做的就是让head元素指向node元素。下一个node元素将会自动成为null（请看5.1节的源代码）。

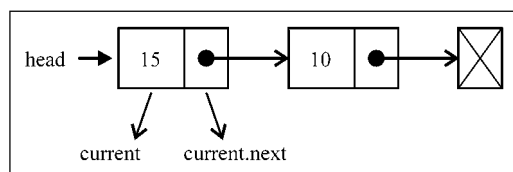


列表最后一个节点的下一个元素始终是null。

好了，我们已经说完了第一种场景。再来看看第二个，也就是向一个不为空的列表尾部添加元素。

要向列表的尾部添加一个元素，首先需要找到最后一个元素。记住，我们只有第一个元素的引用（行{4}），因此需要循环访问列表，直到找到最后一项。为此，我们需要一个指向列表中current项的变量（行{2}）。

循环访问列表时，当current.next元素为null时，我们就知道已经到达列表尾部了。然后要做的就是让当前（也就是最后一个）元素的next指针指向想要添加到列表的节点（行{5}）。下图展示了这个行为：



而当一个Node元素被创建时，它的next指针总是null。这没问题，因为我们知道它会是列表的最后一项。

当然，别忘了递增列表的长度，这样就能控制它，轻松地得到列表的长度（行{6}）。

我们可以通过以下代码来使用和测试目前创建的数据结构：

```
let list = new LinkedList();
list.append(15);
list.append(10);
```

5.2.2 从链表中移除元素

5

现在，让我们看看如何从LinkedList对象中移除元素。移除元素也有两种场景：第一种是移除第一个元素，第二种是移除第一个以外的任一元素。我们要实现两种remove方法：第一种是从特定位置移除一个元素，第二种是根据元素的值移除元素（稍后我们会展示第二种remove方法）。

```
this.removeAt = function(position){

  //检查越界值
  if (position > -1 && position < length){ // {1}

    let current = head, // {2}
        previous, // {3}
        index = 0; // {4}

    //移除第一项
    if (position === 0){ // {5}
      head = current.next;
    } else {

      while (index++ < position){ // {6}

        previous = current; // {7}
        current = current.next; // {8}
      }

      //将previous与current的下一项链接起来：跳过current，从而移除它
      previous.next = current.next; // {9}
    }

    length--; // {10}

    return current.element;
```



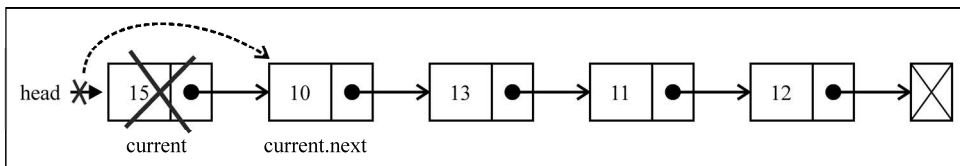
```

    } else {
        return null; // {11}
    }
};

```

一步一步来看这段代码。该方法要得到需要移除的元素的位置，就需要验证这个位置是有效的（行{1}）。从0（包括0）到列表的长度（`size - 1`，因为索引是从零开始的）都是有效的位置。如果不是有效的位置，就返回`null`（意即没有从列表中移除元素）。

一起来为第一种场景编写代码：我们要从列表中移除第一个元素（`position === 0`——行{5}）。下图展示了这个过程：



因此，如果想移除第一个元素，要做的就是让`head`指向列表的第二个元素。我们将用`current`变量创建一个对列表中第一个元素的引用（行{2}——我们还会用它来迭代列表，但稍等一下再说）。这样`current`变量就是对列表中第一个元素的引用。如果把`head`赋为`current.next`，就会移除第一个元素。

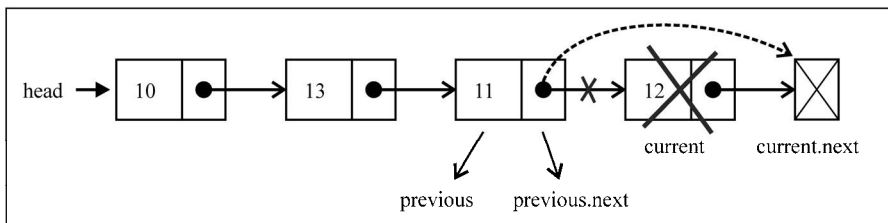
现在，假设我们要移除列表的最后一项或者中间某一项。为此，需要依靠一个细节来迭代列表，直到到达目标位置（行{6}——我们会使用一个用于内部控制和递增的`index`变量）；`current`变量总是为对所循环列表的当前元素的引用（行{8}）。我们还需要一个对当前元素的前一个元素的引用（行{7}）；它被命名为`previous`（行{3}）。

因此，要从列表中移除当前元素，要做的就是将`previous.next`和`current.next`链接起来（行{9}）。这样，当前元素就会被丢弃在计算机内存中，等着被垃圾回收器清除。



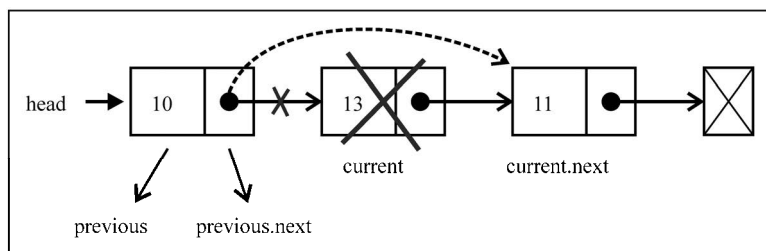
要更好地理解JavaScript垃圾回收器如何工作，请阅读https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management。

我们试着通过一些图表来更好地理解。首先考虑移除最后一个元素：



对于最后一个元素，当我们在行{6}跳出循环时，`current`变量将是对列表中最后一个元素的引用（要移除的元素）。`current.next`的值将是`null`（因为它是最后一个元素）。由于还保留了对`previous`元素的引用（当前元素的前一个元素），`previous.next`就指向了`current`。那么要移除`current`，要做的就是将`previous.next`的值改变为`current.next`。

现在来看看，对于列表中间的元素是否可以应用相同的逻辑：



`current`变量是对要移除元素的引用。`previous`变量是对要移除元素的前一个元素的引用。那么要移除`current`元素，需要做的就是将`previous.next`与`current.next`链接起来。因此，我们的逻辑对这两种情况都管用。

5

5.2.3 在任意位置插入元素

接下来，我们要实现`insert`方法。使用这个方法可以在任意位置插入一个元素。我们来看一看它的实现：

```
this.insert = function(position, element){

  //检查越界值
  if (position >= 0 && position <= length){ //{1}

    let node = new Node(element),
        current = head,
        previous,
        index = 0;

    if (position === 0){ //在第一个位置添加

      node.next = current; //{2}
      head = node;

    } else {
      while (index++ < position){ //{3}
        previous = current;
        current = current.next;
      }
      node.next = current; //{4}
      previous.next = node; //{5}
    }
  }
}
```

```

    }

    length++; //更新列表的长度

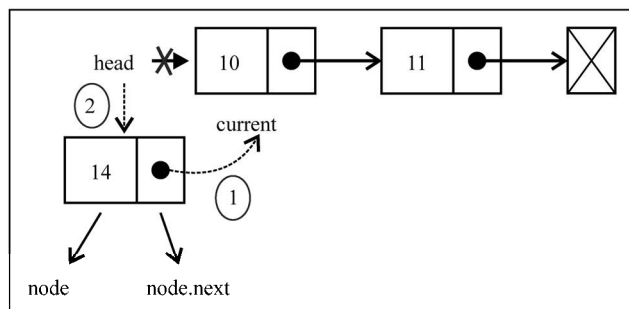
    return true;

} else {
    return false; //{6}
}
};

```

由于我们处理的是位置，就需要检查越界值（行{1}，跟remove方法类似）。如果越界了，就返回false值，表示没有添加项到列表中（行{6}）。

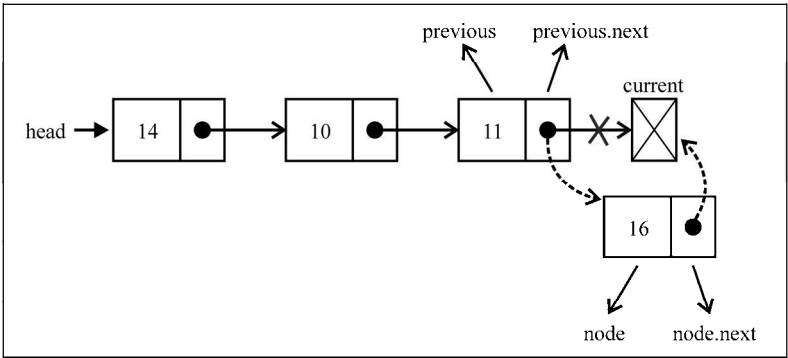
现在我们要处理不同的场景。第一种场景，需要在列表的起点添加一个元素，也就是**第一个位置**。下图展示了这种场景：



current变量是对列表中第一个元素的引用。我们需要做的是把node.next的值设为current（列表中第一个元素）。现在head和node.next都指向了current。接下来要做的就是将head的引用改为node（行{2}），这样列表中就有了一个新元素。

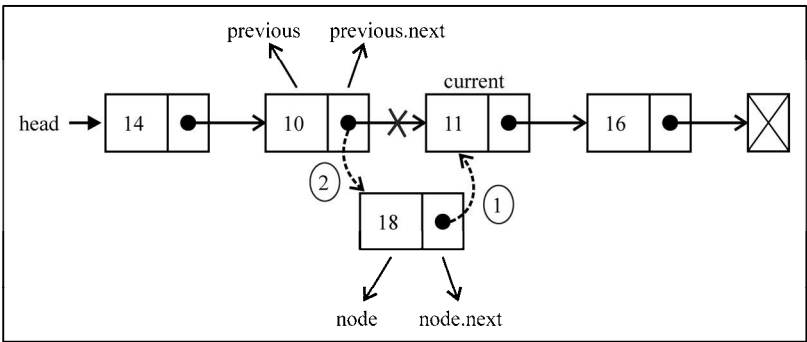
现在来处理第二种场景：在列表中间或尾部添加一个元素。首先，我们需要循环访问列表，找到目标位置（行{3}）。当跳出循环时，current变量将是对想要插入新元素的位置之后一个元素的引用，而previous将是对想要插入新元素的位置之前一个元素的引用。在这种情况下，我们要在previous和current之间添加新项。因此，首先需要把新项（node）和当前项链接起来（行{4}），然后需要改变previous和current之间的链接。我们还需要让previous.next指向node（行{5}）。

我们通过一张图表来看看代码所做的事：



如果我们试图向最后一个位置添加一个新元素，`previous`将是对列表最后一项的引用，而`current`将是`null`。在这种情况下，`node.next`将指向`current`，而`previous.next`将指向`node`，这样列表中就有了一个新的项。

现在来看看如何向列表中间添加一个新元素：



在这种情况下，我们试图将新的项（`node`）插入到`previous`和`current`元素之间。首先，我们需要把`node.next`的值指向`current`。然后把`previous.next`的值设为`node`。这样列表中就有了一个新的项。



使用变量引用我们需要控制的节点非常重要，这样就不会丢失节点之间的链接。我们可以只使用一个变量（`previous`），但那样会很难控制节点之间的链接。由于这个原因，最好是声明一个额外的变量来帮助我们处理这些引用。

5.2.4 实现其他方法

在这一节中，我们将会学习如何实现`toString`、`indexOf`、`isEmpty`和`size`等其他`LinkedList`类的方法。

1. toString方法

toString方法会把LinkedList对象转换成一个字符串。下面是toString方法的实现：

```
this.toString = function(){  
  
    let current = head, //{1}  
    string = '';      //{2}  
  
    while (current) {    //{3}  
        string +=current.element +(current.next ? 'n' : ''); //{4}  
        current = current.next;      //{5}  
    }  
    return string;        //{6}  
};
```

首先，要循环访问列表中的所有元素，就需要有一个起点，也就是head。我们会把current变量当作索引（行{1}），控制循环访问列表。我们还需要初始化用于拼接元素值的变量（行{2}）。

接下来就是循环访问列表中的每个元素（行{3}）。我们要用current来检查元素是否存在（如果列表为空，或是到达列表中最后一个元素的下一位（null），while循环中的代码就不会执行）。然后我们就得到了元素的内容，将其拼接到字符串中（行{4}）。最后，继续迭代下一个元素（行{5}）。最后，返回列表内容的字符串（行{6}）。

2. indexOf方法

indexOf是我们下一个要实现的方法。indexOf方法接收一个元素的值，如果在列表中找到它，就返回元素的位置，否则返回-1。

来看看它的实现：

```
this.indexOf = function(element){  
  
    let current = head, //{1}  
    index = -1;  
  
    while (current) { //{2}  
        if (element === current.element) {  
            return index;      //{3}  
        }  
        index++;                //{4}  
        current = current.next; //{5}  
    }  
  
    return -1;  
};
```

一如既往，我们需要一个变量来帮助我们循环访问列表，这个变量是current，它的初始值是head（列表的第一个元素——我们还需要一个index变量来计算位置数（行{1}））。然后循环

访问元素（行{2}），检查当前元素是否是我们要找的。如果是，就返回它的位置（行{3}）；如果不是，就继续计数（行{4}），检查列表中下一个节点（行{5}）。

如果列表为空，或是到达列表的尾部（`current = current.next`将是`null`），循环就不会执行。如果没有找到值，就返回-1。

实现了这个方法，我们就可以实现`remove`等其他的方法：

```
this.remove = function(element){
  let index = this.indexOf(element);
  return this.removeAt(index);
};
```

我们已经有一个移除给定位置的一个元素的`removeAt`方法了。现在有了`indexOf`方法，如果传入元素的值，就能找到它的位置，然后调用`removeAt`方法并传入找到的位置。这样非常简单，如果需要更改`removeAt`方法的代码，这样也更容易——两个方法都会被更改（这就是重用代码的妙处）。这样，我们就不需要维护两个从列表中移除一项的方法，只需要一个！同时，`removeAt`方法将会检查边界约束。

5

3. `isEmpty`、`size`和`getHead`方法

`isEmpty`和`size`方法跟我们在上一章实现的一模一样。但我们还是来看一下：

```
this.isEmpty = function() {
  return length === 0;
};
```

如果列表中没有元素，`isEmpty`方法就返回`true`，否则返回`false`。

```
this.size = function() {
  return length;
};
```

`size`方法返回列表的`length`。和我们之前几章实现的类有所不同，列表的`length`是内部控制的，因为`LinkedList`是从头构建的。

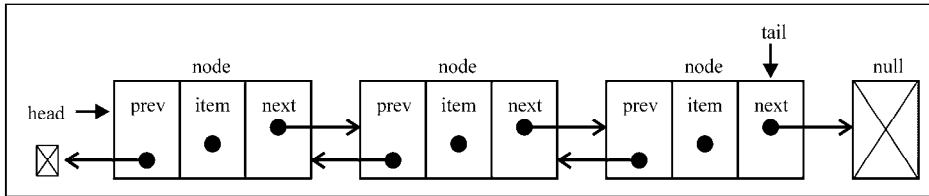
最后还有`getHead`方法：

```
this.getHead = function(){
  return head;
};
```

`head`变量是`LinkedList`类的私有变量（这意味着它不能在`LinkedList`实例外部被访问和更改，只有通过`LinkedList`实例才可以）。但是，如果我们需要在类的实现外部循环访问列表，就需要提供一种获取类的第一个元素的方法。

5.3 双向链表

链表有多种不同的类型，这一节介绍**双向链表**。双向链表和普通链表的区别在于，在链表中，一个节点只有链向下一个节点的链接，而在双向链表中，链接是双向的：一个链向下一个元素，另一个链向前一个元素，如下图所示：



先从实现DoublyLinkedList类所需的变动开始：

```
function DoublyLinkedList() {

  let Node = function(element){

    this.element = element;
    this.next = null;
    this.prev = null; //新增的
  };

  let length = 0;
  let head = null;
  let tail = null; //新增的

  //这里是方法
}
```

在代码中可以看到，LinkedList类和DoublyLinkedList类之间的区别标为**新增的**。在Node类里有prev属性（一个新指针），在DoublyLinkedList类里也有用来保存对列表最后一项的引用的tail属性。

双向链表提供了两种迭代列表的方法：从头到尾，或者反过来。我们也可以访问一个特定节点的下一个或前一个元素。在单向链表中，如果迭代列表时错过了要找的元素，就需要回到列表起点，重新开始迭代。这是双向链表的一个优点。

5.3.1 在任意位置插入新元素

向双向链表中插入一个新项跟（单向）链表非常类似。区别在于，链表只要控制一个next指针，而双向链表则要同时控制next和prev（previous，前一个）这两个指针。

这是向任意位置插入一个新元素的算法：

```

this.insert = function(position, element){

    //检查越界值
    if (position >= 0 && position <= length){

        let node = new Node(element),
            current = head,
            previous,
            index = 0;

        if (position === 0){ //在第一个位置添加

            if (!head){ //新增的 {1}
                head = node;
                tail = node;
            } else {
                node.next = current;
                current.prev = node; //新增的 {2}
                head = node;
            }
        } else if (position === length) { //最后一项 //新增的

            current = tail; // {3}
            current.next = node;
            node.prev = current;
            tail = node;

        } else {
            while (index++ < position){ //{4}
                previous = current;
                current = current.next;
            }
            node.next = current; //{5}
            previous.next = node;

            current.prev = node; //新增的
            node.prev = previous; //新增的
        }

        length++; //更新列表的长度

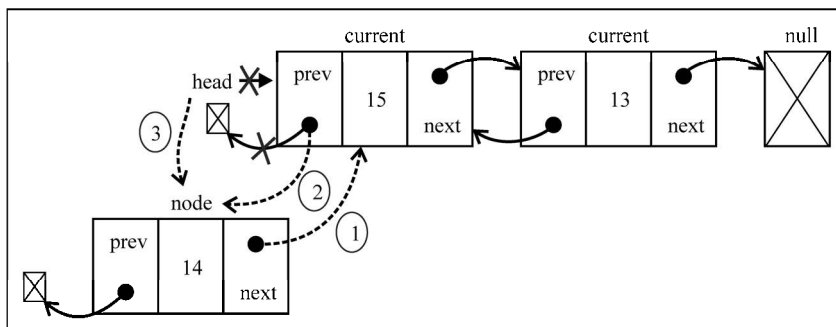
        return true;

    } else {
        return false;
    }
};

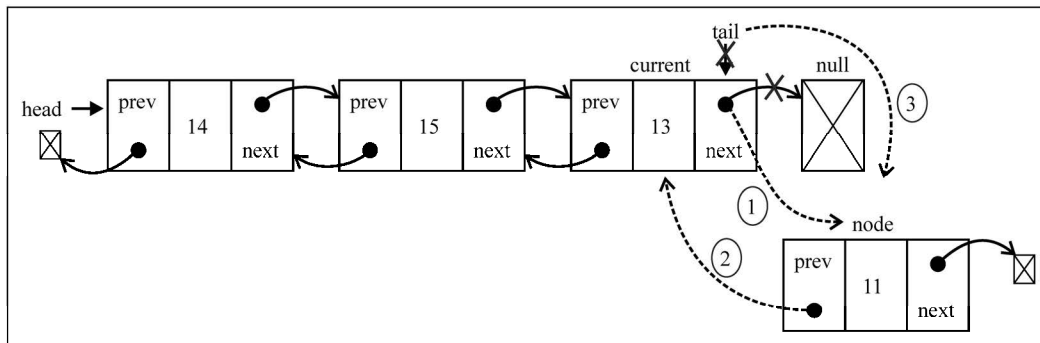
```

我们来分析第一种场景：在列表的第一个位置（列表的起点）插入一个新元素。如果列表为空（行{1}），只需要把head和tail都指向这个新节点。如果不为空，current变量将是对列表中第一个元素的引用。就像我们在链表中所做的，把node.next设为current，而head将指向node（它将成为列表中的第一个元素）。不同之处在于，我们还需要为指向上一个元素的指针设

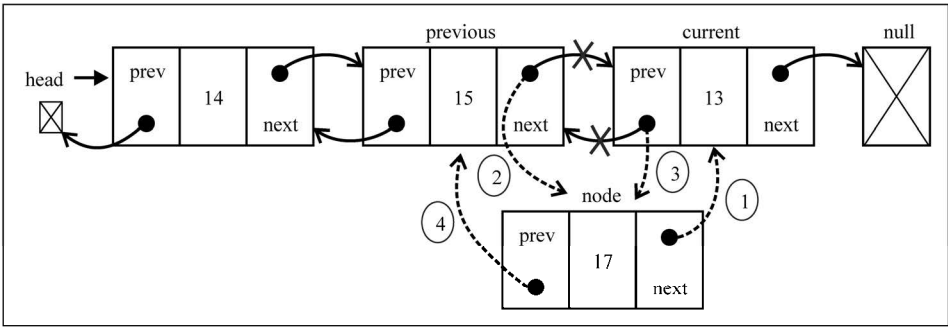
一个值。`current.prev`指针将由指向`null`变为指向新元素（`node`——行{2}）。`node.prev`指针已经是`null`，因此不需要再更新任何东西。下图演示了这个过程：



现在来分析一下，假如我们要在列表最后添加一个新元素。这是一个特殊情况，因为我们还控制着指向最后一个元素的指针（`tail`）。`current`变量将引用最后一个元素（行{3}）。然后开始建立第一个链接：`node.prev`将引用`current`。`current.next`指针（指向`null`）将指向`node`（由于构造函数，`node.next`已经指向了`null`）。然后只剩一件事了，就是更新`tail`，它将由指向`current`变为指向`node`。下图展示了这些行为：



然后还有第三种场景：在列表中间插入一个新元素。就像我们在之前的方法中所做的，迭代列表，直到到达要找的位置（行{4}）。我们将会在`current`和`previous`元素之间插入新元素。首先，`node.next`将指向`current`（行{5}），而`previous.next`将指向`node`，这样就不会丢失节点之间的链接。然后需要处理所有的链接：`current.prev`将指向`node`，而`node.prev`将指向`previous`。下图展示了这一过程：



我们可以对insert和remove这两个方法的实现做一些改进。在结果为否的情况下，我们可以把元素插入到列表的尾部。性能也可以有所改进，比如，如果position大于length/2，就最好从尾部开始迭代，而不是从头开始（这样就能迭代更少列表中的元素）。

5

5.3.2 从任意位置移除元素

从双向链表中移除元素跟链表非常类似。唯一的区别就是还需要设置前一个位置的指针。我们来看一下它的实现：

```
this.removeAt = function(position){  
  
  //检查越界值  
  if (position > -1 && position < length){  
  
    let current = head,  
        previous,  
        index = 0;  
  
    //移除第一项  
    if (position === 0){  
  
      head = current.next; // {1}  
  
      //如果只有一项，更新tail //新增的  
      if (length === 1){ // {2}  
        tail = null;  
      } else {  
        head.prev = null; // {3}  
      }  
    }  
  
    } else if (position === length-1){ //最后一项 //新增的  
  
      current = tail; // {4}  
      tail = current.prev;  
      tail.next = null;  
    }  
  }  
}
```

```

    } else {

        while (index++ < position){ // {5}

            previous = current;
            current = current.next;
        }

        //将previous与current的下一项链接起来——跳过current
        previous.next = current.next; // {6}
        current.next.prev = previous; //新增的
    }

    length--;

    return current.element;

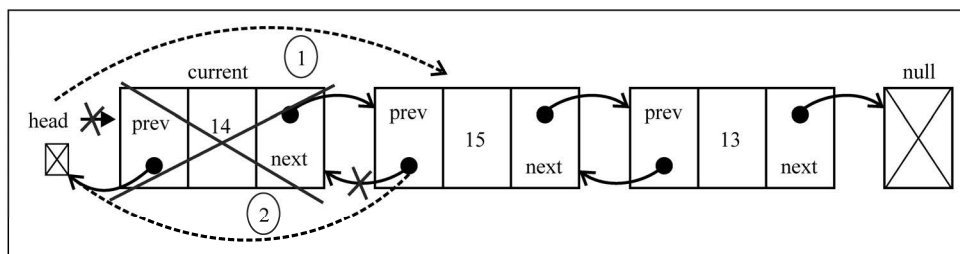
} else {
    return null;
}
};

```

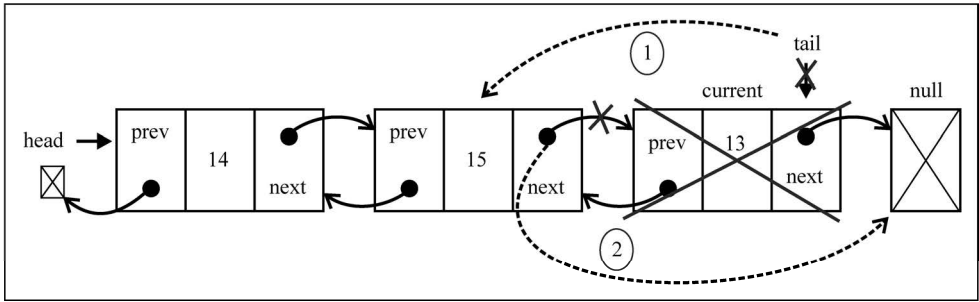
我们需要处理三种场景：从头部、从中间和从尾部移除一个元素。

我们来看看如何移除第一个元素。current变量是对列表中第一个元素的引用，也就是我们想移除的元素。需要做的就是改变head的引用，将其从current改为下一个元素（current.next——行{1}）。但我们还需要更新current.next指向上一个元素的指针（因为第一个元素的prev指针是null）。因此，把head.prev的引用改为null（行{3}——因为head也指向列表中新的第一个元素，或者也可以用current.next.prev）。由于还需要控制tail的引用，我们可以检查要移除的元素是否是第一个元素，如果是，只需要把tail也设为null（行{2}）。

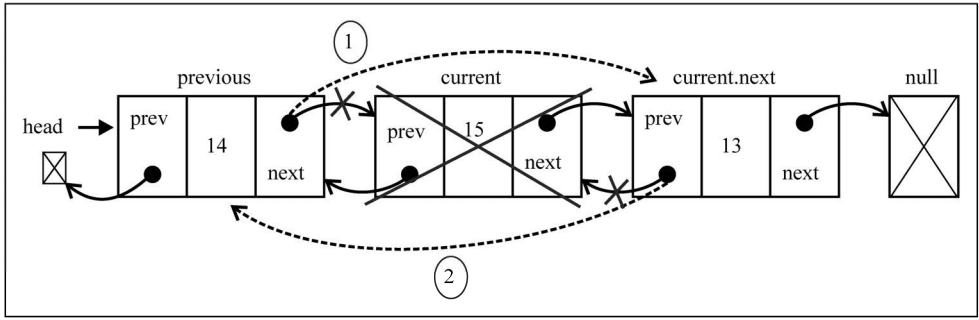
下图勾画了从双向链表移除第一个元素的过程：



下一种场景是从最后一个位置移除元素。既然已经有了对最后一个元素的引用（tail），我们就不需要为找到它而迭代列表。这样我们也就可以把tail的引用赋给current变量（行{4}）。接下来，需要把tail的引用更新为列表中倒数第二个元素（current.prev，或者tail.prev也可以）。既然tail指向了倒数第二个元素，我们就只需要把next指针更新为null（tail.next = null）。下图演示了这一行为：



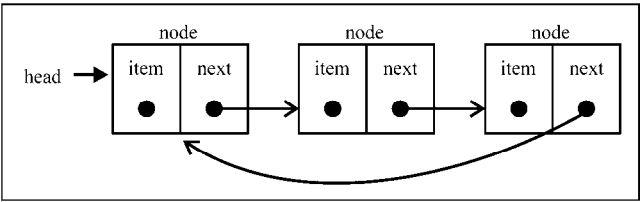
第三种也是最后一种场景：从列表中间移除一个元素。首先需要迭代列表，直到到达要找的位置（行{5}）。current变量所引用的就是要移除的元素。那么要移除它，我们可以通过更新previous.next和current.next.prev的引用，在列表中跳过它。因此，previous.next将指向current.next，而current.next.prev将指向previous，如下图所示：



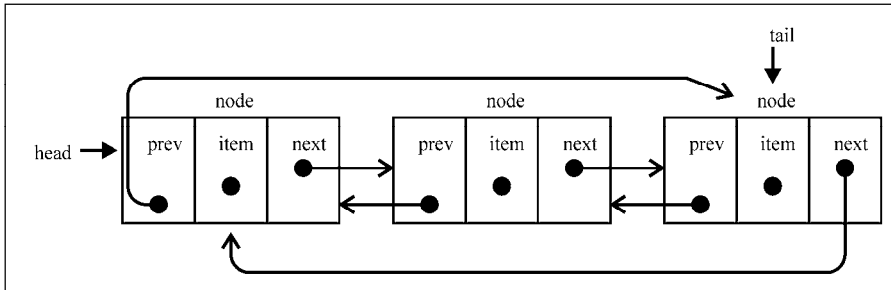
要了解双向链表的其他方法的实现，请参阅本书源代码。源代码的下载链接见本书前言。

5.4 循环链表

循环链表可以像链表一样只有单向引用，也可以像双向链表一样有双向引用。循环链表和链表之间唯一的区别在于，最后一个元素指向下一个元素的指针（tail.next）不是引用null，而是指向第一个元素（head），如下图所示。



双向循环链表有指向head元素的tail.next，和指向tail元素的head.prev。



我们并不打算在这本书中完整地介绍CircularLinkedList算法（源代码与LinkedList和DoublyLinkedList非常类似）。不过，你可以下载本书的源代码来访问这部分代码。在本书的源代码中，你还会找到所有三个链表类的ECMAScript 6版本。

5.5 小结

在这一章中，你学习了链表这种数据结构，及其变体双向链表和循环链表。你学习了如何在任意位置添加和移除元素，以及如何循环访问链表。你还学习了链表相比数组最重要的优点，那就是无需移动链表中的元素，就能轻松地添加和移除元素。因此，当你需要添加和移除很多元素时，最好的选择就是链表，而非数组。

在下一章中，你将学习集合，这是我们要在本书中介绍的最后一种顺序数据结构。

迄今为止，我们已经学习了数组（列表）、栈、队列和链表（及其变种）等顺序数据结构。在这一章中，我们要学习集合，这是一种不允许值重复的顺序数据结构。

本章，你会学到如何创建集合这种数据结构，如何添加和移除值，如何搜索值是否存在。你也会学到如何进行并集、交集、差集等数学操作，还会学到如何使用ES6（ECMAScript 6）原生的Set类。

6.1 构建数据集合

6

集合是由一组无序且唯一（即不能重复）的项组成的。这个数据结构使用了与有限集合相同的数学概念，但应用在计算机科学的数据结构中。

在深入学习集合的计算机科学实现之前，我们先看看它的数学概念。在数学中，集合是一组不同的对象（的集）。

比如说，一个由大于或等于0的整数组成的自然数集合： $N = \{0, 1, 2, 3, 4, 5, 6, \dots\}$ 。集合中的对象列表用“{}”（大括号）包围。

还有一个概念叫**空集**。空集就是不包含任何元素的集合。比如24和29之间的素数集合。由于24和29之间没有素数（除了1和自身，没有其他正因数的大于1的自然数），这个集合就是空集。空集用“{}”表示。

你也可以把集合想象成一个既没有重复元素，也没有顺序概念的数组。

在数学中，集合也有并集、交集、差集等基本操作。在这一章中我们也会介绍这些操作。

6.2 创建集合

目前的JavaScript实现是基于2011年6月发布的ECMAScript 5.1（现代浏览器均已支持），它包

括了我们在之前章节已经提到过的Array类的实现。ECMAScript 6也包括了Set类的实现，本章稍后会介绍它的用法。本章中，我们要实现的类就是以ECMAScript 6中Set类的实现为基础的。

以下是我们的Set类的骨架：

```
function Set() {  
  let items = {};  
}
```

有一个非常重要的细节，我们使用对象而不是数组来表示集合（items）。但也可以用数组实现。在这里我们用对象来实现，稍微有点儿不一样，也学习一下实现相似数据结构的新方法。同时，JavaScript的对象不允许一个键指向两个不同的属性，也保证了集合里的元素都是唯一的。

接下来，需要声明一些集合可用的方法（我们会尝试模拟与ECMAScript 6实现相同的Set类）。

- ❑ add(value)：向集合添加一个新的项。
- ❑ delete(value)：从集合移除一个值。
- ❑ has(value)：如果值在集合中，返回true，否则返回false。
- ❑ clear()：移除集合中的所有项。
- ❑ size()：返回集合所包含元素的数量。与数组的length属性类似。
- ❑ values()：返回一个包含集合中所有值的数组。

6.2.1 has(value)方法

首先要实现的是has(value)方法。这是因为它会被add、remove等其他方法调用。下面看看它的实现：

```
this.has = function(value){  
  return value in items;  
};
```

既然我们使用对象来存储集合的值，就可以用JavaScript的in操作符来验证给定的值是否是items对象的属性。

但这个方法还有更好的实现方式，如下：

```
this.has = function(value){  
  return items.hasOwnProperty(value);  
};
```

所有JavaScript对象都有hasOwnProperty方法。这个方法返回一个表明对象是否具有特定属性的布尔值。

6.2.2 add 方法

接下来要实现add方法：

```
this.add = function(value){
  if (!this.has(value)){
    items[value] = value; //{1}
    return true;
  }
  return false;
};
```

对于给定的value，可以检查它是否存在于集合中。如果不存在，就把value添加到集合中（行{1}），返回true，表示添加了这个值。如果集合中已经有这个值，就返回false，表示没有添加它。



添加一个值的时候，把它同时作为键和值保存，因为这样有利于查找这个值。

6.2.3 remove 和 clear 方法

6

下面要实现remove方法：

```
this.remove = function(value){
  if (this.has(value)){
    delete items[value]; //{2}
    return true;
  }
  return false;
};
```

在remove方法中，我们会验证给定的value是否存在于集合中。如果存在，就从集合中移除value（行{2}），返回true，表示值被移除；否则返回false。

既然用对象来存储集合的items对象，就可以简单地使用delete操作符从items对象中移除属性（行{2}）。

使用Set类的示例代码如下：

```
let set = new Set();
set.add(1);
set.add(2);
```

出于好奇，如果在执行以上代码之后，在控制台（console.log）输出items变量，谷歌Chrome就会输出如下内容：

```
Object {1: 1, 2: 2}
```




可以看到，这是一个有两个属性的对象。属性名就是添加到集合的值，同时它也是属性值。

如果想移除集合中的所有值，可以用`clear`方法：

```
this.clear = function(){
    items = {}; // {3}
};
```

要重置`items`对象，需要做的只是把一个空对象重新赋值给它（行{3}）。我们也可以迭代集合，用`remove`方法依次移除所有的值，但既然有更简单的方法，那样做就太麻烦了。

6.2.4 size 方法

下一个要实现的是`size`方法（返回集合中有多少项）。这个方法有三种实现方式。

第一种方法是使用一个`length`变量，每当使用`add`或`remove`方法时控制它，就像在上一章中使用`LinkedList`类一样。

第二种方法，使用JavaScript内建的`Object`类的一个内建函数（ECMAScript 5以上版本）：

```
this.size = function(){
    return Object.keys(items).length; //{4}
};
```

JavaScript的`Object`类有一个`keys`方法，它返回一个包含给定对象所有属性的数组。在这种情况下，可以使用这个数组的`length`属性（行{4}）来返回`items`对象的属性个数。以上代码只能在现代浏览器中运行（比如IE9以上版本、Firefox 4以上版本、Chrome 5以上版本、Opera 12以上版本、Safari 5以上版本，等等）。

第三种方法是手动提取`items`对象的每一个属性，记录属性的个数并返回这个数字。这个方法可以在任何浏览器上运行，和之前的代码是等价的：

```
this.sizeLegacy = function(){
    let count = 0;
    for(let key in items) { //{5}
        if(items.hasOwnProperty(key)) //{6}
            ++count; //{7}
    }
    return count;
};
```

遍历`items`对象的所有属性（行{5}），检查它们是否是对象自身的属性（避免重复计数——行{6}）。如果是，就递增`count`变量的值（行{7}），最后在方法结束时返回这个数字。



不能简单地使用`for-in`语句遍历`items`对象的属性，并递增`count`变量的值。还需要使用`hasOwnProperty`方法（以验证`items`对象具有该属性），因为对象的原型包含了额外的属性（属性既有继承自JavaScript的`Object`类的，也有属于对象自身，未用于数据结构的）。

6.2.5 values 方法

`values`方法也应用了相同的逻辑，提取`items`对象的所有属性，以数组的形式返回：

```
this.values = function(){
  let values = [];
  for (let i=0, keys=Object.keys(items); i<keys.length; i++) {
    values.push(items[keys[i]]);
  }
  return values;
};
```

以上代码只能在现代浏览器中运行。在本书中我们使用的测试浏览器是Chrome和Firefox，因此代码可以工作。

如果想让代码在任何浏览器中都能执行，可以用与之前代码等价的下面这段代码：

```
this.valuesLegacy = function(){
  let values = [];
  for(let key in items) { //{7}
    if(items.hasOwnProperty(key)) { //{8}
      values.push(items[key]);
    }
  }
  return values;
};
```

所以，首先遍历`items`对象的所有属性（行{7}），把它们添加一个数组中（行{8}），并返回这个数组。该方法类似于我们开发的`sizeLegacy`方法，但我们添加一个数组，而不是计算属性个数。

6.2.6 使用 set 类

现在数据结构已经完成了，看看如何使用它吧。试着执行一些命令，测试我们的`Set`类：

```
let set = new Set();

set.add(1);
console.log(set.values()); //输出["1"]
console.log(set.has(1)); //输出true
console.log(set.size()); //输出1
```

```
set.add(2);
console.log(set.values()); //输出["1", "2"]
console.log(set.has(2)); //true
console.log(set.size()); //2

set.remove(1);
console.log(set.values()); //输出["2"]

set.remove(2);
console.log(set.values()); //输出[]
```

现在我们有了一个和ECMAScript 6中非常类似的Set类实现。如前所述，也可以用数组替代对象，存储元素。既然我们在第2章、第3章和第4章都用过数组，知道有不同方式实现同样的东西，这也不错。

6.3 集合操作

对集合可以进行如下操作。

- **并集**：对于给定的两个集合，返回一个包含两个集合中所有元素的新集合。
- **交集**：对于给定的两个集合，返回一个包含两个集合中共有元素的新集合。
- **差集**：对于给定的两个集合，返回一个包含所有存在于第一个集合且不存在于第二个集合的元素的新集合。
- **子集**：验证一个给定集合是否是另一集合的子集。

6.3.1 并集

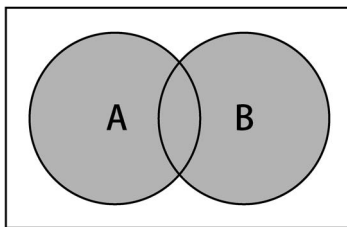
并集的数学概念是集合A和集合B的并集，表示为：

$$A \cup B$$

该集合定义如下：

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

意思是x（元素）存在于A中，或x存在于B中。下图展示了并集操作：



现在来实现Set类的union方法：

```
this.union = function(otherSet){
  let unionSet = new Set(); //{1}

  let values = this.values(); //{2}
  for (let i=0; i<values.length; i++){
    unionSet.add(values[i]);
  }

  values = otherSet.values(); //{3}
  for (let i=0; i<values.length; i++){
    unionSet.add(values[i]);
  }

  return unionSet;
};
```

首先需要创建一个新的集合，代表两个集合的并集（行{1}）。接下来，获取第一个集合（当前的Set类实例）所有的值（values），遍历并全部添加到代表并集的集合中（行{2}）。然后对第二个集合做同样的事（行{3}）。最后返回结果。

测试一下上面的代码：

```
let setA = new Set();
setA.add(1);
setA.add(2);
setA.add(3);

let setB = new Set();
setB.add(3);
setB.add(4);
setB.add(5);
setB.add(6);

let unionAB = setA.union(setB);
console.log(unionAB.values());
```

输出为["1", "2", "3", "4", "5", "6"]。注意元素3同时存在于A和B中，它在结果的集合中只出现一次。

6.3.2 交集

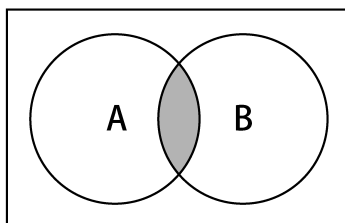
交集的数学概念是集合A和集合B的交集，表示为：

$$A \cap B$$

该集合定义如下：

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

意思是 x （元素）存在于 A 中，且 x 存在于 B 中。下图展示了交集操作：



现在来实现Set类的intersection方法：

```
this.intersection = function(otherSet){
  let intersectionSet = new Set(); //{1}

  let values = this.values();
  for (let i=0; i<values.length; i++){ //{2}
    if (otherSet.has(values[i])){ //{3}
      intersectionSet.add(values[i]); //{4}
    }
  }

  return intersectionSet;
}
```

intersection方法需要找到当前Set实例中，所有也存在于给定Set实例中的元素。首先创建一个新的Set实例，这样就能用它返回共有的元素（行{1}）。接下来，遍历当前Set实例所有的值（行{2}），验证它们是否也存在于otherSet实例（行{3}）之中。可以用这一章前面实现的has方法来验证元素是否存在于Set实例中。然后，如果这个值也存在于另一个Set实例中，就将其添加到创建的intersectionSet变量中（行{4}），最后返回它。

做些测试：

```
let setA = new Set();
setA.add(1);
setA.add(2);
setA.add(3);

let setB = new Set();
setB.add(2);
setB.add(3);
setB.add(4);

let intersectionAB = setA.intersection(setB);
console.log(intersectionAB.values());
```

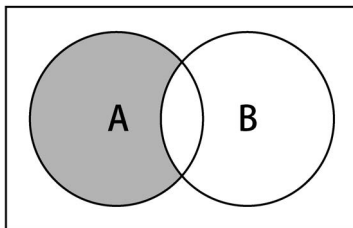
输出为["2", "3"]，因为2和3同时存在于两个集合中。

6.3.3 差集

差集的数学概念是集合 A 和集合 B 的差集，表示为： $A-B$ ，定义如下：

$$A-B = \{x | x \in A \wedge x \notin B\}$$

意思是 x （元素）存在于 A 中，且 x 不存在于 B 中。下图展示了集合 A 和 B 的差集操作：



现在来实现Set类的difference方法：

```
this.difference = function(otherSet){
  let differenceSet = new Set(); //{1}

  let values = this.values();
  for (let i=0; i<values.length; i++){ //{2}
    if (!otherSet.has(values[i])){ //{3}
      differenceSet.add(values[i]); //{4}
    }
  }

  return differenceSet;
};
```

intersection方法会得到所有同时存在于两个集合中的值。而difference方法会得到所有存在于集合 A 但不存在于 B 的值。因此这两个方法在实现上唯一的区别就是行{3}。只获取不存在于otherSet实例中的值，而不是也存在于其中的值。行{1}、{2}和{4}是完全相同的。

（用跟intersection部分相同的集合）做些测试：

```
let setA = new Set();
setA.add(1);
setA.add(2);
setA.add(3);

let setB = new Set();
setB.add(2);
setB.add(3);
setB.add(4);

let differenceAB = setA.difference(setB);
console.log(differenceAB.values());
```

输出为["1"]，因为1是唯一一个仅存在于setA的元素。

6.3.4 子集

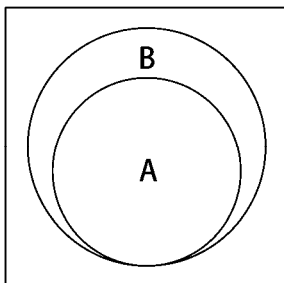
我们要介绍的最后一个集合操作是子集。子集的数学概念是集合 A 是集合 B 的子集（或集合 B 包含了 A ），表示为：

$$A \subseteq B$$

该集合定义如下：

$$\forall x \{ x \in A \rightarrow x \in B \}$$

意思是集合 A 中的每一个 x （元素），也需要存在于 B 中。下图展示了集合 A 是集合 B 的子集：



现在来实现Set类的subset方法：

```
this.subset = function(otherSet){
    if (this.size() > otherSet.size()){ //{1}
        return false;
    } else {
        let values = this.values();
        for (let i=0; i<values.length; i++){ //{2}
            if (!otherSet.has(values[i])){ //{3}
                return false; //{4}
            }
        }
        return true; //{5}
    }
};
```

首先需要验证的是当前Set实例的大小。如果当前实例中的元素比otherSet实例更多，它就不是一个子集（行{1}）。子集的元素个数需要小于或等于要比较的集合。

接下来要遍历集合中的所有元素（行{2}），验证这些元素也存在于otherSet中（行{3}）。如果有任何元素不存在于otherSet中，就意味着它不是一个子集，返回false（行{4}）。如果所有元素都存在于otherSet中，行{4}就不会被执行，那么就返回true（行{5}）。

检验一下上面的代码效果如何：

```
let setA = new Set();
setA.add(1);
```

```

setA.add(2);

let setB = new Set();
setB.add(1);
setB.add(2);
setB.add(3);

let setC = new Set();
setC.add(2);
setC.add(3);
setC.add(4);

console.log(setA.subset(setB));
console.log(setA.subset(setC));

```

我们有三个集合：setA是setB的子集(因此输出为true)，然而setA不是setC的子集(setC只包含了setA中的2，而不包含1)，因此输出为false。

6.4 ES6——set 类

ECMAScript 2015新增了Set类。我们可以基于ES6的Set开发我们的Set类。



关于ECMAScript 6的Set类的实现细节，请查阅<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global-Objects/Set>（或<http://goo.gl/2li2a5>）。

6

我们先看看原生的Set类怎么用。

还是用我们原来测试Set类的例子：

```

let set = new Set();
set.add(1);
console.log(set.values()); // 输出@Iterator
console.log(set.has(1));   // 输出true
console.log(set.size);     // 输出1

```

和我们的Set不同，ES6的Set的values方法返回Iterator（第2章提到过），而不是值构成的数组。另一个区别是，我们实现的size方法返回set中存储的值的个数，而ES6的Set则有一个size属性。

可以用delete方法删除set中的元素：

```
set.delete(1);
```

clear方法会重置set数据结构，这跟我们实现的功能一样。

ES6 set 类的操作

我们的Set类实现了并集、交集、差集、子集等数学操作，然而ES6原生的Set并没有这些功能。不过，有需要的话，我们也可以模拟。

我们的例子会用到下面两个集合：

```
let setA = new Set();
setA.add(1);
setA.add(2);
setA.add(3);

let setB = new Set();
setB.add(2);
setB.add(3);
setB.add(4);
```

1. 模拟并集操作

我们可以创建一个新的集合，用来添加两个集合中所有的元素（行{1}）。迭代这两个集合（行{2}、行{3}），把所有元素都添加到并集的集合中。代码如下：

```
let unionAb = new Set(); //{1}
for (let x of setA) unionAb.add(x); //{2}
for (let x of setB) unionAb.add(x); //{3}
```

2. 模拟交集操作

模拟交集操作需要创建一个辅助函数，来生成包含setA和setB都有的元素的新集合（行{1}）。代码如下：

```
let intersection = function(setA, setB) {
  let intersectionSet = new Set();
  for (let x of setA) {
    if (setB.has(x)) { //{1}
      intersectionSet.add(x);
    }
  }
  return intersectionSet;
};
let intersectionAB = intersection(setA, setB);
```

交集可以用更简单的语法实现，代码如下：

```
intersectionAb = new Set([x for (x of setA) if (setB.has(x))]);
```

这和intersection函数的效果完全一样。



目前只有Firefox支持简化的语法，但在所有支持ES6的现代浏览器中都可以执行intersection函数。

3. 模拟差集操作

交集操作创建的集合包含setA和setB都有的元素，差集操作创建的集合包含的则是setA有而setB没有的元素。看下面的代码：

```
let difference = function(setA, setB) {
  let differenceSet = new Set();
  for (let x of setA) {
    if (!setB.has(x)) { //{1}
      differenceSet.add(x);
    }
  }
  return differenceSet;
};
let differenceAB = difference(setA, setB);
```

intersection函数和difference函数只有行{1}不同，因为差集中只添加setA有而setB没有的元素。

差集也可以用更简单的语法实现，代码如下：

```
differenceAB = new Set([x for (x of setA) if (!setB.has(x))]);
```



目前只有Firefox支持简化的语法，但在所有支持ES6的现代浏览器中都可以执行difference函数。

6.5 小结

在这一章中，我们学习了如何从头实现一个与ECMAScript 6中定义的类似的Set类。我们还介绍了在其他编程语言的集合数据结构的实现中不常见的一些方法，比如并集、交集、差集和子集。因此，相比于其他编程语言目前的Set实现，我们实现了一个非常完备的Set类。

在下一章中，我们将会介绍散列和字典这两种非顺序数据结构。

在上一章中，我们学习了集合。本章我们会继续使用字典和散列表来存储唯一值（不重复的值）的数据结构。

集合、字典和散列表可以存储不重复的值。在集合中，我们感兴趣的是每个值本身，并把它当作主要元素。在字典中，我们用[键，值]的形式来存储数据。在散列表中也是一样（也是以[键，值]对的形式来存储数据）。但是两种数据结构的实现方式略有不同，本章中将会介绍。

7.1 字典

你已经知道，集合表示一组互不相同的元素（不重复的元素）。在字典中，存储的是[键，值]对，其中键名是用来查询特定元素的。字典和集合很相似，集合以[值，值]的形式存储元素，字典则是以[键，值]的形式来存储元素。字典也称作映射。

在本章中，我们会介绍几个在现实问题上使用字典数据结构的例子：一个实际的字典（单词和它们的释义）以及一个地址簿。

7.1.1 创建字典

与Set类相似，ECMAScript 6同样包含了一个Map类的实现，即我们所说的字典。

我们在本章中将要实现的类就是以ECMAScript 6中Map类的实现为基础的。你会发现它和Set类很相似（但不同于存储[值，值]对的形式，我们将要存储的是[键，值]对）。

这是我们的Dictionary类的骨架：

```
function Dictionary() {  
  var items = {};  
}
```

与Set类类似，我们将在一个Object的实例而不是数组中存储元素。

然后，我们需要声明一些映射/字典所能使用的方法。

- ❑ `set(key, value)`: 向字典中添加新元素。
- ❑ `delete(key)`: 通过使用键值来从字典中移除键值对应的数据值。
- ❑ `has(key)`: 如果某个键值存在于这个字典中，则返回`true`，反之则返回`false`。
- ❑ `get(key)`: 通过键值查找特定的数值并返回。
- ❑ `clear()`: 将这个字典中的所有元素全部删除。
- ❑ `size()`: 返回字典所包含元素的数量。与数组的`length`属性类似。
- ❑ `keys()`: 将字典所包含的所有键名以数组形式返回。
- ❑ `values()`: 将字典所包含的所有数值以数组形式返回。

1. has和set方法

我们首先来实现`has(key)`方法。之所以要先实现这个方法，是因为它会被`set`和`remove`等其他方法调用。我们可以通过如下代码来实现：

```
this.has = function(key) {
  return key in items;
};
```

这个方法的实现和我们之前在`Set`类中的实现是一样的。我们使用JavaScript中的`in`操作符来验证一个`key`是否是`items`对象的一个属性。

然后是`set`方法的实现：

```
this.set = function(key, value) {
  items[key] = value; //{1}
};
```

该方法接受一个`key`和一个`value`作为参数。我们直接将`value`设为`items`对象的`key`属性的值。它可以用来给字典添加一个新的值，或者用来更新一个已有的值。

2. delete方法

接下来，我们实现`delete`方法。它和`Set`类中的`delete`方法很相似，唯一的不同点在于我们将先搜索`key`（而不是`value`）：

```
this.delete= function(key) {
  if (this.has(key)) {
    delete items[key];
    return true;
  }
  return false;
};
```

然后我们可以使用JavaScript的`remove`操作符来从`items`对象中移除`key`属性。

3. get和values方法

如果我们想在字典中查找一个特定的项，并检索它的值，可以使用下面的方法：

```
this.get = function(key) {  
    return this.has(key) ? items[key] : undefined;  
};
```

get方法首先会验证我们想要检索的值是否存在（通过查找key值），如果存在，将返回该值，反之将返回一个undefined值（请记住undefined值和null值是不一样的，第1章中提到过这个概念）。

下一个是values方法。这个方法以数组的形式返回字典中所有values实例的值：

```
this.values = function() {  
    var values = [];  
    for (var k in items) { //{1}  
        if (this.has(k)) {  
            values.push(items[k]); //{2}  
        }  
    }  
    return values;  
};
```

首先，我们遍历items对象的所有属性值（行{1}）。为了确定值存在，我们使用has函数来验证key确实存在，然后将它的值加入values数组（行{2}）。最后，我们就能返回所有找到的值。



我们不能仅仅使用for-in语句来遍历items对象的所有属性，还需要使用hasOwnProperty方法（验证items对象是否包含某个属性），因为对象的原型也会包含对象的其他属性（JavaScript基本的Object类中的属性将会被继承，并存在于当前对象中，而对于这个数据结构来说，我们并不需要它们）。

4. clear、size、keys和getItems方法

clear和size方法与第6章Set类中对应的方法是完全一样的，因此我们就不在本章讨论了。

keys方法返回在Dictionary类中所有用于标识值的键名。要取出一个JavaScript对象中所有的键名，可以把这个对象作为参数传入Object类的keys方法（到目前为止，书中创建的类，包括Dictionary在内，都是JavaScript对象），如下：

```
this.keys = function() {  
    return Object.keys(items);  
};
```

最后，我们来验证items属性的输出值。我们可以实现一个返回items变量的方法，叫作

getItems:

```
this.getItems = function() {
    return items;
}
```

7.1.2 使用 Dictionary 类

首先，我们来创建一个Dictionary类的实例，然后给它添加三条电子邮件地址。我们将会使用这个dictionary实例来实现一个电子邮件地址簿。

使用我们创建的类来执行如下代码：

```
var dictionary = new Dictionary();
dictionary.set('Gandalf', 'gandalf@email.com');
dictionary.set('John', 'johnsnow@email.com');
dictionary.set('Tyrion', 'tyrion@email.com');
```

如果执行了如下代码，输出结果将会是true：

```
console.log(dictionary.has('Gandalf'));
```

下面的代码将会输出3，因为我们向字典实例中添加了三个元素：

```
console.log(dictionary.size());
```

现在，执行下面的几行代码：

```
console.log(dictionary.keys());
console.log(dictionary.values());
console.log(dictionary.get('Tyrion'));
```

输出结果分别如下所示：

```
["Gandalf", "John", "Tyrion"]
["gandalf@email.com", "johnsnow@email.com", "tyrion@email.com"]
tyrion@email.com
```

最后，再执行几行代码：

```
dictionary.delete('John');
```

再执行下面的代码：

```
console.log(dictionary.keys());
console.log(dictionary.values());
console.log(dictionary.getItems());
```

输出结果如下所示：

```
["Gandalf", "Tyrion"]
```

```
["gandalf@email.com", "tyrion@email.com"]
Object {Gandalf: "gandalf@email.com", Tyrion:
"tyrion@email.com"}
```

移除了一个元素后,现在的dictionary实例中只包含两个元素了。加粗的一行表现了items对象的内部结构。

7.2 散列表

在本节中,你将会学到HashTable类,也叫HashMap类,它是Dictionary类的一种散列表实现方式。

散列算法的作用是尽可能快地在数据结构中找到一个值。在之前的章节中,你已经知道如果要在数据结构中获得一个值(使用get方法),需要遍历整个数据结构来找到它。如果使用散列函数,就知道值的具体位置,因此能够快速检索到该值。散列函数的作用是给定一个键值,然后返回值在表中的地址。

举个例子,我们继续使用在前一节中使用的电子邮件地址簿。我们将要使用最常见的散列函数——“lose lose”散列函数,方法是简单地将每个键值中的每个字母的ASCII值相加。

名称/键	散列函数	散列值	散列表
Gandalf	71 + 97 + 110 + 100 + 97 + 108 + 102	685	[...] [399] johnsnow@email.com
John	74 + 111 + 104 + 110	399	[...] tyrion@email.com
Tyrion	84 + 121 + 114 + 105 + 111 + 110	645	[...] gandalf@email.com
			[...]

7.2.1 创建散列表

我们将使用数组来表示我们的数据结构,该数据结构和上一章中的图表所用的非常相似。

和之前一样,我们从搭建类的骨架开始:

```
function HashTable() {
  var table = [];
}
```

然后,给类添加一些方法。我们给每个类实现三个基本方法。

- ❑ `put(key, value)`: 向散列表增加一个新的项（也能更新散列表）。
- ❑ `remove(key)`: 根据键值从散列表中移除值。
- ❑ `get(key)`: 返回根据键值检索到的特定的值。

在实现这三个方法之前，要实现的第一个方法是散列函数，它是`HashTable`类中的一个私有方法：

```
var loseloseHashCode = function (key) {
  var hash = 0; //{1}
  for (var i = 0; i < key.length; i++) { //{2}
    hash += key.charCodeAt(i); //{3}
  }
  return hash % 37; //{4}
};
```

给定一个`key`参数，我们就能根据组成`key`的每个字符的ASCII码值的和得到一个数字。所以，首先需要有一个变量来存储这个总和（行{1}）。然后，遍历`key`（行{2}）并将从ASCII表中查到的每个字符对应的ASCII值加到`hash`变量中（可以使用JavaScript的`String`类中的`charCodeAt`方法——行{3}）。最后，返回`hash`值。为了得到比较小的数值，我们会使用`hash`值和一个任意数做除法的余数（`mod`）。



要了解更多关于ASCII的信息，请访问<http://www.asciitable.com/>。

7

现在，有了散列函数，我们就可以实现`put`方法了：

```
this.put = function(key, value) {
  var position = loseloseHashCode(key); //{5}
  console.log(position + ' - ' + key); //{6}
  table[position] = value; //{7}
};
```

首先，根据给定的`key`，我们需要根据所创建的散列函数计算出它在表中的位置（行{5}）。为了便于展示信息，我们将计算出的位置输出至控制台（行{6}）。由于它不是必需的，我们也可以将这行代码移除。然后要做的，是将`value`参数添加到用散列函数计算出的对应的位置上。（行{7}）。

从`HashTable`实例中查找一个值也很简单。为此，我们将会实现一个`get`方法：

```
this.get = function (key) {
  return table[loseloseHashCode(key)];
};
```

首先，我们会使用所创建的散列函数来求出给定`key`所对应的位置。这个函数会返回值的位置，因此我们所要做的就是根据这个位置从数组`table`中获得这个值。

我们要实现的最后一个方法是remove方法：

```
this.remove = function(key) {  
    table[loseloseHashCode(key)] = undefined;  
};
```

要从HashTable实例中移除一个元素，只要求出元素的位置（可以使用散列函数来获取）并赋值为undefined。

对于HashTable类来说，我们不需要像ArrayList类一样从table数组中将位置也移除。由于元素分布于整个数组范围内，一些位置会没有任何元素占据，并默认为undefined值。我们也不能将位置本身从数组中移除（这会改变其他元素的位置），否则，当下次需要获得或移除一个元素的时候，这个元素会不在我们用散列函数求出的位置上。

7.2.2 使用 HashTable 类

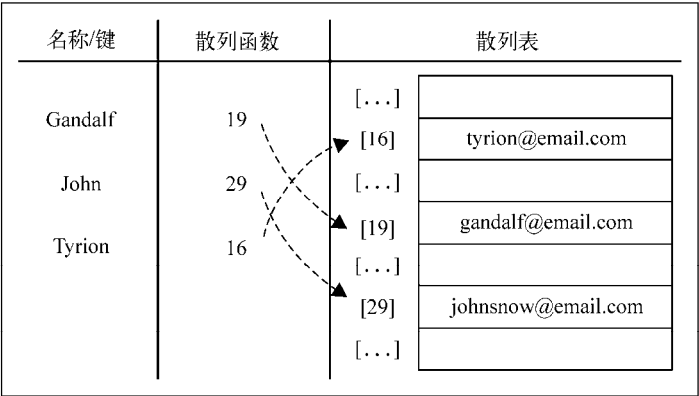
让我们执行一些代码来测试HashTable类：

```
var hash = new HashTable();  
hash.put('Gandalf', 'gandalf@email.com');  
hash.put('John', 'johnsnow@email.com');  
hash.put('Tyrion', 'tyrion@email.com');
```

执行上述代码，会在控制台中获得如下输出：

```
19 - Gandalf  
29 - John  
16 - Tyrion
```

下面的图表展现了包含这三个元素的HashTable数据结构：



现在来测试get方法：

```
console.log(hash.get('Gandalf'));  
console.log(hash.get('Loiane'));
```

获得如下的输出：

```
gandalf@email.com  
undefined
```

由于Gandalf是一个在散列表中存在的键，get方法将会返回它的值。而由于Loiane是一个不存在的键，当我们试图在数组中根据位置获取值的时候（一个由散列函数生成的位置），返回值将会是undefined（即不存在）。

然后，我们试试从散列表中移除Gandalf：

```
hash.remove('Gandalf');  
console.log(hash.get('Gandalf'));
```

由于Gandalf不再存在于表中，hash.get('Gandalf')方法将会在控制台上给出undefined的输出结果。

7.2.3 散列表和散列集合

散列表和散列映射是一样的，我们已经在本章中介绍了这种数据结构。

在一些编程语言中，还有一种叫作**散列集合**的实现。散列集合由一个集合构成，但是插入、移除或获取元素时，使用的是散列函数。我们可以重用本章中实现的所有代码来实现散列集合，不同之处在于，不再添加键值对，而是只插入值而没有键。例如，可以使用散列集合来存储所有的英语单词（不包括它们的定义）。和集合相似，散列集合只存储唯一的不重复的值。

7

7.2.4 处理散列表中的冲突

有时候，一些键会有相同的散列值。不同的值在散列表中对应相同位置的时候，我们称其为冲突。例如，我们看看下面的代码会得到怎样的输出结果：

```
var hash = new HashTable();  
hash.put('Gandalf', 'gandalf@email.com');  
hash.put('John', 'johnsnow@email.com');  
hash.put('Tyrion', 'tyrion@email.com');  
hash.put('Aaron', 'aaron@email.com');  
hash.put('Donnie', 'donnie@email.com');  
hash.put('Ana', 'ana@email.com');  
hash.put('Jonathan', 'jonathan@email.com');  
hash.put('Jamie', 'jamie@email.com');  
hash.put('Sue', 'sue@email.com');  
hash.put('Mindy', 'mindy@email.com');  
hash.put('Paul', 'paul@email.com');  
hash.put('Nathan', 'nathan@email.com');
```

输出结果如下：

```
19 - Gandalf
29 - John
16 - Tyrion
16 - Aaron
13 - Donnie
13 - Ana
5 - Jonathan
5 - Jamie
5 - Sue
32 - Mindy
32 - Paul
10 - Nathan
```



注意，Tyrion和Aaron有相同的散列值（16）。Donnie和Ana有相同的散列值（13），Jonathan、Jamie和Sue有相同的散列值（5），Mindy和Paul也有相同的散列值（32）。

那HashTable实例会怎样呢？执行之前的代码后散列表中会有哪些值呢？

为了获得结果，我们来实现一个叫作print的辅助方法，它会在控制台上输出HashTable中的值：

```
this.print = function() {
  for (var i = 0; i < table.length; ++i) { //{1}
    if (table[i] !== undefined) { //{2}
      console.log(i + ": " + table[i]); //{3}
    }
  }
};
```

首先，遍历数组中的所有元素（行{1}）。当某个位置上有值的时候（行{2}），会在控制台上输出位置和对应的值（行{3}）。

现在来使用这个方法：

```
hash.print();
```

在控制台上得到如下的输出结果：

```
5: sue@email.com
10: nathan@email.com
13: ana@email.com
16: aaron@email.com
19: gandalf@email.com
29: johnsnow@email.com
32: paul@email.com
```

Jonathan、Jamie和Sue有相同的散列值，也就是5。由于Sue是最后一个被添加的，Sue

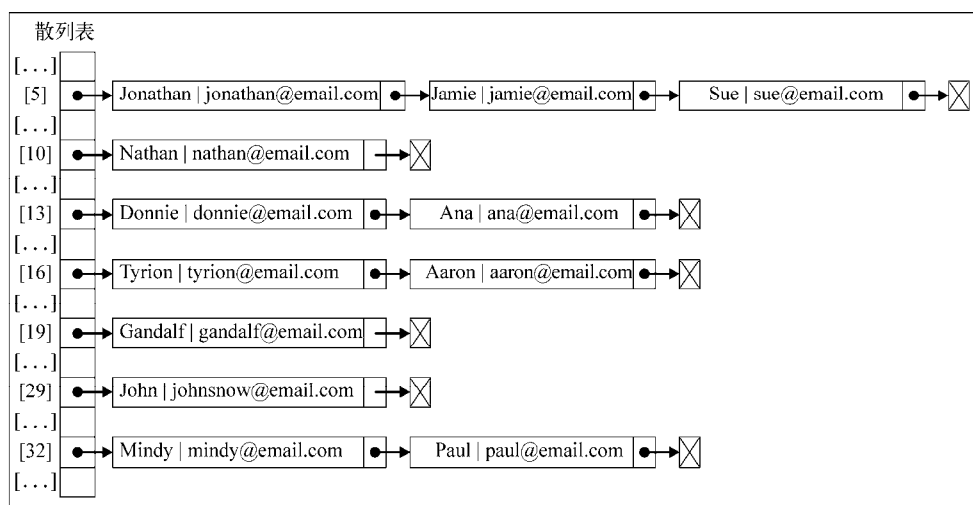
将是在HashTable实例中占据位置5的元素。首先，Jonathan会占据这个位置，然后Jamie会覆盖它，然后Sue会再次覆盖。这对于其他发生冲突的元素来说也是一样的。

使用一个数据结构来保存数据的目的显然不是去丢失这些数据，而是通过某种方法将它们全部保存起来。因此，当这种情况发生的时候就要去解决它。处理冲突有几种方法：分离链接、线性探查和双散列法。在本书中，我们会介绍前两种方法。

1. 分离链接

分离链接法包括为散列表的每一个位置创建一个链表并将元素存储在里面。它是解决冲突的最简单的方法，但是它在HashTable实例之外还需要额外的存储空间。

例如，我们在之前的测试代码中使用分离链接的话，输出结果将会是这样：



在位置5上，将会有包含三个元素的LinkedList实例；在位置13、16和32上，将会有包含两个元素的LinkedList实例；在位置10、19和29上，将会有包含单个元素的LinkedList实例。

对于分离链接和线性探查来说，只需要重写三个方法：put、get和remove。这三个方法在每种技术实现中都是不同的。

为了实现一个使用了分离链接的HashTable实例，我们需要一个新的辅助类来表示将要加入LinkedList实例的元素。我们管它叫ValuePair类（在HashTable类内部定义）：

```

var ValuePair = function(key, value){
  this.key = key;
  this.value = value;

  this.toString = function() {
    return '[' + this.key + ' - ' + this.value + ']';
  };
};

```

```

    }
};

```

这个类只会将key和value存储在一个Object实例中。我们也重写了toString方法，以便之后在浏览器控制台中输出结果。

(1) put方法

我们来实现第一个方法，put方法，代码如下：

```

this.put = function(key, value){
    var position = loseloseHashCode(key);

    if (table[position] == undefined) { //{1}
        table[position] = new LinkedList();
    }
    table[position].append(new ValuePair(key, value)); //{2}
};

```

在这个方法中，将验证要加入新元素的位置是否已经被占据（行{1}）。如果这个位置是第一次被加入元素，我们会在这个位置上初始化一个LinkedList类的实例（你已经在第5章中学习过）。然后，使用第5章中实现的append方法向LinkedList实例中添加一个ValuePair实例（键和值）（行{2}）。

(2) get方法

然后，我们实现用来获取特定值的get方法：

```

this.get = function(key) {
    var position = loseloseHashCode(key);

    if (table[position] !== undefined){ //{3}

        //遍历链表来寻找键/值
        var current = table[position].getHead(); //{4}

        while(current.next){ //{5}
            if (current.element.key === key){ //{6}
                return current.element.value; //{7}
            }
            current = current.next; //{8}
        }

        //检查元素在链表第一个或最后一个节点的情况
        if (current.element.key === key){ //{9}
            return current.element.value;
        }
    }
    return undefined; //{10}
};

```

我们要做的第一个验证，是确定在特定的位置上是否有元素存在（行{3}）。如果没有，则返回一个undefined表示在HashTable实例中没有找到这个值（行{10}）。如果在这个位置上有值存在，我们知道这是一个LinkedList实例。现在要做的是遍历这个链表来寻找我们需要的元素。在遍历之前先要获取链表表头的引用（行{4}），然后就可以从链表的头部遍历到尾部（行{5}，current.next将会是null）。

Node链表包含next指针和element属性。而element属性又是ValuePair的实例，所以它又有value和key属性。可以通过current.element.key来获得Node链表的key属性，并通过比较它来确定它是否就是我们要找的键（行{6}）。（这就是要使用ValuePair这个辅助类来存储元素的原因。我们不能简单地存储值本身，这样就不能确定哪个值对应着特定的键。）如果key值相同，就返回Node的值（行{7}）；如果不相同，就继续遍历链表，访问下一个节点（行{8}）。

如果要找的元素是链表的第一个或最后一个节点，那么就不会进入while循环的内部。因此，需要在行{9}处理这种情况。

(3) remove方法

使用分离链接法从HashTable实例中移除一个元素和之前在本章实现的remove方法有一些不同。现在使用的是链表，我们需要从链表中移除一个元素。来看看remove方法的实现：

```
this.remove = function(key){
    var position = loseloseHashCode(key);

    if (table[position] !== undefined){

        var current = table[position].getHead();
        while(current.next){
            if (current.element.key === key){ //{11}
                table[position].remove(current.element); //{12}
                if (table[position].isEmpty()){ //{13}
                    table[position] = undefined; //{14}
                }
                return true; //{15}
            }
            current = current.next;
        }

        // 检查是否为第一个或最后一个元素
        if (current.element.key === key){ //{16}
            table[position].remove(current.element);
            if (table[position].isEmpty()){
                table[position] = undefined;
            }
            return true;
        }
    }

    return false; //{17}
};
```

在`remove`方法中,我们使用和`get`方法一样的步骤找到要找的元素。遍历`LinkedList`实例时,如果链表中的`current`元素就是要找的元素(行{11}),使用`remove`方法将其从链表中移除。然后进行一步额外的验证:如果链表为空了(行{13}——链表中不再有任何元素了),就将散列表这个位置的值设为`undefined`(行{14}),这样搜索一个元素或打印它的内容的时候,就可以跳过这个位置了。最后,返回`true`表示这个元素已经被移除(行{15})或者在最后返回`false`表示这个元素在散列表中不存在(行{17})。同样,需要和`get`方法一样,处理元素在第一个或最后一个的情况(行{16})。

重写了这三个方法后,我们就拥有了一个使用了分离链接法来处理冲突的`HashMap`实例。

2. 线性探查

另一种解决冲突的方法是线性探查。当想向表中某个位置加入一个新元素的时候,如果索引为`index`的位置已经被占据了,就尝试`index+1`的位置。如果`index+1`的位置也被占据了,就尝试`index+2`的位置,以此类推。

● put方法

让我们继续实现需要重写的三个方法。第一个是`put`方法:

```
this.put = function(key, value){
  var position = loseloseHashCode(key); // {1}

  if (table[position] == undefined) { // {2}
    table[position] = new ValuePair(key, value); // {3}
  } else {
    var index = ++position; // {4}
    while (table[index] != undefined){ // {5}
      index++; // {6}
    }
    table[index] = new ValuePair(key, value); // {7}
  }
};
```

和之前一样,先获得由散列函数生成的位置(行{1}),然后验证这个位置是否有元素存在(如果这个位置被占据了,将会通过行{2}的验证)。如果没有元素存在,就在这个位置加入新元素(行{3}——一个`ValuePair`的实例)。

如果这个位置已经被占据了,需要找到下一个没有被占据的位置(`position`的值是`undefined`),因此我们声明一个`index`变量并赋值为`position+1`(行{4}——在变量名前使用自增运算符`++`会先递增变量值然后再将其赋值给`index`)。然后验证这个位置是否被占据(行{5}),如果被占据了,继续将`index`递增(行{6}),直到找到一个没有被占据的位置。然后要做的,就是将值分配到这个位置(行{7})。



在一些编程语言中，我们需要定义数组的大小。如果使用线性探查的话，需要注意的一个问题是数组的可用位置可能会被用完。在JavaScript中，我们不需要担心这个问题，因为我们不需要定义数组的大小，它可以根据需要自动改变大小——这是JavaScript内置的一个功能。

如果再次执行7.2.4节中插入数据的代码，下图展示使用了线性探查的散列表的最终结果：

散列表	
[...]	
[5]	Jonathan jonathan@email.com
[6]	Jamie jamie@email.com
[7]	Sue sue@email.com
[...]	
[10]	Nathan nathan@email.com
[...]	
[13]	Donnie donnie@email.com
[14]	Ana ana@email.com
[...]	
[16]	Tyrion tyrion@email.com
[17]	Aaron aaron@email.com
[18]	
[19]	Gandalf gandalf@email.com
[...]	
[19]	John johnsnow@email.com
[...]	

让我们来模拟一下散列表中的插入操作。

- (1) 试着插入**Gandalf**。它的散列值是**19**，由于散列表刚刚被创建，位置**19**还是空的——可以在这里插入数据。
- (2) 试着在位置**29**插入**John**。它也是空的，所以可以插入这个姓名。
- (3) 试着在位置**16**插入**Tyrion**。它是空的，所以可以插入这个姓名。
- (4) 试着插入**Aaron**，它的散列值也是**16**。位置**16**已经被**Tyrion**占据了，所以需要检查索引值为**position+1**的位置（**16+1**）。位置**17**是空的，所以可以在位置**17**插入**Aaron**。
- (5) 接着，试着在位置**13**插入**Donnie**。它是空的，所以可以插入这个姓名。
- (6) 想在位置**13**插入**Ana**，但是这个位置被占据了。因此在位置**14**进行尝试，它是空的，所

以可以在这里插入姓名。

(7) 然后，在位置5插入**Jonathan**，这个位置是空的，所以可以插入这个姓名。

(8) 试着在位置5插入**Jamie**，但是这个位置被占了。所以跳至位置6，这个位置是空的，因此可以在这个位置插入姓名。

(9) 试着在位置5插入**Sue**，但是位置被占据了。所以跳至位置6，但也被占了。接着跳至位置7，这里是空的，所以可以在这里插入姓名。以此类推。

● get方法

现在插入了所有的元素，让我们实现get方法来获取它们的值吧：

```
this.get = function(key) {
    var position = loseloseHashCode(key);

    if (table[position] !== undefined){ //{8}
        if (table[position].key === key) { //{9}
            return table[position].value; //{10}
        } else {
            var index = ++position;
            while (table[index] === undefined
                || table[index].key !== key){ //{11}
                index++;
            }
            if (table[index].key === key) { //{12}
                return table[index].value; //{13}
            }
        }
    }
    return undefined; //{14}
};
```

要获得一个键对应的值，先要确定这个键存在（行{8}）。如果这个键不存在，说明要查找的值不在散列表中，因此可以返回undefined（行{14}）。如果这个键存在，需要检查我们要找的值是否就是这个位置上的值（行{9}）。如果是，就返回这个值（行{10}）。

如果不是，就在散列表中的下一个位置继续查找，直到找到一个键值与我们要找的键值相同的元素（行{11}）。然后，验证一下当前项就是我们要找的项（行{12}——只是为了确认一下）并且将它的值返回（行{13}）。

我们无法确定要找的元素实际上在哪个位置，这就是使用ValuePair来表示HashTable元素的原因。

● remove方法

remove方法和get方法基本相同，不同之处在于行{10}和{13}，它们将会由下面的代码代替：

```
table[index] = undefined;
```

要移除一个元素，只需要给其赋值为`undefined`，来表示这个位置不再被占据并且可以在必要时接受一个新元素。

7.2.5 创建更好的散列函数

我们实现的“lose lose”散列函数并不是一个表现良好的散列函数，因为它会产生太多的冲突。如果我们使用这个函数的话，会产生各种各样的冲突。一个表现良好的散列函数是由几个方面构成的：插入和检索元素的时间（即性能），当然也包括较低的冲突可能性。我们可以在网上找到一些不同的实现方法，或者也可以实现自己的散列函数。

另一个可以实现的比“lose lose”更好的散列函数是`djb2`：

```
var djb2HashCode = function (key) {
  var hash = 5381; //{1}
  for (var i = 0; i < key.length; i++) { //{2}
    hash = hash * 33 + key.charCodeAt(i); //{3}
  }
  return hash % 1013; //{4}
};
```

它包括初始化一个`hash`变量并赋值为一个质数（行{1}——大多数实现都使用5381），然后迭代参数`key`（行{2}），将`hash`与33相乘（用来当作一个魔力数），并和当前迭代到的字符的ASCII码值相加（行{3}）。

最后，我们将使用相加的和与另一个随机质数（比我们认为的散列表的大小要大——在本例中，我们认为散列表的大小为1000）相除的余数。

如果再次执行7.2.4节中插入数据的代码，这将是使用`djb2HashCode`代替`loseloseHashCode`的最终结果：

```
798 - Gandalf
838 - John
624 - Tyrion
215 - Aaron
278 - Donnie
925 - Ana
288 - Jonathan
962 - Jamie
502 - Sue
804 - Mindy
54 - Paul
223 - Nathan
```

没有冲突！

这并不是最好的散列函数，但这是最受社区推崇的散列函数之一。



也有一些为数字键值准备的散列函数，你可以在<http://goo.gl/VtdN2x>找到一系列的实现。

7.3 ES6——Map 类

ECMAScript 2015新增了Map类。我们可以基于ES6的Map类开发我们的Dictionary类。



关于ECMAScript 6的Map类的实现细节，请查阅https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map（或<http://goo.gl/dm8VP6>）。

我们看看原生的Map类怎么用。

还是用我们原来测试Dictionary类的例子：

```
var map = new Map();

map.set('Gandalf', 'gandalf@email.com');
map.set('John', 'johnsnow@email.com');
map.set('Tyrion', 'tyrion@email.com');

console.log(map.has('Gandalf')); //输出true
console.log(map.size); //输出3
console.log(map.keys()); //输出["Gandalf", "John", "Tyrion"]
console.log(map.values()); //输出["gandalf@email.com",
"johnsnow@email.com", "tyrion@email.com"]
console.log(map.get('Tyrion')); //输出tyrion@email.com
```

和我们的Dictionary类不同，ES6的Map类的values方法和keys方法都返回Iterator（第2章提到过），而不是值或键构成的数组。另一个区别是，我们实现的size方法返回字典中存储的值的个数，而ES6的Map类则有一个size属性。

删除map中的元素可以用delete方法：

```
map.delete('John');
```

clear方法会重置map数据结构，这跟我们在Dictionary类里实现的一样。

7.4 ES6——WeakMap 类和 WeakSet 类

除了Set和Map这两种新的数据结构，ES6还增加了它们的弱化版本，WeakSet和WeakMap。

基本上，Map和Set与其弱化版本之间仅有的区别是：

❑ WeakSet或WeakMap类没有entries、keys和values等方法；

❑ 只能用对象作为键。

创建和使用这两个类主要是为了性能。WeakSet和WeakMap是弱化的（用对象作为键），没有强引用的键。这使得JavaScript的垃圾回收器可以从中清除整个入口。

另一个优点是，必须用键才可以取出值。这些类没有entries、keys和values等迭代器方法，因此，除非你知道键，否则没有办法取出值。这印证了我们在第3章的做法，即使用WeakMap类封装ES6类的私有属性。

使用WeakMap类的例子如下：

```
var map = new WeakMap();

var ob1 = {name:'Gandalf'}, //{1}
    ob2 = {name:'John'},
    ob3 = {name:'Tyrion'};

map.set(ob1, 'gandalf@email.com'); //{2}
map.set(ob2, 'johnsnow@email.com');
map.set(ob3, 'tyrion@email.com');

console.log(map.has(ob1)); //{3} 输出true
console.log(map.get(ob3)); //{4} 输出tyrion@email.com
map.delete(ob2); //{5}
```

WeakMap类也可以用set方法，但不能使用数字、字符串、布尔值等基本数据类型，需要将名字转换为对象（行{1}和行{2}）。

搜索（行{3}）、读取（行{4}）和删除值（行{5}），也要传入作为键的对象。

同样的逻辑也适用于WeakSet类。

7.5 小结

在本章中，我们学习了字典的相关知识，了解了如何添加、移除和获取元素以及其他的一些方法。我们还了解了字典和集合的不同之处。

我们也学习了散列运算，怎样创建一个散列表（或者说散列映射）数据结构，如何添加、移除和获取元素，以及如何创建散列函数。我们学习了怎样使用两种不同的方法解决散列表中的冲突问题。

我们还介绍了如何使用ES6的Map、WeakMap和WeakSet类。

在下一章中，我们将学习一种新的数据结构——树。

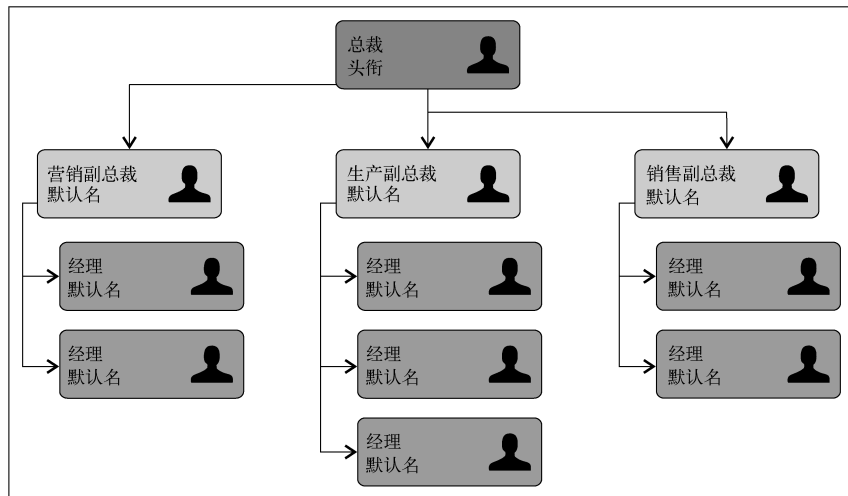
到目前为止，本书介绍了一些顺序数据结构，介绍的第一个非顺序数据结构是散列表。在本章，我们将要学习另一种非顺序数据结构——树，它对于存储需要快速查找的数据非常有用。

本章内容包括：

- ❑ 树的相关术语
- ❑ 创建树数据结构
- ❑ 树的遍历
- ❑ 添加和移除节点
- ❑ AVL树

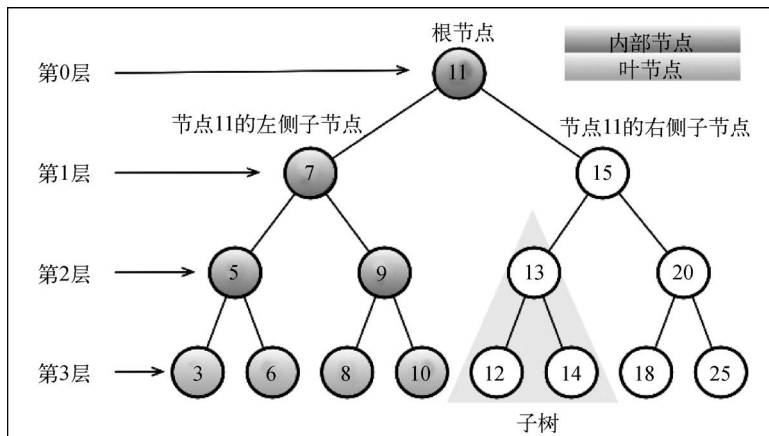
8.1 树数据结构

树是一种分层数据的抽象模型。现实生活中最常见的树的例子是家谱，或是公司的组织架构图，如下图所示：



8.2 树的相关术语

一个树结构包含一系列存在父子关系的节点。每个节点都有一个父节点（除了顶部的第一个节点）以及零个或多个子节点：



位于树顶部的节点叫作**根节点**（11）。它没有父节点。树中的每个元素都叫作节点，节点分为内部节点和外部节点。至少有一个子节点的节点称为内部节点（7、5、9、15、13和20是内部节点）。没有子元素的节点称为外部节点或叶节点（3、6、8、10、12、14、18和25是叶节点）。

一个节点可以有祖先和后代。一个节点（除了根节点）的祖先包括父节点、祖父节点、曾祖父节点等。一个节点的后代包括子节点、孙子节点、曾孙节点等。例如，节点5的祖先有节点7和节点11，后代有节点3和节点6。

有关树的另一个术语是子树。子树由节点和它的后代构成。例如，节点13、12和14构成了上图中树的一棵子树。

节点的一个属性是深度，节点的深度取决于它的祖先节点的数量。比如，节点3有3个祖先节点（5、7和11），它的深度为3。

树的高度取决于所有节点深度的最大值。一棵树也可以被分解成层级。根节点在第0层，它的子节点在第1层，以此类推。上图中的树的高度为3（最大高度已在图中表示——第3层）。

现在我们知道了与树相关的一些最重要的概念，下面来学习更多有关树的知识。

8.3 二叉树和二叉搜索树

二叉树中的节点最多只能有两个子节点：一个是左侧子节点，另一个是右侧子节点。这些定

义有助于我们写出更高效的向/从树中插入、查找和删除节点的算法。二叉树在计算机科学中的应用非常广泛。

二叉搜索树（BST）是二叉树的一种，但是它只允许你在左侧节点存储（比父节点）小的值，在右侧节点存储（比父节点）大（或者等于）的值。上一节的图中就展现了一棵二叉搜索树。

二叉搜索树将是我们在本章中要研究的数据结构。

8.3.1 创建 BinarySearchTree 类

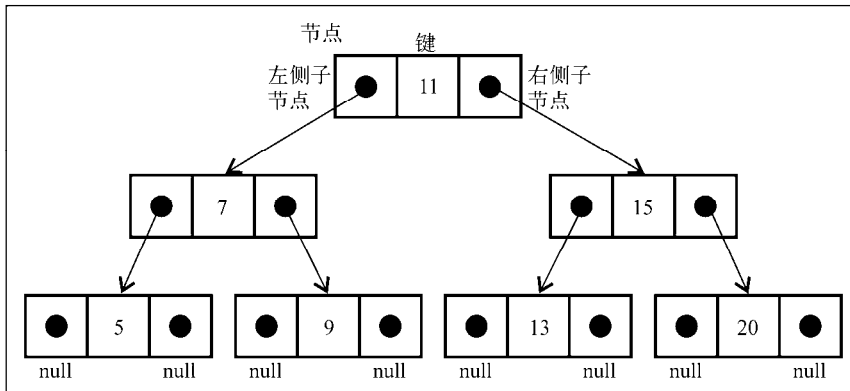
让我们开始创建自己的BinarySearchTree类。首先，声明它的结构：

```
function BinarySearchTree() {

  var Node = function(key){ //{1}
    this.key = key;
    this.left = null;
    this.right = null;
  };

  var root = null; //{2}
}
```

下图展现了二叉搜索树数据结构的组织方式：



和链表一样，将通过指针来表示节点之间的关系（术语称其为边）。在双向链表中，每个节点包含两个指针，一个指向下一个节点，另一个指向上一个节点。对于树，使用同样的方式（也使用两个指针）。但是，一个指向左侧子节点，另一个指向右侧子节点。因此，将声明一个Node类来表示树中的每个节点（行{1}）。值得注意的一个小细节是，不同于在之前的章节中将节点本身称作节点或项，我们将会称其为键。键是树相关的术语中对节点的称呼。

我们将会遵循和LinkedList类中相同的模式（第5章），这表示也将声明一个变量以控制此数据结构的第一个节点。在树中，它不再是头节点，而是根元素（行{2}）。

然后，我们需要实现一些方法。下面是将要在树类中实现的方法。

- ❑ insert(key)：向树中插入一个新的键。
- ❑ search(key)：在树中查找一个键，如果节点存在，则返回true；如果不存在，则返回false。
- ❑ inOrderTraverse：通过中序遍历方式遍历所有节点。
- ❑ preOrderTraverse：通过先序遍历方式遍历所有节点。
- ❑ postOrderTraverse：通过后序遍历方式遍历所有节点。
- ❑ min：返回树中最小的值/键。
- ❑ max：返回树中最大的值/键。
- ❑ remove(key)：从树中移除某个键。

我们将在后面的小节中实现每个方法。

8.3.2 向树中插入一个键

本章要实现的方法会比前几章实现的方法稍微复杂一些。我们将会使用很多递归。如果你对递归还不熟悉的话，请先参考11.1节。

下面的代码是用来向树插入一个新键的算法的第一部分：

```
this.insert = function(key){

    var newNode = new Node(key); //{1}

    if (root === null){ //{2}
        root = newNode;
    } else {
        insertNode(root,newNode); //{3}
    }
};
```

要向树中插入一个新的节点（或项），要经历三个步骤。

第一步是创建用来表示新节点的Node类实例（行{1}）。只需要向构造函数传递我们想用来插入树的节点值，它的左指针和右指针的值会由构造函数自动设置为null。

第二步要验证这个插入操作是否为一种特殊情况。这个特殊情况就是我们要插入的节点是树的第一个节点（行{2}）。如果是，就将根节点指向新节点。

第三步是将节点加在非根节点的其他位置。这种情况下,需要一个私有的辅助函数(行{3}),函数定义如下:

```
var insertNode = function(node, newNode){
  if (newNode.key < node.key){ //{4}
    if (node.left === null){ //{5}
      node.left = newNode; //{6}
    } else {
      insertNode(node.left, newNode); //{7}
    }
  } else {
    if (node.right === null){ //{8}
      node.right = newNode; //{9}
    } else {
      insertNode(node.right, newNode); //{10}
    }
  }
};
```

insertNode函数会帮助我们找到新节点应该插入的正确位置。下面是这个函数实现的步骤。

- ❑ 如果树非空,需要找到插入新节点的位置。因此,在调用insertNode方法时要通过参数传入树的根节点和要插入的节点。
- ❑ 如果新节点的键小于当前节点的键(现在,当前节点就是根节点)(行{4}),那么需要检查当前节点的左侧子节点。如果它没有左侧子节点(行{5}),就在那里插入新的节点。如果有左侧子节点,需要通过递归调用insertNode方法(行{7})继续找到树的下一层。在这里,下次将要比较的节点将会是当前节点的左侧子节点。
- ❑ 如果节点的键比当前节点的键大,同时当前节点没有右侧子节点(行{8}),就在那里插入新的节点(行{9})。如果有右侧子节点,同样需要递归调用insertNode方法,但是是用来和新节点比较的节点将会是右侧子节点。

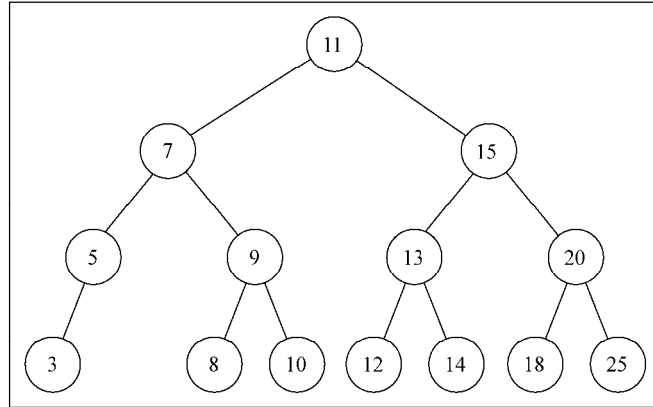
让我们通过一个例子来更好地理解这个过程。

考虑下面的情景:我们有一个新的树,并且想要向它插入第一个值。

```
var tree = new BinarySearchTree();
tree.insert(11);
```

这种情况下,树中有一个单独的节点,根指针将会指向它。源代码的行{2}将会执行。

现在,来考虑下图所示树结构的情况:



创建上图所示的树的代码如下，它们接着上面一段代码（插入了键为11的节点）之后输入执行：

```

tree.insert(7);
tree.insert(15);
tree.insert(5);
tree.insert(3);
tree.insert(9);
tree.insert(8);
tree.insert(10);
tree.insert(13);
tree.insert(12);
tree.insert(14);
tree.insert(20);
tree.insert(18);
tree.insert(25);

```

同时我们想要插入一个值为6的键，执行下面的代码：

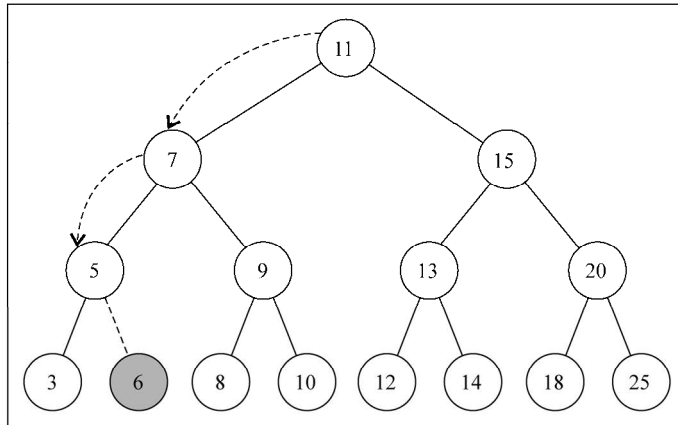
```
tree.insert(6);
```

下面的步骤将会被执行。

- (1) 树不是空的，行{3}的代码将会执行。`insertNode`方法将会被调用 (`root, key[6]`)。
- (2) 算法将会检测行{4} (`key[6] < root[11]`为真)，并继续检测行{5} (`node.left[7]`不是null)，然后将到达行{7}并调用 `insertNode (node.left[7], key[6])`。
- (3) 将再次进入 `insertNode` 方法内部，但是使用了不同的参数。它会再次检测行{4} (`key[6] < node[7]`为真)，然后再检测行{5} (`node.left[5]`不是null)，接着到达行{7}，调用 `insertNode (node.left[5], key[6])`。
- (4) 将再一次进入 `insertNode` 方法内部。它会再次检测行{4} (`key[6] < node[5]`为假)，然后到达行{8} (`node.right`是null——节点5没有任何右侧的子节点)，然后将会执行行{9}，在节点5的右侧子节点位置插入键6。

(5) 然后，方法调用会依次出栈，代码执行过程结束。

这是插入键6后的结果：



8.4 树的遍历

遍历一棵树是指访问树的每个节点并对它们进行某种操作的过程。但是我们应该怎么去做呢？应该从树的顶端还是底端开始呢？从左开始还是从右开始呢？访问树的所有节点有三种方式：中序、先序和后序。

在后面的小节中，我们将会深入了解这三种遍历方式的用法和实现。

8.4.1 中序遍历

中序遍历是一种以上行顺序访问BST所有节点的遍历方式，也就是以从最小到最大的顺序访问所有节点。中序遍历的一种应用就是对树进行排序操作。我们来看它的实现：

```

this.inOrderTraverse = function(callback){
    inOrderTraverseNode(root, callback); //{1}
};
  
```

`inOrderTraverse`方法接收一个回调函数作为参数。回调函数用来定义我们对遍历到的每个节点进行的操作（这也叫作访问者模式，要了解更多关于访问者模式的信息，请参考http://en.wikipedia.org/wiki/Visitor_pattern）。由于我们在BST中最常实现的算法是递归，这里使用了一个私有的辅助函数，来接收一个节点和对应的回调函数作为参数（行{1}）。

```

var inOrderTraverseNode = function (node, callback) {
    if (node !== null) { //{2}
  
```

```
inOrderTraverseNode(node.left, callback); //{3}  
callback(node.key); //{4}  
inOrderTraverseNode(node.right, callback); //{5}  
}  
};
```

要通过中序遍历的方法遍历一棵树，首先要检查以参数形式传入的节点是否为null（这就是停止递归继续执行的判断条件——行{2}——递归算法的基本条件）。

然后，递归调用相同的函数来访问左侧子节点（行{3}）。接着对这个节点进行一些操作（callback），然后再访问右侧子节点（行{5}）。

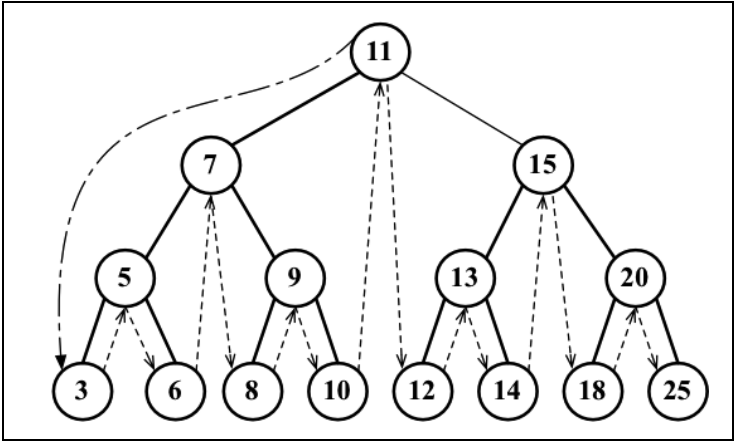
我们试着在之前展示的树上执行下面的方法：

```
function printNode(value){ //{6}  
    console.log(value);  
}  
tree.inOrderTraverse(printNode); //{7}
```

但首先，需要创建一个回调函数（行{6}）。我们要做的，是在浏览器的控制台上输出节点的值。然后，调用inOrderTraverse方法并将回调函数作为参数传入（行{7}）。当执行上面的代码后，下面的结果将会在控制台上输出（每个数字将会输出在不同的行）：

```
3 5 6 7 8 9 10 11 12 13 14 15 18 20 25
```

下面的图描绘了inOrderTraverse方法的访问路径：



8.4.2 先序遍历

先序遍历是以优先于后代节点的顺序访问每个节点的。先序遍历的一种应用是打印一个结构化的文档。

我们来看实现：

```
this.preOrderTraverse = function(callback){
    preOrderTraverseNode(root, callback);
};
```

preOrderTraverseNode方法的实现如下：

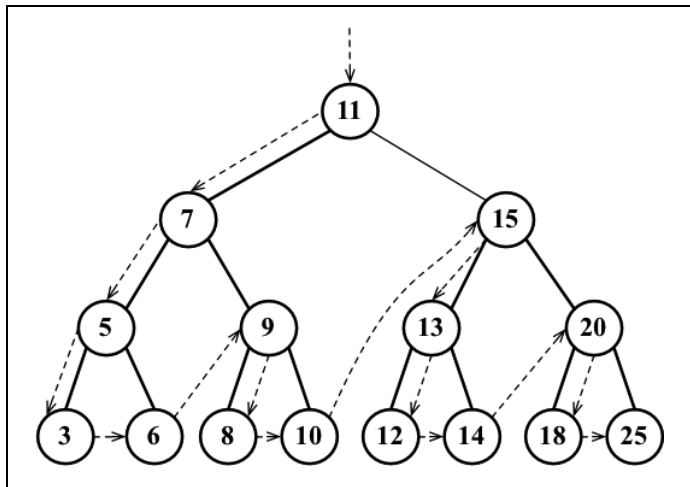
```
var preOrderTraverseNode = function (node, callback) {
    if (node !== null) {
        callback(node.key); //{1}
        preOrderTraverseNode(node.left, callback); //{2}
        preOrderTraverseNode(node.right, callback); //{3}
    }
};
```

先序遍历和中序遍历的不同点是，先序遍历会先访问节点本身（行{1}），然后再访问它的左侧子节点（行{2}），最后是右侧子节点（行{3}），而中序遍历的执行顺序是：{2}、{1}和{3}。

下面是控制台上的输出结果（每个数字将会输出在不同的行）：

```
11 7 5 3 6 9 8 10 15 13 12 14 20 18 25
```

下面的图描绘了preOrderTraverse方法的访问路径：



8.4.3 后序遍历

后序遍历则是先访问节点的后代节点，再访问节点本身。后序遍历的一种应用是计算一个目录和它的子目录中所有文件所占空间的大小。

我们来看它的实现：

```
this.postOrderTraverse = function(callback){
  postOrderTraverseNode(root, callback);
};
```

postOrderTraverseNode方法的实现如下：

```
var postOrderTraverseNode = function (node, callback) {
  if (node !== null) {
    postOrderTraverseNode(node.left, callback); // {1}
    postOrderTraverseNode(node.right, callback); // {2}
    callback(node.key);                          // {3}
  }
};
```

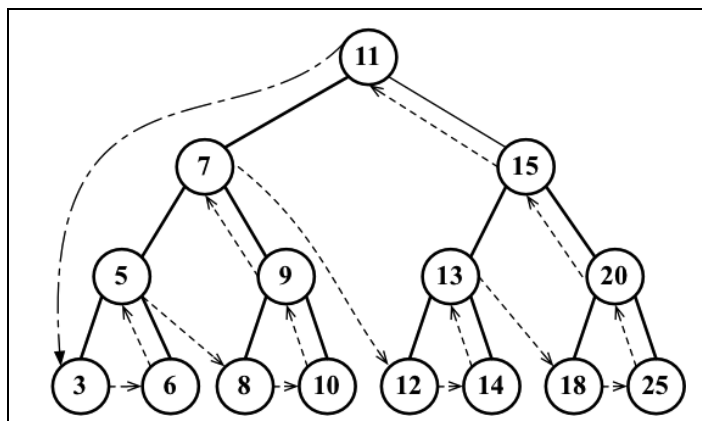
这个例子中，后序遍历会先访问左侧子节点（行{1}），然后是右侧子节点（行{2}），最后是父节点本身（行{3}）。

你会发现，中序、先序和后序遍历的实现方式是很相似的，唯一不同的是行{1}、{2}和{3}的执行顺序。

下面是控制台的输出结果（每个数字将会输出在不同行）：

```
3 6 5 8 10 9 7 12 14 13 18 25 20 15 11
```

下面的图描绘了postOrderTraverse方法的访问路径：



8.5 搜索树中的值

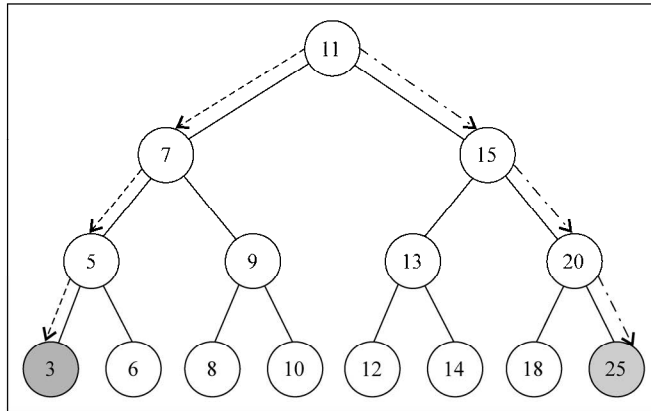
在树中，有三种经常执行的搜索类型：

- ☐ 搜索最小值
- ☐ 搜索最大值
- ☐ 搜索特定的值

我们依次来看。

8.5.1 搜索最小值和最大值

我们使用下面的树作为示例：



只用眼睛看这张图，你能一下找到树中的最小值和最大值吗？

如果你看一眼树最后一层最左侧的节点，会发现它的值为3，这是这棵树中最小的键。如果你再看一眼树最右端的节点（同样是树的最后一层），会发现它的值为25，这是这棵树中最大的键。这条信息在我们实现搜索树节点的最小值和最大值的方法时能给予我们很大的帮助。

首先，我们来看寻找树的最小键的方法：

```
this.min = function() {  
    return minNode(root); //{1}  
};
```

min方法将会暴露给用户。这个方法调用了minNode方法（行{1}）：

```
var minNode = function (node) {  
    if (node){  
        while (node && node.left !== null) { //{2}  
            node = node.left;           //{3}  
        }  
  
        return node.key;  
    }  
    return null; //{4}  
};
```

minNode方法允许我们从树中任意一个节点开始寻找最小的键。我们可以使用它来找到一棵树或它的子树中最小的键。因此，我们在调用minNode方法的时候传入树的根节点（行{1}），

因为我们想要找到整棵树的最小键。

在minNode内部，我们会遍历树的左边（行{2}和行{3}）直到找到树的最下层（最左端）。

以相似的方式，可以实现max方法：

```
this.max = function() {
    return maxNode(root);
};

var maxNode = function (node) {
    if (node){
        while (node && node.right !== null) { //{5}
            node = node.right;
        }

        return node.key;
    }
    return null;
};
```

要找到最大的键，我们要沿着树的右边进行遍历（行{5}）直到找到最右端的节点。

因此，对于寻找最小值，总是沿着树的左边；而对于寻找最大值，总是沿着树的右边。

8.5.2 搜索一个特定的值

在之前的章节中，我们同样实现了find、search或get方法来查找数据结构中的一个特定的值（和之前章节中实现的has方法相似）。我们将同样在BST中实现搜索的方法，来看它的实现：

```
this.search = function(key){
    return searchNode(root, key); //{1}
};

var searchNode = function(node, key){

    if (node === null){ //{2}
        return false;
    }
    if (key < node.key){ //{3}
        return searchNode(node.left, key); //{4}
    } else if (key > node.key){ //{5}
        return searchNode(node.right, key); //{6}
    } else {
        return true; //{7}
    }
};
```

我们要做的第一件事，是声明search方法。和BST中声明的其他方法的模式相同，我们将

会使用一个辅助函数（行{1}）。

`searchNode`方法可以用来寻找一棵树或它的任意子树中的一个特定的值。这也是为什么在行{1}中调用它的时候传入树的根节点作为参数。

在开始算法之前，先要验证作为参数传入的`node`是否合法（不是`null`）。如果是`null`的话，说明要找的键没有找到，返回`false`。

如果传入的节点不是`null`，需要继续验证。如果要找的键比当前的节点小（行{3}），那么继续在左侧的子树上搜索（行{4}）。如果要找的键比当前的节点大，那么就从右侧子节点开始继续搜索（行{6}），否则就说明要找的键和当前节点的键相等，就返回`true`来表示找到了这个键（行{7}）。

可以通过下面的代码来测试这个方法：

```
console.log(tree.search(1) ? 'Key 1 found.' : 'Key 1 not found.');
```

```
console.log(tree.search(8) ? 'Key 8 found.' : 'Key 8 not found.');
```

输出结果如下：

```
Value 1 not found.  
Value 8 found.
```

让我们详细展示查找1这个键的时候方法是如何执行的。

(1) 调用`searchNode`方法，传入根节点作为参数（行{1}）。(`node[root[11]]`)不是`null`（行{2}），因此我们执行到行{3}。

(2) (`key[1] < node[11]`)为`true`（行{3}），因此来到行{4}并再次调用`searchNode`方法，传入(`node[7]`, `key[1]`)作为参数。

(3) (`node[7]`)不是`null`（{2}），因此继续执行行{3}。

(4) (`key[1] < node[7]`)为`true`（行{3}），因此来到行{4}并再次调用`searchNode`方法，传入(`node[5]`, `key[1]`)作为参数。

(5) (`node[5]`)不是`null`（行{2}），因此继续执行行{3}。

(6) (`key[1] < node[5]`)为`true`（行{3}），因此来到行{4}并再次调用`searchNode`方法，传入(`node[3]`, `key[1]`)作为参数。

(7) (`node[3]`)不是`null`（行{2}），因此来到行{3}。

(8) (`key[1] < node[3]`)为真（行{3}），因此来到行{4}并再次调用`searchNode`方法，传入(`null`, `key[1]`)作为参数。`null`被作为参数传入是因为`node[3]`是一个叶节点（它没有子节点，所以它的左侧子节点的值为`null`）。

(9) 节点(`null`)的值为`null`（行{2}），这时要搜索的节点为`null`），因此返回`false`。

(10) 然后，方法调用会依次出栈，代码执行过程结束。

让我们再来查找值为8的节点。

(1) 调用searchNode方法，传入root作为参数（行{1}）。(node[root[11]])不是null（行{2}），因此我们来到行{3}。

(2) (key[8] < node[11])为真（行{3}），因此执行到行{4}并再次调用searchNode方法，传入(node[7], key[8])作为参数。

(3) (node[7])不是null，因此来到行{3}。

(4) (key[8] < node[7])为假（行{3}），因此来到行{5}。

(5) (key[8] > node[7])为真（行{5}），因此来到行{6}并再次调用searchNode方法，传入(node[9], key[8])作为参数。

(6) (node[9])不是null（行{2}），因此来到行{3}。

(7) (key[8] < node[9])为真（行{3}），因此来到行{4}并再次调用searchNode方法，传入(node[8], key[8])作为参数。

(8) (node[8])不是null（行{2}），因此来到行{3}。

(9) (key[8] < node[8])为假（行{3}），因此来到行{5}。

(10) (key[8] > node[8])为假（行{5}），因此来到行{7}并返回true，因为node[8]就是要找的键。

(11) 然后，方法调用会依次出栈，代码执行过程结束。

8.5.3 移除一个节点

我们要为BST实现的下一个、也是最后一个方法是remove方法。这是我们在本书中要实现的最复杂的方法。我们先创建这个方法，使它能够在树的实例上被调用：

```
this.remove = function(key){ root = removeNode(root, key); //{1} };
```

这个方法接收要移除的键并且它调用了removeNode方法，传入root和要移除的键作为参数（行{1}）。我要提醒大家的一件非常重要的事情是，root被赋值为removeNode方法的返回值。我们稍后会明白其中的原因。

removeNode方法的复杂之处在于我们要处理不同的运行场景，当然也包括它同样是通过递归来实现的。

我们来看removeNode方法的实现：

```
var removeNode = function(node, key){

    if (node === null){ //{2}
        return null;
    }
    if (key < node.key){ //{3}
        node.left = removeNode(node.left, key); //{4}
```

```

    return node; //{5}

} else if (key > node.key){ //{6}
    node.right = removeNode(node.right, key); //{7}
    return node; //{8}

} else { //键等于node.key

    //第一种情况——一个叶节点
    if (node.left === null && node.right === null){ //{9}
        node = null; //{10}
        return node; //{11}
    }

    //第二种情况——一个只有一个子节点的节点
    if (node.left === null){ //{12}
        node = node.right; //{13}
        return node; //{14}

    } else if (node.right === null){ //{15}
        node = node.left; //{16}
        return node; //{17}
    }

    //第三种情况——一个有两个子节点的节点
    var aux = findMinNode(node.right); //{18}
    node.key = aux.key; //{19}
    node.right = removeNode(node.right, aux.key); //{20}
    return node; //{21}
}
};

```

我们来看行{2}，如果正在检测的节点是null，那么说明键不存在于树中，所以返回null。

然后，我们要做的第一件事，就是在树中找到要移除的节点。因此，如果要找的键比当前节点的值小（行{3}），就沿着树的左边找到下一个节点（行{4}）。如果要找的键比当前节点的值大（行{6}），那么就沿着树的右边找到下一个节点（行{7}）。

如果我们找到了要找的键（键和node.key相等），就需要处理三种不同的情况。

findMinNode方法如下：

```

var findMinNode = function(node){
    while (node && node.left !== null) {
        node = node.left;
    }
    return node;
};

```

1. 移除一个叶节点

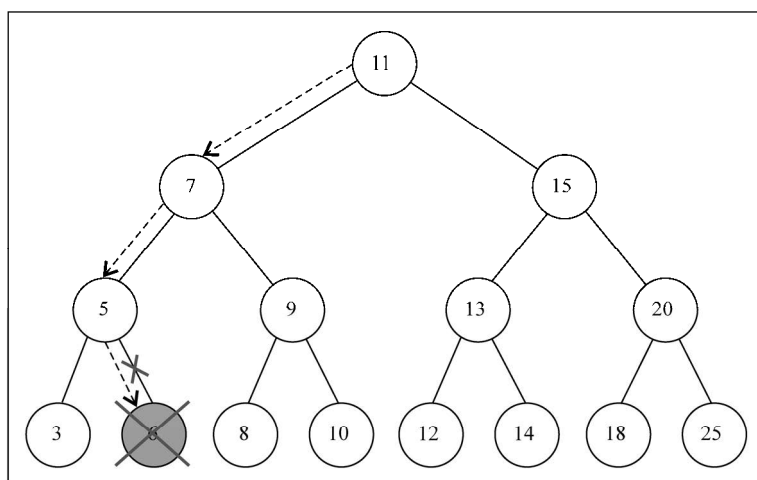
第一种情况是该节点是一个没有左侧或右侧子节点的叶节点——行{9}。在这种情况下，我

们要做的就是给这个节点赋予null值来移除它（行{9}）。但是当学习了链表的实现之后，我们知道仅仅赋一个null值是不够的，还需要处理指针。在这里，这个节点没有任何子节点，但是它有一个父节点，需要通过返回null来将对应的父节点指针赋予null值（行{11}）。

现在节点的值已经是null了，父节点指向它的指针也会接收到这个值，这也是我们要在函数中返回节点的值的原因。父节点总是会接收到函数的返回值。另一种可行的办法是将父节点和节点本身都作为参数传入方法内部。

如果回头来看方法的第一行代码，会发现我们在行{4}和行{7}更新了节点左右指针的值，同样也在行{5}和行{8}返回了更新后的节点。

下图展现了移除一个叶节点的过程：

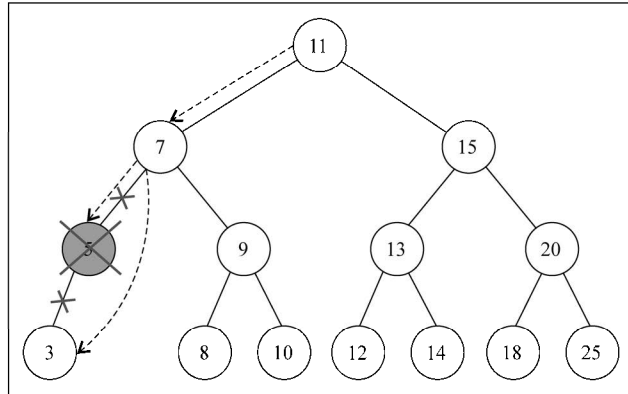


2. 移除有一个左侧或右侧子节点的节点

现在我们来看第二种情况，移除有一个左侧子节点或右侧子节点的节点。这种情况下，需要跳过这个节点，直接将父节点指向它的指针指向子节点。

如果这个节点没有左侧子节点（行{12}），也就是说它有一个右侧子节点。因此我们把对它的引用改为对它右侧子节点的引用（行{13}）并返回更新后的节点（行{14}）。如果这个节点没有右侧子节点，也是一样——把对它的引用改为对它左侧子节点的引用（行{16}）并返回更新后的值（行{17}）。

下图展现了移除只有一个左侧子节点或右侧子节点的节点的过程：



3. 移除有两个子节点的节点

现在是第三种情况，也是最复杂的情况，那就是要移除的节点有两个子节点——左侧子节点和右侧子节点。要移除有两个子节点的节点，需要执行四个步骤。

(1) 当找到了需要移除的节点后，需要找到它右边子树中最小的节点（它的继承者——行{18}）。

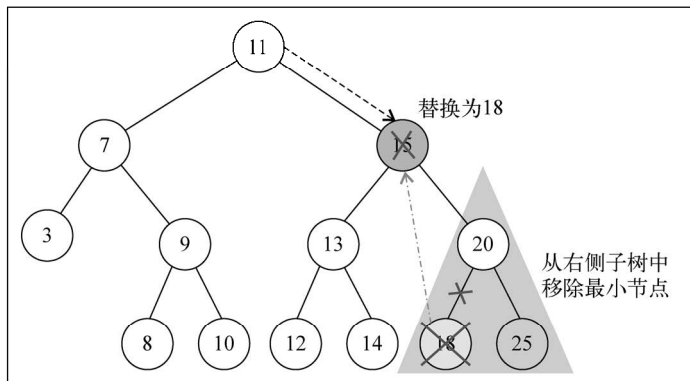
(2) 然后，用它右侧子树中最小节点的键去更新这个节点的值（行{19}）。通过这一步，我们改变了这个节点的键，也就是说它被移除了。

(3) 但是，这样在树中就有两个拥有相同键的节点了，这是不行的。要继续把右侧子树中的最小节点移除，毕竟它已经被移至要移除的节点的位置了（行{20}）。

(4) 最后，向它的父节点返回更新后节点的引用（行{21}）。

`findMinNode`方法的实现和`min`方法的实现方式是一样的。唯一不同之处在于，在`min`方法中只返回键，而在`findMinNode`中返回了节点。

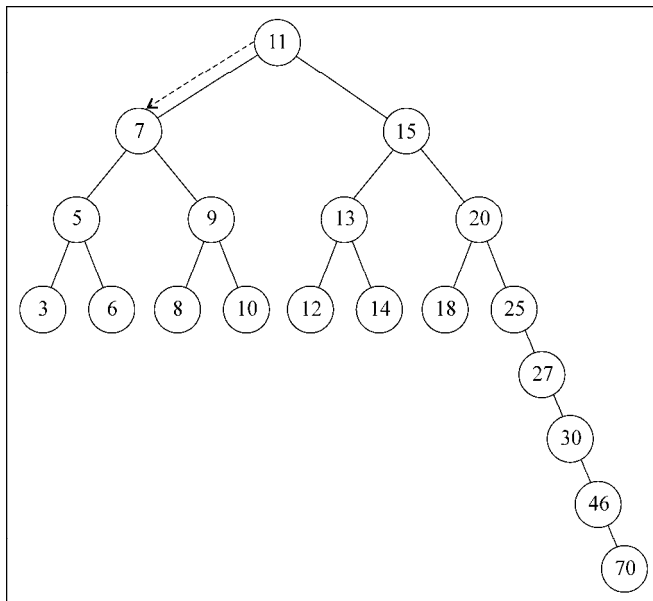
下图展现了移除有两个子节点的节点的过程：



8.6 自平衡树

现在你知道如何使用二叉搜索树了，如果愿意的话，可以继续去学习更多关于树的知识。

BST存在一个问题：取决于你添加的节点数，树的一条边可能会非常深；也就是说，树的一条分支会有很多层，而其他的分支却只有几层，如下图所示：



这会在需要在某条边上添加、移除和搜索某个节点时引起一些性能问题。为了解决这个问题，有一种树叫作Adelson-Velskii-Landi树（AVL树）。AVL树是一种自平衡二叉搜索树，意思是任何一个节点左右两侧子树的高度之差最多为1。也就是说这种树会在添加或移除节点时尽量试着成为一棵完全树。

8

8.6.1 Adelson-Velskii-Landi 树（AVL 树）

AVL树是一种自平衡树。添加或移除节点时，AVL树会尝试自平衡。任意一个节点（不论深度）的左子树和右子树高度最多相差1。添加或移除节点时，AVL树会尽可能尝试转换为完全树。

1. 在AVL树中插入节点

在AVL树中插入或移除节点和BST完全相同。然而，AVL树的不同之处在于我们需要检验它的平衡因子，如果有需要，则将其逻辑应用于树的自平衡。

下面的代码是向AVL树插入新节点的例子：

```
var insertNode = function(node, element) {
  if (node === null) {
    node = new Node(element);
  } else if (element < node.key) {
    node.left = insertNode(node.left, element);

    if (node.left !== null) {
      // 确认是否需要平衡 {1}
    }
  } else if (element > node.key) {
    node.right = insertNode(node.right, element);

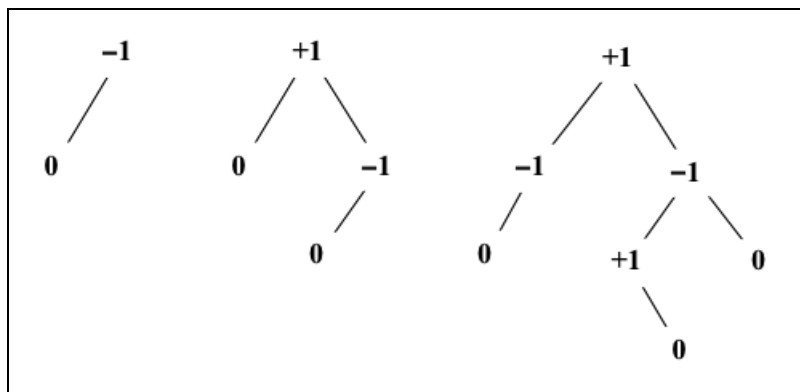
    if (node.right !== null) {
      // 确认是否需要平衡 {2}
    }
  }
  return node;
};
```

然而，插入新节点时，还要检查是否需要平衡树（行{1}和行{2}）。

● 计算平衡因子

在AVL树中，需要对每个节点计算右子树高度（hr）和左子树高度（hl）的差值，该值（hr - hl）应为0、1或-1。如果结果不是这三个值之一，则需要平衡该AVL树。这就是平衡因子的概念。

下图举例说明了一些树的平衡因子（所有的树都是平衡的）：



计算节点高度的代码如下：

```
var heightNode = function(node) {
  if (node === null) {
    return -1;
  }
};
```

```

    } else {
        return Math.max(heightNode(node.left),
            heightNode(node.right)) + 1;
    }
};

```

因此，向左子树插入新节点时，需要计算其高度；如果高度大于1（即不为-1、0和1之一），就需要平衡左子树。代码如下：

```

// 替换insertNode方法的行{1}
if ((heightNode(node.left) - heightNode(node.right)) > 1) {
    // 旋转 {3}
}

```

向右子树插入新节点时，应用同样的逻辑，代码如下：

```

// 替换insertNode方法的行{2}
if ((heightNode(node.right) - heightNode(node.left)) > 1) {
    // 旋转 {4}
}

```

● AVL旋转

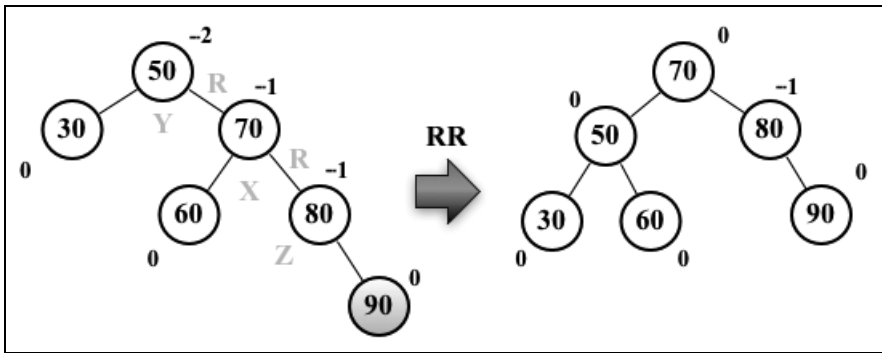
向AVL树插入节点时，可以执行单旋转或双旋转两种平衡操作，分别对应四种场景。

- ❑ 右-右（RR）：向左的单旋转
- ❑ 左-左（LL）：向右的单旋转
- ❑ 左-右（LR）：向右的双旋转
- ❑ 右-左（RL）：向左的双旋转

我们依次看看它们是如何工作的。

右-右（RR）：向左的单旋转

如下图所示：



假设向AVL树插入节点90, 这会造成树失衡(节点50-Y高度为-2), 因此需要恢复树的平衡。下面是我们执行的操作:

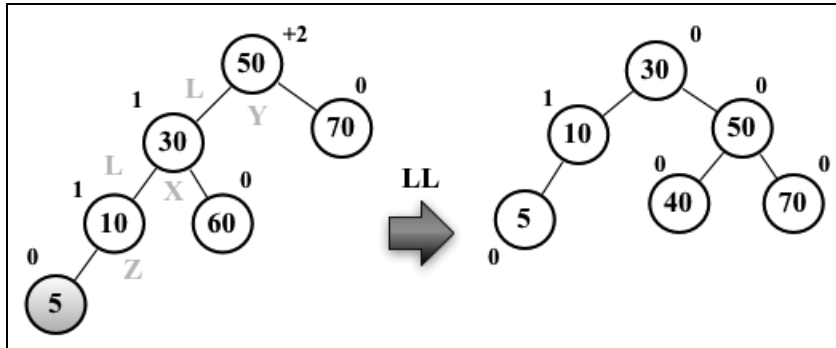
- ❑ 与平衡操作相关的节点有三个(X、Y、Z), 将节点X置于节点Y(平衡因子为-2)所在的位置(行{1});
- ❑ 节点X的右子树保持不变;
- ❑ 将节点Y的右子节点置为节点X的左子节点Z(行{2});
- ❑ 将节点X的左子节点置为节点Y(行{3})。

下面的代码举例说明了整个过程:

```
var rotationRR = function(node) {
    var tmp = node.right; // {1}
    node.right = tmp.left; // {2}
    tmp.left = node;      // {3}
    return tmp;
};
```

左-左(LL): 向右的单旋转

如下图所示:



假设向AVL树插入节点5, 这会造成树失衡(节点50-Y高度为+2), 需要恢复树的平衡。下面是我们执行的操作:

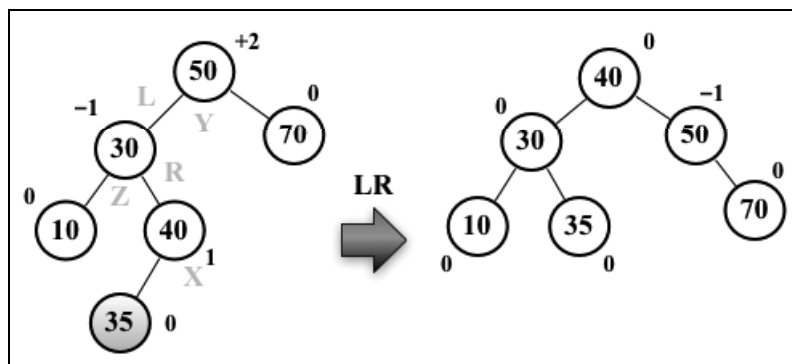
- ❑ 与平衡操作相关的节点有三个(X、Y、Z), 将节点X置于节点Y(平衡因子为+2)所在的位置(行{1});
- ❑ 节点X的左子树保持不变;
- ❑ 将节点Y的左子节点置为节点X的右子节点Z(行{2});
- ❑ 将节点X的右子节点置为节点Y(行{3})。

下面的代码举例说明了整个过程:

```
var rotationLL = function(node) {
  var tmp = node.left; // {1}
  node.left = tmp.right; // {2}
  tmp.right = node; // {3}
  return tmp;
};
```

左-右 (LR): 向右的双旋转

如下图所示:



假设向AVL树插入节点35，这会造成树失衡（节点50-Y高度为+2），需要恢复树的平衡。下面是我们执行的操作：

- ❑ 将节点X置于节点Y（平衡因子为+2）所在的位置；
- ❑ 将节点Y的左子节点置为节点X的右子节点；
- ❑ 将节点Z的右子节点置为节点X的左子节点；
- ❑ 将节点X的右子节点置为节点Y；
- ❑ 将节点X的左子节点置为节点Z。

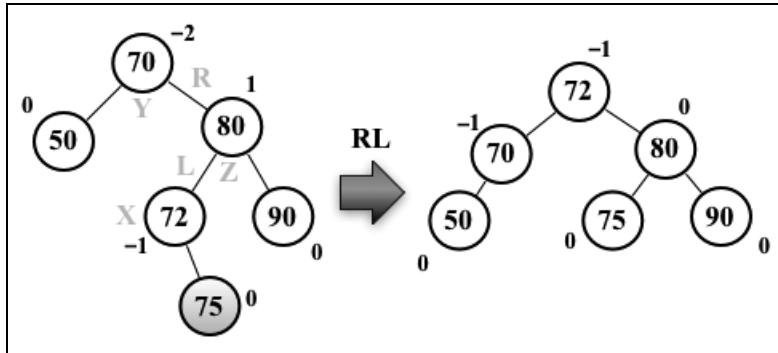
基本上，就是先做一次RR旋转，再做一次LL旋转。

下面的代码举例说明了整个过程：

```
var rotationLR = function(node) {
  node.left = rotationRR(node.left);
  return rotationLL(node);
};
```

右-左 (RL): 向左的双旋转

如下图所示：



假设向AVL树插入节点75，这会造成树失衡（节点70-Y高度为-2），需要恢复树的平衡。下面是我们执行的操作：

- ❑ 将节点X置于节点Y（平衡因子为-2）所在的位置；
- ❑ 将节点Z的左子节点置为节点X的右子节点；
- ❑ 将节点Y的右子节点置为节点X的左子节点；
- ❑ 将节点X的左子节点置为节点Y；
- ❑ 将节点X的右子节点置为节点Z。

基本上，就是先做一次LL旋转，再做一次RR旋转。

下面的代码举例说明了整个过程：

```
var rotationRL = function(node) {
  node.right = rotationLL(node.right);
  return rotationRR(node);
};
```

2. 完成insertNode方法

确认树需要平衡后，就需要对每种情况分别应用正确的旋转。

向左子树插入新节点，且节点的值小于其左子节点时，应进行LL旋转。否则，进行LR旋转。该过程的代码如下：

```
// 替换insertNode方法的行{1}
if ((heightNode(node.left) - heightNode(node.right)) > 1){
  // 旋转 {3}
  if (element < node.left.key){
    node = rotationLL(node);
  } else {
    node = rotationLR(node);
  }
}
```

向右子树插入新节点，且节点的值大于其右子节点时，应进行RR旋转。否则，进行RL旋转。该过程的代码如下：

```
// 替换insertNode方法的行{2}
if ((heightNode(node.right) - heightNode(node.left)) > 1){
    // 旋转 {4}
    if (element > node.right.key){
        node = rotationRR(node);
    } else {
        node = rotationRL(node);
    }
}
```

8.6.2 更多关于二叉树的知识

尽管AVL树是自平衡的，其插入或移除节点的性能并不总是最好的。更好的选择是红黑树。

红黑树可以高效有序地遍历其节点（<http://goo.gl/OxED8K>）。本书不打算讲解红黑树，但也提供了它的源代码。

如果感兴趣的话，你还可以了解堆积树（<http://goo.gl/SFlhW6>）。

8.7 小结

在本章中，我们介绍了在计算机科学中被广泛使用的基本树数据结构——二叉搜索树中添加、搜索和移除项的算法。我们同样介绍了访问树中每个节点的三种遍历方式。此外还学习了如何开发名叫AVL的自平衡树。

在下一章中，我们将会学习图的基本概念，它也是一种非线性的数据结构。

在本章，你将学习另一种非线性数据结构——图。这是我们要讲的最后一种数据结构，下一章将深入学习排序和搜索算法。

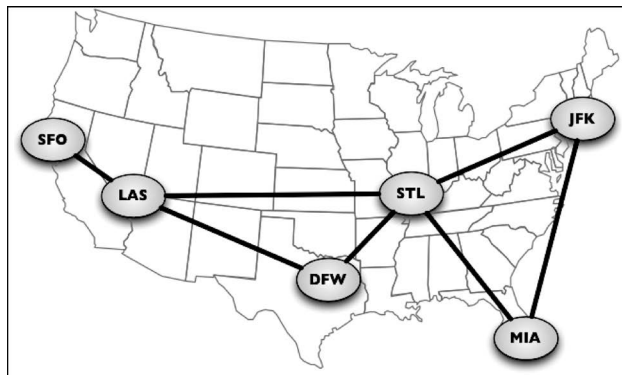
本章将会包含不少图的巧妙运用。图是一个庞大的主题，深入探索图的奇妙世界都足够写一本书了。

9.1 图的相关术语

图是网络结构的抽象模型。图是一组由边连接的节点（或顶点）。学习图是重要的，因为任何二元关系都可以用图来表示。

任何社交网络，例如Facebook、Twitter和Google plus，都可以用图来表示。

我们还可以使用图来表示道路、航班以及通信状态，如下图所示：

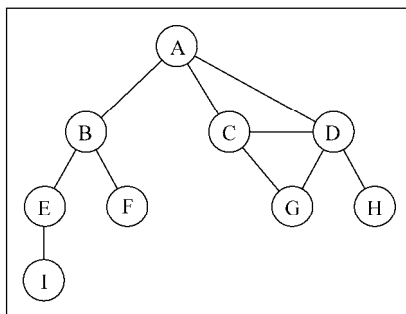


让我们来学习一下图在数学及技术上的概念。

一个图 $G = (V, E)$ 由以下元素组成。

- V : 一组顶点
- E : 一组边，连接 V 中的顶点

下图表示一个图：



在着手实现算法之前，让我们先了解一下图的一些术语。

由一条边连接在一起的顶点称为**相邻顶点**。比如，A和B是相邻的，A和D是相邻的，A和C是相邻的，A和E不是相邻的。

一个顶点的**度**是其相邻顶点的数量。比如，A和其他三个顶点相连接，因此，A的度为3；E和其他两个顶点相连，因此，E的度为2。

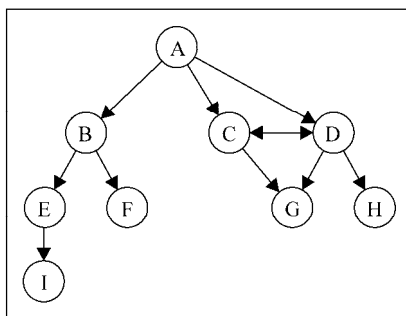
路径是顶点 $v_1, v_2, \dots, v_k$ 的一个连续序列，其中 v_i 和 v_{i+1} 是相邻的。以上一示意图中的图为例，其中包含路径A B E I和A C D G。

简单路径要求不包含重复的顶点。举个例子，A D G是一条简单路径。除去最后一个顶点（因为它和第一个顶点是同一个顶点），**环**也是一个简单路径，比如A D C A（最后一个顶点重新回到A）。

如果图中不存在环，则称该图是**无环的**。如果图中每两个顶点间都存在路径，则该图是**连通的**。

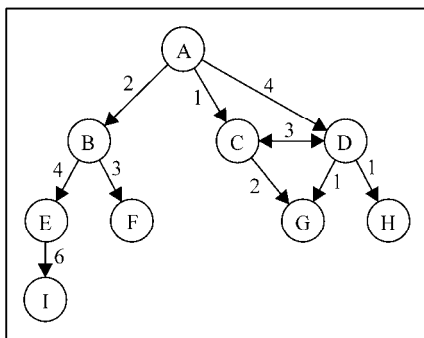
有向图和无向图

图可以是**无向的**（边没有方向）或是**有向的**（有向图）。如下图所示，有向图的边有一个方向：



如果图中每两个顶点间在双向上都存在路径,则该图是强连通的。例如,C和D是强连通的,而A和B不是强连通的。

图还可以是未加权的(目前为止我们看到的图都是未加权的)或是加权的。如下图所示,加权图的边被赋予了权值:



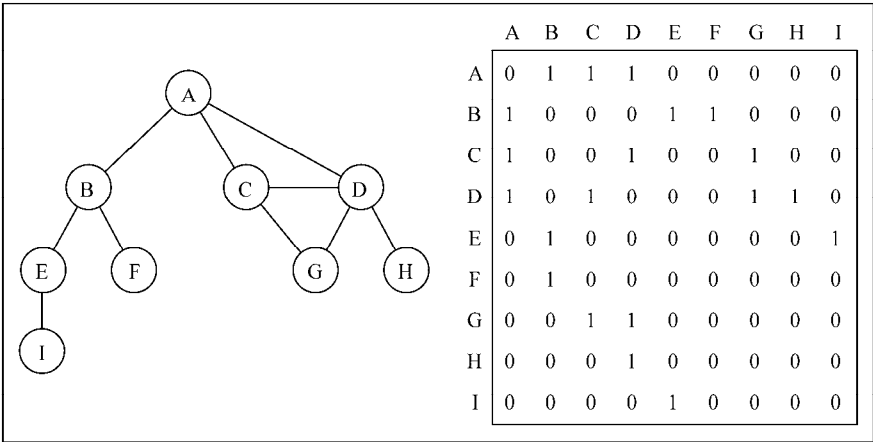
我们可以使用图来解决计算机科学世界中的很多问题,比如搜索图中的一个特定顶点或搜索一条特定边,寻找图中的一条路径(从一个顶点到另一个顶点),寻找两个顶点之间的最短路径,以及环检测。

9.2 图的表示

从数据结构的角度来说,我们有多种方式来表示图。在所有的表示法中,不存在绝对正确的方式。图的正确表示法取决于待解决的问题和图的类型。

9.2.1 邻接矩阵

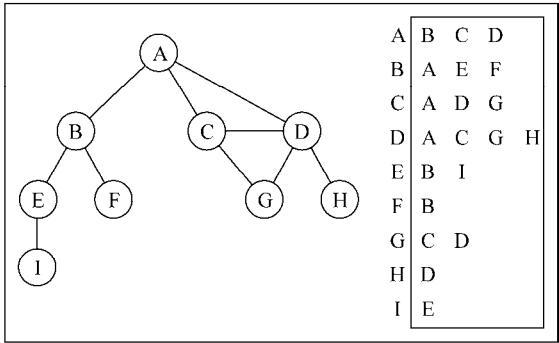
图最常见的实现是邻接矩阵。每个节点都和一个整数相关联,该整数将作为数组的索引。我们用一个二维数组来表示顶点之间的连接。如果索引为 i 的节点和索引为 j 的节点相邻,则 $\text{array}[i][j] == 1$,否则 $\text{array}[i][j] == 0$,如下图所示:



不是强连通的图（稀疏图）如果用邻接矩阵来表示，则矩阵中将会有很多0，这意味着我们浪费了计算机存储空间来表示根本不存在的边。例如，找给定顶点的相邻顶点，即使该顶点只有一个相邻顶点，我们也不得不迭代一整行。邻接矩阵表示法不够好的另一个理由是，图中顶点的数量可能会改变，而2维数组不太灵活。

9.2.2 邻接表

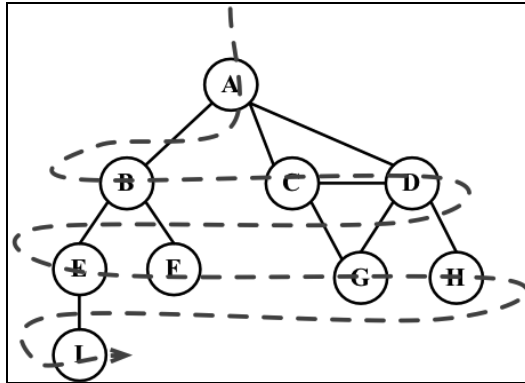
我们也可以使用一种叫作邻接表的动态数据结构来表示图。邻接表由图中每个顶点的相邻顶点列表所组成。存在好几种方式来表示这种数据结构。我们可以用列表（数组）、链表，甚至是散列表或是字典来表示相邻顶点列表。下面的示意图展示了邻接表数据结构。



尽管邻接表可能对大多数问题来说都是更好的选择，但以上两种表示法都很有用，且它们有着不同的性质（例如，要找出顶点v和w是否相邻，使用邻接矩阵会比较快）。在本书的示例中，我们将会使用邻接表表示法。

9.2.3 关联矩阵

我们还可以用关联矩阵来表示图。在关联矩阵中，矩阵的行表示顶点，列表示边。如下图所示，我们使用二维数组来表示两者之间的连通性，如果顶点 v 是边 e 的入射点，则 $\text{array}[v][e] == 1$ ；否则， $\text{array}[v][e] == 0$ 。



关联矩阵通常用于边的数量比顶点多的情况下，以节省空间和内存。

9.3 创建 Graph 类

照例，我们声明类的骨架：

```
function Graph() {
  var vertices = []; //{1}
  var adjList = new Dictionary(); //{2}
}
```

我们使用一个数组来存储图中所有顶点的名字（行{1}），以及一个字典（在第7章中已经实现）来存储邻接表（行{2}）。字典将会使用顶点的名字作为键，邻接顶点列表作为值。vertices数组和adjList字典两者都是我们Graph类的私有属性。

接着，我们将实现两个方法：一个用来向图中添加一个新的顶点（因为图实例化后是空的），另外一个方法用来添加顶点之间的边。我们先实现addVertex方法：

```
this.addVertex = function(v){
  vertices.push(v); //{3}
  adjList.set(v, []); //{4}
};
```

这个方法接受顶点 v 作为参数。我们将该顶点添加到顶点列表中（行{3}），并且在邻接表中，设置顶点 v 作为键对应的字典值为一个空数组（行{4}）。

现在，我们来实现addEdge方法：

```
this.addEdge = function(v, w){
  adjList.get(v).push(w); //{5}
  adjList.get(w).push(v); //{6}
};
```

这个方法接受两个顶点作为参数。首先，通过将w加入到v的邻接表中，我们添加了一条自顶点v到顶点w的边。如果你想实现一个有向图，则行{5}就足够了。由于本章中大多数的例子都是基于无向图的，我们需要添加一条自w向v的边（行{6}）。



请注意我们只是往数组里新增元素，因为数组已经在行{4}被初始化了。

让我们测试这段代码：

```
var graph = new Graph();
var myVertices = ['A','B','C','D','E','F','G','H','I']; //{7}
for (var i=0; i<myVertices.length; i++){ //{8}
  graph.addVertex(myVertices[i]);
}
graph.addEdge('A', 'B'); //{9}
graph.addEdge('A', 'C');
graph.addEdge('A', 'D');
graph.addEdge('C', 'D');
graph.addEdge('C', 'G');
graph.addEdge('D', 'G');
graph.addEdge('D', 'H');
graph.addEdge('B', 'E');
graph.addEdge('B', 'F');
graph.addEdge('E', 'I');
```

为方便起见，我们创建了一个数组，包含所有我们想添加到图中的顶点（行{7}）。接下来，我们只要遍历vertices数组并将其中的值逐一添加到我们的图中（行{8}）。最后，我们添加想要的边（行{9}）。这段代码将会创建一个图，也就是到目前为止本章的示意图所使用的。

为了方便一些，让我们来实现一下Graph类的toString方法，以便于在控制台输出图。

```
this.toString = function(){
  var s = '';
  for (var i=0; i<vertices.length; i++){ //{10}
    s += vertices[i] + ' -> ';
    var neighbors = adjList.get(vertices[i]); //{11}
    for (var j=0; j<neighbors.length; j++){ //{12}
      s += neighbors[j] + ' ';
    }
    s += '\n'; //{13}
  }
  return s;
};
```

我们为邻接表表示法构建了一个字符串。首先，迭代vertices数组列表（行{10}），将顶点的名字加入字符串中。接着，取得该顶点的邻接表（行{11}），同样也迭代该邻接表（行{12}），将相邻顶点加入我们的字符串。邻接表迭代完成后，给我们的字符串添加一个换行符（行{13}），这样就可以在控制台看到一个漂亮的输出了。运行如下代码：

```
console.log(graph.toString());
```

输出如下：

```
A -> B C D
B -> A E F
C -> A D G
D -> A C G H
E -> B I
F -> B
G -> C D
H -> D
I -> E
```

一个漂亮的邻接表！从该输出中，我们知道顶点A有这几个相邻顶点：B、C和D。

9.4 图的遍历

和树数据结构类似，我们可以访问图的所有节点。有两种算法可以对图进行遍历：**广度优先搜索**（Breadth-First Search, BFS）和**深度优先搜索**（Depth-First Search, DFS）。图遍历可以用来寻找特定的顶点或寻找两个顶点之间的路径，检查图是否连通，检查图是否含有环等。

在实现算法之前，让我们来更好地理解一下图遍历的思想方法。

图遍历算法的思想是必须追踪每个第一次访问的节点，并且追踪有哪些节点还没有被完全探索。对于两种图遍历算法，都需要明确指出第一个被访问的顶点。

完全探索一个顶点要求我们查看该顶点的每一条边。对于每一条边所连接的没有被访问过的顶点，将其标注为被发现的，并将其加进待访问顶点列表中。

为了保证算法的效率，务必访问每个顶点至多两次。连通图中每条边和顶点都会被访问到。

广度优先搜索算法和深度优先搜索算法基本上是相同的，只有一点不同，那就是待访问顶点列表的数据结构。

算 法	数据结构	描 述
深度优先搜索	栈	通过将顶点存入栈中（在第3章中学习过），顶点是沿着路径被探索的，存在新的相邻顶点就去访问
广度优先搜索	队列	通过将顶点存入队列中（在第4章中学习过），最先入队列的顶点先被探索

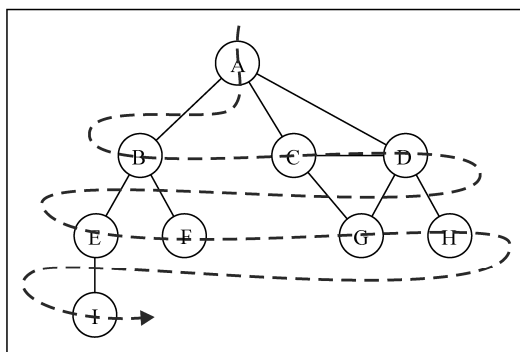
当要标注已经访问过的顶点时，我们用三种颜色来反映它们的状态。

- 白色：表示该顶点还没有被访问。
- 灰色：表示该顶点被访问过，但并未被探索过。
- 黑色：表示该顶点被访问过且被完全探索过。

这就是之前提到的务必访问每个顶点最多两次的原因。

9.4.1 广度优先搜索

广度优先搜索算法会从指定的第一个顶点开始遍历图，先访问其所有的相邻点，就像一次访问图的一层。换句话说，就是先宽后深地访问顶点，如下图所示：



以下是从顶点 v 开始的广度优先搜索算法所遵循的步骤。

- (1) 创建一个队列 Q 。
- (2) 将 v 标注为被发现的（灰色），并将 v 入队列 Q 。
- (3) 如果 Q 非空，则运行以下步骤：
 - (a) 将 u 从 Q 中出队列；
 - (b) 将标注 u 为被发现的（灰色）；
 - (c) 将 u 所有未被访问过的邻点（白色）入队列；
 - (d) 将 u 标注为已被探索的（黑色）。

让我们来实现广度优先搜索算法：

```
var initializeColor = function(){
  var color = [];
  for (var i=0; i<vertices.length; i++){
    color[vertices[i]] = 'white'; //{1}
  }
  return color;
};
```

```

this.bfs = function(v, callback){

    var color = initializeColor(), //{2}
    queue = new Queue();           //{3}
    queue.enqueue(v);              //{4}

    while (!queue.isEmpty()){      //{5}
        var u = queue.dequeue(),    //{6}
        neighbors = adjList.get(u); //{7}
        color[u] = 'grey';          //{8}
        for (var i=0; i<neighbors.length; i++){ // {9}
            var w = neighbors[i];    //{10}
            if (color[w] === 'white'){ // {11}
                color[w] = 'grey';   //{12}
                queue.enqueue(w);     //{13}
            }
        }
        color[u] = 'black'; // {14}
        if (callback) {           //{15}
            callback(u);
        }
    }
};

```

广度优先搜索和深度优先搜索都需要标注被访问过的顶点。为此，我们将使用一个辅助数组 `color`。由于当算法开始执行时，所有的顶点颜色都是白色（行{1}），所以我们可以创建一个辅助函数 `initializeColor`，为这两个算法执行此初始化操作。

让我们深入学习广度优先搜索方法的实现。我们要做的第一件事情是用 `initializeColor` 函数来将 `color` 数组初始化为 `white`（行{2}）。我们还需要声明和创建一个 `Queue` 实例（行{3}），它将会存储待访问和待探索的顶点。

照着本章开头解释过的步骤，`bfs` 方法接受一个顶点作为算法的起始点。起始顶点是必要的，我们将此顶点入队列（行{4}）。

如果队列非空（行{5}），我们将通过出队列（行{6}）操作从队列中移除一个顶点，并取得一个包含其所有邻点的邻接表（行{7}）。该顶点将被标注为 `grey`（行{8}），表示我们发现了它（但还未完成对其的探索）。

对于 `u`（行{9}）的每个邻点，我们取得其值（该顶点的名字——行{10}），如果它还未被访问过（颜色为 `white`——行{11}），则将其标注为我们已经发现了它（颜色设置为 `grey`——行{12}），并将这个顶点加入队列中（行{13}），这样当其从队列中出列的时候，我们可以完成对其的探索。

当完成探索该顶点和其相邻顶点后，我们将该顶点标注为已探索过的（颜色设置为 `black`——行{14}）。

我们实现的这个**dfs**方法也接受一个回调（我们在第8章中遍历树时使用了一个相似的方法）。这个参数是可选的，如果我们传递了回调函数（行{15}），会用到它。

让我们执行下面这段代码来测试一下这个算法：

```
function printNode(value){ //{16}
  console.log('Visited vertex: ' + value); //{17}
}
graph.bfs(myVertices[0], printNode); //{18}
```

首先，我们声明了一个回调函数（行{16}），它仅仅在浏览器控制台上输出已经被完全探索过的顶点的名字。接着，我们会调用**dfs**方法，给它传递第一个顶点（A——从本章开头声明的**myVertices**数组）和回调函数。当我们执行这段代码时，该算法会在浏览器控制台输出下示的结果：

```
Visited vertex: A
Visited vertex: B
Visited vertex: C
Visited vertex: D
Visited vertex: E
Visited vertex: F
Visited vertex: G
Visited vertex: H
Visited vertex: I
```

如你所见，顶点被访问的顺序和本节开头的示意图中所展示的一致。

1. 使用BFS寻找最短路径

到目前为止，我们只展示了BFS算法的工作原理。我们可以用该算法做更多事情，而不只是输出被访问顶点的顺序。例如，考虑如何解决下面这个问题。

给定一个图*G*和源顶点*v*，找出对每个顶点*u*，*u*和*v*之间最短路径的距离（以边的数量计）。

对于给定顶点*v*，广度优先算法会访问所有与其距离为1的顶点，接着是距离为2的顶点，以此类推。所以，可以用广度优先算法来解这个问题。我们可以修改**dfs**方法以返回给我们一些信息：

- 从*v*到*u*的距离 $d[u]$ ；
- 前溯点 $pred[u]$ ，用来推导出从*v*到其他每个顶点*u*的最短路径。

让我们来看看改进过的广度优先方法的实现：

```
this.BFS = function(v){

  var color = initializeColor(),
      queue = new Queue(),
      d = [],    //{1}

```

```

pred = []; //{2}
queue.enqueue(v);

for (var i=0; i<vertices.length; i++){ //{3}
    d[vertices[i]] = 0;                //{4}
    pred[vertices[i]] = null;          //{5}
}

while (!queue.isEmpty()){
    var u = queue.dequeue();
    neighbors = adjList.get(u);
    color[u] = 'grey';
    for (i=0; i<neighbors.length; i++){
        var w = neighbors[i];
        if (color[w] === 'white'){
            color[w] = 'grey';
            d[w] = d[u] + 1;            //{6}
            pred[w] = u;                //{7}
            queue.enqueue(w);
        }
    }
    color[u] = 'black';
}
return { //{8}
    distances: d,
    predecessors: pred
};
};

```

这个版本的BFS方法有些什么改变？



本章源代码中包含两个bfs方法：bfs（第一个）和BFS（改进版）。

我们还需要声明数组d（行{1}）来表示距离，以及pred数组来表示前溯点。下一步则是对图中的每一个顶点，用0来初始化数组d（行{4}），用null来初始化数组pred。

当我们发现顶点u的邻点w时，则设置w的前溯点值为u（行{7}）。我们还通过给d[u]加1来设置v和w之间的距离（u是w的前溯点，d[u]的值已经有了）。

方法最后返回了一个包含d和pred的对象（行{8}）。

现在，我们可以再次执行BFS方法，并将其返回值存在一个变量中：

```

var shortestPathA = graph.BFS(myVertices[0]);
console.log(shortestPathA);

```

对顶点A执行BFS方法，以下将会是输出：

```

distances: [A: 0, B: 1, C: 1, D: 1, E: 2, F: 2, G: 2, H: 2, I: 3],
predecessors: [A: null, B: "A", C: "A", D: "A", E: "B", F: "B", G: "C", H: "D", I: "E"]

```

这意味着顶点A与顶点B、C和D的距离为1；与顶点E、F、G和H的距离为2；与顶点I的距离为3。

通过前溯点数组，我们可以用下面这段代码来构建从顶点A到其他顶点的路径：

```
var fromVertex = myVertices[0]; //{9}
for (var i=1; i<myVertices.length; i++){ //{10}
    var toVertex = myVertices[i], //{11}
    path = new Stack(); //{12}
    for (var v=toVertex; v!=fromVertex;
    v=shortestPathA.predecessors[v]) { //{13}
        path.push(v); //{14}
    }
    path.push(fromVertex); //{15}
    var s = path.pop(); //{16}
    while (!path.isEmpty()){ //{17}
        s += ' - ' + path.pop(); //{18}
    }
    console.log(s); //{19}
}
```

我们用顶点A作为源顶点（行{9}）。对于每个其他顶点（除了顶点A——行{10}），我们会计算顶点A到它的路径。我们从顶点数组得到toVertex（行{11}），然后会创建一个栈来存储路径值（行{12}）。

接着，我们追溯toVertex到fromVertex的路径（行{13}）。变量v被赋值为其前溯点的值，这样我们能够反向追溯这条路径。将变量v添加到栈中（行{14}）。最后，源顶点也会被添加到栈中，以得到完整路径。

这之后，我们创建了一个s字符串，并将源顶点赋值给它（它是最后一个加入栈中的，所以它是第一个被弹出的项——行{16}）。当栈是非空的，我们就从栈中移出一个项并将其拼接到字符串s的后面（行{18}）。最后（行{19}）在控制台上输出路径。

执行该代码段，我们会得到如下输出：

```
A - B
A - C
A - D
A - B - E
A - B - F
A - C - G
A - D - H
A - B - E - I
```

这里，我们得到了从顶点A到图中其他顶点的最短路径（衡量标准是边的数量）。

2. 深入学习最短路径算法

本章中的图不是加权图。如果要计算加权图中的最短路径（例如，城市A和城市B之间的最

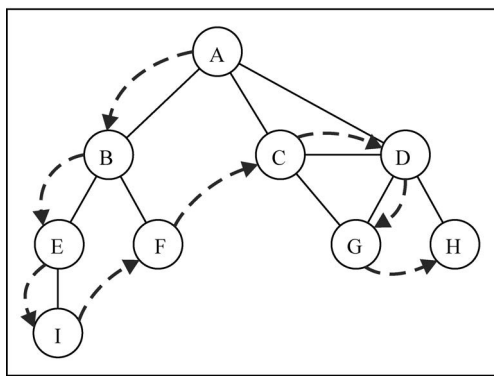
短路径——GPS和Google Maps中用到的算法)，广度优先搜索未必合适。

举些例子，Dijkstra算法解决了单源最短路径问题。Bellman-Ford算法解决了边权值为负的单源最短路径问题。A*搜索算法解决了求仅一对顶点间的最短路径问题，它用经验法则来加速搜索过程。Floyd-Warshall算法解决了求所有顶点对间的最短路径这一问题。

如本章开头提到的，图是一个广泛的主题，对最短路径问题及其变种问题，我们有很多的解决方案。但在开始学习这些其他解决方案前，我们需要掌握好图的基本概念，这是本章涵盖的内容。而这些其他解决方案则不会在本章讲述，但你可以自行探索图的奇妙世界。

9.4.2 深度优先搜索

深度优先搜索算法将会从第一个指定的顶点开始遍历图，沿着路径直到这条路径最后一个顶点被访问了，接着原路回退并探索下一条路径。换句话说，它是先深度后广度地访问顶点，如下图所示：



深度优先搜索算法不需要一个源顶点。在深度优先搜索算法中，若图中顶点 v 未访问，则访问该顶点 v 。

要访问顶点 v ，照如下步骤做。

- (1) 标注 v 为被发现的（灰色）。
- (2) 对于 v 的所有未访问的邻点 w ，访问顶点 w ，标注 v 为已被探索的（黑色）。

如你所见，深度优先搜索的步骤是递归的，这意味着深度优先搜索算法使用栈来存储函数调用（由递归调用所创建的栈）。

让我们来实现一下深度优先算法：

```

this.dfs = function(callback){
    var color = initializeColor(); //{1}

```

```

    for (var i=0; i<vertices.length; i++){ //{2}
        if (color[vertices[i]] === 'white'){ //{3}
            dfsVisit(vertices[i], color, callback); //{4}
        }
    }
};

var dfsVisit = function(u, color, callback){
    color[u] = 'grey'; //{5}
    if (callback) { //{6}
        callback(u);
    }
    var neighbors = adjList.get(u); //{7}
    for (var i=0; i<neighbors.length; i++){ //{8}
        var w = neighbors[i]; //{9}
        if (color[w] === 'white'){ //{10}
            dfsVisit(w, color, callback); //{11}
        }
    }
    color[u] = 'black'; //{12}
};

```

首先，我们创建颜色数组（行{1}），并用值white为图中的每个顶点对其做初始化，广度优先搜索也这么做的。接着，对于图实例中每一个未被访问过的顶点（行{2}和{3}），我们调用私有的递归函数dfsVisit，传递的参数为顶点、颜色数组以及回调函数（行{4}）。

当访问u顶点时，我们标注其为被发现的（grey——行{5}）。如果有callback函数的话（行{6}），则执行该函数输出已访问过的顶点。接下来一步是取得包含顶点u所有邻点的列表（行{7}）。对于顶点u的每一个未被访问过（颜色为white——行{10}和行{8}）的邻点w（行{9}），我们将调用dfsVisit函数，传递w和其他参数（行{11}——添加顶点w入栈，这样接下来就能访问它）。最后，在该顶点和邻点按深度访问之后，我们回退，意思是该顶点已被完全探索，并将其标注为black（行{12}）。

让我们执行下面的代码段来测试一下dfs方法：

```
graph.dfs(printNode);
```

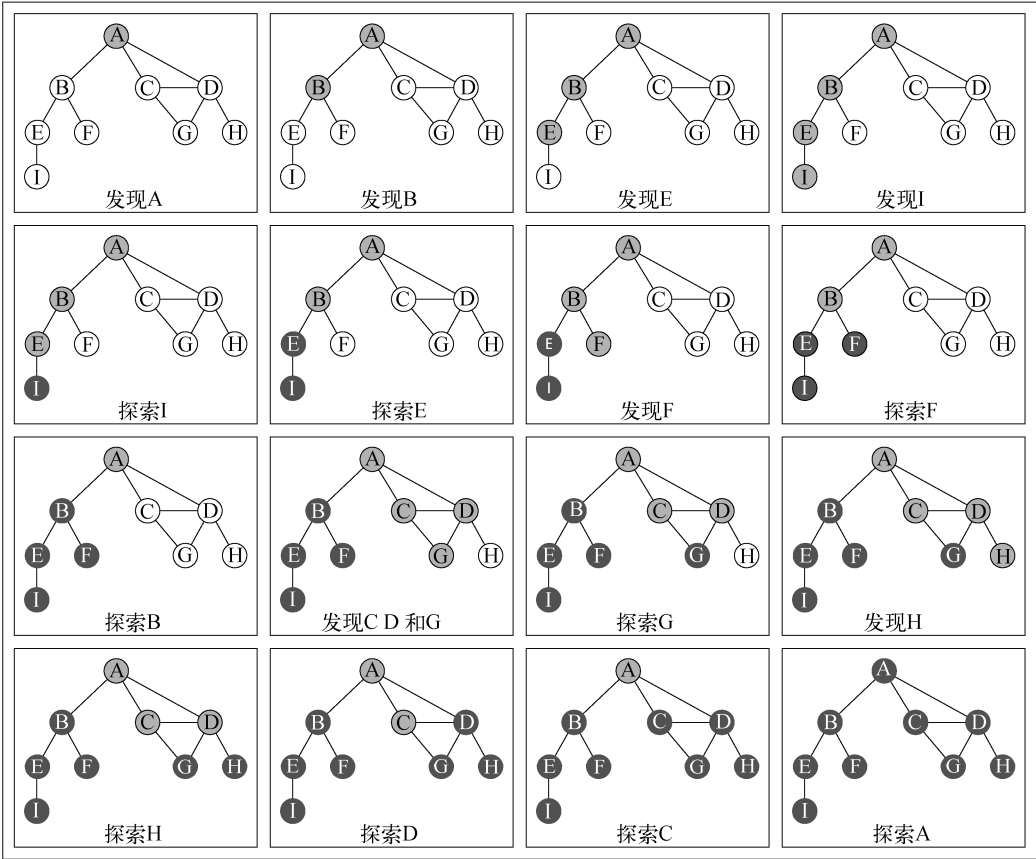
输出如下：

```

Visited vertex: A
Visited vertex: B
Visited vertex: E
Visited vertex: I
Visited vertex: F
Visited vertex: C
Visited vertex: D
Visited vertex: G
Visited vertex: H

```

这个顺序和本节开头处示意图所展示的一致。下面这个示意图展示了该算法每一步的执行过程：



在我们示例所用的图中，行{4}只会被执行一次，因为所有其他的顶点都有路径到第一个调用dfsVisit函数的顶点（顶点A）。如果顶点B第一个调用函数，则行{4}将会为其他顶点再执行一次（比如顶点A）。

1. 探索深度优先算法

到目前为止，我们只是展示了深度优先搜索算法的工作原理。我们可以用该算法做更多的事情，而不只是输出被访问顶点的顺序。

对于给定的图G，我们希望深度优先搜索算法遍历图G的所有节点，构建“森林”（有根树的一个集合）以及一组源顶点（根），并输出两个数组：发现时间和完成探索时间。我们可以修改dfs方法来返回给我们一些信息：

- ❑ 顶点 u 的发现时间 $d[u]$;
- ❑ 当顶点 u 被标注为黑色时, u 的完成探索时间 $f[u]$;
- ❑ 顶点 u 的前溯点 $p[u]$ 。

让我们来看看改进了的DFS方法的实现:

```
var time = 0; //{1}
this.DFS = function(){
  var color = initializeColor(), //{2}
  d = [],
  f = [],
  p = [];
  time = 0;

  for (var i=0; i<vertices.length; i++){ //{3}
    f[vertices[i]] = 0;
    d[vertices[i]] = 0;
    p[vertices[i]] = null;
  }
  for (i=0; i<vertices.length; i++){
    if (color[vertices[i]] === 'white'){
      DFSVisit(vertices[i], color, d, f, p);
    }
  }
  return {
    discovery: d,
    finished: f,
    predecessors: p
  };
};

var DFSVisit = function(u, color, d, f, p){
  console.log('discovered ' + u);
  color[u] = 'grey';
  d[u] = ++time; //{5}
  var neighbors = adjList.get(u);
  for (var i=0; i<neighbors.length; i++){
    var w = neighbors[i];
    if (color[w] === 'white'){
      p[w] = u;
      DFSVisit(w,color, d, f, p);
    }
  }
  color[u] = 'black';
  f[u] = ++time; //{7}
  console.log('explored ' + u);
};
```

我们需要一个变量来追踪发现时间和完成探索时间(行{1})。时间变量不能被作为参数传递,因为非对象的变量不能作为引用传递给其他JavaScript方法(将变量作为引用传递的意思是如果该变量在其他方法内部被修改,新值会在原始变量中反映出来)。接下来,我们声明数组 d 、

f 和 p （行{2}）。我们需要为图的每一个顶点来初始化这些数组（行{3}）。在这个方法结尾处返回这些值（行{4}），之后我们要用到它们。

当一个顶点第一次被发现时，我们追踪其发现时间（行{5}）。当它是由引自顶点 u 的边而被发现的，我们追踪它的前溯点（行{6}）。最后，当这个顶点被完全探索后，我们追踪其完成时间（行{7}）。

深度优先算法背后的思想是什么？边是从最近发现的顶点 u 处被向外探索的。只有连接到未发现的顶点的边被探索了。当 u 所有的边都被探索了，该算法回退到 u 被发现的地方去探索其他的边。这个过程持续到我们发现了所有从原始顶点能够触及的顶点。如果还留有任何其他未被发现的顶点，我们对新源顶点重复这个过程。重复该算法，直到图中所有的顶点都被探索了。

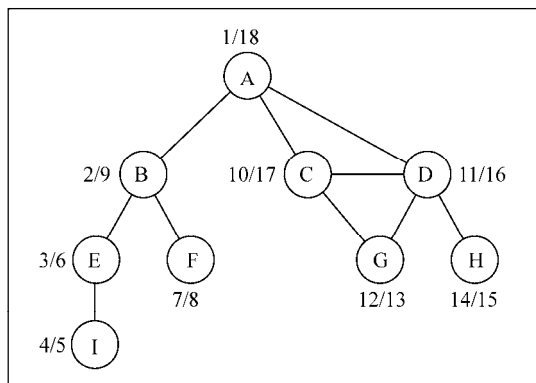
对于改进过的深度优先搜索，有两点需要我们注意：

- ❑ 时间（time）变量值的范围只可能在图顶点数量的一倍到两倍之间；
- ❑ 对于所有的顶点 u ， $d[u] < f[u]$ （意味着，发现时间的值比完成时间的值小，完成时间意思是所有顶点都被探索过了）。

在这两个假设下，我们有如下的规则：

$$1 \leq d[u] < f[u] \leq 2|V|$$

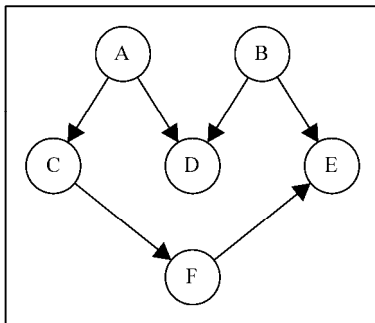
如果对同一个图再跑一遍新的深度优先搜索方法，对图中每个顶点，我们会得到如下的发现/完成时间：



但我们能用这些新信息来做什么呢？来看下一节。

2. 拓扑排序——使用深度优先搜索

给定下图，假定每个顶点都是一个我们需要去执行的任务：



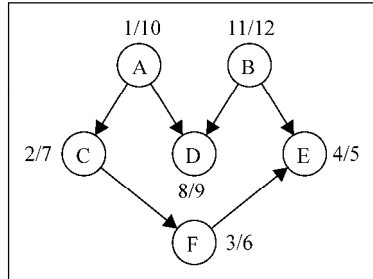
这是一个有向图，意味着任务的执行是有顺序的。例如，任务F不能在任务A之前执行。注意这个图没有环，意味着这是一个无环图。所以，我们可以说该图是一个**有向无环图**（DAG）。

当我们需要编排一些任务或步骤的执行顺序时，这称为**拓扑排序**（topological sorting，英文亦写作topsort或是toposort）。在日常生活中，这个问题在不同情形下都会出现。例如，当我们开始学习一门计算机科学课程，在学习某些知识之前得按顺序完成一些知识储备（你不可以先上算法I前上算法II）。当我们在开发一个项目时，需要按顺序执行一些步骤，例如，首先我们得从客户那里得到需求，接着开发客户要求的东西，最后交付项目。你不能先交付项目再去收集需求。

拓扑排序只能应用于DAG。那么，如何使用深度优先搜索来实现拓扑排序呢？让我们在本节开头的示意图上执行一下深度优先搜索。

```
graph = new Graph();
myVertices = ['A','B','C','D','E','F'];
for (i=0; i<myVertices.length; i++){
    graph.addVertex(myVertices[i]);
}
graph.addEdge('A', 'C');
graph.addEdge('A', 'D');
graph.addEdge('B', 'D');
graph.addEdge('B', 'E');
graph.addEdge('C', 'F');
graph.addEdge('F', 'E');
var result = graph.DFS();
```

这段代码将创建图，添加边，执行改进版本的深度优先搜索算法，并将结果保存到result变量。下图展示了深度优先搜索算法执行后，该图的发现和完成时间。



现在要做的仅仅是以倒序来排序完成时间数组，这便得出了该图的拓扑排序：

B - A - D - C - F - E

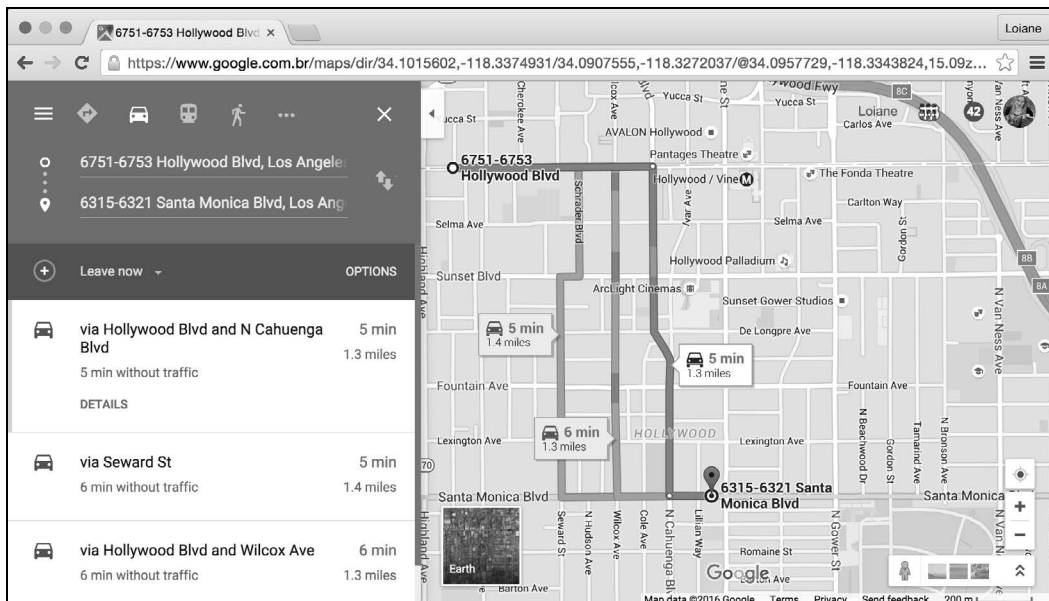
注意之前的拓扑排序结果仅是多种可能性之一。如果我们稍微修改一下算法，就会有不同的结果，比如下面这个结果也是众多其他可能性中的一个：

A - B - C - D - F - E

这也是一个可以接受的结果。

9.5 最短路径算法

设想你要从街道地图上的A点，通过可能的最短路径到达B点。举例来说，从洛杉矶的圣莫尼卡大道到好莱坞大道，如下图所示：



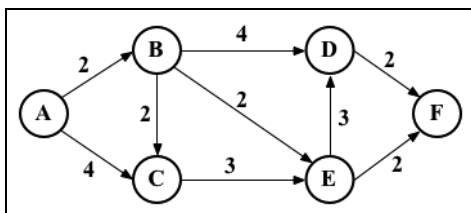
这种问题在生活中非常常见，我们（特别是生活在大城市的人们）会求助于苹果地图、谷歌地图、Waze等应用程序。当然，我们也有其他的考虑，如时间或路况，但根本的问题仍然是：从A到B的最短路径是什么？

我们可以用图来解决这个问题，相应的算法被称为最短路径。本节我们将介绍两种非常著名的算法，即Dijkstra算法和Floyd-Warshall算法。

9.5.1 Dijkstra 算法

Dijkstra算法是一种计算从单个源到所有其他源的最短路径的贪心算法（你可以在第11章了解到更多关于贪心算法的内容），这意味着我们可以用它来计算从图的一个顶点到其余各顶点的最短路径。

考虑下图：



我们来看看如何找到顶点A和其余顶点之间的最短路径。但首先，我们需要声明表示上图的邻接矩阵，如下所示：

```

var graph = [[0, 2, 4, 0, 0, 0],
             [0, 0, 1, 4, 2, 0],
             [0, 0, 0, 0, 3, 0],
             [0, 0, 0, 0, 0, 2],
             [0, 0, 0, 3, 0, 2],
             [0, 0, 0, 0, 0, 0]];

```

现在，通过下面的代码来看看Dijkstra算法是如何工作的：

```

this.dijkstra = function(src) {
  var dist = [], visited = [],
      length = this.graph.length;

  for (var i = 0; i < length; i++) { //{1}
    dist[i] = INF;
    visited[i] = false;
  }
  dist[src] = 0; //{2}

  for (var i = 0; i < length-1; i++) { //{3}
    var u = minDistance(dist, visited); //{4}

```



```

    visited[u] = true; //{5}

    for (var v = 0; v < length; v++) {
        if (!visited[v] &&
            this.graph[u][v] != 0 && dist[u] != INF &&
            dist[u] + this.graph[u][v] < dist[v]) { //{6}
            dist[v] = dist[u] + this.graph[u][v]; //{7}
        }
    }
    return dist; //{8}
};

```

下面是对算法过程的描述。

- 行{1}: 首先,把所有的距离(`dist`)初始化为无限大(JavaScript最大的数`INF = Number.MAX_SAFE_INTEGER`),将`visited[]`初始化为`false`。
- 行{2}: 然后,把源顶点到自己的距离设为0。
- 行{3}: 接下来,要找出到其余顶点的最短路径。
- 行{4}: 为此,我们需要从尚未处理的顶点中选出距离最近的顶点。
- 行{5}: 把选出的顶点标为`visited`,以免重复计算。
- 行{6}: 如果找到更短的路径,则更新最短路径的值(行{7})。
- 行{8}: 处理完所有顶点后,返回从源顶点(`src`)到图中其他顶点最短路径的结果。

要计算顶点间的`minDistance`,就要搜索`dist`数组中的最小值,返回它在数组中的索引:

```

var minDistance = function(dist, visited) {
    var min = INF, minIndex = -1;

    for (var v = 0; v < dist.length; v++) {
        if (visited[v] == false && dist[v] <= min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
};

```

对本节开始的图执行以上算法,会得到如下输出:

```

0    0
1    2
2    3
3    6
4    4
5    6

```



也可以修改算法，将最短路径的值和路径一同返回。

9.5.2 Floyd-Warshall 算法

Floyd-Warshall算法是一种计算图中所有最短路径的**动态规划算法**(你可在第11章了解到更多关于动态规划算法的内容)。通过该算法，我们可以找出从所有源到所有顶点的最短路径。

Floyd-Warshall算法实现如下：

```
this.floydWarshall = function() {
  var dist = [],
      length = this.graph.length,
      i, j, k;

  for (i = 0; i < length; i++) { //{1}
    dist[i] = [];
    for (j = 0; j < length; j++) {
      dist[i][j] = this.graph[i][j];
    }
  }

  for (k = 0; k < length; k++) { //{2}
    for (i = 0; i < length; i++) {
      for (j = 0; j < length; j++) {
        if (dist[i][k] + dist[k][j] < dist[i][j]) { //{3}
          dist[i][j] = dist[i][k] + dist[k][j]; //{4}
        }
      }
    }
  }
  return dist;
};
```

下面是对算法过程的描述。

- 行{1}：首先，把dist数组初始化为每个顶点之间的权值，因为i到j可能的最短距离就是这些顶点间的权值。
- 行{2}：通过k，得到i途径顶点0至k，到达j的最短路径。
- 行{3}：判断i经过顶点k到达j的路径是否比已有的最短路径更短。
- 行{4}：如果是更短的路径，则更新最短路径的值。

行{3}是Floyd-Warshall算法的核心。对本节开始的图执行以上算法，会得到如下输出：

```
0   2   3   6   4   6
INF 0   1   4   2   4
INF INF 0   6   3   5
INF INF INF 0   INF 2
```

```
INF INF INF 3 0 2
INF INF INF INF INF 0
```

其中，INF代表顶点*i*到*j*的最短路径不存在。

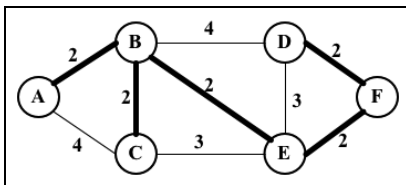
对图中每一个顶点执行Dijkstra算法，也可以得到相同的结果。

9.6 最小生成树

最小生成树（MST）问题是网络设计中常见的问题。想象一下，你的公司有几间办公室，要以最低的成本实现办公室电话线路相互连通，以节省资金，最好的办法是什么？

这也可以应用于岛桥问题。设想你要在*n*个岛屿之间建造桥梁，想用最低的成本实现所有岛屿相互连通。

这两个问题都可以用MST算法来解决，其中的办公室或者岛屿可以表示为图中的一个顶点，边代表成本。这里我们有一个图的例子，其中较粗的边是一个MST的解决方案。



本节我们将学习两种主要的求最小生成树的算法：Prim算法和Kruskal算法。

9.6.1 Prim 算法

Prim算法是一种求解加权无向连通图的MST问题的贪心算法。它能找出一个边的子集，使得其构成的树包含图中所有顶点，且边的权值之和最小。

现在，通过下面的代码来看看Prim算法是如何工作的：

```
this.prim = function() {
  var parent = [],
      key = [],
      visited = [];
  length = this.graph.length,
  i;

  for (i = 0; i < length; i++) { //{1}
    key[i] = INF;
    visited[i] = false;
  }
}
```

```

key[0] = 0; //{2}
parent[0] = -1;

for (i = 0; i < length-1; i++) { //{3}
    var u = minKey(key, visited); //{4}
    visited[u] = true; //{5}

    for (var v = 0; v < length; v++) {
        if (this.graph[u][v] && visited[v] == false
            && this.graph[u][v] < key[v]) { //{6}
            parent[v] = u; //{7}
            key[v] = this.graph[u][v]; //{8}
        }
    }
}
return parent; //{9}
};

```

下面是对算法过程的描述。

- 行{1}: 首先,把所有顶点(key)初始化为无限大(JavaScript最大的数`INF = Number.MAX_SAFE_INTEGER`), `visited[]`初始化为`false`。
- 行{2}: 其次, 选择第一个`key`作为第一个顶点, 同时, 因为第一个顶点总是MST的根节点, 所以`parent[0] = -1`。
- 行{3}: 然后, 对所有顶点求MST。
- 行{4}: 从未处理的顶点集合中选出`key`值最小的顶点(与Dijkstra算法中使用的函数一样, 只是名字不同)。
- 行{5}: 把选出的顶点标为`visited`, 以免重复计算。
- 行{6}: 如果得到更小的权值, 则保存MST路径 (`parent`, 行{7}) 并更新其权值 (行{8})。
- 行{9}: 处理完所有顶点后, 返回包含MST的结果。



比较Prim算法和Dijkstra算法, 我们会发现除了行{7}和行{8}之外, 两者非常相似。行{7}用`parent`数组保存MST的结果。行{8}用`key`数组保存权值最小的边, 而在Dijkstra算法中, 用`dist`数组保存距离。我们可以修改Dijkstra算法, 加入`parent`数组。这样, 就可以在求出距离的同时得到路径。

对如下的图执行以上算法:

```

var graph = [[0, 2, 4, 0, 0, 0],
             [2, 0, 2, 4, 2, 0],
             [4, 2, 0, 0, 3, 0],
             [0, 4, 0, 0, 3, 2],
             [0, 2, 3, 3, 0, 2],
             [0, 0, 0, 2, 2, 0]];

```

我们会得到如下输出：

Edge	Weight
0 - 1	2
1 - 2	2
5 - 3	2
1 - 4	2
4 - 5	2

9.6.2 Kruskal 算法

和Prim算法类似，Kruskal算法也是一种求加权无向连通图的MST的贪心算法。

现在，通过下面的代码来看看Kruskal算法是如何工作的：

```
this.kruskal = function() {
    var length = this.graph.length,
        parent = [], cost,
        ne = 0, a, b, u, v, i, j, min;
    cost = initializeCost(); //{1}

    while (ne < length-1) { //{2}
        for (i = 0, min = INF; i < length; i++) { //{3}
            for (j = 0; j < length; j++) {
                if (cost[i][j] < min) {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }

        u = find(u, parent); //{4}
        v = find(v, parent); //{5}

        if (union(u, v, parent)) { //{6}
            ne++;
        }

        cost[u][v] = cost[v][u] = INF; //{7}
    }
    return parent;
}
```

下面是对算法过程的描述。

- 行{1}：首先，把邻接矩阵的值复制到cost数组，以方便修改且可以保留原始值行{7}。
- 行{2}：当MST的边数小于顶点总数减1时。
- 行{3}：找出权值最小的边。
- 行{4}和行{5}：检查MST中是否已存在这条边，以避免环路。

- 行{6}: 如果 u 和 v 是不同的边, 则将其加入MST。
- 行{7}: 从列表中移除这些边, 以免重复计算。
- 行{8}: 返回MST。

下面是find函数的定义。它能防止MST出现环路:

```
var find = function(i, parent) {  
    while (parent[i]) {  
        i = parent[i];  
    }  
    return i;  
};
```

union函数的定义如下:

```
var union = function(i, j, parent) {  
    if (i !== j) {  
        parent[j] = i;  
        return true;  
    }  
    return false;  
};
```

这个算法有几种变体。这取决于对边的权值排序时所使用的数据结构 (如优先队列), 以及图是如何表示的。

9.7 小结

本章涵盖了图的基本概念。我们学习了几种不同的方式来表示这一数据结构, 并实现了用邻接表表示图的算法。你还学到了如何用广度优先搜索和深度优先搜索来遍历图。本章还包括了广度优先搜索和深度优先搜索的两个实际应用, 它们分别是使用广度优先搜索来找到最短路径, 以及使用深度优先搜索来做拓扑排序。

本章还介绍了一些著名的算法, 如计算最短路径的Dijkstra算法和Floyd-Warshall算法, 以及计算图的最小生成树的Prim算法和Kruskal算法。

下一章, 我们将会学习计算机科学中最常用的排序算法。

假设我们有一个没有任何排列顺序的电话号码表（或号码簿）。当需要添加联络人和电话时，你只能将其写在下一个空位上。假定你的联系人列表上有很多人，某天，你要找某个联系人及其电话号码。但是由于联系人列表没有按照任何顺序来组织，你只能逐个检查，直到找到那个你想要的联系人为止。这个方法太吓人了，难道你不这么认为？想象一下你要在黄页上搜寻一个联系人，但是那本黄页没有进行任何组织，那得花多久时间啊？！

因此（还有其他原因），我们需要组织信息集，比如那些存储在数据结构里的信息。排序和搜索算法广泛地运用在待解决的日常问题中。

本章，你会学到最常用的排序和搜索算法，如冒泡排序、选择排序、插入排序、归并排序、快速排序和堆排序，以及顺序搜索和二分搜索算法。

10.1 排序算法

从上面的引言中，你应该理解，对给定信息得先排序再搜索。本节会介绍一些在计算机科学中最著名的排序算法。我们会从最慢的一个开始，接着是一些性能较好的算法。

在开始排序算法之前，我们先创建一个数组（列表）来表示待排序和搜索的数据结构。

```
function ArrayList(){  
  
    var array = []; //{1}  
  
    this.insert = function(item){ //{2}  
        array.push(item);  
    };  
  
    this.toString= function(){ //{3}  
        return array.join();  
    };  
}
```

如你所见，ArrayList是一个简单的数据结构，它将项存储在数组中（行{1}）。我们只需要一个插入方法来向数据结构中添加元素（行{2}），用第2章中介绍的JavaScript Array类原生的

push方法即可。最后,为了帮助我们验证结果,toString方法使用JavaScript原生Array类的join方法,来拼接数组中的所有元素至一个单一的字符串,这样我们就可以轻松地在浏览器的控制台输出结果了。



join方法拼接数组元素至一个字符串,并返回该字符串。

注意ArrayList类并没有任何方法来移除数据或插入数据到特定位置。我们刻意保持简单是为了能够专注于排序和搜索算法。所有的排序和搜索算法会添加至这个类中。

我们现在开始吧!

10.1.1 冒泡排序

人们开始学习排序算法时,通常都先学冒泡算法,因为它在所有排序算法中最简单。然而,从运行时间的角度来看,冒泡排序是最差的一个,接下来你会知晓原因。

冒泡排序比较任何两个相邻的项,如果第一个比第二个大,则交换它们。元素项向上移动至正确的顺序,就好像气泡升至表面一样,冒泡排序因此得名。

让我们来实现一下冒泡排序:

```
this.bubbleSort = function(){
    var length = array.length;           //{1}
    for (var i=0; i<length; i++){         //{2}
        for (var j=0; j<length-1; j++ ){ //{3}
            if (array[j] > array[j+1]){    //{4}
                swap(array, j, j+1);      //{5}
            }
        }
    }
};
```

首先,声明一个名为length的变量,用来存储数组的长度(行{1})。这一步可选,它能帮助我们在行{2}和行{3}时直接使用数组的长度。接着,外循环(行{2})会从数组的第一位迭代至最后一位,它控制了数组中经过多少轮排序(应该是数组中每项都经过一轮,轮数和数组长度一致)。然后,内循环将从第一位迭代至倒数第二位,内循环实际上进行当前项和下一项的比较(行{4})。如果这两项顺序不对(当前项比下一项大),则交换它们(行{5}),意思是位置为j+1的值将会被换置到位置j处,反之亦然。

现在我们需要声明swap函数(一个私有函数,只能用在ArrayList类的内部代码中):

```
var swap = function(array, index1, index2){
    var aux = array[index1];
```



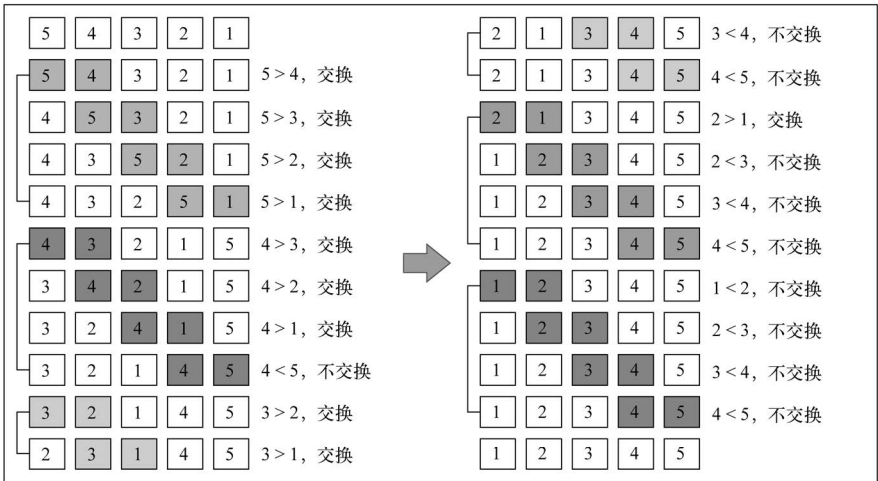
```
array[index1] = array[index2];
array[index2] = aux;
};
```

交换时，我们用一个中间值来存储某一交换项的值。其他排序法也会用到这个方法，因此我们声明一个方法放置这段交换代码以便重用。

如果使用在第1章学过的ES6（ECMAScript 2015）增强的对象属性，这个函数可以写成下面这样：

```
[array[index1], array[index2]] = [array[index2], array[index1]];
```

下面这个示意图展示了冒泡排序的工作过程：



该示意图中每一小段表示外循环的一轮（行{2}），而相邻两项的比较则是在内循环中进行的（行{3}）。

我们将使用下面这段代码来测试冒泡排序算法，看结果是否和示意图所示一致：

```
function createNonSortedArray(size){ //{6}
  var array = new ArrayList();
  for (var i = size; i > 0; i--){
    array.insert(i);
  }
  return array;
}

var array = createNonSortedArray(5); //{7}
console.log(array.toString());        //{8}
array.bubbleSort();                    //{9}
console.log(array.toString());        //{10}
```

为了辅助测试本章将要学习的排序算法，我们将创建一个函数来自动地创建一个未排序的数

组，数组的长度由函数参数指定（行{6}）。如果传递5作为参数，该函数会创建如下数组：[5, 4, 3, 2, 1]。调用这个函数并将返回值存储在一个变量中，该变量将包含这个以某些数字来初始化的ArrayList类实例（行{7}）。我们在控制台上输出这个数组内容，确保这是一个未排序数组（行{8}），接着我们调用冒泡排序方法（行{9}）并再次在控制台上输出数组内容以验证数组已被排序了（行{10}）。



你可以从书本的支持页面（或GitHub仓库）所下载的源码中找到完整的ArrayList源码和测试代码（带有补充注释的）。

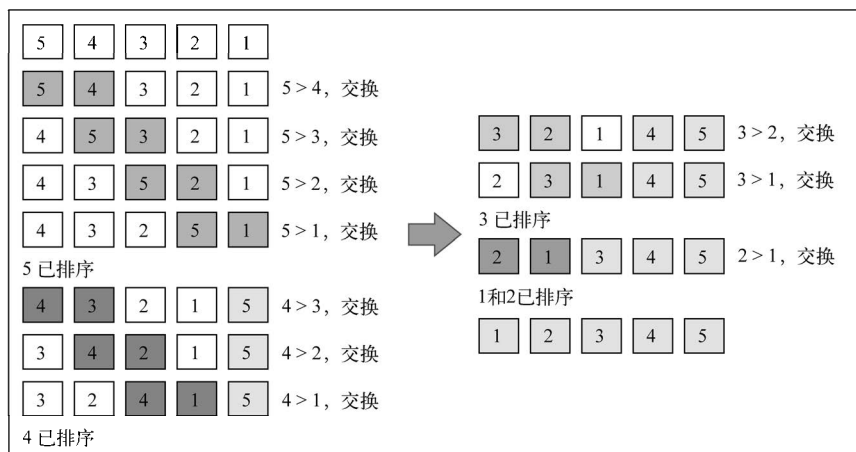
注意当算法执行外循环的第二轮的时候，数字4和5已经是正确排序的了。尽管如此，在后续比较中，它们还一直在进行比较，即使这是不必要的。因此，我们可以稍稍改进一下冒泡排序算法。

改进后的冒泡排序

如果从内循环减去外循环中已跑过的轮数，就可以避免内循环中所有不必要的比较（行{1}）。

```
this.modifiedBubbleSort = function(){
    var length = array.length;
    for (var i=0; i<length; i++){
        for (var j=0; j<length-1-i; j++ ){ //{1}
            if (array[j] > array[j+1]){
                swap(j, j+1);
            }
        }
    }
};
```

下面这个示意图展示了改进后的冒泡排序算法是如何执行的：





注意已经在正确位置上的数字没有被比较。即便我们做了这个小改变，改进了一下冒泡排序算法，但我们还是不推荐该算法，它的复杂度是 $O(n^2)$ 。

我们将在第12章详细介绍大 O 表示法，对算法做更多的讨论。

10.1.2 选择排序

选择排序算法是一种原址比较排序算法。选择排序大致的思路是找到数据结构中的最小值并将其放置在第一位，接着找到第二小的值并将其放在第二位，以此类推。

下面是选择排序算法的源代码：

```
this.selectionSort = function(){
  var length = array.length,           //{1}
      indexMin;
  for (var i=0; i<length-1; i++){       //{2}
    indexMin = i;                       //{3}
    for (var j=i; j<length; j++){       //{4}
      if(array[indexMin]>array[j]){      //{5}
        indexMin = j;                  //{6}
      }
    }
    if (i !== indexMin){                 //{7}
      swap(i, indexMin);
    }
  }
};
```

首先声明一些将在算法内使用的变量（行{1}）。接着，外循环（行{2}）迭代数组，并控制迭代轮次（数组的第 n 个值——下一个最小值）。我们假设本迭代轮次的第一个值为数组最小值（行{3}）。然后，从当前 i 的值开始至数组结束（行{4}），我们比较是否位置 j 的值比当前最小值小（行{5}）；如果是，则改变最小值至新最小值（行{6}）。当内循环结束（行{4}），将得出数组第 n 小的值。最后，如果该最小值和原最小值不同（行{7}），则交换其值。

用以下代码段来测试选择排序算法：

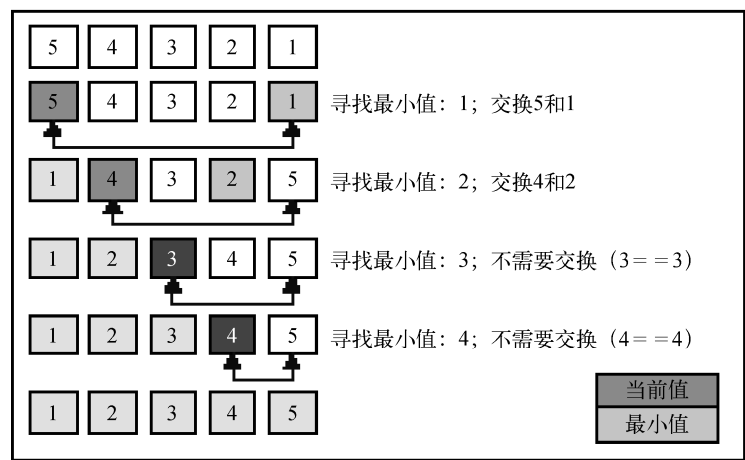
```
array = createNonSortedArray(5);
console.log(array.toString());
array.selectionSort();
console.log(array.toString());
```

下面的示意图展示了选择排序算法，此例基于之前代码中所用的数组。

数组底部的箭头指示出当前迭代轮寻找最小值的数组范围（内循环{4}），示意图中的每一步则表示外循环。

选择排序同样也是一个复杂度为 $O(n^2)$ 的算法。和冒泡排序一样，它包含有嵌套的两个循环，

这导致了二次方的复杂度。然而，接下来要学的插入排序比选择排序性能要好。



10.1.3 插入排序

插入排序每次排一个数组项，以此方式构建最后的排序数组。假定第一项已经排序了，接着，它和第二项进行比较，第二项是应该待在原位还是插到第一项之前呢？这样，头两项就已正确排序，接着和第三项比较（它是该插入到第一、第二还是第三的位置呢？），以此类推。

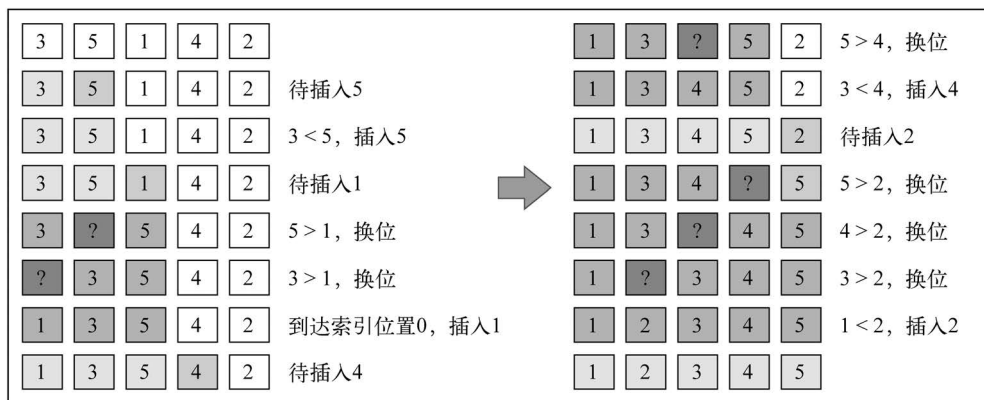
下面这段代码表示插入排序算法：

```
this.insertionSort = function(){
  var length = array.length,           //{1}
      j, temp;
  for (var i=1; i<length; i++){          //{2}
    j = i;                                //{3}
    temp = array[i];                      //{4}
    while (j>0 && array[j-1] > temp){    //{5}
      array[j] = array[j-1];             //{6}
      j--;
    }
    array[j] = temp;                      //{7}
  }
};
```

照例，算法的第一行用来声明代码中使用的变量（行{1}）。接着，迭代数组来给第*i*项找到正确的位置（行{2}）。注意，算法是从第二个位置（索引1）而不是0位置开始的（我们认为第一项已排序了）。然后，用*i*的值来初始化一个辅助变量（行{3}）并将其值亦存储于一临时变量中（行{4}），便于之后将其插入到正确的位置上。下一步是要找到正确的位置来插入项目。只要变量*j*比0大（因为数组的第一个索引是0——没有负值的索引）并且数组中前面的值比待比较的值大（行{5}），我们就把这个值移到当前位置上（行{6}）并减小*j*。最终，该项目能插入到

正确的位置上。

下面的示意图展示了一个插入排序的实例：



举个例子，假定待排序数组是[3, 5, 1, 4, 2]。这些值将被插入排序算法按照下面形容的步骤进行排序。

(1) 3已被排序，所以我们从数组第二个值5开始。3比5小，所以5待在原位（数组的第二位）。3和5排序完毕。

(2) 下一个待排序和插到正确位置上去的值是1（目前在数组的第三位）。5比1大，所以5被移至第三位去了。我们得分析1是否应该被插入到第二位——1比3大吗？不，所以3被移到第二位去了。接着，我们得证明1应该插入到数组的第一位上。因为0是第一个位置且没有负数位，所以1必须被插入到第一位。1、3、5三个数字已经排序。

(3) 4应该在当前位置（索引3）还是要移动到索引较低的位置上去呢？4比5小，所以5移动到索引3位置上去。那么应该把4插到索引2的位置上去吗？4要比3大，所以4插入到数组的位置3上。

(4) 下一个待插入的数字是2（数组的位置4）。5比2大，所以5移动至索引4。4比2大，所以4也得移动（位置3）。3也比2大，所以3还得移动。1比2小，所以2插入到数组的第二位置上。至此，数组已排序完成。

排序小型数组时，此算法比选择排序和冒泡排序性能要好。

10.1.4 归并排序

归并排序是第一个可以被实际使用的排序算法。你在本书中学到的前三个排序算法性能不好，但归并排序性能不错，其复杂度为 $O(n\log n)$ 。



JavaScript的Array类定义了一个sort函数(Array.prototype.sort)用以排序JavaScript数组(我们不必自己实现这个算法)。ECMAScript没有定义用哪个排序算法,所以浏览器厂商可以自行去实现算法。例如,Mozilla Firefox使用归并排序作为Array.prototype.sort的实现,而Chrome使用了一个快速排序(下面我们会学习的)的变体。

归并排序是一种分治算法。其思想是将原始数组切分成较小的数组,直到每个小数组只有一个位置,接着将小数组归并成较大的数组,直到最后只有一个排序完毕的大数组。

由于是分治法,归并排序也是递归的:

```
this.mergeSort = function(){
    array = mergeSortRec(array);
};
```

像之前的章节一样,每当要实现一个递归函数,我们都会实现一个实际被执行的辅助函数。对归并排序我们也会这么做。我们将声明mergeSort方法以供随后使用,而mergeSort方法将会调用mergeSortRec,该函数是一个递归函数:

```
var mergeSortRec = function(array){
    var length = array.length;
    if(length === 1) {           //{1}
        return array;           //{2}
    }
    var mid = Math.floor(length / 2),    //{3}
        left = array.slice(0, mid),      //{4}
        right = array.slice(mid, length); //{5}

    return merge(mergeSortRec(left), mergeSortRec(right)); //{6}
};
```

归并排序将一个大数据组转化为多个小数组直到只有一个项。由于算法是递归的,我们需要一个停止条件,在这里此条件是判断数组的长度是否为1(行{1})。如果是,则直接返回这个长度为1的数组(行{2}),因为它已排序了。

如果数组长度比1大,那么我们得将其分成小数组。为此,首先得找到数组的中间位(行{3}),找到后我们将数组分成两个小数组,分别叫作left(行{4})和right(行{5})。left数组由索引0至中间索引的元素组成,而right数组由中间索引至原始数组最后一个位置的元素组成。

下面的步骤是调用merge函数(行{6}),它负责合并和排序小数组来产生大数据组,直到回到原始数组并已排序完成。为了不断将原始数组分成小数组,我们得再次对left数组和right数组递归调用mergeSortRec,并同时作为参数传递给merge函数。

```
var merge = function(left, right){
    var result = [], //{7}
        il = 0,
        ir = 0;
```

```

while(il < left.length && ir < right.length) { // {8}
    if(left[il] < right[ir]) {
        result.push(left[il++]); // {9}
    } else{
        result.push(right[ir++]); // {10}
    }
}

while (il < left.length){ // {11}
    result.push(left[il++]);
}

while (ir < right.length){ // {12}
    result.push(right[ir++]);
}

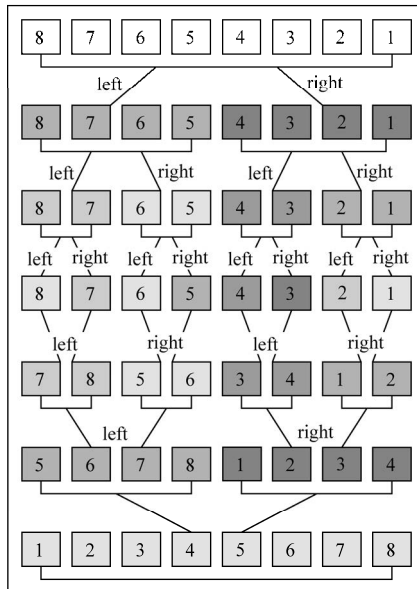
return result; // {13}
};

```

merge函数接受两个数组作为参数，并将它们归并至一个大数组。排序发生在归并过程中。首先，需要声明归并过程要创建的新数组以及用来迭代两个数组（left和right数组）所需的两个变量（行{7}）。迭代两个数组的过程中（行{8}），我们比较来自left数组的项是否比来自right数组的项小。如果是，将该项从left数组添加至归并结果数组，并递增迭代数组的控制变量（行{9}）；否则，从right数组添加项并递增相应的迭代数组的控制变量（行{10}）。

接下来，将left数组或者right数组所有剩余的项添加到归并数组中（行{11}和行{12}）。最后，将归并数组作为结果返回（行{13}）。

如果执行mergeSort函数，下图是具体的执行过程：



可以看到，算法首先将原始数组分割直至只有一个元素的子数组，然后开始归并。归并过程也会完成排序，直至原始数组完全合并并完成排序。

10.1.5 快速排序

快速排序也许是最常用的排序算法了。它的复杂度为 $O(n\log^n)$ ，且它的性能通常比其他的复杂度为 $O(n\log^n)$ 的排序算法要好。和归并排序一样，快速排序也使用分治的方法，将原始数组分为较小的数组（但它没有像归并排序那样将它们分割开）。

快速排序比到目前为止你学过的其他排序算法要复杂一些。让我们一步步地来学习。

(1) 首先，从数组中选择中间一项作为主元。

(2) 创建两个指针，左边一个指向数组第一个项，右边一个指向数组最后一个项。移动左指针直到我们找到一个比主元大的元素，接着，移动右指针直到找到一个比主元小的元素，然后交换它们，重复这个过程，直到左指针超过了右指针。这个过程将使得比主元小的值都排在主元之前，而比主元大的值都排在主元之后。这一步叫作划分操作。

(3) 接着，算法对划分后的小数组（较主元小的值组成的子数组，以及较主元大的值组成的子数组）重复之前的两个步骤，直至数组已完全排序。

让我们开始快速排序的实现吧：

```
this.quickSort = function(){
  quick(array, 0, array.length - 1);
};
```

就像归并算法那样，开始我们声明一个主方法来调用递归函数，传递待排序数组，以及索引0及其最末的位置（因为我们要排整个数组，而不是一个子数组）作为参数。

```
var quick = function(array, left, right){

  var index; //{1}

  if (array.length > 1) { //{2}

    index = partition(array, left, right); //{3}

    if (left < index - 1) {                //{4}
      quick(array, left, index - 1);      //{5}
    }

    if (index < right) { //{6}
      quick(array, index, right);        //{7}
    }
  }
};
```

首先声明`index`（行{1}），该变量能帮助我们z子数组分离为较小值数组和较大值数组，这

样，我们就能再次递归的调用quick函数了。partition函数返回值将赋值给index（行{3}）。

如果数组的长度比1大（因为只有一个元素的数组必然是已排序了的（行{2}），我们将对给定子数组执行partition操作（第一次调用是针对整个数组）以得到index（行{3}）。如果子数组存在较小值的元素（行{4}），则对该数组重复这个过程（行{5}）。同理，对存在较大值得子数组也是如此，如果存在子数组存在较大值，我们也将重复快速排序过程（行{7}）。

1. 划分过程

第一件要做的事情是选择主元（pivot），有好几种方式。最简单的一种是选择数组的第一项（最左项）。然而，研究表明对于几乎已排序的数组，这不是一个好的选择，它将导致该算法的最差表现。另外一种方式是随机选择一个数组项或是选择中间项。

现在，让我们看看划分过程：

```
var partition = function(array, left, right) {

    var pivot = array[Math.floor((right + left) / 2)], //{8}
        i = left,                                //{9}
        j = right;                                //{10}

    while (i <= j) {                               //{11}
        while (array[i] < pivot) { //{12}
            i++;
        }
        while (array[j] > pivot) { //{13}
            j--;
        }
        if (i <= j) { //{14}
            swap(array, i, j); //{15}
            i++;
            j--;
        }
    }
    return i; //{16}
};
```

在本实现中，我们选择中间项作为主元（行{8}）。我们初始化两个指针：left（低——行{9}），初始化为数组第一个元素；right（高——行{10}），初始化为数组最后一个元素。

只要left和right指针没有相互交错（行{11}），就执行划分操作。首先，移动left指针直到找到一个元素比主元大（行{12}）。对right指针，我们做同样的事情，移动right指针直到我们找到一个元素比主元小。

当左指针指向的元素比主元大且右指针指向的元素比主元小，并且此时左指针索引没有右指针索引大（行{14}），意思是左项比右项大（值比较）。我们交换它们，然后移动两个指针，并重复此过程（从行{11}再次开始）。

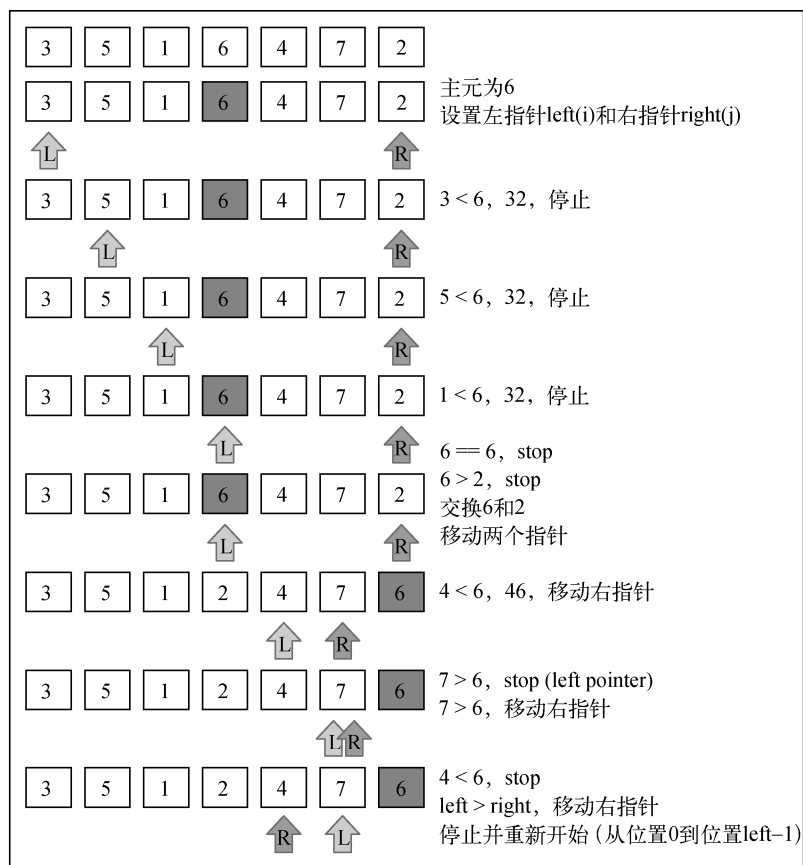
在划分操作结束后，返回左指针的索引，用来在行{3}处创建子数组。

swap函数和我们在本章开始为冒泡排序算法实现的相同。我们也可以将此函数替换为以下ES6代码。

```
[array[index1], array[index2]] = [array[index2], array[index1]];
```

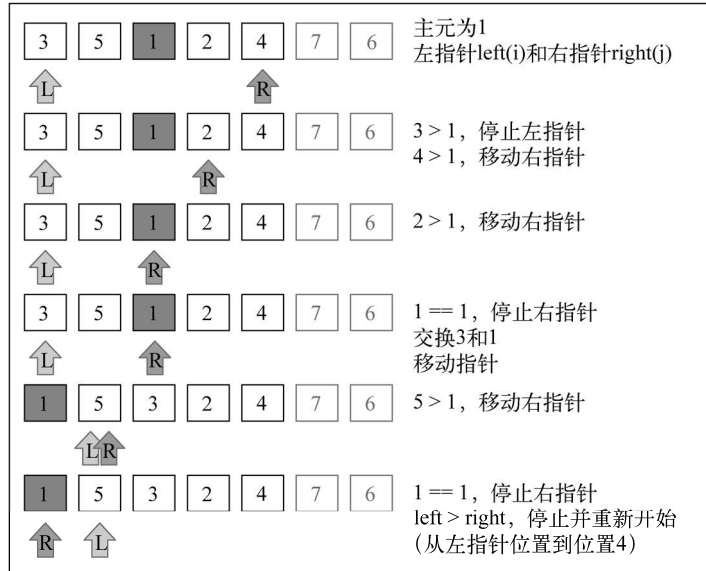
2. 快速排序实战

让我来一步步地看一个快速排序的实际例子：

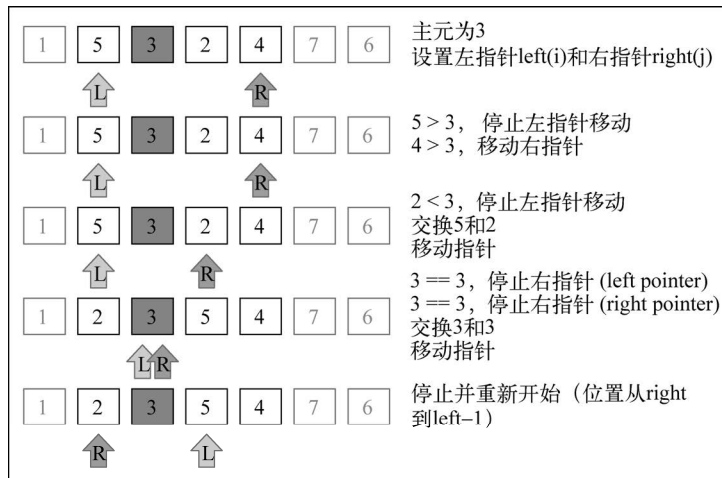


给定数组 [3, 5, 1, 6, 4, 7, 2]，前面的示意图展示了划分操作的第一次执行。

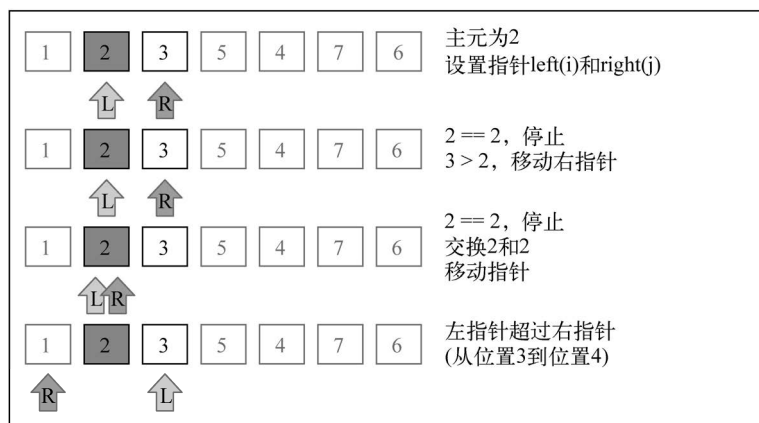
下面的示意图展示了对有较小值的子数组执行的划分操作（注意7和6不包含在子数组之内）：



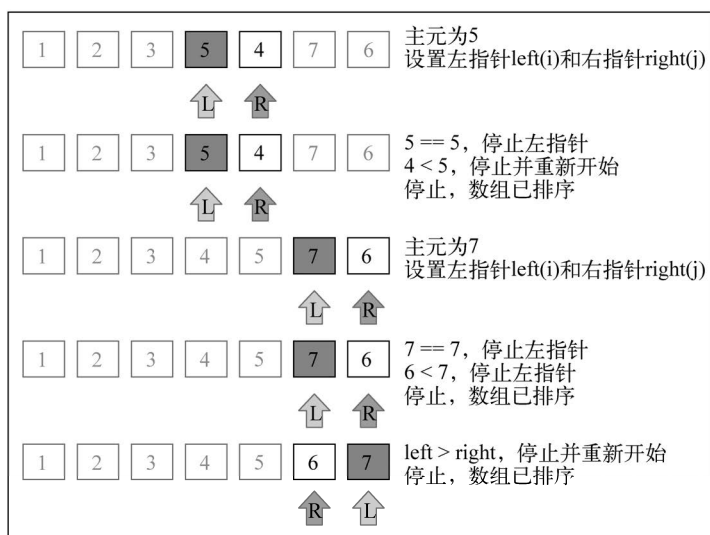
接着，我们继续创建子数组，请看下图，但是这次操作是针对上图中有较大值的子数组（有1的那个较小子数组不用再划分了，因为它仅含有一个项）。



子数组 $[2, 3, 5, 4]$ 中的较小子数组 $[2, 3]$ 继续进行划分（算法代码中的行{5}）：



然后子数组 $[2, 3, 5, 4]$ 中的较大子数组 $[5, 4]$ 也继续进行划分（算法中的行 {7}），示意图如下：



最终，较大子数组 $[6, 7]$ 也会进行划分（partition）操作，快速排序算法的操作执行完成。

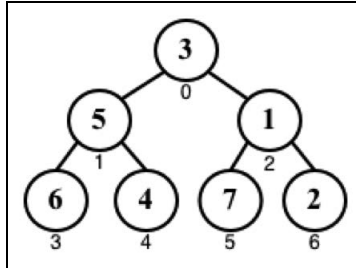
10.1.6 堆排序

堆排序也是一种很高效的算法，因其把数组当作二叉树来排序而得名。这个算法会根据以下信息，把数组当作二叉树来管理。

- ❑ 索引0是树的根节点；
- ❑ 除根节点外，任意节点N的父节点是 $N/2$ ；

- 节点L的左子节点是 $2*L$;
- 节点R的右子节点是 $2*R+1$ 。

举例来说,可以将数组[3, 5, 1, 6, 4, 7, 2]想象成下面的树:



堆排序算法实现如下:

```

this.heapSort = function() {
    var heapSize = array.length;
    buildHeap(array); //{1}

    while (heapSize > 1) {
        heapSize--;
        swap(array, 0, heapSize); //{2}
        heapify(array, heapSize, 0); //{3}
    }
};
  
```

第一步,构造一个满足 $\text{array}[\text{parent}(i)] \geq \text{array}[i]$ 的堆结构(行{1})。

第二步,交换堆里第一个元素(数组中较大的值)和最后一个元素的位置(行{2})。这样,最大的值就会出现在它已排序的位置。

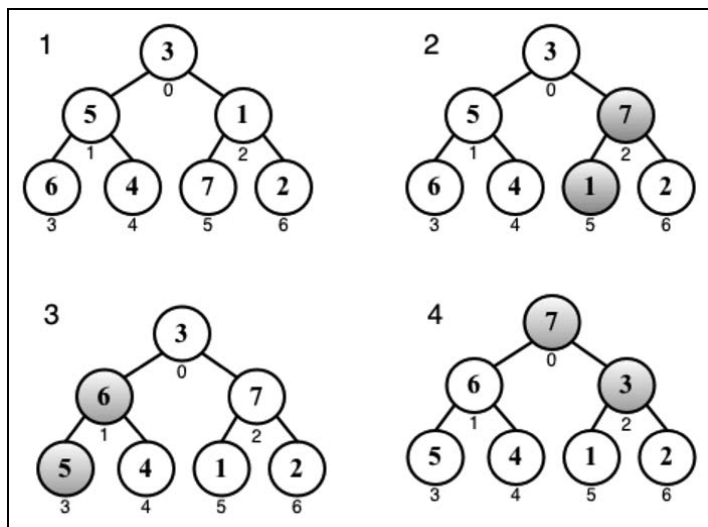
第二步可能会丢掉堆的属性。因此,我们还需要执行一个heapify函数,再次将数组转换成堆,也就是说,它会找到当前堆的根节点(较小的值),重新放到树的底部。

buildHeap函数实现如下:

```

var buildHeap = function(array){
    var heapSize = array.length;
    for (var i = Math.floor(array.length / 2); i >= 0; i--) {
        heapify(array, heapSize, i);
    }
};
  
```

如果对数组[3, 5, 1, 6, 4, 7, 2]调用buildHeap函数,堆的构建过程如下:



最后，heapify函数实现如下：

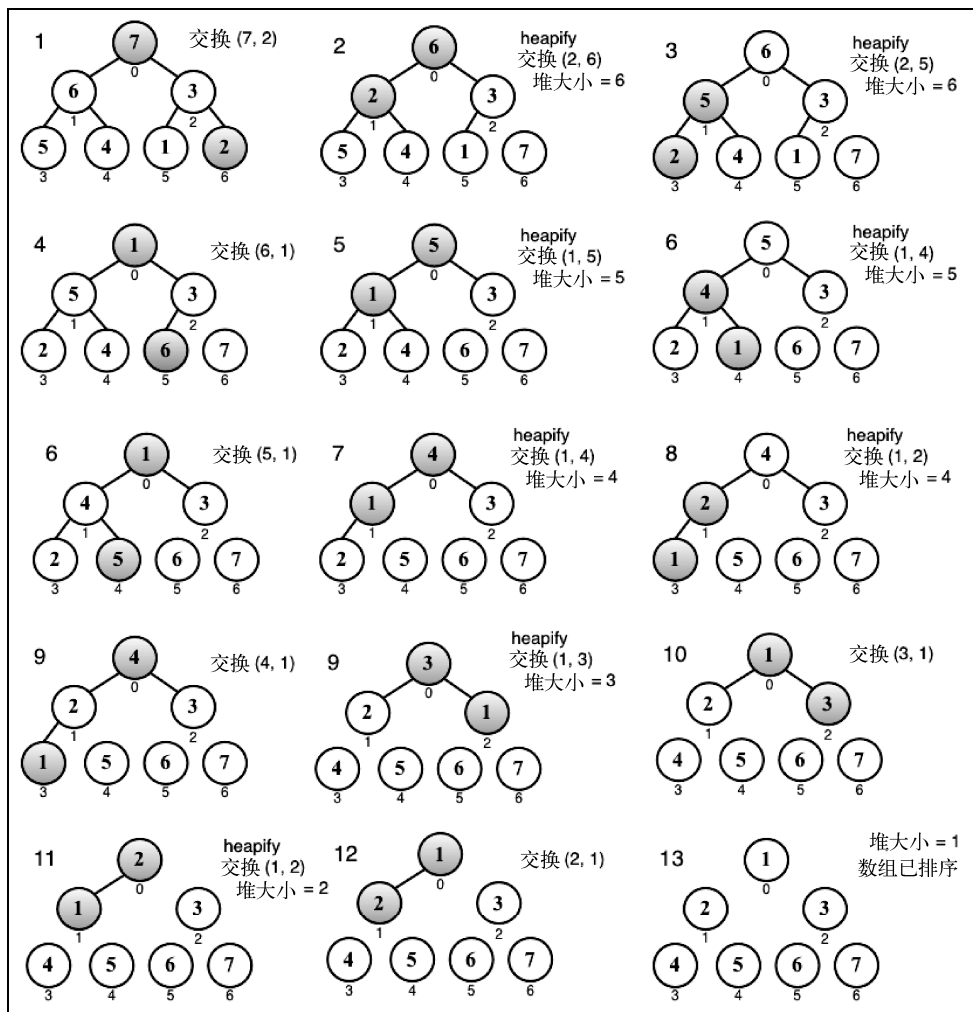
```
var heapify = function(array, heapSize, i){
    var left = i * 2 + 1,
        right = i * 2 + 2,
        largest = i;

    if (left < heapSize && array[left] > array[largest]) {
        largest = left;
    }

    if (right < heapSize && array[right] > array[largest]) {
        largest = right;
    }

    if (largest !== i) {
        swap(array, i, largest);
        heapify(array, heapSize, largest);
    }
};
```

堆构造好之后，就可以应用堆排序的算法了，也就是行{2}和行{3}。



10.1.7 计数排序、桶排序和基数排序（分布式排序）

到目前为止，你已经学习了如何在不借助任何辅助数据结构的情况下对数组进行排序。还有一类被称为分布式排序的算法，原始数组中的数据会分发到多个中间结构（桶），再合起来放回原始数组。

最著名的分布式算法有计数排序、桶排序和基数排序。这三种算法非常相似。



本书不打算介绍这几种算法。不过，你可以在本书源码包或GitHub项目 <https://github.com/loiane/javascript-datastructures-algorithms> 中找到算法的例子。

10.2 搜索算法

现在,让我们来谈谈搜索算法。回顾一下之前章节所实现的算法,我们会发现BinarySearchTree类的search方法(第8章),以及LinkedList类的indexOf方法(第5章)等,都是搜索算法,当然,它们每一个都是根据其各自的数据结构来实现的。所以,我们已经熟悉两个搜索算法了,只是还不知道它们“正式”的名称而已。

10.2.1 顺序搜索

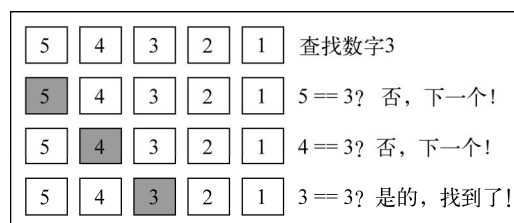
顺序或线性搜索是最基本的搜索算法。它的机制是,将每一个数据结构中的元素和我们要找的元素做比较。顺序搜索是最低效的一种搜索算法。

以下是其实现:

```
this.sequentialSearch = function(item){
  for (var i=0; i<array.length; i++){ //{1}
    if (item === array[i])           //{2}
      return i;                     //{3}
    }
  }
  return -1; //{4}
};
```

顺序搜索迭代整个数组(行{1}),并将每个数组元素和搜索项作比较(行{2})。如果搜索到了,算法将用返回值来标示搜索成功。返回值可以是该搜索项本身,或是true,又或是搜索项的索引(行{3})。如果没有找到该项,则返回-1(行{4}),表示该索引不存在;也可以考虑返回false或者null。

假定有数组[5, 4, 3, 2, 1]和待搜索值3,下图展示了顺序搜索的示意图:



10.2.2 二分搜索

二分搜索算法的原理和猜数字游戏类似,就是那个有人说“我正想着一个1到100的数字”的游戏。我们每回应一个数字,那个人就会说这个数字是高了、低了还是对了。

这个算法要求被搜索的数据结构已排序。以下是该算法遵循的步骤。

- (1) 选择数组的中间值。
- (2) 如果选中值是待搜索值，那么算法执行完毕（值找到了）。
- (3) 如果待搜索值比选中值要小，则返回步骤1并在选中值左边的子数组中寻找。
- (4) 如果待搜索值比选中值要大，则返回步骤1并在选中值右边的子数组中寻找。

以下是其实现：

```
this.binarySearch = function(item){
    this.quickSort(); //{1}

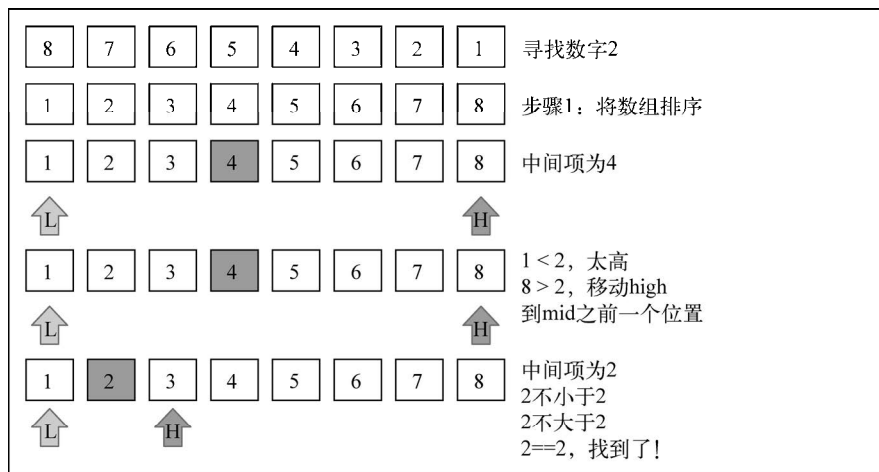
    var low = 0,                //{2}
        high = array.length - 1, //{3}
        mid, element;

    while (low <= high){ //{4}
        mid = Math.floor((low + high) / 2); //{5}
        element = array[mid];                //{6}
        if (element < item) {                //{7}
            low = mid + 1;                    //{8}
        } else if (element > item) {          //{9}
            high = mid - 1;                   //{10}
        } else {
            return mid;                       //{11}
        }
    }
    return -1; //{12}
};
```

开始前需要先将数组排序，我们可以选择任何一个在10.1节中实现的排序算法。这里我们选择了快速排序。在数组排序之后，我们设置low（行{2}）和high（行{3}）指针（它们是边界）。

当low比high小时（行{4}），我们计算得到中间项索引并取得中间项的值，此处如果low比high大，则意思是该待搜索值不存在并返回-1（行{12}）。接着，我们比较选中项的值和搜索值（行{7}）。如果小了，则选择数组低半边并重新开始。如果选中项的值比搜索值大了，则选择数组高半边并重新开始。若两者都不是，则意味着选中项的值和搜索值相等，因此，直接返回该索引（行{11}）。

给定下图所示数组，让我们试试看搜索2。这些是算法将会执行的步骤：



第8章中，我们实现的BinarySearchTree类有一个search方法，和这个二分搜索完全一样，只不过它是针对树数据结构的。

10.3 小结

本章介绍了排序和搜索算法。你学到了冒泡、选择、插入、归并、快速以及堆排序算法，它们是用来排序数据结构的。你还学到了顺序搜索和二分搜索（需要数据结构已排序）。

本章中你所掌握的方法可以运用到任何数据结构或数据类型上，你只需在源代码上做一些必要的修改即可。

下一章，我们要学习一些高级算法技巧。

到现在为止，我们愉快地学习了不同的数据结构的实现，其中包括常用的排序和搜索算法。算法和编程的世界很有意思。本章，你会进一步了解这个世界，并且我们将探讨进一步深入其中的途径（如果你感兴趣的话）。

我们将介绍第8章中提到过的递归，还将学习动态规划和贪心算法。此外，我们还会介绍一些著名的问题。

11.1 递归

递归是一种解决问题的方法，它解决问题的各个小部分，直到解决最初的大问题。递归通常涉及函数调用自身。

递归函数是像下面这样能够直接调用自身的方法或函数：

```
function recursiveFunction(someParam) {  
    recursiveFunction(someParam);  
};
```

能够像下面这样间接调用自身的函数，也是递归函数：

```
function recursiveFunction1(someParam) {  
    recursiveFunction2(someParam);  
};  
  
function recursiveFunction2(someParam) {  
    recursiveFunction1(someParam);  
};
```

假设现在必须要执行`recursiveFunction`，结果是什么？单单就上述情况而言，它会一直执行下去。因此，每个递归函数都必须要有边界条件，即一个不再递归调用的条件（停止点），以防止无限递归。

11.1.1 JavaScript 调用栈大小的限制

如果忘记加上用以停止函数递归调用的边界条件,会发生什么呢?递归并不会无限地执行下去;浏览器会抛出错误,也就是所谓的栈溢出错误 (stack overflow error)。

每个浏览器都有自己的上限,可用以下代码测试:

```
var i = 0;

function recursiveFn () {
    i++;
    recursiveFn();
}

try {
    recursiveFn();
} catch (ex) {
    alert('i = ' + i + ' error: ' + ex);
}
```

在Chrome v37中,这个函数执行了20 955次,而后浏览器抛出错误RangeError: Maximum call stack size exceeded (超限错误: 超过最大调用栈大小)。在Firefox v27中,函数执行了343 429次,然后浏览器抛出错误 InternalError: too much recursion (内部错误: 递归次数过多)。



根据操作系统和浏览器的不同,具体数值会所有不同,但区别不大。

ECMAScript 6有尾调用优化 (tail call optimization)。如果函数内最后一个操作是调用函数(就像示例中加粗的那行),会通过“跳转指令”(jump)而不是“子程序调用”(subroutine call)来控制。也就是说,在ECMAScript 6中,这里的代码可以一直执行下去。所以,具有停止递归的边界条件非常重要。



有关尾调用优化的更多相关信息,请访问<http://goo.gl/ZdTZzg>。

11.1.2 斐波那契数列

回到第10章提到的斐波那契问题。斐波那契数列的定义如下:

- 1和2的斐波那契数是 1;
- n ($n > 2$) 的斐波那契数是 $(n-1)$ 的斐波那契数加上 $(n-2)$ 的斐波那契数。

那么,让我们开始实现斐波那契函数:

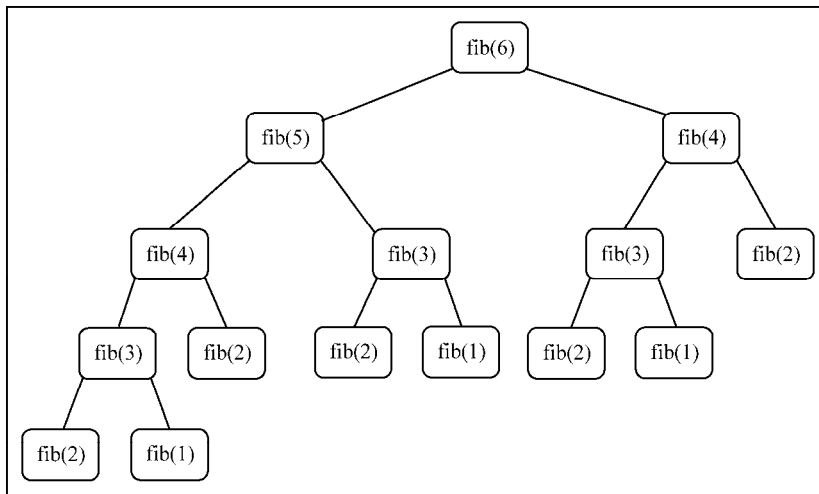
```
function fibonacci(num){
  if (num === 1 || num === 2){ //{1}
    return 1;
  }
}
```

边界条件是已知的，1和2的斐波那契数（行{1}）是1。现在，让我们完成斐波那契函数的实现：

```
function fibonacci(num){
  if (num === 1 || num === 2){
    return 1;
  }
  return fibonacci(num - 1) + fibonacci(num - 2);
}
```

我们已经知道，当 n 大于2时， $Fibonacci(n)$ 等于 $Fibonacci(n-1)+Fibonacci(n-2)$ 。

现在，斐波那契函数实现完毕。让我们试着找出6的斐波那契数，其会产生如下函数调用：



我们也可以用非递归的方式实现斐波那契函数：

```
function fib(num){
  var n1 = 1,
      n2 = 1,
      n = 1;
  for (var i = 3; i<=num; i++){
    n = n1 + n2;
    n1 = n2;
    n2 = n;
  }
  return n;
}
```

为何用递归呢？更快吗？递归并不比普通版本更快，反倒更慢。但要知道，递归更容易理解，并且它所需的代码量更少。



在ECMAScript 6中，因为尾调用优化的缘故，递归并不会更慢。但是在其他语言中，递归通常更慢。

所以，我们用递归，通常是因为它更容易解决问题。

11.2 动态规划

动态规划（Dynamic Programming，DP）是一种将复杂问题分解成更小的子问题来解决的优化技术。

本书之前提到过几次动态规划技术。用动态规划解决的一个问题是第9章中的深度优先搜索。



要注意动态规划和分而治之（归并排序和快速排序算法中用到的那种）是不同的方法。分而治之方法是把问题分解成相互独立的子问题，然后组合它们的答案，而动态规划则是将问题分解成相互依赖的子问题。

另一个例子是上一节解决的斐波那契问题。我们将斐波那契问题分解成如该节图示的小问题。

用动态规划解决问题时，要遵循三个重要步骤：

- (1) 定义子问题；
- (2) 实现要反复执行来解决子问题的部分（这一步要参考前一节讨论的递归的步骤）；
- (3) 识别并求解出边界条件。

能用动态规划解决的一些著名的问题如下。

- ❑ **背包问题**：给出一组项目，各自有值和容量，目标是找出总值最大的项目的集合。这个问题的限制是，总容量必须小于等于“背包”的容量。
- ❑ **最长公共子序列**：找出一组序列的最长公共子序列（可由另一序列删除元素但不改变余下元素的顺序而得到）。
- ❑ **矩阵链相乘**：给出一系列矩阵，目标是找到这些矩阵相乘的最高效办法（计算次数尽可能少）。相乘操作不会进行，解决方案是找到这些矩阵各自相乘的顺序。
- ❑ **硬币找零**：给出面额为 $d_1 \cdots d_n$ 的一定数量的硬币和要找零的钱数，找出有多少种找零的方法。
- ❑ **图的全源最短路径**：对所有顶点对 (u, v) ，找出从顶点 u 到顶点 v 的最短路径。我们在第9章已经学习过这个问题的Floyd-Warshall算法。

接下来几节，我们会一一讲解这些问题。



在Google、Amazon、Microsoft、Oracle等大公司的编程面试中，这些问题及其解决方案非常常见。

11.2.1 最少硬币找零问题

最少硬币找零问题是硬币找零问题的一个变种。硬币找零问题是给出要找零的钱数，以及可用的硬币面额 $d_1 \cdots d_n$ 及其数量，找出有多少种找零方法。最少硬币找零问题是给出要找零的钱数，以及可用的硬币面额 $d_1 \cdots d_n$ 及其数量，找到所需的最少的硬币个数。

例如，美国有以下面额（硬币）： $d_1=1$ ， $d_2=5$ ， $d_3=10$ ， $d_4=25$ 。

如果要找36美分的零钱，我们可以用1个25美分、1个10美分和1个便士（1美分）。

如何将这个解答转化成算法？

最少硬币找零的解决方案是找到 n 所需的最小硬币数。但要做到这一点，首先得找到对每个 $x < n$ 的解。然后，我们将解建立在更小的值的解的基础上。

来看看算法：

```
function MinCoinChange(coins){
  var coins = coins; //{1}
  var cache = {};    //{2}

  this.makeChange = function(amount) {
    var me = this;
    if (!amount) { //{3}
      return [];
    }
    if (cache[amount]) { //{4}
      return cache[amount];
    }
    var min = [], newMin, newAmount;
    for (var i=0; i<coins.length; i++){ //{5}
      var coin = coins[i];
      newAmount = amount - coin; //{6}
      if (newAmount >= 0){
        newMin = me.makeChange(newAmount); //{7}
      }
      if (
        newAmount >= 0 && //{8}
        (newMin.length < min.length-1 || !min.length) //{9}
        && (newMin.length || !newAmount) //{10}
      ) {
        min = [coin].concat(newMin); //{11}
        console.log('new Min ' + min + ' for ' + amount);
      }
    }
  }
}
```

```

    }
  }
  return (cache[amount] = min); //{12}
};
}

```

为了更有条理，我们创建了一个类，解决给定面额的最少硬币找零问题。让我们一步步解读这个算法。

`MinCoinChange`类接收`coins`参数（行{1}），该参数代表问题中的面额。对美国的硬币系统而言，它是`[1, 5, 10, 25]`。我们可以随心所欲传递任何面额。此外，为了更加高效且不重复计算值，我们使用了`cache`（行{2}）。

接下来是`makeChange`方法，它也是一个递归函数，找零问题由它解决。首先，若`amount`不为正（`<0`），就返回空数组（行{3}）；方法执行结束后，会返回一个数组，包含用来找零的各个面额的硬币数量（最少硬币数）。接着，检查`cache`缓存。若结果已缓存（行{4}），则直接返回结果；否则，执行算法。

我们基于`coins`参数（面额）解决问题。因此，对每个面额（行{5}），我们都计算`newAmount`（行{6}）的值，它的值会一直减小，直到能找零的最小钱数（别忘了本算法对所有的`x < amount`都会计算`makeChange`结果）。若`newAmount`是合理的值（正值），我们也会计算它的找零结果（行{7}）。

最后，我们判断`newAmount`是否有效，`minValue`（最少硬币数）是否是最优解，与此同时`minValue`和`newAmount`是否是合理的值（行{10}）。若以上判断都成立，意味着有一个比之前更优的答案（行{11}）。以5美分为例，可以给5便士或者1个5美分镍币，1个5美分镍币是最优解。最后，返回最终结果（行{12}）。

测试一下这个算法：

```

var minCoinChange = new MinCoinChange([1, 5, 10, 25]);
console.log(minCoinChange.makeChange(36));

```

要知道，如果我们检查`cache`变量，会发现它存储了从1到36美分的所有结果。以上代码的结果是`[1, 10, 25]`。

本书的源码中会有几行多余的代码，输出算法的步骤。例如，使用面额`[1, 3, 4]`，并对钱数6执行算法，会产生以下输出：

```

new Min 1 for 1
new Min 1,1 for 2
new Min 1,1,1 for 3
new Min 3 for 3
new Min 1,3 for 4
new Min 4 for 4

```



```
new Min 1,4 for 5
new Min 1,1,4 for 6
new Min 3,3 for 6
[3, 3]
```

所以，找零钱数为6时，最佳答案是两枚价值为3的硬币。

11.2.2 背包问题

背包问题是一个组合优化问题。它可以描述如下：给定一个固定大小、能够携重W的背包，以及一组有价值 and 重量的物品，找出一个最佳解决方案，使得装入背包的物品总重量不超过W，且总价值最大。

下面是一个例子：

物 品#	重 量	价 值
1	2	3
2	3	4
3	4	5

考虑背包能够携带的重量只有5。对于这个例子，我们可以说最佳解决方案是往背包里装入物品1和物品2，这样，总重量为5，总价值为7。



这个问题有两个版本。0-1版本只能往背包里装完整的物品，而**分数背包问题**则允许装入分数物品。在这个例子里，我们将处理该问题的0-1版本。动态规划对分数版本无能为力，但本章稍后要学习的贪心算法可以解决它。

我们来看看下面这个背包算法：

```
function knapSack(capacity, weights, values, n) {

  var i, w, a, b, kS = [];

  for (i = 0; i <= n; i++) { //{1}
    kS[i] = [];
  }

  for (i = 0; i <= n; i++) {
    for (w = 0; w <= capacity; w++) {
      if (i == 0 || w == 0) { //{2}
        kS[i][w] = 0;
      } else if (weights[i-1] <= w) { //{3}
        a = values[i-1] + kS[i-1][w-weights[i-1]];
        b = kS[i-1][w];
        kS[i][w] = (a > b) ? a : b; //{4} max(a,b)
      } else {
        kS[i][w] = kS[i-1][w]; //{5}
      }
    }
  }
}
```

```

    }
  }
}
return kS[n][capacity]; //{6}
}

```

我们来看看这个算法是如何工作的。

- 行{1}：首先，初始化将用于寻找解决方案的矩阵 $ks[n+1][capacity+1]$ 。
- 行{2}：忽略矩阵的第一列和第一行，只处理索引不为0的列和行。
- 行{3}：物品 i 的重量必须小于约束($capacity$)才有可能成为解决方案的一部分；否则，总重量就会超出背包能够携带的重量，这是不可能发生的。发生这种情况时，只要忽略它，用之前的值就可以了（行{5}）。
- 行{4}：当找到可以构成解决方案的物品时，选择价值最大的那个。
- 行{6}：最后，问题的解决方案就在这个二维表格右下角的最后一个格子里。

我们可以用开头的例子来测试这个算法：

```

var values = [3, 4, 5],
    weights = [2, 3, 4],
    capacity = 5,
    n = values.length;
console.log(knapSack(capacity, weights, values, n)); //输出 7

```

下图举例说明了例子中 ks 矩阵的构造：

1 i/w							2 i/w						
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0						1	0	0	3	3	3	3
2	0						2	0					
3	0						3	0					

物品
1: (2,3)
2: (3,4)
3: (4,5)

3 i/w							4 i/w						
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	1	0	0	3	3	3	3
2	0	0	3	4	4	7	2	0	0	3	4	4	7
3	0						3	0	0	3	4	5	7

物品
1: (2,3)
2: (3,4)
3: (4,5)

物品
1: (2,3)
2: (3,4)
3: (4,5)

请注意，这个算法只输出背包携带物品价值的最大值，而不列出实际的物品。我们可以增加下面的附加函数来找出构成解决方案的物品：

```

function findValues(n, capacity, kS, weights, values) {
  var i = n, k = capacity;

```

```

console.log('解决方案包含以下物品: ');

while (i > 0 && k > 0) {
  if (kS[i][k] !== kS[i-1][k]) {
    console.log('物品' + i + ', 重量: ' + weights[i-1] + ', 价值: ' + values[i-1]);
    i--;
    k = k - kS[i][k];
  } else {
    i--;
  }
}
}

```

我们可以在knapsack函数的行{6}之前调用这个函数。执行完整的算法，会得到如下输出：

```

解决方案包含以下物品:
物品2, 重量: 4, 价值: 3
物品1, 重量: 3, 价值: 2
总价值: 7

```



背包问题也可以写成递归形式。你可以在本书的源码包中找到它的递归版本。

11.2.3 最长公共子序列

另一个经常被当作编程挑战问题的动态规划问题是最长公共子序列（LCS）：找出两个字符串序列的最长子序列的长度。最长子序列是指，在两个字符串序列中以相同顺序出现，但不要求连续（非字符串子串）的字符串序列。

考虑如下例子：

字符串1	a	c	b	a	c	d
字符串2	a	b	c	a	d	f
LCS: 长度为4的 “acad”						

再看看下面这个算法：

```

function lcs(wordX, wordY) {
  var m = wordX.length,
      n = wordY.length,
      l = [],
      i, j, a, b;

  for (i = 0; i <= m; ++i) {
    l[i] = [];
    // {1}
  }
}

```

```

    for (j = 0; j <= n; ++j) {
        l[i][j] = 0;
        //{2}
    }
}

for (i = 0; i <= m; i++) {
    for (j = 0; j <= n; j++) {
        if (i == 0 || j == 0) {
            l[i][j] = 0;
        } else if (wordX[i-1] == wordY[j-1]) {
            l[i][j] = l[i-1][j-1] + 1;
            //{3}
        } else {
            a = l[i-1][j];
            b = l[i][j-1];
            l[i][j] = (a > b) ? a : b; //max(a, b)
            //{4}
        }
    }
}
//{5}
return l[m][n];
}

```

比较背包问题和LCS算法，我们会发现两者非常相似。这项从顶部开始构建解决方案的技术被称为记忆，而解决方案就在表格或矩阵的右下角。

像背包问题算法一样，这种方法只输出LCS的长度，而不包含LCS的实际结果。要提取这个信息，需要对算法稍作修改，声明一个新的solution矩阵。注意，代码中有一些注释，我们需要用以下代码替换这些注释。

```

□ 行{1}: solution[i] = [];
□ 行{2}: solution[i][j] = '0';
□ 行{3}: solution[i][j] = 'diagonal';
□ 行{4}: solution[i][j]=(l[i][j] == l[i-1][j]) ? 'top' : 'left';
□ 行{5}: printSolution(solution, l, wordX, wordY, m, n);

```

printSolution函数如下：

```

function printSolution(solution, l, wordX, wordY, m, n) {
    var a = m, b = n, i, j,
        x = solution[a][b],
        answer = '';

    while (x !== '0') {
        if (solution[a][b] === 'diagonal') {
            answer = wordX[a - 1] + answer;
            a--;
            b--;
        }
    }
}

```

```

    } else if (solution[a][b] === 'left') {
      b--;
    } else if (solution[a][b] === 'top') {
      a--;
    }
    x = solution[a][b];
  }
  console.log('lcs: ' + answer);
}

```

当解矩阵的方向为对角线时，我们可以将字符添加到答案中。

如果用 'acbaed' 和 'abcadf' 两个字符串执行上面的算法，我们将得到输出 4。用于构建结果的矩阵 1 看起来像下面这样。我们也可以用附加的算法来跟踪 LCS 的值（如下图高亮所示）。

		a	b	c	a	d	f
	0	0	0	0	0	0	0
a	0	1	1	1	1	1	1
c	0	1	1	2	2	2	2
b	0	1	2	2	2	2	2
a	0	1	2	2	3	3	3
e	0	1	2	2	3	3	3
d	0	1	2	2	3	4	4

通过上面的矩阵，我们知道 LCS 算法的结果是长度为 4 的 `acad`。



LCS 问题也可以写成递归形式。你可以在本书的源码包中找到它的递归版本。

11.2.4 矩阵链相乘

矩阵链相乘是另一个可以用动态规划解决的著名问题。这个问题是要找出一组矩阵相乘的最佳方式（顺序）。

让我们试着更好地理解这个问题。 n 行 m 列的矩阵 A 和 m 行 p 列的矩阵 B 相乘，结果是 n 行 p 列的矩阵 C 。

考虑我们想做 $A*B*C*D$ 的乘法。因为乘法满足结合律，所以我们可以让这些矩阵以任意顺序相乘。因此，考虑如下情况：

- ❑ A 是一个 10 行 100 列的矩阵
- ❑ B 是一个 100 行 5 列的矩阵

- C是一个5行50列的矩阵
- D是一个50行1列的矩阵
- $A*B*C*D$ 的结果是一个10行1列的矩阵

在这个例子里，相乘的方式有五种。

- (1) $(A(B(CD)))$: 乘法运算的次数是1750次。
- (2) $((AB)(CD))$: 乘法运算的次数是5300次。
- (3) $((AB)C)D$: 乘法运算的次数是8000次。
- (4) $((A(BC))D)$: 乘法运算的次数是75 500次。
- (5) $(A((BC)D))$: 乘法运算的次数是31 000次。

相乘的顺序不一样，要进行的乘法运算总数也有很大差异。那么，要如何构建一个算法，求出最少的乘法运算操作次数？矩阵链相乘的算法如下：

```
function matrixChainOrder(p, n) {
  var i, j, k, l, q, m = [];

  for (i = 1; i <= n; i++) {
    m[i] = [];
    m[i][i] = 0;
  }

  for (l = 2; l < n; l++) {
    for (i = 1; i <= n-l+1; i++) {
      j = i+l-1;
      m[i][j] = Number.MAX_SAFE_INTEGER;
      for (k = i; k <= j-1; k++) {
        q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]; //{1}
        if (q < m[i][j]){
          m[i][j] = q;
          //{2}
        }
      }
    }
  }
  //{3}
  return m[1][n-1];
}
```

整个算法中最重要的是行{1}，神奇之处全都在这一行。它计算了给定括号顺序的乘法运算次数，并将值保存在辅助矩阵m中。

对开头的例子执行上面的算法，会得到结果7500；正如我们前面提到的，这是最少的操作次数。看看这个：

```
var p = [10, 100, 5, 50, 1],
    n = p.length;
console.log(matrixChainOrder(p, n));
```

然而，这个算法也不会给出最优解的括号顺序。为了得到这些信息，我们可以对代码做一些改动。

首先，我们需要通过以下代码声明并初始化一个辅助矩阵s：

```
var s = [];  
for (i = 0; i <= n; i++) {  
  s[i] = [];  
  for (j = 0; j <= n; j++) {  
    s[i][j] = 0;  
  }  
}
```

然后，在matrixChainOrder函数的行{2}添加下面的代码：

```
s[i][j] = k;
```

在行{3}，我们调用打印括号的函数，如下：

```
printOptimalParenthesis(s, 1, n-1);
```

最后，我们的printOptimalParenthesis函数如下：

```
function printOptimalParenthesis(s, i, j) {  
  if (i == j) {  
    console.log("A[" + i + "]");  
  } else {  
    console.log("(");  
    printOptimalParenthesis(s, i, s[i][j]);  
    printOptimalParenthesis(s, s[i][j] + 1, j);  
    console.log(")");  
  }  
}
```

执行修改后的算法，也能得到括号的最佳顺序(A[1] (A[2] (A[3]A[4]))))，并可以转化为(A(B(CD)))。

11.3 贪心算法

贪心算法遵循一种近似解决问题的技术，期盼通过每个阶段的局部最优选择（当前最好的解），从而达到全局的最优（全局最优解）。它不像动态规划算法那样计算更大的格局。

我们来看看如何用贪心算法解决11.2节中的最少硬币找零问题和背包问题。



我们在第9章介绍了一些其他的贪心算法，比如Dijkstra算法、Prim算法和Kruskal算法。

11.3.1 最少硬币找零问题

最少硬币找零问题也能用贪心算法解决。大部分情况下的结果是最优的，不过对有些面额而言，结果不会是最优的。

来看看算法：

```
function MinCoinChange(coins){
  var coins = coins; //{1}

  this.makeChange = function(amount) {
    var change = [],
        total = 0;
    for (var i=coins.length; i>=0; i--){ //{2}
      var coin = coins[i];
      while (total + coin <= amount) { //{3}
        change.push(coin); //{4}
        total += coin; //{5}
      }
    }
    return change;
  };
}
```

不得不说贪心版本的MinCoinChange比动态规划版本的简单多了。和动态规划方法相似，我们传递面额参数，实例化MinCoinChange（行{1}）。

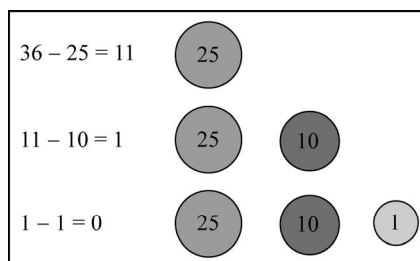
对每个面额（行{2}——从大到小），把它的值和total相加后，total需要小于amount（行{3}）。我们会将当前面额coin添加到结果中（行{4}），也会将它和total相加（行{5}）。

如你所见，这个解法很简单。从最大面额的硬币开始，拿尽可能多的这种硬币找零。当无法再拿更多这种价值的硬币时，开始拿第二大价值的硬币，依次继续。

用和DP方法同样的测试代码测试：

```
var minCoinChange = new MinCoinChange([1, 5, 10, 25]);
console.log(minCoinChange.makeChange(36));
```

结果依然是[25, 10, 1]，和用DP得到的一样。下图阐释了算法的执行过程：



然而，如果用[1, 3, 4]面额执行贪心算法，会得到结果[4, 1, 1]。如果用动态规划的解法，会得到最优的结果[3, 3]。

比起动态规划算法而言，贪心算法更简单、更快。然而，如我们所见，它并不总是得到最优答案。但是综合来看，它相对执行时间来说，输出了一个可以接受的解。

11.3.2 分数背包问题

求解分数背包问题的算法与动态规划版本稍有不同。在0-1背包问题中，只能向背包里装入完整的物品，而在分数背包问题中，我们可以装入分数的物品。我们用前面用过的例子来比较两者的差异，如下所示：

物品#	重 量	价 值
1	2	3
2	3	4
3	4	5

在动态规划的例子中，我们考虑背包能够携带的重量只有5。而在这个例子中，我们可以说最佳解决方案是往背包里装入物品1和物品2，总重量为5，总价值为7。

如果在分数背包问题中考虑相同的容量，得到的结果是一样的。因此，我们考虑容量为6的情况。

在这种情况下，解决方案是装入物品1和物品2，还有25%的物品3。这样，重量为6的物品总价值为8.25。

我们来看看下面这个算法：

```
function knapSack(capacity, values, weights) {
  var n = values.length,
      load = 0, i = 0, val = 0;

  for (i = 0; i < n && load < capacity; i++) { //{1}
    if (weights[i] <= (capacity - load)) { //{2}
      val += values[i];
      load += weights[i];
    } else {
      var r = (capacity - load) / weights[i]; //{3}
      val += r * values[i];
      load += weights[i];
    }
  }
  return w;
}
```

下面是对算法的解释。

- 行{1}: 总重量少于背包容量, 继续迭代, 装入物品。
- 行{2}: 如果物品可以完整地装入背包, 就将其价值和重量分别计入背包已装入物品的总价值 (val) 和总重量 (load)。
- 行{3}: 如果物品不能完整地装入背包, 计算能够装入部分的比例 (r)。

如果在0-1背包问题中考虑同样的容量6, 我们就会看到, 物品1和物品3组成了解决方案。在这种情况下, 对同一个问题应用不同的解决方法, 会得到两种不同的结果。

11.4 函数式编程简介

到目前为止, 我们在本书中所用的编程范式都是命令式编程。在命令式编程中, 我们按部就班地编写程序代码, 详细描述要完成的事情以及完成的顺序。

在本节中, 我们会介绍一种新的范式, 叫作函数式编程。函数式编程是一种曾经主要用于学术领域的范式, 多亏了Python和Ruby等现代语言, 它才开始在行业开发者中流行起来。值得欣慰的是, 借助ES6的能力, JavaScript也能够进行函数式编程。

11.4.1 函数式编程与命令式编程

以函数式范式进行开发并不简单; 关键在于习惯这种范式的机制。我们编写一个例子来说明差异。

假设我们想打印一个数组中所有的元素。我们可以用命令式编程, 声明的函数如下:

```
var printArray = function(array) {  
  for (var i = 0; i < array.length; i++) {  
    console.log(array[i]);  
  }  
};  
printArray([1, 2, 3, 4, 5]);
```

在上面的代码中, 我们迭代数组, 打印每一项。

现在, 我们试着把这个例子转换成函数式编程。在函数式编程中, 函数就是摇滚明星。我们关注的重点是需要描述什么, 而不是如何描述。回到这一句: “我们迭代数组, 打印每一项”。那么, 我们首先要关注的是迭代数据, 然后进行操作, 即打印数组项。下面的函数负责迭代数组:

```
var forEach = function(array, action) {  
  for (var i = 0; i < array.length; i++) {  
    action(array[i]);  
  }  
};
```

接下来，我们要创建另一个负责把数组元素打印到控制台的函数（考虑为回调函数），如下：

```
var logItem = function (item) {  
    console.log(item);  
};
```

最后，像下面这样使用声明的函数：

```
forEach([1, 2, 3, 4, 5], logItem);
```

只需要上面这一行代码，我们就能描述我们要把数组的每一项打印到控制台。这是我们的第一个函数式编程的例子！

有几点要注意：

- ❑ 主要目标是描述数据，以及要对数据应用的转换；
- ❑ 程序执行顺序的重要性很低，而在命令式编程中，步骤和顺序是非常重要的；
- ❑ 函数和数据集合是函数式编程的核心；
- ❑ 在函数式编程中，我们可以使用和滥用函数和递归，而在命令式编程中，则使用循环、赋值、条件和函数。

11.4.2 ES2015 和函数式编程

有了ES2015的新功能，用JavaScript进行函数式编程变得更加容易了。我们来看一个例子。

考虑我们要找出数组中最小的值。要用命令式编程完成这个任务，只要迭代数组，检查当前的最小值是否大于数组元素；如果是，就更新最小值，代码如下：

```
var findMinArray = function(array) {  
    var minValue = array[0];  
    for (var i = 1; i < array.length; i++) {  
        if (minValue > array[i]) {  
            minValue = array[i];  
        }  
    }  
    return minValue;  
};  
console.log(findMinArray([8, 6, 4, 5, 9])); //输出4
```

用函数式编程完成相同的任务，可以使用Math.min函数，传入所有要比较的数组元素。我们可以像下面的例子里这样，使用ES2015的解构操作符（...），把数组转换成单个的元素：

```
const min_ = function(array) {  
    return Math.min(...array)  
};  
console.log(min_([8, 6, 4, 5, 9])); //输出4
```

使用ES2015的箭头函数，我们可以进一步简化上面的代码：

```
const min = arr => Math.min(...arr);
console.log(min([8, 6, 4, 5, 9]));
```

11.4.3 JavaScript 函数式工具箱——map、filter 和 reduce

map、filter和reduce函数（第2章已经学习过）是函数式编程的基础。

我们可以使用map函数，把一个数据集转换或映射成另一个数据集。先看一个命令式编程的例子：

```
var daysOfWeek = [
  {name: 'Monday', value: 1},
  {name: 'Tuesday', value: 2},
  {name: 'Wednesday', value: 7}
];

var daysOfWeekValues_ = [];
for (var i = 0; i < daysOfWeek.length; i++) {
  daysOfWeekValues_.push(daysOfWeek[i].value);
}
```

再以函数式编程来考虑同样的例子，代码如下：

```
var daysOfWeekValues = daysOfWeek.map(function(day) {
  return day.value;
});
console.log(daysOfWeekValues);
```

我们可以使用filter函数过滤一个集合的值。来看一个例子：

```
var positiveNumbers_ = function(array) {
  var positive = [];
  for (var i = 0; i < array.length; i++) {
    if (array[i] >= 0) {
      positive.push(array[i]);
    }
  }
  return positive;
}
console.log(positiveNumbers_([-1, 1, 2, -2]));
```

我们可以把同样的代码写成函数式的，如下：

```
var positiveNumbers = function(array) {
  return array.filter(function(num) {
    return num >= 0;
  });
};
console.log(positiveNumbers([-1, 1, 2, -2]));
```

我们也可以使用`reduce`函数，把一个集合归约成一个特定的值。比如，对一个数组中的值求和：

```
var sumValues = function(array) {
  var total = array[0];
  for (var i = 1; i < array.length; i++) {
    total += array[i];
  }
  return total;
};
console.log(sumValues([1, 2, 3, 4, 5]));
```

上面的代码也可以写成这样：

```
var sum_ = function(array) {
  return array.reduce(function(a, b) {
    return a + b;
  });
};
console.log(sum_([1, 2, 3, 4, 5]));
```

我们还可以把这些函数与ES2015的功能结合起来，比如解构操作符和箭头函数，代码如下：

```
const sum = arr => arr.reduce((a, b) => a + b);
console.log(sum([1, 2, 3, 4, 5]));
```

我们再看另一个例子。考虑我们需要写一个函数，把几个数组连接起来。为此，可以创建另一个数组，用于存放其他数组的元素。我们可以执行以下命令式的代码：

```
var mergeArrays_ = function(arrays) {
  var count = arrays.length,
      newArray = [],
      k = 0;
  for (var i = 0; i < count; i++) {
    for (var j = 0; j < arrays[i].length; j++) {
      newArray[k++] = arrays[i][j];
    }
  }
  return newArray;
};
console.log(mergeArrays_([[1, 2, 3], [4, 5], [6]]));
```

注意，在这个例子中，我们声明了变量，还使用了循环。现在，我们用JavaScript函数式编程把上面的代码重写如下：

```
var mergeArraysConcat = function(arrays) {
  return arrays.reduce(function(p, n) {
    return p.concat(n);
  });
};
console.log(mergeArraysConcat([[1, 2, 3], [4, 5], [6]]));
```

上面的代码完成了同样的任务，但它是面向函数的。我们也可以用ES2015使代码更加精简，如下所示：

```
const mergeArrays = (...arrays) => [].concat(...arrays);
console.log(mergeArrays([1, 2, 3], [4, 5], [6]));
```

从11行代码变成了只有一行！



如果你想更多地练习JavaScript函数式编程，可以试试这些习题，非常有意思：<http://reactivex.io/learnrx/>。

11.4.4 JavaScript 函数式类库和数据结构

有一些很棒的JavaScript类库借助工具函数和函数式数据结构，对函数式编程提供支持。通过下面的列表，你可以找到一些最有名的JavaScript函数式类库。

- ❑ **Underscore.js**: <http://underscorejs.org/>
- ❑ **Bilby.js**: <http://bilby.brianmckenna.org/>
- ❑ **Lazy.js**: <http://danieltao.com/lazy.js/>
- ❑ **Bacon.js**: <https://baconjs.github.io/>
- ❑ **: <http://eliperelman.com/fn.js/>**
- ❑ **Functional.js**: <http://functionaljs.com/>
- ❑ **Ramda.js**: <http://ramdajs.com/0.20.1/index.html>
- ❑ **Mori**: <http://swannodette.github.io/mori/>



如果你对学习JavaScript函数式编程感兴趣，可以看看Packt出版的另一本书：<https://www.packtpub.com/web-development/functional-programming-javascript>。

11.5 小结

在本章中，你了解了更多递归的知识，以及它如何帮助我们解决一些动态规划问题。我们介绍了最著名的动态规划问题，如最少硬币找零问题、背包问题、最长公共子序列和矩阵链相乘。

你学习了贪心算法，以及如何用贪心算法解决最少硬币找零问题和分数背包问题。

你还学习了函数式编程，并通过一些例子了解了如何以这种范式使用JavaScript的功能。

下一章，我们会介绍大O表示法，并讨论如何计算一个算法的复杂性。你还将学习更多存在于算法世界里的概念。

第 12 章

算法复杂度

12

本章，我们要学习著名的大O表示法（第10章提到过）和NP完全理论，还要看看如何用算法增添乐趣，巩固知识，提高我们编程和解决问题的能力。

12.1 大 O 表示法

第10章引入了大O表示法的概念。它的确切含义是什么？它用于描述算法的性能和复杂程度。

分析算法时，时常遇到以下几类函数：

符 号	名 称
$O(1)$	常数的
$O(\log(n))$	对数的
$O((\log(n))^c)$	对数多项式的
$O(n)$	线性的
$O(n^2)$	二次的
$O(n^c)$	多项式的
$O(c^n)$	指数的

12.1.1 理解大 O 表示法

如何衡量算法的效率？通常是用资源，例如CPU（时间）占用、内存占用、硬盘占用和网络占用。当讨论大O表示法时，一般考虑的是CPU（时间）占用。

让我们试着用一些例子来理解大O表示法的规则。

1. $O(1)$

考虑以下函数：

```
function increment(num) {  
    return ++num;  
}
```

假设运行 `increment(1)` 函数，执行时间等于 X 。如果再用不同的参数（例如2）运行一次 `increment` 函数，执行时间依然是 X 。和参数无关，`increment` 函数的性能都一样。因此，我们说上述函数的复杂度是 $O(1)$ （常数）。

2. $O(n)$

现在以第10章中实现的顺序搜索算法为例：

```
function sequentialSearch(array, item){
  for (var i=0; i<array.length; i++){
    if (item === array[i]){ //{1}
      return i;
    }
  }
  return -1;
}
```

如果将含10个元素的数组（`[1, ..., 10]`）传递给该函数，假如搜索1这个元素，那么，第一次判断时就能找到想要搜索的元素。在这里我们假设每执行一次行{1}，开销是1。

现在，假如要搜索元素11。行{1}会执行10次（遍历数组中所有的值，并且找不到要搜索的元素，因而结果返回-1）。如果行{1}的开销是1，那么它执行10次的开销就是10，10倍于第一种假设。

现在，假如该数组有1000个元素（`[1, ..., 1000]`）。搜索1001的结果是行{1}执行了1000次（然后返回-1）。

注意，`sequentialSearch` 函数执行的总开销取决于数组元素的个数（数组大小），而且也和搜索的值有关。如果是查找数组中存在的值，行{1}会执行几次呢？如果查找的是数组中不存在的值，那么行{1}就会执行和数组大小一样多次，这就是通常所说的最坏情况。

最坏情况下，如果数组大小是10，开销就是10；如果数组大小是1000，开销就是1000。可以得出 `sequentialSearch` 函数的时间复杂度是 $O(n)$ ， n 是（输入）数组的大小。

回到之前的例子，修改一下算法的实现，使之计算开销：

```
function sequentialSearch(array, item){
  var cost = 0;
  for (var i=0; i<array.length; i++){
    cost++;
    if (item === array[i]){ //{1}
      return i;
    }
  }
  console.log('cost for sequentialSearch with input size ' +
    array.length + ' is ' + cost);
  return -1;
}
```


用不同大小的输入数组执行以上算法，可以看到不同的输出。

3. $O(n^2)$

用冒泡排序做 $O(n^2)$ 的例子：

```
function swap(array, index1, index2){
    var aux = array[index1];
    array[index1] = array[index2];
    array[index2] = aux;
}

function bubbleSort(array){
    var length = array.length;
    for (var i=0; i<length; i++){           //{1}
        for (var j=0; j<length-1; j++ ){ //{2}
            if (array[j] > array[j+1]){
                swap(array, j, j+1);
            }
        }
    }
}
```

假设行{1}和行{2}的开销分别是1。修改算法的实现使之计算开销：

```
function bubbleSort(array){
    var length = array.length;
    var cost = 0;
    for (var i=0; i<length; i++){ //{1}
        cost++;
        for (var j=0; j<length-1; j++ ){ //{2}
            cost++;
            if (array[j] > array[j+1]){
                swap(array, j, j+1);
            }
        }
    }
    console.log('cost for bubbleSort with input size ' + length + '
    is ' + cost);
}
```

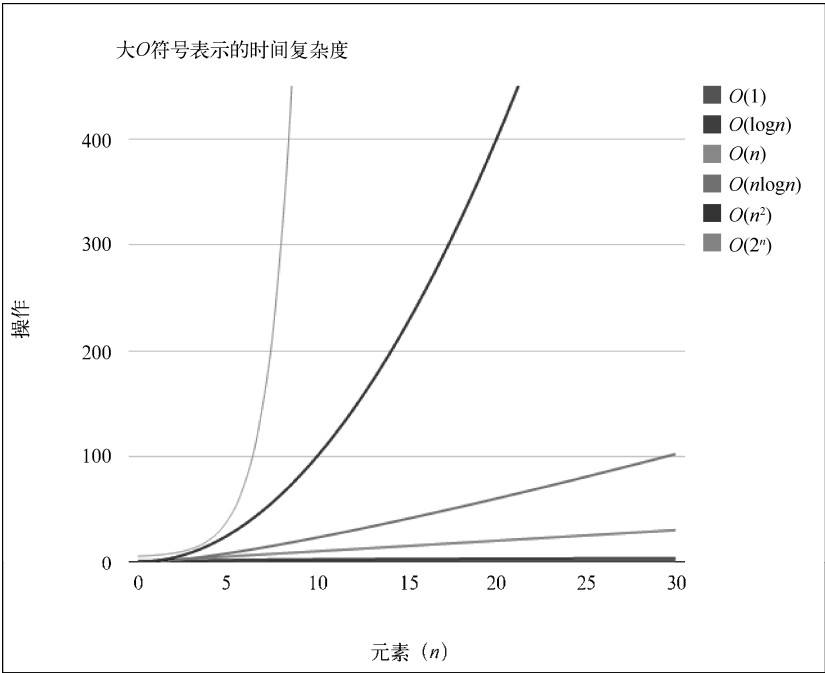
如果用大小为10的数组执行bubbleSort，开销是 100 (10^2)。如果用大小为100的数组执行bubbleSort，开销就是 10 000 (100^2)。需要注意，我们每次增加输入的大小，执行都会越来越久。



时间复杂度 $O(n)$ 的代码只有一层循环，而 $O(n^2)$ 的代码有双层嵌套循环。如果算法有三层遍历数组的嵌套循环，它的时间复杂度很可能就是 $O(n^3)$ 。

12.1.2 时间复杂度比较

下图比较了前述各个大 O 符号表示的时间复杂度：



这个图表是用JavaScript绘制的哦！在本书示例代码中，你可以到Chapter12下的bigOChart目录中找到绘制本图表的源代码。

在接下来的部分，你可以找到本书实现的所有算法的时间复杂度的速查表。

1. 数据结构

下表是常用数据结构的时间复杂度：

数据结构	一般情况			最差情况		
	插入	删除	搜索	插入	删除	搜索
数组/栈/队列	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
链表	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
双向链表	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
散列表	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
二分搜索树	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$
AVL树	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$

2. 图

下表是图的时间复杂度：

节点/边的管理方式	存储空间	增加顶点	增加边	删除顶点	删除边	轮 询
邻接表	$O(V + E)$	$O(1)$	$O(1)$	$O(V + E)$	$O(E)$	$O(V)$
邻接矩阵	$O(V ^2)$	$O(V ^2)$	$O(1)$	$O(V ^2)$	$O(1)$	$O(1)$

3. 排序算法

下表是排序算法的时间复杂度：

算法（用于数组）	时间复杂度		
	最好情况	一般情况	最差情况
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$
归并排序	$O(n\log(n))$	$O(n\log(n))$	$O(n\log(n))$
快速排序	$O(n\log(n))$	$O(n\log(n))$	$O(n^2)$
堆排序	$O(n\log(n))$	$O(n\log(n))$	$O(n\log(n))$
桶排序	$O(n+k)$	$O(n+k)$	$O(n^2)$
基数排序	$O(nk)$	$O(nk)$	$O(nk)$

4. 搜索算法

下表是搜索算法的时间复杂度：

算 法	数据结构	最差情况
顺序搜索	数组	$O(n)$
二分搜索	已排序的数组	$O(\log(n))$
深度优先搜索（DPS）	顶点数为 $ V $ ，边数为 $ E $ 的图	$O(V + E)$
广度优先搜索（BFS）	顶点数为 $ V $ ，边数为 $ E $ 的图	$O(V + E)$

12.1.3 NP 完全理论概述

一般来说，如果一个算法的复杂度为 $O(n^k)$ ，其中 k 是常数，我们就认为这个算法是高效的，这就是多项式算法。

对于给定的问题，如果存在多项式算法，则计为 P （polynomial，多项式）。

还有一类NP（nondeterministic polynomial，非确定性多项式）算法。如果一个问题可以在多项式时间内验证解是否正确，则计为NP。

如果一个问题存在多项式算法，自然可以在多项式时间内验证其解。因此，所有的P都是NP。然而， $P = NP$ 是否成立，仍然不得而知。

NP问题中最难的是NP完全问题，它满足以下两个条件：

- (1) 是NP问题，也就是说，可以在多项式时间内验证解，但还没有找到多项式算法；
- (2) 所有的NP问题都能在多项式时间内归约为它。

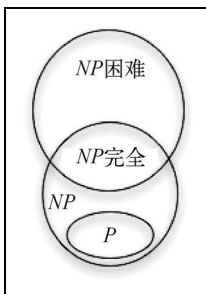
为了理解问题的归约，考虑两个决策问题L和M。假设法A可以解决问题L，算法B可以验证输入y是否为M的解。目标是找到一个把L转化为M的方法，使得算法B可以用于构造算法A。

还有一类问题，只需满足NP完全问题的第二个条件，称为NP困难问题。因此，NP完全问题也是NP困难问题的子集。



$P = NP$ 是否成立，是计算机科学中最重要的难题之一。如果能找到答案，对密码学、算法研究、人工智能等诸多领域都会产生重大影响。

下面是满足 $P \neq NP$ 时，P、NP、NP完全和NP困难问题的欧拉图：



非NP完全的NP困难问题的例子有停机问题和布尔可满足性问题（SAT）。

NP完全问题的例子有子集和问题、旅行商问题、顶点覆盖问题，等等。



关于这些问题，详情请查阅<https://en.wikipedia.org/wiki/NP-completeness>。

不可解问题与启发式算法

我们提到的有些问题是不可解的。然而，仍然有办法在符合要求的时间内找到一个近似解。

启发式算法就是其中之一。启发式算法得到的未必是最优解，但足够解决问题了。

启发式算法的例子有局部搜索、遗传算法、启发式导航、机器学习等。详情请查阅 <http://goo.gl/gxlu9w>。



启发式算法可以很巧妙地解决一些问题。你可以尝试把研究启发式算法作为学士或硕士学位的论文主题。

12.2 用算法娱乐身心

我们学习算法并不单单是因为它是大学必修课，也不单单是因为我们想成为开发者。通过用在本书中学到的算法来解决问题，我们可以提高解决问题的能力，进而成为更棒的专业人士。

增长（解题）知识的最好方式是练习，而练习不一定是枯燥的。本节将展示一些网站，你可以访问它们并尝试从算法中获到快乐（甚至小赚一笔）。

这里列出一些有用的网站（有些不支持用JavaScript提交解答，但是我们依然可以将从本书中所学到的逻辑应用到其他语言上）。

- ❑ **UVa Online Judge** (<http://uva.onlinejudge.org/>): 这个网站包含了世界各大赛事的题目，包括由IBM赞助的ACM国际大学生程序竞赛（ICPC。若你依然在校，应尽量参与这项赛事，如果团队获胜，则有可能免费享受一次国际旅行）。这个网站包括了成百上千的题目，可以应用本书所学的算法。
- ❑ **Sphere Online Judge** (<http://www.spoj.com/>): 这个网站和UVa Online Judge差不多，但支持用更多语言解题（包括JavaScript）。
- ❑ **Coder Byte** (<http://coderbyte.com/>): 这个网站包含了74个可以用JavaScript解答的题目（简单、中等难度和非常困难）。
- ❑ **Project Euler** (<https://projecteuler.net/>): 这个网站包含了一系列数学/计算机的编程题目。你所要做的就是输入那些题目的答案，不过我们可以用算法来找到正确的解答。
- ❑ **Hacker Rank** (<https://www.hackerrank.com>): 这个网站包含了263个挑战，分为16个类别（可以应用本书中的算法和更多其他算法）。它也支持JavaScript和其他语言。
- ❑ **Code Chef** (<http://www.codechef.com/>): 这个网站包含一些题目，并会举办在线比赛。
- ❑ **Top Coder** (<http://www.topcoder.com/>): 此网站会举办算法联赛，这些联赛通常由NASA、Google、Yahoo!、Amazon和Facebook这样的公司赞助。参加其中一些赛事，你可以获得到赞助公司工作的机会，而参与另一些赛事会赢得奖金。这个网站也提供很棒的解题和算法教程。

以上网站的另一个好处是，它们通常给出的是真实世界中的问题，而我们需要鉴别用哪一个

算法解决它。通过这样的方式也能让我们明白本书中的算法并非局限于学术，而是能应用到现实问题上。

如果你想从事技术工作，强烈推荐你创建一个免费的GitHub (<https://github.com>) 账号，你可以将上述网站的解答代码提交上去。如果你没有任何专业经验，GitHub可以帮助你建立一个作品集，还会对你找到第一份工作有帮助！

12.3 小结

本章，我们学习了大 O 表示法，以及如何运用它计算算法的复杂度。我们也介绍了NP完全理论。如果你想进一步了解如何面对无解难题，如何用启发式算法得到一个近似满足的方案，这是你可以深入探索的一个领域。

另外我们还列出了一些网站。你可以免费注册这些网站，并应用你从本书中学到的知识，甚至还可能得到第一份IT行业的工作！

编程快乐！

延伸阅读



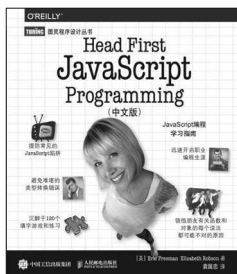
- 打通JavaScript语言的任督二脉
- 领略语言内部的绝美风光

书号: 978-7-115-38573-4
定价: 49.00 元



- 深入挖掘JavaScript语言本质
- 简练形象地解释抽象概念

书号: 978-7-115-43116-5
定价: 79.00 元



- Head First系列经典畅销书
- 囊括JavaScript语言的方方面面
- 为独立打造Web应用程序打下坚实的基础

书号: 978-7-115-45841-4
定价: 129.00 元



- 掌握JavaScript基础知识要点及其现代技术和工具
- 用正确的编码风格开发Web应用

书号: 978-7-115-46569-6
定价: 39.00 元



微信连接



回复“JavaScript” 查看相关书单



微博连接

关注@图灵教育 每日分享IT好书



QQ连接

图灵读者官方群I: 218139230

图灵读者官方群II: 164939616

图灵社区
iTuring.cn

在线出版,电子书,《码农》杂志,图灵访谈

数据结构是计算机为了高效地利用资源而组织数据的一种方式。数据结构和算法是解决一切编程问题的基础。

本书首先介绍了JavaScript语言的基础知识，接着讨论了数组、栈、队列和链表等重要的数据结构，随后分析了集合、字典和散列表的工作原理，接下来阐述了什么是树以及如何使用二叉树和二叉搜索树，然后介绍了图、DFS和BFS算法，以及各种排序（冒泡排序、选择排序、插入排序、归并排序、快速排序等）和搜索（顺序搜索、二分搜索）算法，最后介绍了动态规划和贪心算法等高级算法。

相较上一版，这一版新增了ES6和ES7的新功能介绍，补充了ES6的当前实现。同时拓展了对树、图、排序算法、动态规划和贪心算法的讨论，增加了AVL树、Dijkstra算法、Floyd-Warshall算法、Prim算法、Kruskal算法、堆排序、分布式排序、背包问题、矩阵链相乘等内容。此外还概述了函数式编程、NP完全理论。

如果你是计算机科学专业的学生，或是刚刚开启职业生涯的技术人员，想探索JavaScript的能力，这本书一定适合你。

Amazon读者评论

“这本书非常适合用来学习数据结构与算法。书中的例子写得很好，易于学习和实践。其教学方法也比一般的C/C++图书好得多。我向很多人推荐了这本书，尤其是从其他语言转到JavaScript的人。我看过各种编程语言的很多书籍和参考指南，这一本是其中难得的佳作。”

“如果你没上过算法课程，但是想学习实现常用的JavaScript数据结构和算法，或者拥有JavaScript背景，并想提升技能，那么一定要看看这本书！”

[PACKT]
PUBLISHING

图灵社区: iTuring.cn

热线: (010)51095186转600

分类建议 计算机/Web开发/JavaScript

人民邮电出版社网址: www.ptpress.com.cn

ISBN 978-7-115-46753-9



9 787115 467539 >

ISBN 978-7-115-46753-9

定价: 49.00元

看完了

如果您对本书内容有疑问，可发邮件至 contact@turingbook.com，会有编辑或译者协助答疑。也可访问图灵社区，参与本书讨论。

如果是有关电子书的建议或问题，请联系专用客服邮箱：
ebook@turingbook.com。

在这可以找到我们：

微博 @图灵教育：好书、活动每日播报

微博 @图灵社区：电子书和好文章的消息

微博 @图灵新知：图灵教育的科普小组

微信 图灵访谈：ituring_interview，讲述码农精彩人生

微信 图灵教育：turingbooks